



BUKU KERJA PRAKTIK MAHASISWA (BKPM)

**WORKSHOP SISTEM INFORMASI
BERBASIS WEB 2**

**TIF430805
SEMESTER 4**

OLEH :

- 1. Rifqi Aji Widarso, S.T., M.T.**
- 2. Sholihah Ayu Wulandari, S.ST., M.Tr.T.**



**PROGRAM STUDI TEKNIK INFORMATIKA PSDKU SIDOARJO
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER TAHUN 2026**

KEMENTERIAN PENDIDIKAN TINGGI SAINS DAN TEKNOLOGI
POLITEKNIK NEGERI JEMBER

LEMBAR PENGESAHAN

BUKU KERJA PRAKTIKUM MAHSISWA (BPKM)
WORKSHOP SISTEM INFORMASI BERBASIS WEB 2
TIF430805

Mengetahui,
Koordinator Program Studi



Rani Purbaningtyas, S.Kom, M.T
NIP. 19820312 2005 01 2 002

Koord./Tim Matakuliah



Sholihah Ayu Wulandari, S.ST., M.Tr.T.
NIP. 199311242024062003

Jember, 21 Juli 2026
Penulis



Rifqi Aji Widarso, S.T., M.T.
NIP. 199012072024061001

Menyetujui,
Ketua Jurusan,



Hendra Yufit Riskiawan, S.Kom, M.Cs
NIP. 19830203 200604 1 003

KATA PENGANTAR

Puji Syukur kehadiran Tuhan Yang Maha Esa karena atas limpahan rahmat-Nya sehingga kami dapat menyelesaikan Rencana Pembelajaran Semester BKPM Workshop Sistem Informasi Berbasis Web 2. Matakuliah ini mengajarkan kepada mahasiswa tentang proses pengembangan aplikasi berbasis Web Framework dengan bahasa pemrograman PHP. Pemilihan Framework berbasis bahasa pemrograman PHP ini didasarkan pada masih tingginya penggunaan bahasa PHP untuk pengembangan WEB, sehingga lulusan diharapkan memiliki kemampuan bekerja di industri pengembangan aplikasi berbasis WEB. Matakuliah ini juga mengajarkan proses pengembangan dengan pendekatan OOP, proses pengembangan yang dimaksud didasarkan pada penggunaan aplikasi version control. Version control adalah sebuah sistem yang mencatat setiap perubahan terhadap sebuah berkas atau kumpulan berkas sehingga pada suatu saat anda dapat kembali kepada salah satu versi dari berkas tersebut. Tujuan akhir dari matakuliah ini adalah mahasiswa yang mampu bersaing dalam industri pengembangan aplikasi berbasis WEB. BKPM ini disusun berdasarkan metode Project Base Learning yaitu pendekatan pengajaran yang dibangun di atas kegiatan pembelajaran dan tugas nyata yang memberikan tantangan bagi peserta didik yang terkait dengan kehidupan sehari-hari untuk dipecahkan secara berkelompok. Kami menyadari masih banyak kekurangan dalam penyusunan BKPM ini. Oleh karena itu, kami sangat mengharapkan kritik dan saran demi perbaikan dan kesempurnaan modul ini. Kami mengucapkan terima kasih kepada berbagai pihak yang telah membantu proses penyelesaian RPS ini. Semoga BKPM ini dapat bermanfaat bagi kita semua.

Sidoarjo, 21 Juli 2026

Penulis,

Tim Penulis.

DAFTAR ISI

KATA PENGANTAR.....	3
Acara 1. Analisis Kebutuhan Sistem.....	11
Acara 2. Use Case Diagram dan Skenario	14
Acara 3. Wireframing dengan Figma	18
Tampilan 1: Login	19
Tampilan 4: Tambah Barang	21
Tampilan 6: Kategori	22
Tampilan 8: Pinjam Barang.....	23
Tampilan 10: Profile.....	24
Acara 4. Perancangan ERD dan Struktur Tabel	25
Legenda ERD.....	28
2.2 Tabel kategoris (Kategori Barang)	29
2.3 Tabel barangs (Barang Inventaris).....	30
2.4 Tabel transaksis (Transaksi Peminjaman)	31
2.5 Tabel barang_user (Favorit Pivot).....	32
3.1 users (1) : transaksis (*)	33
3.2 kategoris (1) : barangs (*).....	33
3.3 barangs (1) : transaksis (*).....	33
3.4 users (*) : barangs (*)	33
Acara 5. Instalasi Laravel dan Konfigurasi Awal.....	35
Acara 6. Konfigurasi Environment dan Git.....	45
Acara 7. Membuat Model dan Migration User	54
Membuat Model.....	57
Mendefinisikan Relationship	57
Membuat Seeder	58
Menjalankan Migration & Seeder.....	58
Migration: users table.....	58
Migration: kategoris table	59
Migration: barangs table	59
Migration: transaksis table	60
Migration: barang_user (pivot).....	60
Model: User.php	61
Model: Kategori.php.....	62
Model: Barang.php	62

Model: Transaksi.php.....	63
Seeder: DatabaseSeeder.php.....	63
Seeder: UserSeeder.php	64
Seeder: KategoriSeeder.php.....	64
Seeder: BarangSeeder.php	65
Acara 8. Implementasi Auth Scaffolding	66
1. Install Laravel Breeze	67
2. Install package Breeze melalui Composer: composer require laravel/breeze -dev.....	67
3. Install stack Blade: php artisan breeze.....	67
4. Install dependencies NPM: npm install && npm run build	67
5. Jalankan migration: php artisan migrate	67
6. Route Auth.....	67
7. Controller Auth.....	68
8. View Auth.....	68
9. Layout Guest & App.....	69
10. Dashboard	69
11. Profile	69
Route: web.php	70
Controller: AuthenticatedSessionController.....	71
Controller: RegisteredUserController	71
Controller: ProfileController.....	72
Request: LoginRequest	72
Layout: guest.blade.php.....	73
View: login.blade.php	73
View: register.blade.php	74
View: dashboard.blade.php	74
Controller: DashboardController.....	75
View: profile/edit.blade.php	75
Acara 9. Middleware dan Role User.....	77
a) Membuat Middleware.....	78
b) Mendaftarkan Middleware	79
c) Menggunakan Middleware di Route	79
d) Role di Model & Migration.....	80
e) Role-Based Scoping di Controller	80

f) Seeder untuk Role.....	80
Middleware: Role.php	81
Model: User.php (role & isAdmin)	82
Config: bootstrap/app.php (register alias)	82
Route: web.php (auth middleware group).....	83
Controller: TransaksiController (role scoping)	83
Migration: users table (role column).....	84
Seeder: UserSeeder (admin & staff).....	84
Controller: RegisteredUserController (role default).....	84
Acara 10. Dashboard Role-Based Access	86
Membuat Layout Utama (app.blade.php)	87
Membuat Sidebar Navigasi	88
Membuat Topbar / Navbar.....	88
Membuat Halaman Dashboard	89
Membuat Halaman CRUD	89
Konfigurasi Routes.....	90
Acara 11. Template Admin dengan Blade	94
Membuat Middleware Role	96
Menambahkan Role di User Model	97
Membuat View Dashboard	97
Mendaftarkan Route	98
Uji Coba Login Admin & Staff	99
Acara 12. Layout dan Component Blade	106
Membuat Komponen Card.....	107
Membuat Komponen FormInput.....	108
Membuat Komponen DataTable.....	109
Menggunakan Komponen di Dashboard	110
Menggunakan Komponen di Halaman CRUD	111
Acara 13. CRUD Kategori - Create dan Read.....	120
Membuat Migration.....	121
Menjalankan Migration	122
Membuat Model Kategori.....	122
Membuat KategoriController	123
Implementasi Method Controller	124
Membuat View Index	124

Membuat View Create & Edit	125
Menambahkan Route	127
Acara 14. CRUD Kategori – Update dan Delete	137
Langkah 1: Membuat Form Request KategoriRequest	138
Langkah 2: Implementasi Method edit() dan update()	139
Langkah 3: Implementasi Method destroy() dengan Pengecekan Relasi	140
Langkah 4: Membuat View edit.blade.php	141
Langkah 5: Menambahkan Tombol Edit dan Delete di Index	143
Langkah 6: Menambahkan Flash Messages	143
Uji Coba Hapus Kategori	145
Acara 15. CRUD Barang dengan Relasi Kategori	148
Langkah 2: Membuat Model Barang	150
Langkah 3: Membuat BarangController	151
Method index()	151
Method create() dan store()	151
Method edit(), update(), destroy()	152
Langkah 4: Menambahkan Route Resource	153
Langkah 5: Membuat View Create dengan Select Kategori	153
Menampilkan Nama Kategori di Index	155
Acara 16. Upload Gambar Barang	158
Langkah 2: Implementasi Upload di Method store()	160
Langkah 3: Implementasi Upload di Method update()	161
Langkah 4: Validasi Upload Gambar	161
Langkah 5: Menambahkan Input File di Form Create	162
Menampilkan Thumbnail di Index	164
Menampilkan Gambar di Detail	165
Acara 17. CRUD Transaksi Peminjaman	166
Membuat Migration Transaksi	167
Menjalankan Migrasi	168
Membuat Model Transaksi	168
Membuat Controller Transaksi	169
Mendaftarkan Route	170
Membuat View Create (Form Peminjaman)	171
Membuat View Index (Daftar Transaksi)	172
Uji Coba Peminjaman Barang	172

Uji Coba Pengembalian Barang	173
Acara 18. CRUD Transaksi Pengembalian	180
Method pengembalian() di Controller	181
Method store() - Stok Berkurang	182
Method destroy() - Stok Recovery.....	182
Route Pengembalian	183
Tombol Kembalikan di View Index.....	183
Riwayat Transaksi di Dashboard.....	184
Uji Coba Pengembalian	184
Acara 19. Relasi One-to-Many dan Query Lanjutan	190
Relasi hasMany di Model Kategori.....	191
Relasi belongsTo + hasMany di Model Barang	191
Relasi hasMany di Model User	192
Eager Loading dengan with() di Controller	192
withCount() di KategoriController	193
Menampilkan Data Relasi di View.....	193
Uji Coba dan Debugbar (N+1 Detection)	195
Acara 20. Relasi Many-to-Many dan Seeder	201
Membuat Migration Pivot Table.....	202
Menjalankan Migrasi.....	203
Relasi belongsToMany di Model User	203
Relasi belongsToMany di Model Barang	203
Membuat DatabaseSeeder.....	204
Menjalankan Seeder.....	204
Uji Coba Relasi dengan Tinker.....	205
Acara 21. Ajax Request dengan jQuery	210
Membuat Route API Endpoint.....	212
Method apiIndex() di Controller.....	212
Membuat View Ajax Test	213
Script jQuery untuk Ajax Request.....	214
Uji Coba dan Inspeksi Network.....	214
Acara 22. DataTables Server Side	218
Instalasi DataTables dan Yajra.....	219
Membuat Route DataTables	219
Method dataTable() di Controller	220

Membuat View dengan DataTables.....	221
Uji Coba Sorting, Searching, Pagination.....	221
Acara 23. Filter Data dan Pencarian.....	226
Filter di BarangController	228
Filter di TransaksiController	228
Membuat Form Filter di View	229
Mempertahankan Filter dengan withQueryString()	230
Filter Transaksi Berdasarkan Status.....	231
Uji Coba Filter dan Pencarian.....	231
Acara 24. Export ke PDF dengan DOMPDF	236
Instalasi DOMPDF.....	237
Membuat LaporanController.....	238
Membuat View PDF Barang	238
Membuat View PDF Transaksi.....	239
Mendaftarkan Route	239
Uji Coba Download PDF	240
Acara 25. Export ke Excel.....	244
Instalasi Laravel Excel.....	245
Membuat Export Class Barang	246
Membuat Export Class Transaksi	246
Method di Controller.....	247
Mendaftarkan Route.....	248
Uji Coba Download Excel.....	248
Acara 26. Debugging dengan Laravel Debugbar	253
Instalasi Debugbar.....	255
Logging di Controller.....	255
Exception Handler	256
Custom View 404	256
Custom View 500	257
Uji Coba Debugbar & Log	257
Acara 27. Unit Test dan Feature Test.....	261
Konfigurasi phpunit.xml.....	263
Membuat Unit Test Model.....	263
Membuat Feature Test Auth.....	264
Membuat Feature Test Profile.....	265

Menjalankan Test.....	266
Membaca Hasil Test.....	267
Acara 28. Deployment ke Hosting Lokal.....	272
Export Database	274
Copy Project ke htdocs.....	274
Konfigurasi .env	274
Konfigurasi .htaccess	274
VirtualHost Apache	275
Import Database	275
Uji Coba Aplikasi.....	276
Acara 29. Deployment ke Hosting Publik	279
Registrasi Hosting & Domain	280
Export Database	281
Upload Project via FTP.....	281
Generate Key & Optimasi.....	282
Set Permission Storage.....	283
Acara 30. Dokumentasi Proyek (README)	287
Membuat README.md.....	288
Konfigurasi .gitignore	289
Inisialisasi Git & Commit	290
Push ke GitHub.....	290
Menambahkan Screenshot.....	290
Update README & Badges.....	291

Acara 1. Analisis Kebutuhan Sistem

Materi Pembelajaran : Analisis kebutuhan dan spesifikasi sistem
Acara Praktikum/Praktik : Minggu 1/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

- Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna (TB430805.1)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.62010 0.001.0 1	Menganalisis Tools	- Mengidentifikasi tools yang akan digunakan - Menggunakan tools perangkat lunak
2	J.62010 0.002.0 1	Menganalisis Skalabilitas Perangkat Lunak	- Menggunakan tools perangkat lunak - Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

Mahasiswa mampu mengidentifikasi kebutuhan sistem

d. Dasar Teori

Analisis kebutuhan adalah fondasi pengembangan sistem. Studi kasus: Sistem Informasi Inventaris Barang Universitas. Sistem ini digunakan untuk mengelola data inventaris barang di lingkungan universitas. Aktor: Admin (mengelola master data, melihat laporan), Staff (mengajukan peminjaman, melihat barang). Kebutuhan fungsional: CRUD Kategori, CRUD Barang, Peminjaman, Pengembalian, Laporan.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code

- Git dan Github
- Composer.

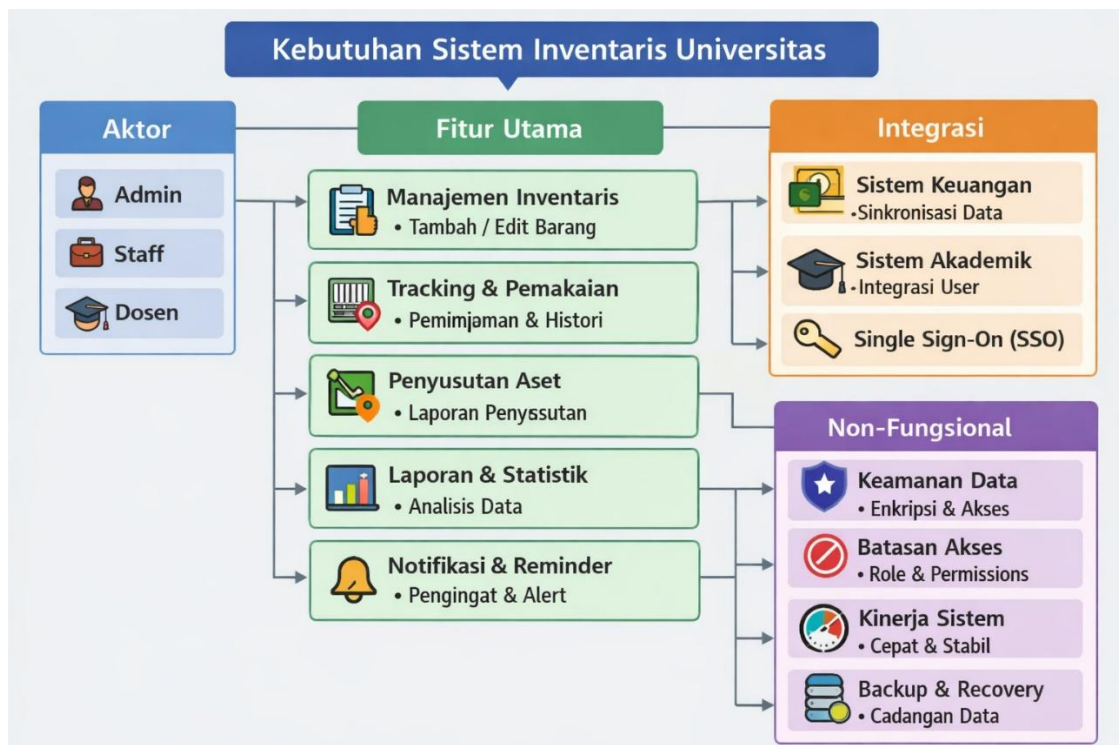
f. Prosedur Kerja

1. Tentukan studi kasus: Sistem Informasi Inventaris Barang Universitas
2. Identifikasi aktor: Admin, Staff, Kepala Lab/Dosen
3. Identifikasi kebutuhan fungsional:
 - Kelola data kategori barang
 - Kelola data barang (nama, kategori, jumlah, kondisi)
 - Peminjaman dan pengembalian barang
 - Laporan inventaris
4. Identifikasi kebutuhan non-fungsional: keamanan, responsif, multi-user
5. Dokumentasikan dalam format spesifikasi kebutuhan

Studi Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Analisis kebutuhan dan spesifikasi sistem". Analisis bagaimana konsep analisis kebutuhan sistem diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan



Gambar diatas menampilkan struktur kebutuhan dalam empat bagian utama:

- o Aktor: Admin, Staff, dan Kepala Lab/Dosen.

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

- o Fitur Utama: Manajemen inventaris, tracking & pemakaian, penyusutan aset, laporan & statistik, serta notifikasi.
- o Integrasi: Sistem keuangan, sistem akademik, dan SSO.
- o Non-Fungsional: Keamanan data, kontrol akses, kinerja sistem, serta backup & recovery.

Tugas

Buatlah analisis kebutuhan untuk aplikasi yang akan di buat pada Mitra PBL.

Kesimpulan

- o Mahasiswa menghasilkan dokumen spesifikasi kebutuhan sistem inventaris universitas
- o Mahasiswa menghasilkan dokumen spesifikasi kebutuhan sistem pada Mitra PBL

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 2. Use Case Diagram dan Skenario

Materi Pembelajaran : Use case diagram, skenario use case
Acara Praktikum/Praktik : Minggu 1/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna (TB430805.1)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.62010 0.001.0 1	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.62010 0.002.0 1	Menganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

Mahasiswa mampu mengidentifikasi kebutuhan sistem

d. Dasar Teori

Use case diagram menggambarkan interaksi aktor dengan sistem. Skenario use case menjelaskan langkah-langkah alur normal dan alternatif. Untuk sistem inventaris: use case Login, Kelola Barang, Pinjam Barang, Kembalikan Barang, Lihat Laporan. Aktor Admin dapat mengelola semua data, Staff hanya meminjam dan melihat.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github

- Composer.

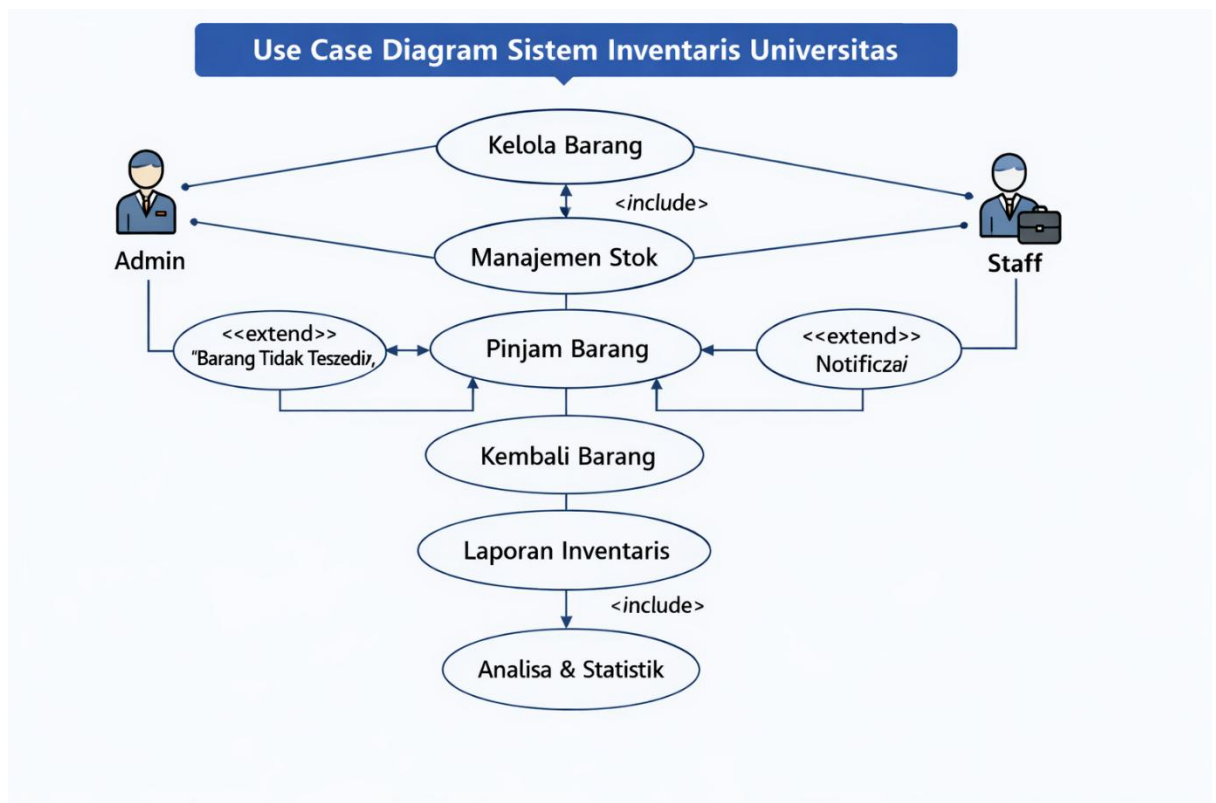
f. Prosedur Kerja

1. Identifikasi use case utama sistem inventaris
2. Buat use case diagram dengan aktor: Admin, Staff
3. Buat skenario use case "Pinjam Barang":
 - Pre-condition: User telah login, barang tersedia
 - Normal flow: Pilih barang -> Input jumlah -> Submit -> Stok berkurang
 - Alternative flow: Barang tidak tersedia -> Notifikasi
4. Buat skenario use case "Kelola Barang"
5. Buat skenario use case "Laporan Inventaris"
6. Dokumentasikan dalam format dokumentasi use case

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Use case diagram, skenario use case". Analisis bagaimana konsep use case diagram dan skenario diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan



o Identifikasi Use Case Utama

No	Use Case	Aktor	Deskripsi Singkat
1	Login	Admin, Staff	Mengakses sistem dengan kredensial valid
2	Kelola Barang	Admin	Menambah, mengedit, dan menghapus data barang
3	Pinjam Barang	Staff, Admin	Melakukan peminjaman barang dari inventaris
4	Kembali Barang	Staff, Admin	Mengembalikan barang yang telah dipinjam
5	Laporan Inventaris	Admin, Staff	Melihat dan mencetak laporan data inventaris

○ **Skenario Use Case: Pinjam Barang**

- Use Case Name: Pinjam Barang
- Aktor: Staff, Admin
- Pre-condition:
 - User telah login
 - Barang tersedia dalam stok
- Normal Flow:
 - User memilih barang yang ingin dipinjam
 - User menginput jumlah barang
 - User menekan tombol Submit
 - Sistem memvalidasi ketersediaan barang
 - Sistem mengurangi stok barang sesuai jumlah yang dipinjam
 - Sistem menampilkan konfirmasi peminjaman berhasil
- Alternative Flow:
 - Jika barang tidak tersedia : Sistem menampilkan notifikasi "Barang tidak tersedia"
- Post-condition:
 - Data peminjaman tersimpan dan stok barang terupdate

○ **Skenario Use Case: Kelola Barang**

- Use Case Name: Kelola Barang
- Aktor: Admin
- Pre-condition: Admin telah login
- Normal Flow:
 - Admin membuka menu *Kelola Barang*
 - Admin menambah, mengedit, atau menghapus data barang
 - Sistem menyimpan perubahan ke database
 - Sistem menampilkan pesan sukses

- Alternative Flow:
 - Jika data tidak valid : Sistem menampilkan pesan error
- Post-condition:
 - Data barang terupdate sesuai perubahan
- **Skenario Use Case: Laporan Inventaris**
 - Use Case Name: Laporan Inventaris
 - Aktor: Admin, Staff
 - Pre-condition: User telah login
 - Normal Flow:
 - User membuka menu *Laporan Inventaris*
 - User memilih periode laporan
 - Sistem menampilkan data inventaris sesuai filter
 - User dapat mencetak atau mengekspor laporan
 - Alternative Flow:
 - Jika tidak ada data : Sistem menampilkan pesan "Data tidak ditemukan"
 - Post-condition:
 - Laporan tersimpan atau dicetak sesuai permintaan

Tugas

Buatlah Use Case Diagram untuk aplikasi yang akan dibuat untuk mitra.

Kesimpulan

- Mahasiswa mampu membuat use case diagram dan skenario sistem inventaris
- Mahasiswa mampu membuat use case diagram dan skenario sistem aplikasi Mitra PBL

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
Total Skor*		Skor x 25			

Acara 3. Wireframing dengan Figma

Materi Pembelajaran : Wireframe, mockup UI
Acara Praktikum/Praktik : Minggu 2/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna (TB430805.1)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.62010 0.001.0 1	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan Menggunakan tools perangkat lunak
2	J.62010 0.002.0 1	Menganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

Mahasiswa mampu membuat wireframe antarmuka pengguna

d. Dasar Teori

Wireframe adalah kerangka visual antarmuka sebelum implementasi. Figma adalah tools kolaboratif untuk desain UI. Wireframe sistem inventaris mencakup: halaman login, dashboard admin, daftar barang, form peminjaman, laporan.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Figma
- Composer.

f. Prosedur Kerja

1. Install Figma desktop atau buka figma.com
2. Buat frame ukuran desktop (1440x900)
3. Wireframe halaman Login (form email + password + tombol)
4. Wireframe Dashboard Admin (sidebar, statistik: total barang, dipinjam, tersedia)
5. Wireframe halaman Daftar Barang (tabel + tombol tambah)
6. Wireframe form Tambah Barang (field: nama, kategori, jumlah, kondisi, gambar)
7. Wireframe halaman Peminjaman (pilih barang, jumlah, tanggal)
8. Export wireframe ke PDF/PNG

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Wireframe, mockup UI". Analisis bagaimana konsep wireframing dengan figma diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Tampilan desain Wireframe

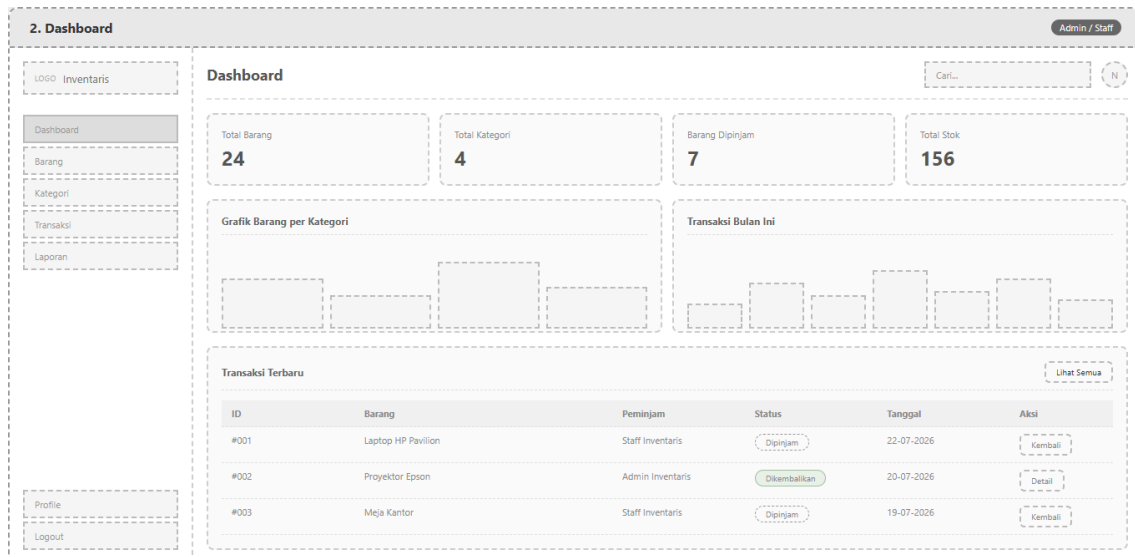
Tampilan 1: Login

Deskripsi: Halaman login untuk autentikasi user. Terdapat form email, password, opsi 'Ingat saya', link lupa password, tombol login, dan link pendaftaran.



Tampilan 2: Dashboard

Deskripsi: Halaman utama setelah login. Menampilkan 4 kartu statistik (Total Barang, Total Kategori, Barang Dipinjam, Total Stok), grafik batang, dan tabel transaksi terbaru. Sidebar navigasi di sebelah kiri.



Tampilan 3: Daftar Barang

Deskripsi: Halaman daftar barang dengan filter kategori, kondisi, dan pencarian. Tabel menampilkan nama barang, kategori, stok, kondisi, dan tombol aksi (Detail, Edit, Hapus). Dilengkapi tombol Tambah Barang, Export PDF/Excel.

The "3. Daftar Barang (Index)" page features a sidebar with a placeholder "[... Sidebar ...]". The main content area is titled "Data Barang" and includes filters for "Semua Kategori", "Semua Kondisi", and a search bar "Cari nama/deskripsi...". There are buttons for "+ Tambah Barang", "Export PDF", and "Export Excel". The table below lists items with their status (Baik, Rusak, Hilang) and action buttons (Detail, Edit, Hapus).

No	Nama Barang	Kategori	Stok	Kondisi	Aksi
1	Laptop HP Pavilion	Elektronik	10	Baik	Detail Edit Hapus
2	Proyektor Epson	Elektronik	3	Rusak	Detail Edit Hapus
3	Meja Kantor	Perabotan	25	Baik	Detail Edit Hapus
4	Kursi Kantor	Perabotan	2	Hilang	Detail Edit Hapus

Tampilan 4: Tambah Barang

Deskripsi: Form tambah barang baru dengan field: nama barang, kategori (dropdown), jumlah, kondisi (dropdown), deskripsi (textarea), dan upload gambar. Tombol Simpan dan Batal.

4. Tambah Barang

Form

← Kembali Tambah Barang

Nama Barang
Masukkan nama barang

Kategori
Pilih kategori

Jumlah
0

Kondisi
Pilih kondisi

Deskripsi
Deskripsi barang...

Gambar
Preview Gambar

Pilih File
Format: JPG, PNG, Maks: 2MB

Batal Simpan

Tampilan 5: Detail Barang

Deskripsi: Halaman detail barang menampilkan foto, informasi lengkap (kategori, stok, kondisi, deskripsi, tanggal), dan riwayat peminjaman barang dalam bentuk tabel.

5. Detail Barang

View

← Kembali Detail Barang

Edit Hapus

Foto Barang

Laptop HP Pavilion

Kategori Elektronik

Jumlah Stok 10 unit

Kondisi Baik

Deskripsi Laptop HP Pavilion untuk keperluan perkantoran dan pengajaran.

Dibuat 15 Januari 2026

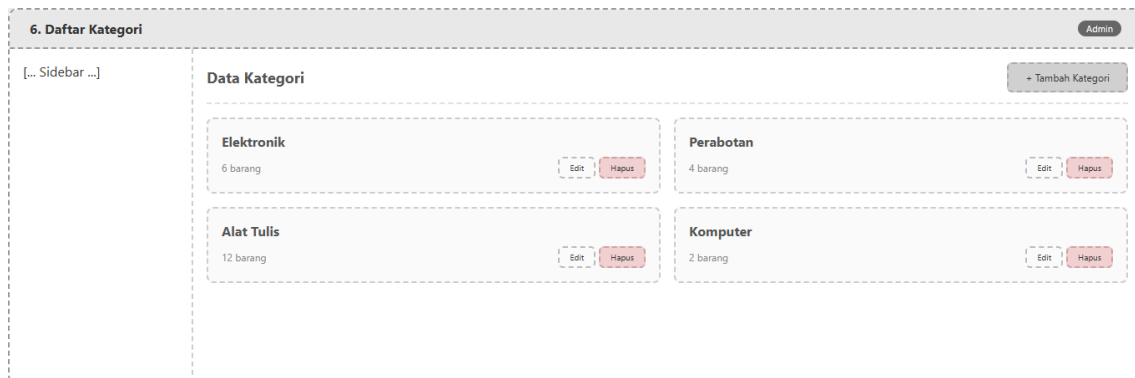
Diperbarui 20 Juli 2026

Riwayat Peminjaman

Peminjam	Tgl Pinjam	Tgl Kembali	Status	Aksi
Staff Inventaris	22-07-2026	-	Dipinjam	Kembali

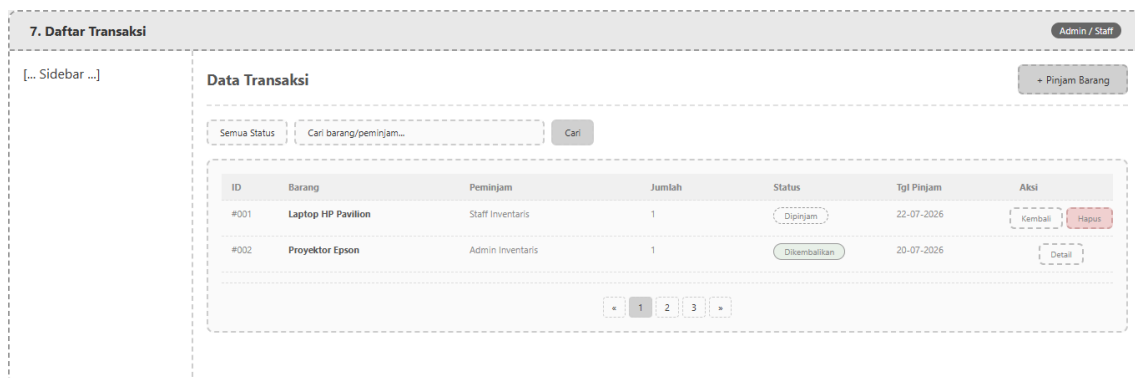
Tampilan 6: Kategori

Deskripsi: Halaman daftar kategori dalam bentuk kartu. Setiap kartu menampilkan nama kategori, jumlah barang, dan tombol aksi (Edit, Hapus). Terdapat tombol Tambah Kategori.



Tampilan 7: Transaksi

Deskripsi: Halaman daftar transaksi peminjaman dengan filter status dan pencarian. Tabel menampilkan ID, barang, peminjam, jumlah, status, tanggal, dan tombol aksi (Kembali, Hapus).



Tampilan 8: Pinjam Barang

Deskripsi: Form peminjaman barang. Memilih barang dari dropdown (dilengkapi info stok), menentukan jumlah, dan tanggal pinjam. Terdapat alert info stok tersedia.

8. Pinjam Barang

[... Sidebar ...]

— Kembali Form Peminjaman Barang

Barang
Pilih barang (tampilkan stok tersedia)

Jumlah
1

Tanggal Pinjam
22-07-2026

Info Stok: Stok tersedia: 10 unit | Kondisi: Baik

Batal Simpan Peminjaman

Tampilan 9: Laporan

Deskripsi: Halaman laporan dengan 4 kartu export: Laporan Barang PDF, Laporan Barang Excel, Laporan Transaksi PDF, dan Laporan Transaksi Excel.

9. Laporan

[... Sidebar ...]

Laporan

[PDF]
Laporan Barang PDF
Export data barang ke format PDF
Download PDF

[XLS]
Laporan Barang Excel
Export data barang ke format Excel
Download Excel

[PDF]
Laporan Transaksi PDF
Export data transaksi ke format PDF
Download PDF

[XLS]
Laporan Transaksi Excel
Export data transaksi ke format Excel
Download Excel

Tampilan 10: Profile

Deskripsi: Halaman profile user dengan form informasi profile (nama, email), form ubah password, dan zona berbahaya untuk hapus akun.

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat wireframe antarmuka sistem inventaris

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seususai	V			
	Total Skor*	Skor x 25			

Acara 4. Perancangan ERD dan Struktur Tabel

Materi Pembelajaran : ERD, relasi, normalisasi
Acara Praktikum/Praktik : Minggu 2/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna (TB430805.1)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.62010 0.001.0 1	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.62010 0.002.0 1	Menganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

Mahasiswa mampu membuat ERD dan struktur tabel

d. Dasar Teori

• Pengertian ERD

Entity Relationship Diagram (ERD) adalah model konseptual yang digunakan untuk menggambarkan struktur logis dari sebuah basis data. ERD menampilkan entitas (objek yang memiliki data), atribut (karakteristik dari entitas), serta hubungan (relationship) antar entitas. Dengan ERD, perancang sistem dapat memahami bagaimana data saling berhubungan sebelum diimplementasikan ke dalam database fisik.

• Komponen Utama ERD

a) Entitas (Entity)

Representasi objek nyata atau konsep yang informasinya perlu disimpan.

Contoh: Mahasiswa, Dosen, Mata Kuliah. Digambarkan dengan persegi panjang.

b) Atribut (Attribute)

Karakteristik atau properti dari entitas.

Contoh: Nama, NIM, Alamat. Digambarkan dengan elips. Ada beberapa jenis atribut:

- Key Attribute: atribut unik (misalnya NIM).
- Multivalued Attribute: atribut dengan lebih dari satu nilai (misalnya nomor telepon).
- Derived Attribute: atribut hasil turunan dari atribut lain (misalnya umur dari tanggal lahir).

c) Relationship (Hubungan)

Menunjukkan keterkaitan antar entitas.

Contoh: Mahasiswa mengambil Mata Kuliah. Digambarkan dengan belah ketupat (diamond).

d) Kardinalitas (Cardinality)

Kardinalitas menunjukkan jumlah minimum dan maksimum entitas yang dapat berpartisipasi dalam suatu hubungan:

1. One to One (1:1) : Satu entitas berhubungan dengan satu entitas lain.
2. One to Many (1:N) : Satu entitas berhubungan dengan banyak entitas lain.
3. Many to Many (M:N) : Banyak entitas berhubungan dengan banyak entitas lain.

• Fungsi ERD

Membantu analisis sistem dalam memahami kebutuhan data. Menjadi alat komunikasi antara analis, desainer, dan pengguna sistem. Menjadi blueprint untuk perancangan database relasional. Meminimalisir redundansi data dan meningkatkan konsistensi.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

1. Identifikasi entitas dari analisis kebutuhan: User, Kategori, Barang, Transaksi
2. Tentukan atribut setiap entitas dan primary key
3. Gambarkan ERD menggunakan draw.io / tools ERD

4. Tentukan relasi antar entitas (one-to-many, many-to-many)
5. Lakukan normalisasi hingga bentuk 3NF
6. Buat skema database lengkap (nama tabel, kolom, tipe data, constraint)
7. Dokumentasikan ERD dan skema database

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "ERD, relasi, normalisasi". Analisis bagaimana konsep perancangan erd dan struktur tabel diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

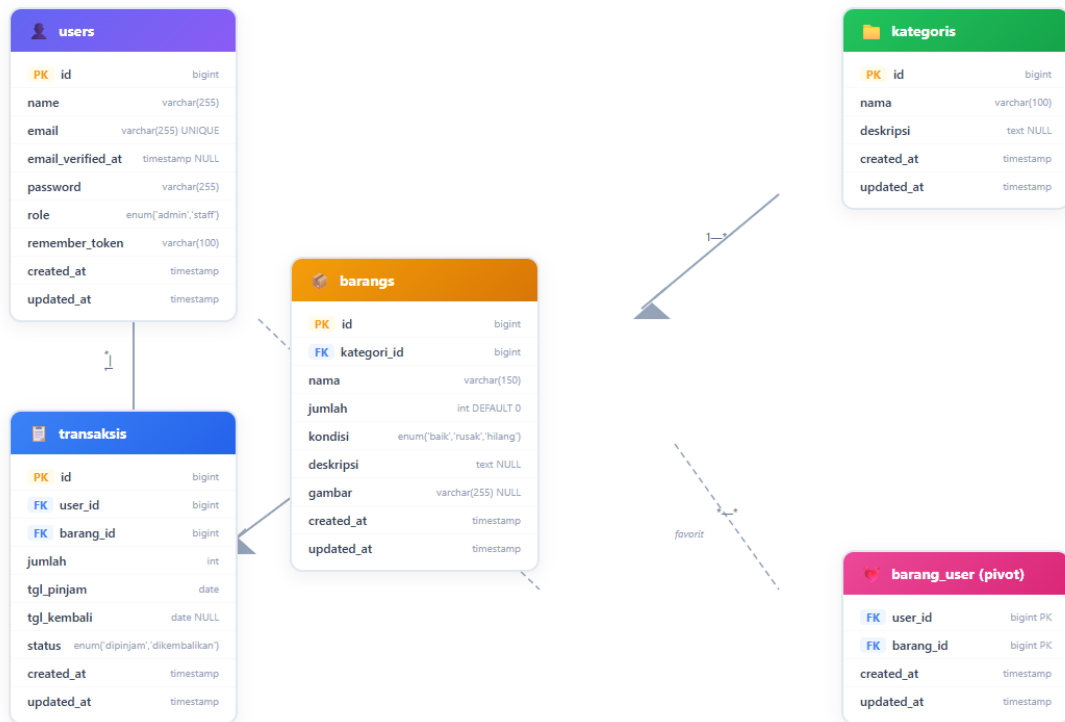
Tampilan Desain ERD

1. Entity Relationship Diagram (ERD)

ERD berikut menggambarkan hubungan antar tabel dalam database Inventaris Universitas. Terdapat 5 tabel utama: **users**, **kategoris**, **barangs**, **transaksis**, **barang_user**.

Entity Relationship Diagram (ERD)

Inventaris Universitas — Database Design



PK Primary Key FK Foreign Key —> One to Many (1—*) —> Many to Many (*—*)

Relationships / Relasi Antar Tabel

users 1—* transaksis → Satu user dapat melakukan banyak transaksi peminjaman
barangs 1—* transaksis → Satu barang dapat dipinjam dalam banyak transaksi

kategoris 1—* barangs → Satu kategori dapat memiliki banyak barang
users *—* barangs → Banyak user dapat memfavoritkan banyak barang (melalui tabel pivot **barang_user**)

Legenda ERD

- PK (Primary Key) : Identifikasi unik setiap record dalam tabel
- FK (Foreign Key) : Menghubungkan tabel dengan tabel lainnya
- 1 : * (One to Many) : Satu record di tabel A dapat memiliki banyak record di tabel B
- * : * (Many to Many) : Banyak record di tabel A berhubungan dengan banyak record di tabel B

2. Desain Tabel Database

2.1 Tabel users (Pengguna)

Menyimpan data pengguna sistem yang terdiri dari admin dan staff. Setiap user dapat melakukan transaksi peminjaman dan menyimpan barang favorit.

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Kolom	Tipe Data	Constraint	Keterangan
id	BIGINT(20)	PK	Primary Key, auto-increment
name	VARCHAR(255)		Nama lengkap pengguna
email	VARCHAR(255)	UNIQUE	Alamat email, unique
email_verified_at	TIMESTAMP	NULL	Tanggal verifikasi email
password	VARCHAR(255)		Password (hashed)
role	ENUM('admin','staff')	DEFAULT 'staff'	Role pengguna: admin atau staff
remember_token	VARCHAR(100)		Token untuk 'remember me'
created_at	TIMESTAMP		Waktu dibuat
updated_at	TIMESTAMP		Waktu diperbarui

SQL DDL:

```
CREATE TABLE `users` (
  `id` BIGINT(20) NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(255) NOT NULL,
  `email` VARCHAR(255) NOT NULL UNIQUE,
  `email_verified_at` TIMESTAMP NULL,
  `password` VARCHAR(255) NOT NULL,
  `role` ENUM('admin','staff') DEFAULT 'staff',
  `remember_token` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP,
  `updated_at` TIMESTAMP,
  PRIMARY KEY (`id`)
);
```

2.2 Tabel kategoris (Kategori Barang)

Menyimpan kategori barang inventaris. Setiap kategori dapat memiliki banyak barang.

Kolom	Tipe Data	Constraint	Keterangan
id	BIGINT(20)	PK	Primary Key, auto-increment
nama	VARCHAR(100)		Nama kategori
deskripsi	TEXT	NULL	Deskripsi kategori

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

created_at	TIMESTAMP		Waktu dibuat
updated_at	TIMESTAMP		Waktu diperbarui

SQL DDL:

```
CREATE TABLE `kategoris` (  
  `id` BIGINT(20) NOT NULL AUTO_INCREMENT,  
  `nama` VARCHAR(100) NOT NULL,  
  `deskripsi` TEXT NULL,  
  `created_at` TIMESTAMP,  
  `updated_at` TIMESTAMP,  
  PRIMARY KEY (`id`)  
);
```

2.3 Tabel barangs (Barang Inventaris)

Menyimpan data barang inventaris universitas. Setiap barang memiliki kategori, jumlah stok, kondisi, dan dapat difavoritkan oleh user.

Kolom	Tipe Data	Constraint	Keterangan
id	BIGINT(20)	PK	Primary Key, auto-increment
kategori_id	BIGINT(20)	FK	Foreign Key ke tabel kategoris
nama	VARCHAR(150)		Nama barang
jumlah	INT(11)	DEFAULT 0	Jumlah stok barang
kondisi	ENUM('baik','rusak','hilang')	DEFAULT 'baik'	Kondisi barang
deskripsi	TEXT	NULL	Deskripsi barang
gambar	VARCHAR(255)	NULL	Path file gambar barang
created_at	TIMESTAMP		Waktu dibuat
updated_at	TIMESTAMP		Waktu diperbarui

SQL DDL:

```
CREATE TABLE `barangs` (  
  `id` BIGINT(20) NOT NULL AUTO_INCREMENT,  
  `kategori_id` BIGINT(20) NOT NULL,  
  `nama` VARCHAR(150) NOT NULL,
```

```

`jumlah` INT(11) DEFAULT 0,
`kondisi` ENUM('baik','rusak','hilang') DEFAULT 'baik',
`deskripsi` TEXT NULL,
`gambar` VARCHAR(255) NULL,
`created_at` TIMESTAMP,
`updated_at` TIMESTAMP,
PRIMARY KEY (`id`)
);

```

2.4 Tabel transaksi (Transaksi Peminjaman)

Menyimpan data transaksi peminjaman dan pengembalian barang. Setiap transaksi mencatat user peminjam, barang, jumlah, dan status peminjaman.

Kolom	Type Data	Constraint	Keterangan
id	BIGINT(20)	PK	Primary Key, auto-increment
user_id	BIGINT(20)	FK	Foreign Key ke tabel users
barang_id	BIGINT(20)	FK	Foreign Key ke tabel barangs
jumlah	INT(11)		Jumlah barang yang dipinjam
tgl_pinjam	DATE		Tanggal peminjaman
tgl_kembali	DATE	NULL	Tanggal pengembalian (NULL jika masih dipinjam)
status	ENUM('dipinjam','dikembalikan')	DEFAULT 'dipinjam'	Status transaksi
created_at	TIMESTAMP		Waktu dibuat
updated_at	TIMESTAMP		Waktu diperbarui

SQL DDL:

```

CREATE TABLE `transaksis` (
  `id` BIGINT(20) NOT NULL AUTO_INCREMENT,
  `user_id` BIGINT(20) NOT NULL,

```

```
`barang_id` BIGINT(20) NOT NULL,  
`jumlah` INT(11) NOT NULL,  
`tgl_pinjam` DATE NOT NULL,  
`tgl_kembali` DATE NULL,  
`status` ENUM('dipinjam','dikembalikan') DEFAULT 'dipinjam',  
`created_at` TIMESTAMP,  
`updated_at` TIMESTAMP,  
PRIMARY KEY (`id`)  
);
```

2.5 Tabel `barang_user` (Favorit Pivot)

Tabel pivot untuk relasi many-to-many antara users dan barangs. Menyimpan data barang favorit yang ditandai oleh user.

Kolom	Tipe Data	Constraint	Keterangan
user_id	BIGINT(20)	FK / PK	Foreign Key ke users, bagian dari composite primary key
barang_id	BIGINT(20)	FK / PK	Foreign Key ke barangs, bagian dari composite primary key
created_at	TIMESTAMP		Waktu dibuat
updated_at	TIMESTAMP		Waktu diperbarui

SQL DDL:

```
CREATE TABLE `barang_user` (  
  `user_id` BIGINT(20) NOT NULL,  
  `barang_id` BIGINT(20) NOT NULL,  
  `created_at` TIMESTAMP,  
  `updated_at` TIMESTAMP,  
  PRIMARY KEY (`user_id`, `barang_id`)  
);
```

3. Relasi Antar Tabel

Berikut adalah penjelasan detail mengenai relasi antar tabel dalam database:

3.1 users (1) : transaksis (*)

Relasi: users (Pengguna) memiliki hubungan 1-to-* dengan transaksis (Transaksi)

Penjelasan: Satu user dapat melakukan banyak transaksi peminjaman. Contoh: Admin dapat meminjam beberapa barang dalam waktu berbeda.

Implementasi: Tabel transaksis memiliki kolom user_id sebagai Foreign Key yang merujuk ke kolom id pada tabel users. Constraint: ON DELETE CASCADE.

3.2 kategoris (1) : barangs (*)

Relasi: kategoris (Kategori) memiliki hubungan 1-to-* dengan barangs (Barang)

Penjelasan: Satu kategori dapat dimiliki oleh banyak barang. Contoh: Kategori 'Elektronik' berisi Laptop, Proyektor, dll.

Implementasi: Tabel barangs memiliki kolom kategori_id sebagai Foreign Key yang merujuk ke kolom id pada tabel kategoris. Constraint: ON DELETE CASCADE.

3.3 barangs (1) : transaksis (*)

Relasi: barangs (Barang) memiliki hubungan 1-to-* dengan transaksis (Transaksi)

Penjelasan: Satu barang dapat dipinjam dalam banyak transaksi (riwayat peminjaman). Contoh: Laptop HP Pavilion dipinjam 5 kali.

Implementasi: Tabel transaksis memiliki kolom barang_id sebagai Foreign Key yang merujuk ke kolom id pada tabel barangs. Constraint: ON DELETE CASCADE.

3.4 users (*) : barangs (*)

Relasi: users (Pengguna) memiliki hubungan *-to-* dengan barangs (Barang)

Penjelasan: Relasi many-to-many melalui tabel pivot barang_user untuk fitur favorit. Banyak user dapat memfavoritkan banyak barang.

Implementasi: Relasi many-to-many diimplementasikan menggunakan tabel pivot barang_user yang memiliki dua Foreign Key: user_id (ke users) dan barang_id (ke barangs). Composite Primary Key: (user_id, barang_id).

4. Ringkasan

Aspek	Detail
Jumlah Tabel	5 tabel: users, kategoris, barangs, transaksis, barang_user
Total Kolom	34 kolom across all tables
Primary Keys	4 tabel dengan single PK (id), 1 tabel dengan composite PK (barang_user)

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Foreign Keys	4 Foreign Key: kategori_id, user_id (x2), barang_id
Enum Types	3 enum: role (admin/staff), kondisi (baik/rusak/hilang), status (dipinjam/dikembalikan)
Relasi One-to-Many	3 relasi: users-transaksis, kategoris-barangs, barangs-transaksis
Relasi Many-to-Many	1 relasi: users-barangs (via barang_user)

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat ERD dan skema database untuk sistem inventaris

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

Acara 5. Instalasi Laravel dan Konfigurasi Awal

Materi Pembelajaran : Laravel install, Composer, XAMPP
Acara Praktikum/Praktik : Minggu 3/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna (TB430805.1)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.62010 0.001.0 1	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.62010 0.002.0 1	Menganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

Mahasiswa mampu membuat project Laravel

d. Dasar Teori

Laravel 12 adalah versi terbaru dari framework PHP yang berlandaskan pada arsitektur Model-View-Controller (MVC) dan dirancang untuk mempermudah pengembangan aplikasi web modern. Framework ini menyediakan berbagai fitur bawaan seperti Eloquent ORM untuk interaksi dengan database secara objektif, Blade Template Engine untuk pengelolaan tampilan yang dinamis, serta Artisan CLI yang mendukung otomatisasi berbagai tugas seperti migrasi, seeding, dan testing. Laravel 12 juga memperkuat sistem middleware untuk keamanan dan kontrol akses, serta menghadirkan integrasi penuh dengan ekosistem frontend modern seperti Vite, Tailwind CSS, Livewire, dan Inertia.

Struktur direktori Laravel 12 tetap konsisten dengan versi sebelumnya, mencakup folder `app/` untuk logika aplikasi, `routes/` untuk definisi jalur akses, `resources/` untuk tampilan dan aset frontend, serta `database/` untuk migrasi dan

seeding. Keunggulan utama Laravel 12 terletak pada sintaks yang ekspresif, keamanan bawaan seperti proteksi CSRF dan hashing, serta ekosistem paket yang luas. Framework ini juga mendukung pengembangan REST API dan integrasi dengan microservices, sehingga cocok untuk aplikasi berskala kecil hingga enterprise..

Laravel 12 membutuhkan PHP 8.2+, Composer, dan web server (XAMPP). Composer adalah dependency manager PHP untuk menginstal Laravel dan package. Project akan bernama "inventaris-universitas".

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

1. Install XAMPP (PHP 8.2+), pastikan php.exe di C:\xampp\php
2. Install Composer dari <https://getcomposer.org>
3. Buka Command Prompt / Terminal
4. Buat project: `composer create-project laravel/laravel inventaris-universitas`
5. Masuk direktori: `cd inventaris-universitas`
6. Uji instalasi: `php artisan serve`
7. Buka `http://127.0.0.1:8000` - pastikan halaman Laravel muncul
8. Catat hasil instalasi dan kendala yang dihadapi

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Laravel install, Composer, XAMPP". Analisis bagaimana konsep instalasi laravel dan konfigurasi awal diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

• Instalasi Composer

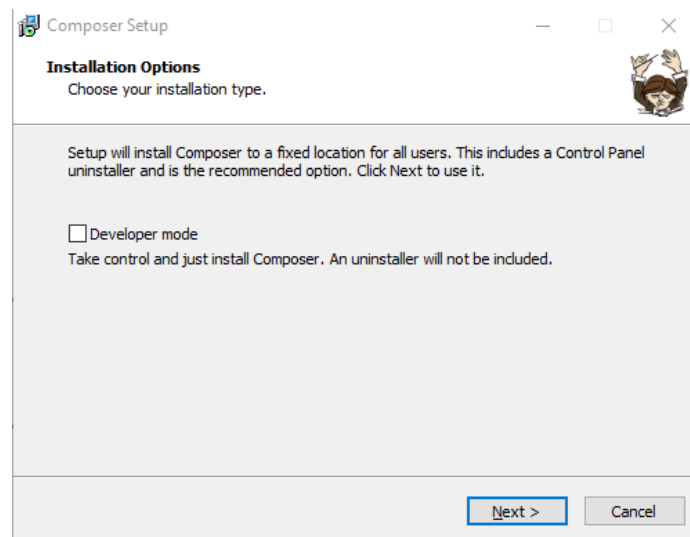
1. Download File Composer



Sebelum melakukan instalasi, hal pertama yang harus dilakukan adalah mengunduh file Composer. Buka link <https://getcomposer.org/> untuk mengunduh file tersebut.

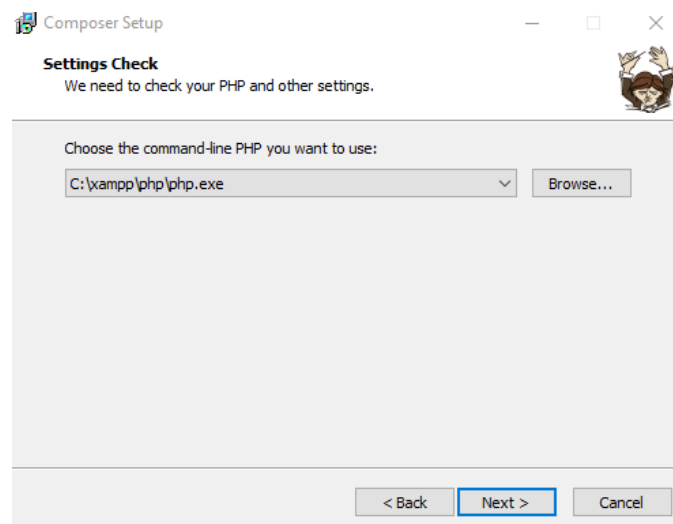
2. Install Composer

Pertama akan muncul halaman seperti pada gambar dibawah ini. Klik Next untuk melanjutkan ke proses instalasi.



3. Tentukan Lokasi File Instalasi

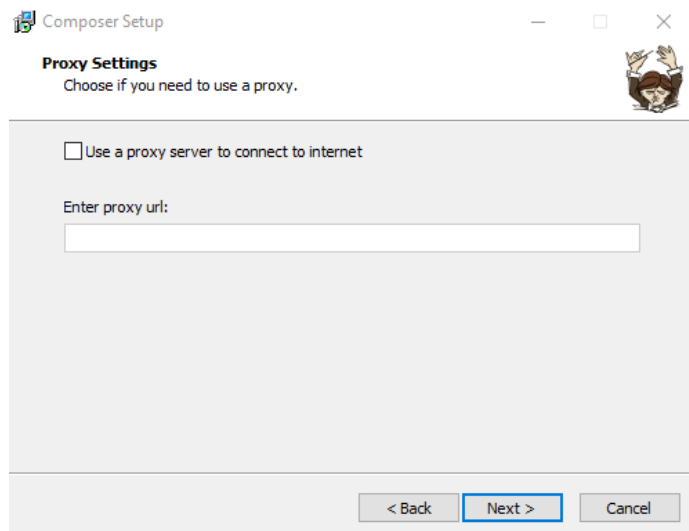
Selanjutnya Anda akan diminta untuk menentukan lokasi file php (php.exe) yang akan digunakan pada command line (PHP CLI- Command Line Interface).



File tersebut biasanya berada di dalam folder php. Jika Anda menggunakan XAMPP, lokasi file ini akan berada di C:\xampp\php. Pastikan pilih lokasi instalasi Composer berjalan di C:\xampp\php\php.exe. Klik Next apabila lokasi file php sudah benar.

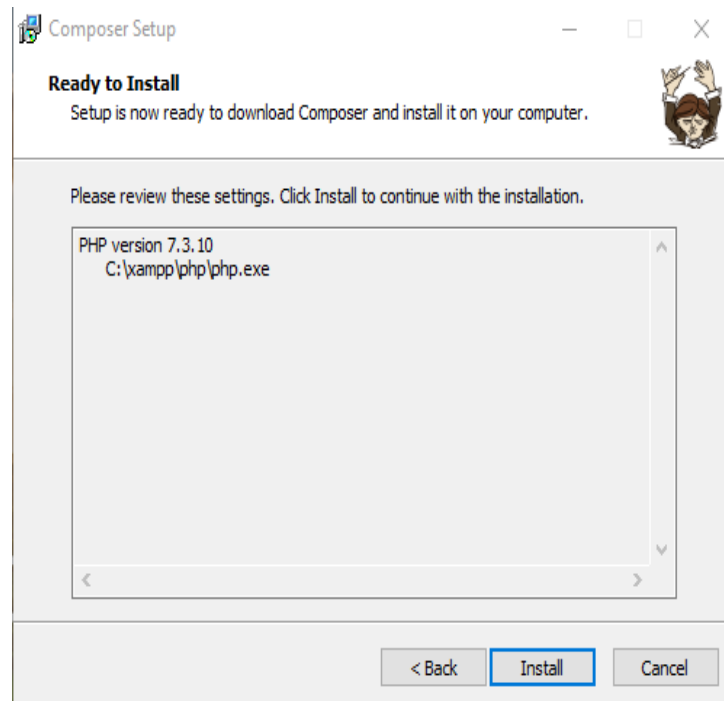
4. Pilih Penggunaan Proxy

Kemudian Anda akan diminta untuk memilih, apakah Anda akan menggunakan proxy atau tidak. Jika Anda ingin menggunakan proxy, klik centang dan masukkan URL proxy Anda. Apabila Anda tidak ingin menggunakan proxy, langsung klik Next untuk melanjutkan instalasi.



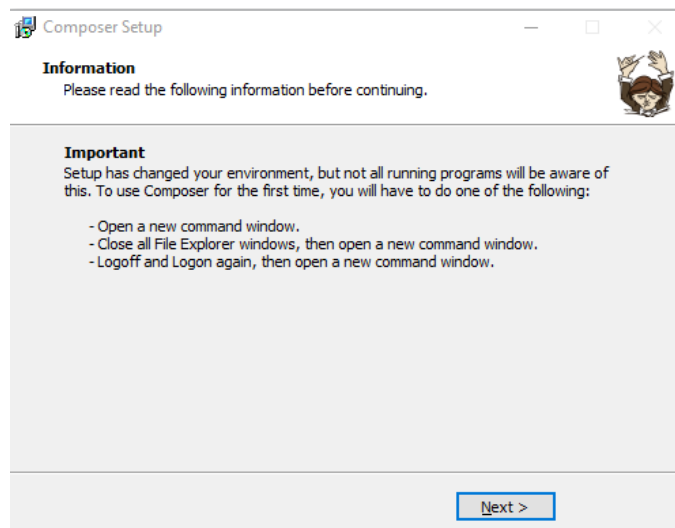
5. Review Lokasi Instalasi

Selanjutnya pastikan kalau proses instalasi berjalan di lokasi file yang seharusnya, yaitu C:\xampp\php\php.exe. Jika sudah benar, klik Install.



6. Review Perubahan Instalasi

Berikutnya adalah tampilan pemberitahuan bahwa telah terjadi perubahan pada Windows environment. Perubahan ini dimaksudkan agar Composer dapat dijalankan pada Command Prompt.



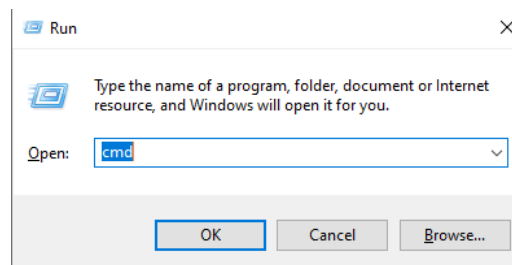
7. Instalasi Selesai

Proses instalasi selesai, klik Finish untuk menutup jendela instalasi Composer.



8. Cek Instalasi Composer

Setelah proses instalasi Composer selesai, langkah selanjutnya adalah cek instalasi Composer menggunakan Command Prompt. Caranya adalah Tekan Win+R lalu ketik cmd dan klik Ok.



Maka Anda akan diarahkan ke jendela Command Prompt masukkan perintah seperti di bawah ini untuk mengecek instalasi sukses atau tidak.

```
Microsoft Windows [Version 10.0.26100.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rifqi>php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies

C:\Users\rifqi>composer -V
Composer version 2.8.10 2025-07-10 19:08:33
PHP version 8.2.12 (C:\xampp\php\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.

C:\Users\rifqi>git --version
git version 2.51.0.windows.1
```

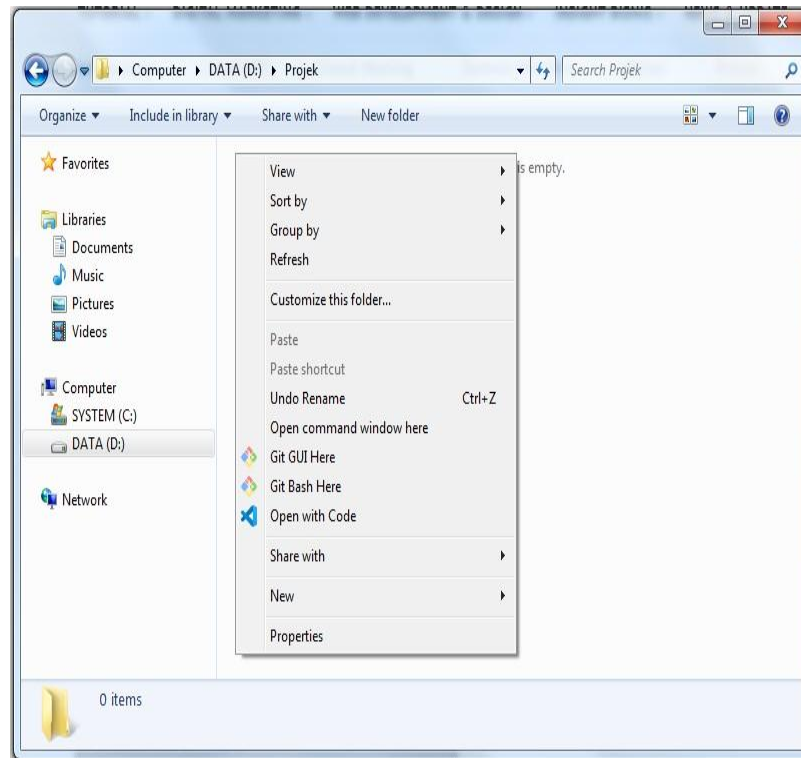
Apabila muncul seperti gambar di atas, instalasi Anda berhasil.

- **Membuat Project Laravel**

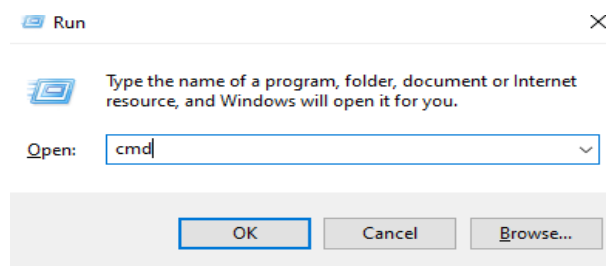
BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

1. Masuk Command Prompt

Buat folder baru dengan nama Project di Drive D, lalu buka folder tersebut. Tekan Shift+ Klik Kanan untuk menampilkan menu command prompt. Klik Open command window here.



Atau masuk Command Prompt melalui Run. Caranya adalah klik Win+R lalu ketik cmd dan klik OK. Ubah direktori ke lokasi folder Project.



cd D:/work/project

```
C:\Users\rifqi>cd D:
D:\
C:\Users\rifqi>D:
D:\>cd work
D:\work>cd project
```

2. Mulai Proses Install Laravel

Selanjutnya jika sudah masuk direktori Project, Anda harus membuat request untuk mengambil (serta menginstall) file Laravel yang telah disediakan dalam repositori Github. Gunakan perintah ini untuk melakukan request:

```
D:\work\project>composer create-project laravel/laravel:^12.0 belajar-laravel
Creating a "laravel/laravel:12.0" project at "./belajar-laravel"
Installing laravel/laravel (v12.0.0)
- Downloading laravel/laravel (v12.0.0)
- Installing laravel/laravel (v12.0.0): Extracting archive
Created project in D:\work\project\belajar-laravel
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
```

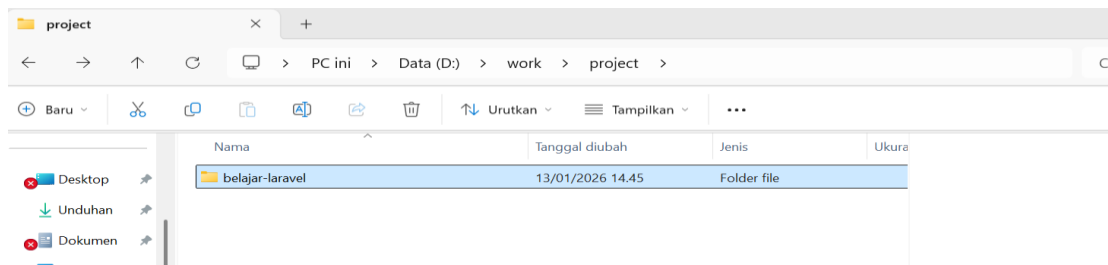
Jika perintah telah berhasil dimasukkan, Composer akan mulai melakukan proses pengambilan data serta instalasi Laravel ke dalam direktori yang telah Anda tentukan. Pastikan bahwa koneksi internet dalam keadaan stabil agar tidak terjadi gangguan pada saat proses pengambilan data Laravel.

```
- Installing doctrine/inflector (2.1.0): Extracting archive
- Installing doctrine/lexer (3.0.1): Extracting archive
- Installing dragonmantank/cron-expression (v3.6.0): Extracting archive
- Installing symfony/deprecation-contracts (v3.6.0): Extracting archive
- Installing psr/container (2.0.2): Extracting archive
- Installing fakerphp/faker (v1.24.1): Extracting archive
- Installing symfony/polyfill-mbstring (v1.33.0): Extracting archive
- Installing symfony/http-foundation (v7.4.3): Extracting archive
- Installing fruitcake/php-cors (v1.4.0): Extracting archive
- Installing psr/http-message (2.0): Extracting archive
- Installing psr/http-client (1.0.3): Extracting archive
- Installing ralouphie/getallheaders (3.0.3): Extracting archive
- Installing psr/http-factory (1.1.0): Extracting archive
- Installing guzzlehttp/psr7 (2.8.0): Extracting archive
- Installing guzzlehttp/promises (2.3.0): Extracting archive
- Installing guzzlehttp/guzzle (7.10.0): Extracting archive
- Installing symfony/polyfill-php80 (v1.33.0): Extracting archive
- Installing guzzlehttp/uri-template (v1.0.5): Extracting archive
- Installing symfony/polyfill-intl-normalizer (v1.33.0): Extracting archive
- Installing symfony/polyfill-intl-grapheme (v1.33.0): Extracting archive
- Installing symfony/polyfill-ctype (v1.33.0): Extracting archive
- Installing symfony/string (v7.4.0): Extracting archive
- Installing symfony/service-contracts (v3.6.1): Extracting archive
- Installing symfony/console (v7.4.3): Extracting archive
- Installing nunomaduro/termwind (v2.3.3): Extracting archive
- Installing voku/portable-ascii (2.0.3): Extracting archive
- Installing phpoption/phpoption (1.9.5): Extracting archive
- Installing graham-campbell/result-type (v1.1.4): Extracting archive
- Installing vlucas/phpdotenv (v5.6.3): Extracting archive
- Installing symfony/css-selector (v7.4.0): Extracting archive
```

3. Cek Instalasi Laravel di Web Browser

Setelah proses download file Laravel selesai, nantinya akan ada folder baru pada direktori file server dengan nama sesuai nama project yang telah Anda tentukan sebelumnya pada folder Project.

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)



Untuk memastikan bahwa Laravel sukses terinstall dan siap untuk digunakan, arahkan Command Prompt atau Terminal menuju direktori yang telah Anda buat sebelumnya. Lalu, masukkan perintah berikut ke dalam Command Prompt atau Terminal:

php artisan serve

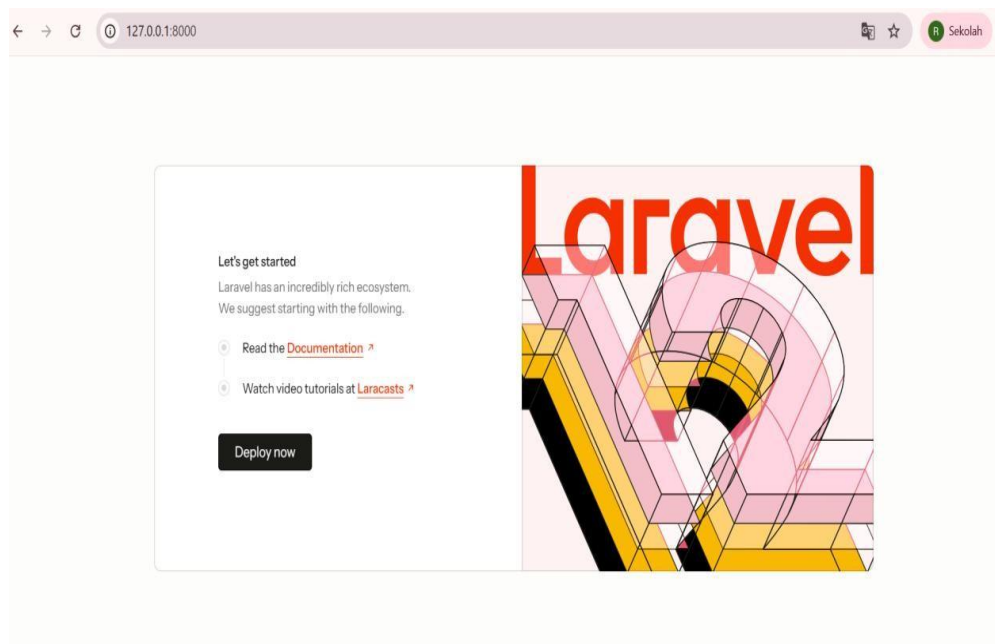
```
D:\work\project>cd belajar-laravel
D:\work\project\belajar-laravel>php artisan serve

[WARN] Unable to respect the 'PHP_CLI_SERVER_WORKERS' environment variable without the '--no-reload' flag. Only creating a single server.

[INFO] Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server
```

Jika muncul tulisan Laravel development server started pada Command Prompt atau Terminal, langkah selanjutnya adalah membuka link yang telah disediakan oleh Laravel. Secara default, Anda akan diarahkan menuju alamat server, yaitu 127.0.0.1:8000. Nantinya, akan muncul tampilan homepage dengan tulisan Laravel di bagian tengah seperti pada gambar di bawah ini:



Jika muncul seperti pada tampilan di atas, proses instalasi Laravel Anda telah berhasil.

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu menginstal Laravel dan membuat project baru

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 6. Konfigurasi Environment dan Git

Materi Pembelajaran : .env, database, Git init
Acara Praktikum/Praktik : Minggu 3/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna (TB430805.1)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.0 03.01	Melakukan Identifikasi Library, Komponen atau Framework yang Diperlukan	• Menganalisis library, komponen, atau framework yang sesuai dengan konteks • Membuat proof of concept library, komponen atau framework berdasarkan konteks kebutuhan • Merancang integrasi dan batasan penggunaan library, komponen atau framework
2	J.620100.0 06.01	Merancang User Experience	• Menganalisis kebutuhan dan skenario penggunaan • Merancang layout tampilan dan skenario penggunaan

c. Indikator Penilaian

Mahasiswa mampu mengkonfigurasi environment dan Git

d. Dasar Teori

File .env adalah file konfigurasi yang digunakan dalam pengembangan aplikasi, termasuk framework seperti Laravel, untuk menyimpan variabel lingkungan (environment variables). File ini berfungsi sebagai tempat penyimpanan informasi penting yang bersifat dinamis dan dapat berbeda antara satu lingkungan dengan lingkungan lainnya, misalnya antara development, testing, dan production. Dengan adanya file .env, developer tidak perlu menuliskan konfigurasi langsung di dalam kode program, sehingga kode menjadi lebih aman, fleksibel, dan mudah dipelihara.

Isi file .env biasanya berupa pasangan key-value yang mendefinisikan konfigurasi aplikasi, seperti pengaturan database (DB_HOST, DB_DATABASE, DB_USERNAME, DB_PASSWORD), konfigurasi server (APP_URL, APP_ENV, APP_DEBUG), serta pengaturan keamanan seperti secret key (APP_KEY). File ini tidak boleh dibagikan secara publik karena sering berisi data sensitif, sehingga biasanya dimasukkan ke dalam .gitignore agar tidak ikut tersimpan di repository. Git digunakan untuk version control project. Setiap perubahan kode akan dicatat dengan Git untuk kolaborasi tim.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

1. Buka file .env di project inventaris-universitas
2. Ubah konfigurasi database:
DB_DATABASE=db_inventaris
DB_USERNAME=root
DB_PASSWORD=
3. Buat database di phpMyAdmin: CREATE DATABASE db_inventaris;
4. Inisialisasi Git: git init
5. Konfigurasi user Git:
git config --global user.name "Nama Anda"
git config --global user.email "email@anda.com"
6. Buat file .gitignore (standard Laravel)
7. Commit pertama: git add . && git commit -m "Initial commit"
8. Buat repository di Github dan push:
git remote add origin https://github.com/username/inventaris-universitas.git
git push -u origin master

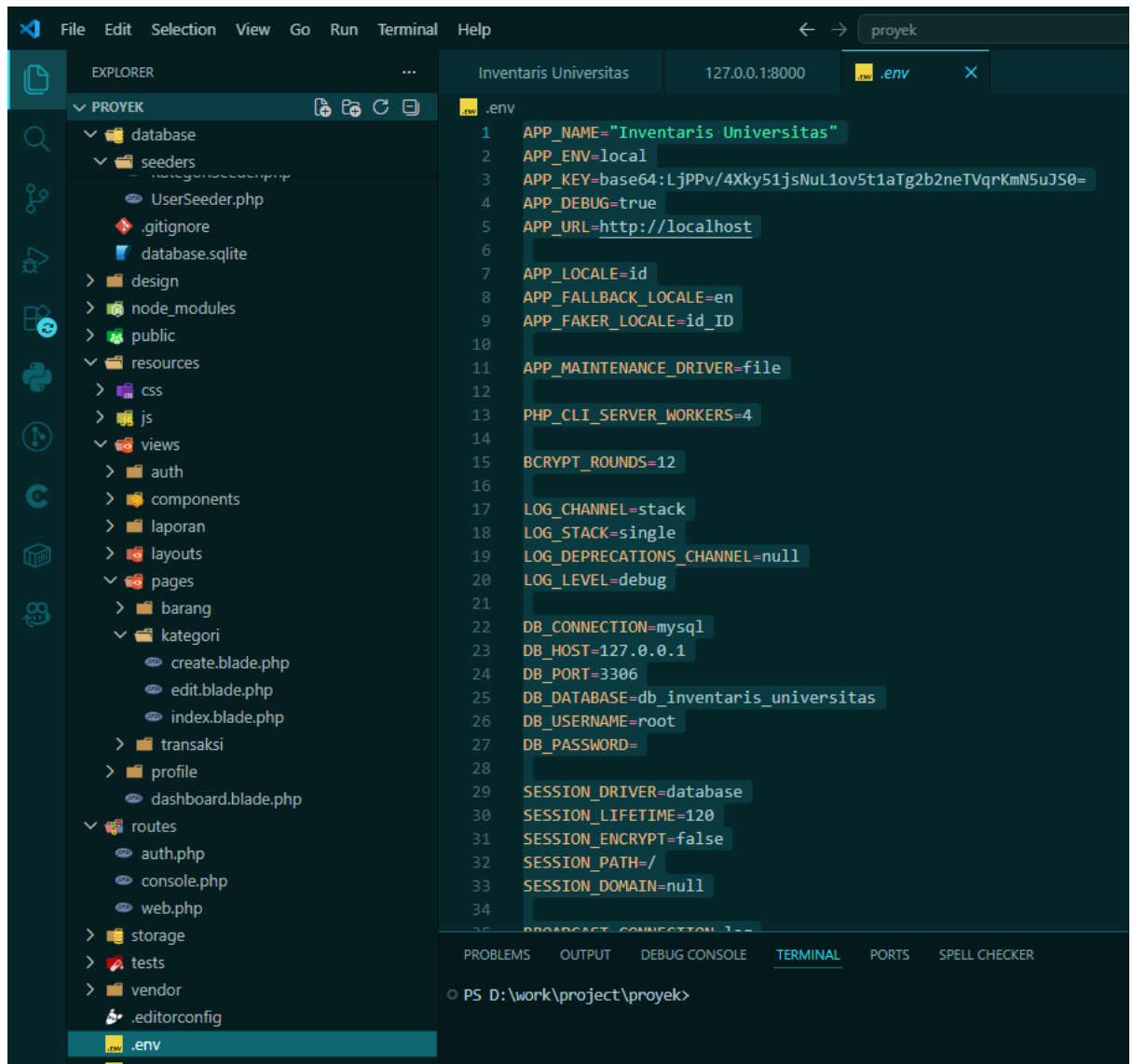
Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi ".env, database, Git init". Analisis bagaimana konsep konfigurasi environment dan git diterapkan dalam konteks manajemen inventaris barang universitas.

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

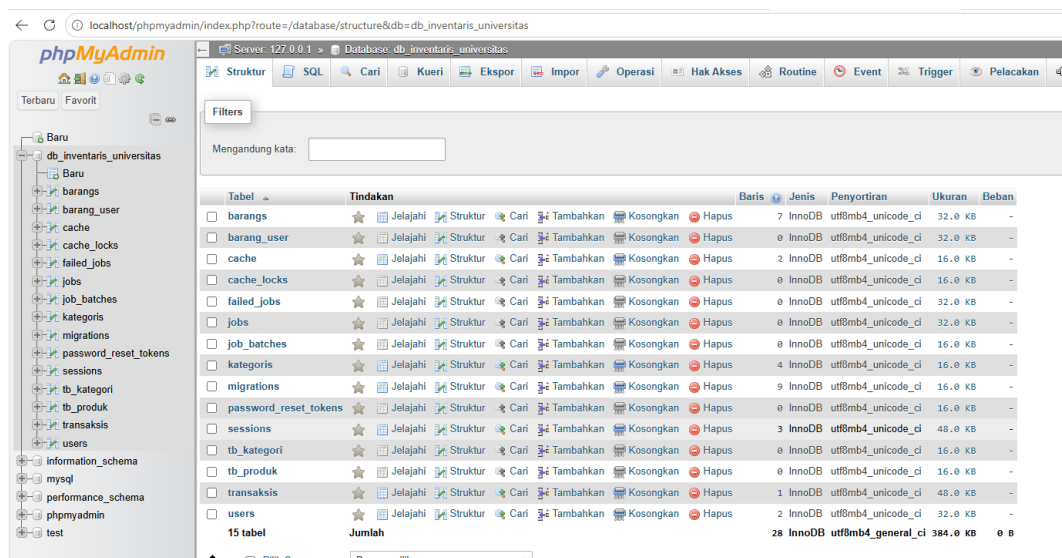
g. Hasil dan Pembahasan

Tampilan Setting .Env



```
1 APP_NAME="Inventaris Universitas"
2 APP_ENV=local
3 APP_KEY=base64:LjPPv/4Xky51jsNuL1ov5t1aTg2b2neTVqrKmN5uJS0=
4 APP_DEBUG=true
5 APP_URL=http://localhost
6
7 APP_LOCALE=id
8 APP_FALLBACK_LOCALE=en
9 APP_FAKER_LOCALE=id_ID
10
11 APP_MAINTENANCE_DRIVER=file
12
13 PHP_CLI_SERVER_WORKERS=4
14
15 BCRYPT_ROUNDS=12
16
17 LOG_CHANNEL=stack
18 LOG_STACK=single
19 LOG_DEPRECATIONS_CHANNEL=null
20 LOG_LEVEL=debug
21
22 DB_CONNECTION=mysql
23 DB_HOST=127.0.0.1
24 DB_PORT=3306
25 DB_DATABASE=db_inventaris_universitas
26 DB_USERNAME=root
27 DB_PASSWORD=
28
29 SESSION_DRIVER=database
30 SESSION_LIFETIME=120
31 SESSION_ENCRYPT=false
32 SESSION_PATH=/
33 SESSION_DOMAIN=null
34
35 BROADCAST_CONNECTION=null
```

Tampilan Database db_inventaris_universitas pada phpMyAdmin

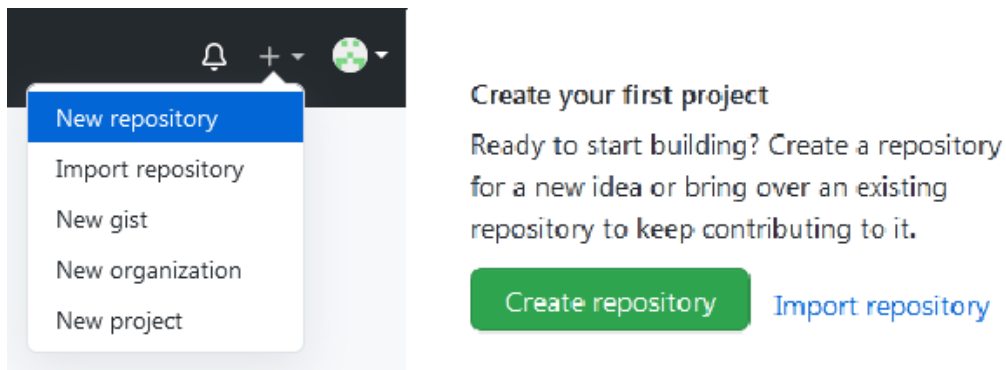


Tabel	Tindakan	Baris	Jenis	Penyortiran	Ukuran	Beban
barangs	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	7	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
barang_user	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
cache	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	2	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
cache_locks	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
failed_jobs	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
jobs	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
job_batches	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
kategori	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	4	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
migrations	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	9	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
password_reset_tokens	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
sessions	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	3	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
tb_kategori	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
tb_produk	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
transaksis	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	1	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
users	Jelajahi Struktur Cari Tambahkan Kosongkan Hapus	2	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
15 tabel	Jumlah	28	InnoDB	utf8mb4_general_ci	384.0 KB	0 B

Konfigurasi GIT

Langkah - langkah membuat repository:

1. Login ke github.com dan buatlah sebuah repository baru dengan mengeklik tombol yang terletak pada kanan atas atau pada sidebar karena pertama kita membuat repository seperti gambar berikut:



2. Buat repository baru, misalnya dengan nama "produktif" kemudian klik tombol create repository
berikut tampilan Create a new repository :

A repository contains all project files, including the revision history. Already have a project repository elsewhere?
[Import a repository.](#)

Create a new repository

Owner *

Repository name *

labksi

/

produktif

Great repository names are short and memorable. Need inspiration? How about [legendary-guacamole?](#)

Description (optional)

Website Pameran Program Studi Teknik Informatika

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☐ Add a README file

This is where you can write a long description for your project. [Learn more.](#)

☐ Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

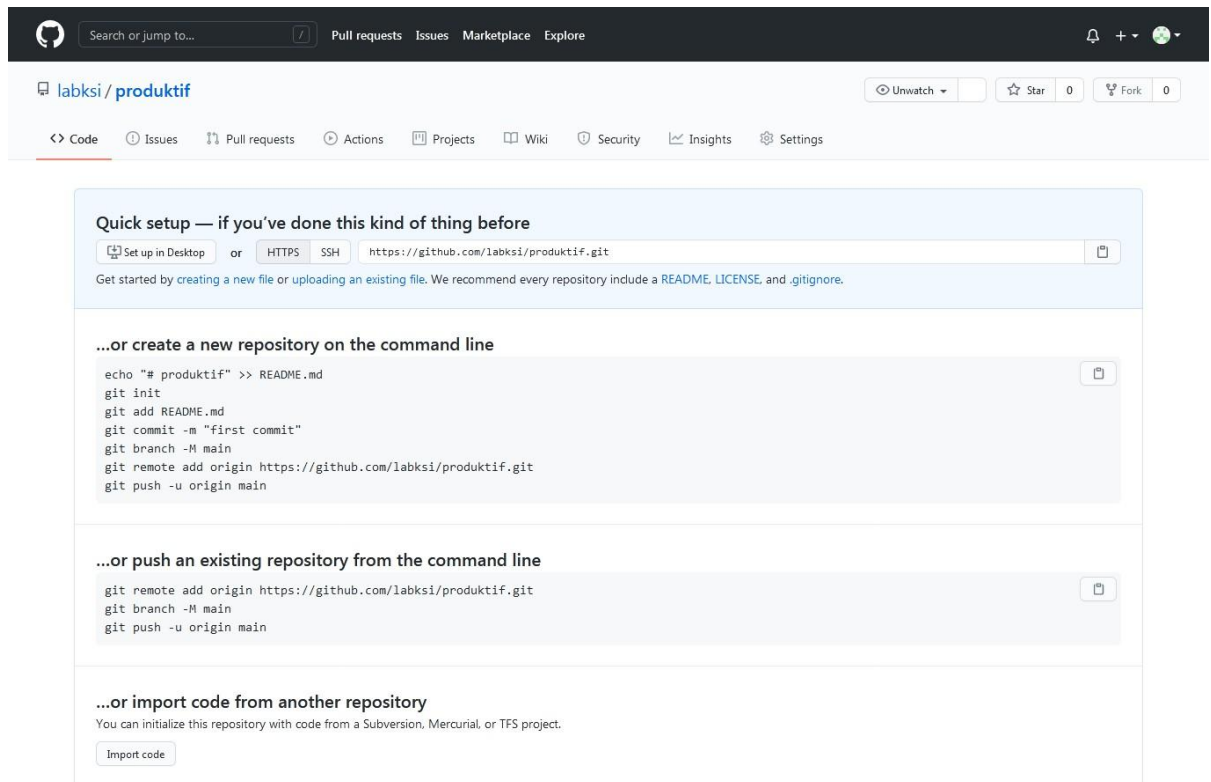
☐ Choose a license

A license tells others what they can and can't do with your code. [Learn more.](#)

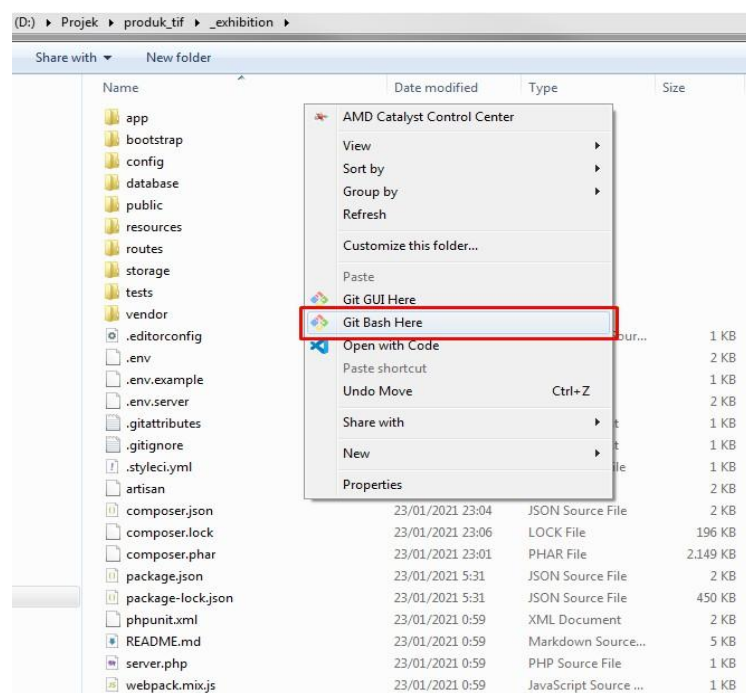
Create repository

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

3. Sekarang kita bisa mengakses remote repository dengan url <https://github.com/labksi/produktif.git>

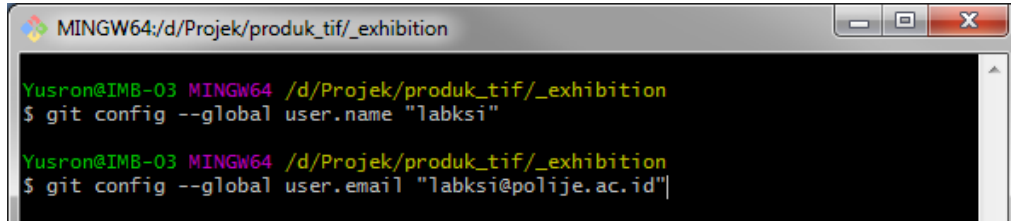


4. Arahkan pada direktori tempat project akan diupload ke repository di Github.
5. Untuk pengguna Windows klik kanan folder project dan pilih Git Bash.
- Berikut gambarnya:



6. Jika baru pertama kali menggunakan software Git, sebaiknya konfigurasi username dan email dulu :

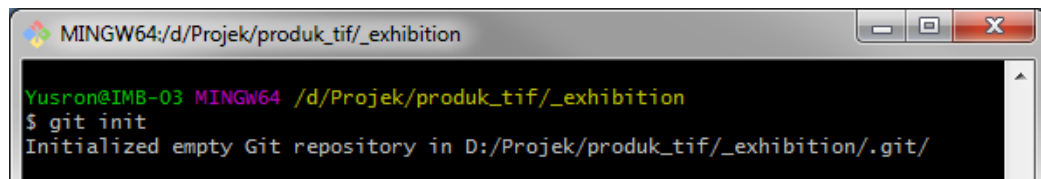
git config --global user.name "username anda" git config --global user.email "email anda"



```
MINGW64:/d/Projek/produk_tif/_exhibition
Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition
$ git config --global user.name "labksi"
Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition
$ git config --global user.email "labksi@polije.ac.id"
```

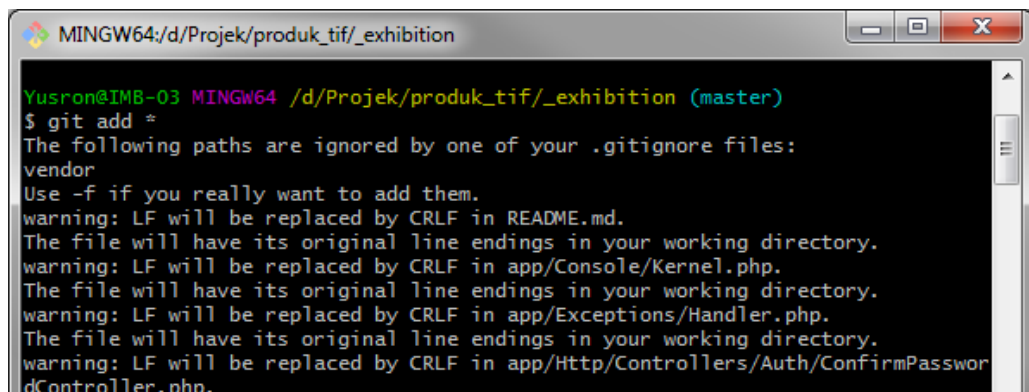
7. Setelah melakukan konfigurasi username dan email, kemudian lakukan inisiasi dengan menulis perintah berikut

git init



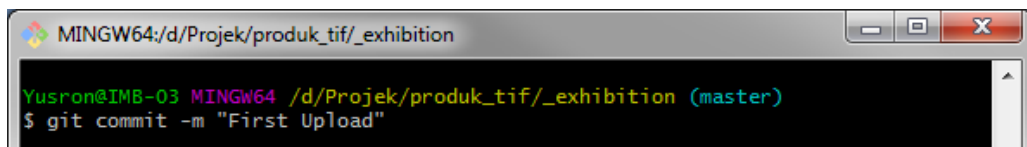
```
MINGW64:/d/Projek/produk_tif/_exhibition
Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition
$ git init
Initialized empty Git repository in D:/Projek/produk_tif/_exhibition/.git/
```

8. Kemudian tambahkan semua file yang ada dalam folder project kita, ketikan git add *



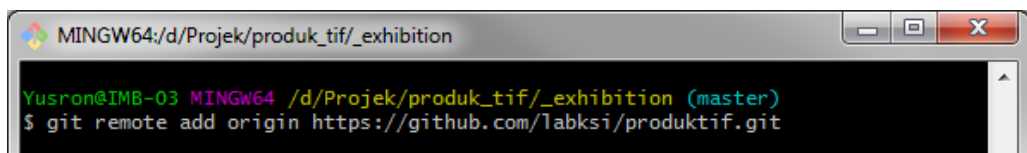
```
MINGW64:/d/Projek/produk_tif/_exhibition
Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition (master)
$ git add *
The following paths are ignored by one of your .gitignore files:
vendor
Use -f if you really want to add them.
warning: LF will be replaced by CRLF in README.md.
The file will have its original line endings in your working directory.
warning: LF will be replaced by CRLF in app/Console/Kernel.php.
The file will have its original line endings in your working directory.
warning: LF will be replaced by CRLF in app/Exceptions/Handler.php.
The file will have its original line endings in your working directory.
warning: LF will be replaced by CRLF in app/Http/Controllers/Auth/ConfirmPasswordController.php.
```

9. Kemudian commit projectnya, sebagai contoh disini commit message "First Upload" git commit -m "First Upload"



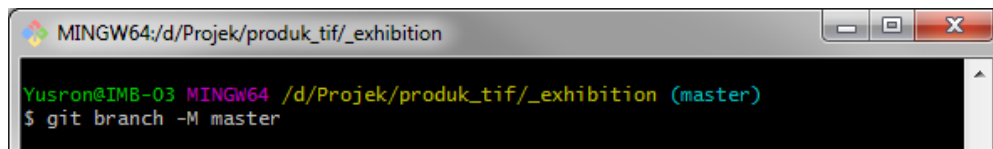
```
MINGW64:/d/Projek/produk_tif/_exhibition
Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition (master)
$ git commit -m "First Upload"
```

10. Setelah kita buat commit untuk projectnya, remote repository yang telah dibuat tadi git remote add origin https://github.com/labksi/produktif.git



```
MINGW64:/d/Projek/produk_tif/_exhibition
Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition (master)
$ git remote add origin https://github.com/labksi/produktif.git
```

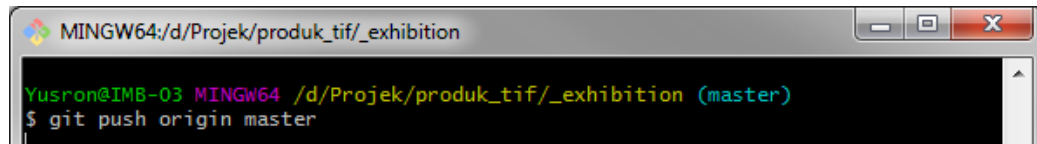
11. Buatlah cabang repository master git branch -M master



```
MINGW64:/d/Projek/produk_tif/_exhibition
Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition (master)
$ git branch -M master
```

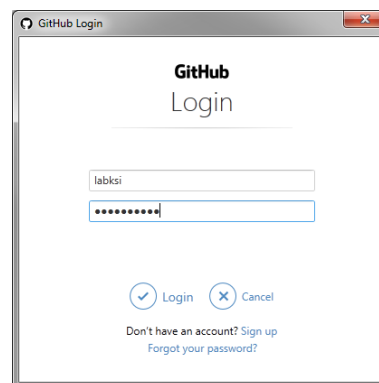
12. Setelah meremote repository, kemudian kita pull projectnya dengan perintah git pull origin master

13. Kirim project ke repository kita. git push origin master

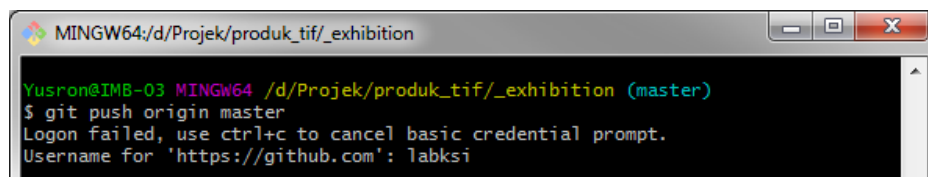


```
MINGW64:/d/Projek/produk_tif/_exhibition
Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition (master)
$ git push origin master
```

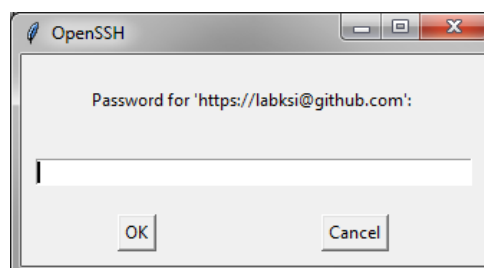
14. Terakhir, kita akan diminta username dan password kita. Untuk password mungkin kita tidak akan melihat password yang kita ketikan



atau jika gagal login tampil seperti berikut :



```
MINGW64:/d/Projek/produk_tif/_exhibition
Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition (master)
$ git push origin master
Logon failed, use ctrl+c to cancel basic credential prompt.
Username for 'https://github.com': labksi
```



15. Tunggu sampai project selesai di upload

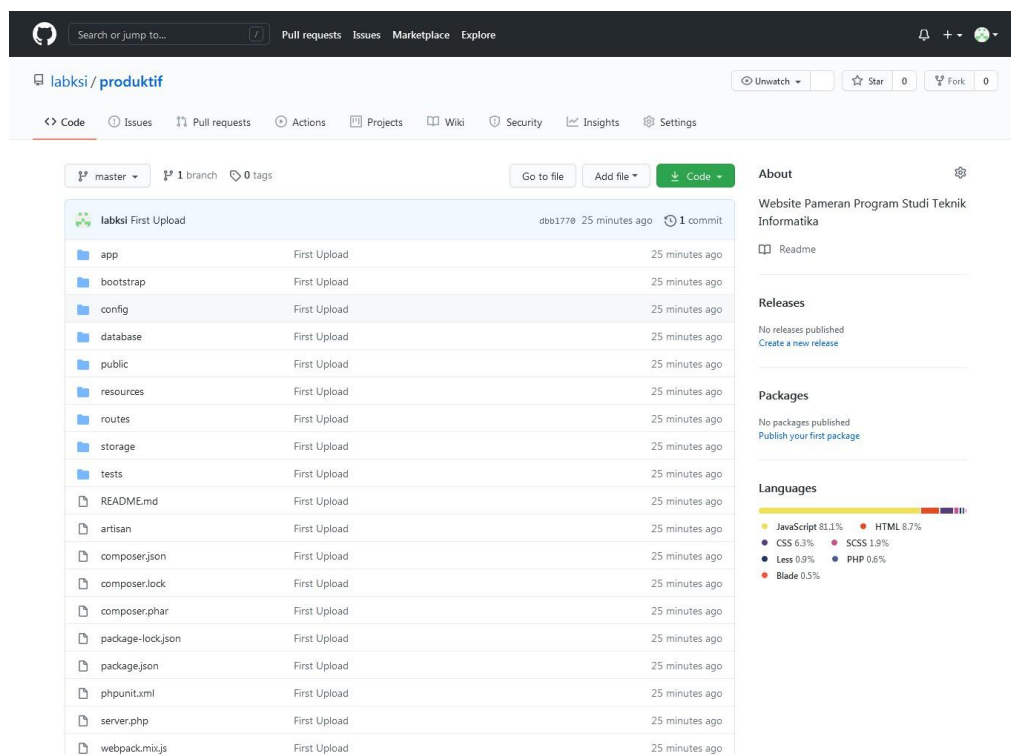
BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

```
MINGW64/d/Projek/produk_tif/_exhibition

Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition (master)
$ git push origin master
Logon failed, use ctrl+c to cancel basic credential prompt.
Username for 'https://github.com': labksi
Enumerating objects: 3684, done.
Counting objects: 100% (3684/3684), done.
Delta compression using up to 8 threads.
Compressing objects: 100% (3643/3643), done.
Writing objects: 100% (3684/3684), 21.09 MiB | 244.00 KiB/s, done.
Total 3684 (delta 903), reused 0 (delta 0)
remote: Resolving deltas: 100% (903/903), done.
To https://github.com/labksi/produktif.git
 * [new branch]      master -> master

Yusron@IMB-03 MINGW64 /d/Projek/produk_tif/_exhibition (master)
```

16. Jika proses upload selesai, cek pada Github lalu refresh repository, maka file kita sudah terupload disana



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu mengkonfigurasi environment dan Git untuk project Laravel

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

Acara 7. Membuat Model dan Migration User

Materi Pembelajaran : Model, Migration, Eloquent
Acara Praktikum/Praktik : Minggu 4/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengembangkan modul aplikasi Web Framework menggunakan fitur routing, controller, model, view (TB430805.2)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.0 17.02	Mengimplementasikan Pemrograman Terstruktur	• Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Membuat program untuk akses file • Membuat program untuk akses file
2	J.620100.0 18.02	Mengimplementasikan Pemrograman Berorientasi Objek	• Membuat program berorientasi objek dengan memanfaatkan class • Menggunakan tipe data dan control program pada metode atau operasi dari suatu kelas • Membuat program dengan konsep berbasis objek • Membuat program object oriented dengan interface dan paket • Mengkompilasi Program

c. Indikator Penilaian

Mampu membuat Model dan Migration untuk entitas User (TB430805.1)

d. Dasar Teori

Model di Laravel menggunakan Eloquent ORM untuk berinteraksi dengan database. Migration mengelola struktur tabel. User model sudah tersedia di Laravel default. Kita akan menambahkan kolom role (enum: admin, staff).

Dalam konteks Laravel, Model dan Migration merupakan dua komponen fundamental yang saling melengkapi dalam pengelolaan data aplikasi.

- Model adalah representasi logis dari tabel dalam database yang digunakan untuk berinteraksi dengan data. Model memanfaatkan Eloquent ORM, sehingga developer dapat bekerja dengan data menggunakan pendekatan berbasis objek tanpa harus menulis query SQL secara manual. Setiap model biasanya mewakili satu tabel, dengan atribut yang sesuai dengan kolom tabel tersebut. Selain itu, model juga dapat mendefinisikan relasi antar tabel, seperti one-to-one, one-to-many, atau many-to-many, sehingga memudahkan pengelolaan hubungan antar data.
- Migration adalah mekanisme untuk mengatur struktur database secara terkontrol dan terdokumentasi. Migration memungkinkan developer membuat, mengubah, atau menghapus tabel dan kolom menggunakan perintah PHP, bukan SQL langsung. Dengan migration, setiap perubahan struktur database dapat dicatat dalam file khusus, sehingga mudah dilacak dan di-rollback bila diperlukan. Migration juga mendukung kolaborasi tim, karena setiap anggota dapat menjalankan perintah `php artisan migrate` untuk menyamakan struktur database sesuai dengan definisi yang ada di file migration.

Secara teori, penggunaan model dan migration mendukung prinsip database versioning dan object-relational mapping. Model berfungsi sebagai jembatan antara logika aplikasi dengan data, sementara migration memastikan struktur database konsisten dan dapat berkembang seiring kebutuhan aplikasi. Kombinasi keduanya menjadikan Laravel lebih efisien, terstruktur, dan aman dalam pengembangan aplikasi berbasis data.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

1. Buka file migration users: `database/migrations/xxxx_create_users_table.php`
2. Tambahkan role setelah password:
`\$table->enum('role', ['admin', 'staff'])->default('staff');`
3. Buka `app/Models/User.php`, tambahkan ke `$fillable`:
`protected $fillable = ['name', 'email', 'password', 'role'];`

4. Jalankan migration: `php artisan migrate`
5. Buat seeder User:
`php artisan make:seeder UserSeeder`
6. Isi UserSeeder dengan:
`User::create(['name'=>'Admin','email'=>'admin@inventaris.test','password'=>bcrypt('password'),'role'=>'admin']);`
`User::create([...role staff...]);`
7. Jalankan seeder: `php artisan db:seed --class=UserSeeder`
8. Cek di database: `SELECT * FROM users;`

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Model, Migration, Eloquent". Analisis bagaimana konsep membuat model dan migration user diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Berikut adalah langkah-langkah untuk membuat Model dan Migration pada project Inventaris Universitas:

Membuat Migration

1. a) Migration Users

Secara default Laravel sudah menyediakan migration users. Kita akan memodifikasinya dengan menambahkan kolom role.

Buka file `database/migrations/0001_01_01_000000_create_users_table.php`
Tambahkan baris berikut pada method `up()` setelah `$table->string('password');`

```
$table->enum('role', ['admin', 'staff'])->default('staff');
```

2. b) Migration Kategori

Jalankan perintah:

```
php artisan make:migration create_kategoris_table
```

Isi method `up()` dengan:

```
Schema::create('kategoris', function (Blueprint $table) {  
    $table->id();  
    $table->string('nama', 100);  
    $table->text('deskripsi')->nullable();  
    $table->timestamps();  
});
```

3. c) Migration Barang

```
php artisan make:migration create_barangs_table
```

Tabel `barang` memiliki foreign key ke `kategoris`:

```
$table->foreignId('kategori_id')->constrained('kategoris')->onDelete('cascade');
```

Kolom lainnya: nama (string 150), jumlah (integer), kondisi (enum: baik/rusak/hilang), deskripsi (text nullable), gambar (string nullable).

4. d) Migration Transaksi

php artisan make:migration create_transaksis_table

Tabel transaksis memiliki foreign key ke users dan barangs:

```
$table->foreignId('user_id')->constrained('users')->onDelete('cascade');
```

```
$table->foreignId('barang_id')->constrained('barangs')->onDelete('cascade');
```

Kolom lainnya: jumlah (integer), tgl_pinjam (date), tgl_kembali (date nullable), status (enum: dipinjam/dikembalikan).

5. e) Migration Pivot Barang_User (Favorit)

php artisan make:migration create_barang_user_table

Tabel pivot untuk relasi many-to-many:

```
$table->foreignId('user_id')->constrained()->onDelete('cascade');
```

```
$table->foreignId('barang_id')->constrained('barangs')->onDelete('cascade');
```

```
$table->primary(['user_id', 'barang_id']);
```

Membuat Model

6. a) Model User (app/Models/User.php)

User sudah tersedia default. Tambahkan 'role' ke \$fillable.

```
protected $fillable = ['name', 'email', 'password', 'role'];
```

7. Tambahkan method untuk transaksi dan favorit:

```
public function transaksis(): HasMany { return $this->hasMany(Transaksi::class); }
```

```
public function favorit(): BelongsToMany { return $this->belongsToMany(Barang::class, 'barang_user'); }
```

8. b) Model Kategori (app/Models/Kategori.php)

php artisan make:model Kategori

Tentukan tabel: protected \$table = 'kategoris';

```
Relasi: public function barangs(): HasMany { return $this->hasMany(Barang::class); }
```

9. c) Model Barang (app/Models/Barang.php)

php artisan make:model Barang

Tentukan tabel dan fillable.

Relasi: belongsTo(Kategori), hasMany(Transaksi), belongsToMany(User).

10. d) Model Transaksi (app/Models/Transaksi.php)

php artisan make:model Transaksi

Relasi: belongsTo(User), belongsTo(Barang).

Date casting untuk tgl_pinjam dan tgl_kembali.

Mendefinisikan Relationship

Relasi antar model pada project Inventaris Universitas:

Model A	Method A	Model B	Method B	Jenis
---------	----------	---------	----------	-------

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

User	transaksis()	Transaksi	user()	1 — *
Kategori	barangs()	Barang	kategori()	1 — *
Barang	transaksis()	Transaksi	barang()	1 — *
User	favorits()	Barang	favoritedBy()	* — * (pivot)

Membuat Seeder

11. a) DatabaseSeeder sebagai entry point yang memanggil semua seeder.
12. b) UserSeeder: membuat 2 user (admin dan staff).
13. c) KategoriSeeder: membuat 4 kategori (Elektronik, Perabotan, Alat Tulis, Komputer).
14. d) BarangSeeder: membuat 6 barang contoh dari berbagai kategori.

Menjalankan Migration & Seeder

15. Jalankan migration: php artisan migrate
16. Jalankan semua seeder: php artisan db:seed
17. Atau jalankan seeder spesifik: php artisan db:seed --class=UserSeeder
18. Refresh migration + seed: php artisan migrate:fresh --seed
19. Cek hasil di database: SELECT * FROM users; SELECT * FROM kategoris; SELECT * FROM barangs;

Berikut adalah tampilan file dari project Inventaris Universitas:

Migration: users table

```
0001_01_01_000000_create_users_table.php Migration

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->enum('role', ['admin', 'staff'])->default('staff');
            $table->rememberToken();
            $table->timestamps();
        });

        Schema::create('password_reset_tokens', function (Blueprint $table) { ... });
        Schema::create('sessions', function (Blueprint $table) { ... });
    }

    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('sessions');
    }
};
```

Migration: kategoris table

2026_07_16_000001_create_kategoris_table.php

Migration

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('kategoris', function (Blueprint $table) {
            $table->id();
            $table->string('nama', 100);
            $table->text('deskripsi')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('kategoris');
    }
};
```

Migration: barangs table

2026_07_16_000002_create_barangs_table.php

Migration

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('barangs', function (Blueprint $table) {
            $table->id();
            $table->foreignId('kategori_id')
                ->constrained('kategoris')
                ->onDelete('cascade');
            $table->string('nama', 150);
            $table->integer('jumlah')->default(0);
            $table->enum('kondisi', ['baik', 'rusak', 'hilang'])
                ->default('baik');
            $table->text('deskripsi')->nullable();
            $table->string('gambar')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('barangs');
    }
};
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Migration: transaksis table

2026_07_16_000003_create_transaksis_table.php

Migration

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('transaksis', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')
                ->constrained('users')
                ->onDelete('cascade');
            $table->foreignId('barang_id')
                ->constrained('barang')
                ->onDelete('cascade');
            $table->integer('jumlah');
            $table->date('tgl_pinjam');
            $table->date('tgl_kembali')->nullable();
            $table->enum('status', ['dipinjam', 'dikembalikan'])
                ->default('dipinjam');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('transaksis');
    }
};
```

Migration: barang_user (pivot)

2026_07_16_000004_create_barang_user_table.php

Migration

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('barang_user', function (Blueprint $table) {
            $table->foreignId('user_id')
                ->constrained()
                ->onDelete('cascade');
            $table->foreignId('barang_id')
                ->constrained('barang')
                ->onDelete('cascade');
            $table->primary(['user_id', 'barang_id']);
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('barang_user');
    }
};
```

Model: User.php

app/Models/User.php

Model

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class User extends Authenticatable
{
    use HasFactory, Notifiable;

    protected $fillable = [
        'name', 'email', 'password', 'role',
    ];

    protected $hidden = [
        'password', 'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }

    public function transaksis(): HasMany
    {
        return $this->hasMany(Transaksi::class);
    }

    public function favorites(): BelongsToMany
    {
        return $this->belongsToMany(Barang::class, 'barang_user');
    }

    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }
}
```

Model: Kategori.php

```
app/Models/Kategori.php

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Kategori extends Model
{
    use HasFactory;
    protected $table = 'kategoris';

    protected $fillable = [
        'nama', 'deskripsi',
    ];

    public function barangs(): HasMany
    {
        return $this->hasMany(Barang::class);
    }
}
```

Model: Barang.php

```
app/Models/Barang.php

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class Barang extends Model
{
    use HasFactory;
    protected $table = 'barangs';

    protected $fillable = [
        'kategori_id', 'nama', 'jumlah',
        'kondisi', 'deskripsi', 'gambar',
    ];

    public function kategori(): BelongsTo
    {
        return $this->belongsTo(Kategori::class);
    }

    public function transaksi(): HasMany
    {
        return $this->hasMany(Transaksi::class);
    }

    public function favoritedBy(): BelongsToMany
    {
        return $this->belongsToMany(User::class, 'barang_user');
    }
}
```

Model: Transaksi.php

```
app/Models/Transaksi.php Model

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Transaksi extends Model
{
    protected $table = 'transaksis';

    protected $fillable = [
        'user_id', 'barang_id', 'jumlah',
        'tgl_pinjam', 'tgl_kembali', 'status',
    ];

    protected function casts(): array
    {
        return [
            'tgl_pinjam' => 'date',
            'tgl_kembali' => 'date',
        ];
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    public function barang(): BelongsTo
    {
        return $this->belongsTo(Barang::class);
    }
}
```

Seeder: DatabaseSeeder.php

```
database/seeder/DatabaseSeeder.php Seeder

<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call([
            UserSeeder::class,
            KategoriSeeder::class,
            BarangSeeder::class,
        ]);
    }
}
```

Seeder: UserSeeder.php

database/seeder/UserSeeder.php

Seeder

```
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class UserSeeder extends Seeder
{
    public function run(): void
    {
        User::create([
            'name' => 'Admin Inventaris',
            'email' => 'admin@inventaris.test',
            'password' => Hash::make('password'),
            'role' => 'admin',
        ]);

        User::create([
            'name' => 'Staff Inventaris',
            'email' => 'staff@inventaris.test',
            'password' => Hash::make('password'),
            'role' => 'staff',
        ]);
    }
}
```

Seeder: KategoriSeeder.php

database/seeder/KategoriSeeder.php

Seeder

```
<?php

namespace Database\Seeders;

use App\Models\Kategori;
use Illuminate\Database\Seeder;

class KategoriSeeder extends Seeder
{
    public function run(): void
    {
        Kategori::create(['nama' => 'Elektronik',
            'deskripsi' => 'Barang elektronik dan perangkat keras']);
        Kategori::create(['nama' => 'Perabotan',
            'deskripsi' => 'Meja, kursi, lemari, dan perabotan lainnya']);
        Kategori::create(['nama' => 'Alat Tulis',
            'deskripsi' => 'Alat tulis kantor dan perlengkapan administrasi']);
        Kategori::create(['nama' => 'Komputer',
            'deskripsi' => 'Desktop, laptop, dan perangkat komputer']);
    }
}
```

Seeder: BarangSeeder.php

database/seeder/BarangSeeder.php

Seeder

```
<?php

namespace Database\Seeders;

use App\Models\Barang;
use Illuminate\Database\Seeder;

class BarangSeeder extends Seeder
{
    public function run(): void
    {
        Barang::create(['kategori_id' => 4, 'nama' => 'Laptop HP Pavilion',
            'jumlah' => 10, 'kondisi' => 'baik',
            'deskripsi' => 'Laptop HP Pavilion 14 inch, RAM 8GB, SSD 256GB']);
        Barang::create(['kategori_id' => 4, 'nama' => 'Desktop PC',
            'jumlah' => 5, 'kondisi' => 'baik',
            'deskripsi' => 'Desktop PC Intel Core i5, RAM 16GB']);
        Barang::create(['kategori_id' => 1, 'nama' => 'Proyektor Epson',
            'jumlah' => 3, 'kondisi' => 'baik',
            'deskripsi' => 'Proyektor Epson EB-X51']);
        Barang::create(['kategori_id' => 2, 'nama' => 'Meja Kantor',
            'jumlah' => 20, 'kondisi' => 'baik',
            'deskripsi' => 'Meja kantor ukuran 120x60 cm']);
        Barang::create(['kategori_id' => 2, 'nama' => 'Kursi Kantor',
            'jumlah' => 25, 'kondisi' => 'baik',
            'deskripsi' => 'Kursi kantor ergonomis']);
        Barang::create(['kategori_id' => 3, 'nama' => 'Spidol Whiteboard',
            'jumlah' => 50, 'kondisi' => 'baik',
            'deskripsi' => 'Spidol whiteboard berbagai warna']);
    }
}
```

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat Model dan Migration untuk entitas User

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusa	V			
Total Skor*		Skor x 25			

Acara 8. Implementasi Auth Scaffolding

Materi Pembelajaran : Laravel Breeze, login, register
Acara Praktikum/Praktik : Minggu 4/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengembangkan modul aplikasi Web Framework menggunakan fitur routing, controller, model, view (TB430805.2)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.0 17.02	Mengimplementasikan Pemrograman Terstruktur	• Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Membuat program untuk akses file • Membuat program untuk akses file
2	J.620100.0 22.02	Mengimplementasikan Algoritma Pemrograman	• Menjelaskan varian dan invariant • Membuat alur logika pemrograman • Menerapkan teknik dasar algoritma umum • Menggunakan prosedur dan fungsi • Mengidentifikasi kompleksitas algoritma

c. Indikator Penilaian

Mahasiswa mampu mengimplementasikan autentikasi

d. Dasar Teori

Laravel Breeze adalah paket starter yang disediakan Laravel untuk mempermudah implementasi autentikasi dasar. Breeze menawarkan scaffolding yang ringan, sederhana, dan mudah dipahami, menggunakan Blade template, Tailwind CSS, serta sistem routing bawaan Laravel. Dengan Breeze, developer

dapat langsung memiliki fitur login, register, reset password, email verification, dan logout tanpa harus membangun semuanya dari nol. Breeze dirancang agar mudah dikustomisasi, sehingga cocok untuk proyek yang membutuhkan fleksibilitas tinggi.

Auth Scaffolding sendiri merujuk pada kerangka kerja yang disediakan Laravel untuk membangun sistem autentikasi. Scaffolding ini mencakup struktur file, controller, route, view, dan logika yang diperlukan agar aplikasi memiliki mekanisme otentikasi pengguna. Laravel sebelumnya menyediakan paket seperti Laravel UI dan Jetstream, namun Breeze hadir sebagai solusi yang lebih sederhana dibanding Jetstream, dengan fokus pada kecepatan dan kemudahan pemahaman.

Secara teori, penggunaan Breeze dan Auth Scaffolding mendukung prinsip security by design dalam pengembangan perangkat lunak. Dengan adanya scaffolding, developer tidak perlu menulis ulang logika autentikasi yang kompleks, sehingga mengurangi risiko kesalahan implementasi dan meningkatkan keamanan aplikasi. Laravel Breeze menyediakan scaffolding autentikasi lengkap (login, register, password reset). Kita akan menginstal Breeze dan blade stack dengan Alpine.js.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

1. Install Laravel Breeze
2. Install package Breeze melalui Composer: `composer require laravel/breeze --dev`
3. Install stack Blade: `php artisan breeze`
4. Install dependencies NPM: `npm install && npm run build`
5. Jalankan migration: `php artisan migrate`

6. Route Auth

Breeze menghasilkan file `routes/auth.php` yang berisi semua route autentikasi.

Route guest (tanpa login):

- GET /register & POST /register : Registrasi user baru

- GET /login & POST /login : Login user
- GET /forgot-password & POST /forgot-password : Lupa password
- GET /reset-password/{token} & POST /reset-password : Reset password

Route auth (sudah login):

- GET /verify-email: Peringatan verifikasi email
- GET/POST /confirm-password : Konfirmasi password
- PUT /password : Update password
- POST /logout : Logout

Route web.php juga menyertakan:

- GET / : Redirect ke halaman login
- GET /dashboard : Halaman dashboard (middleware: auth, verified)
- GET/PATCH/DELETE /profile : Manajemen profile

7. Controller Auth

Controller autentikasi berada di app/Http/Controllers/Auth/:

AuthenticatedSessionController:

- create(): Menampilkan form login
- store(LoginRequest): Memvalidasi credentials, login, redirect ke dashboard
- destroy(Request): Logout, invalidate session, redirect ke '/'

RegisteredUserController:

- create(): Menampilkan form register
- store(Request): Validasi input, create user dengan role 'staff', auto-login

ProfileController:

- edit(): Menampilkan form edit profile
- update(ProfileUpdateRequest): Update nama/email, reset verifikasi jika email berubah
- destroy(Request): Hapus akun (validasi password terlebih dahulu)

LoginRequest: Form request khusus untuk login dengan rate limiting (5 percobaan)

8. View Auth

View Blade untuk autentikasi berada di resources/views/auth/:

- login.blade.php : Form login (email, password, remember me, forgot password link)
- register.blade.php : Form register (name, email, password, confirm password)
- forgot-password.blade.php : Form lupa password (email)
- reset-password.blade.php : Form reset password (token, email, password)

- verify-email.blade.php : Notifikasi verifikasi email
- confirm-password.blade.php : Form konfirmasi password

9. Layout Guest & App

Terdapat dua layout:

guest.blade.php:

- Untuk halaman publik (login, register, forgot/reset password)
- Menggunakan component slot (\$slot) untuk konten
- Tampilan minimal dengan card centered

app.blade.php:

- Untuk halaman terautentikasi (dashboard, profile, CRUD)
- Sidebar navigasi dengan menu Dashboard, Kategori, Barang, Transaksi, Laporan
- Topbar dengan user info
- Custom CSS styling dengan gradient, card, table styles

10. Dashboard

DashboardController mengirim data statistik ke view:

- Total Barang, Total Kategori, Barang Dipinjam, Total Stok
- 5 transaksi terbaru dengan user dan barang (eager loading)
- 5 barang dengan stok tipis (< 5 unit)

View dashboard.blade.php menampilkan:

- 4 stat card dengan gradient warna
- Tabel transaksi terbaru
- Tabel stok tipis
- Aksi cepat (Tambah Barang, Tambah Kategori, Pinjam Barang, Cetak Laporan)

11. Profile

Halaman profile (profile/edit.blade.php) terdiri dari tiga bagian:

1. Update Profile Information : Edit nama dan email
2. Update Password : Ganti password dengan validasi password saat ini
3. Delete User : Hapus akun dengan konfirmasi password

Masing-masing menggunakan partial view di resources/views/profile/partials/

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Laravel Breeze, login, register". Analisis bagaimana konsep implementasi auth scaffolding diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Berikut adalah tampilan file autentikasi dari project Inventaris Universitas:

```
routes/auth.php

<?php

use App\Http\Controllers\Auth\AuthenticatedSessionController;
use App\Http\Controllers\Auth\ConfirmablePasswordController;
use App\Http\Controllers\Auth>EmailVerificationNotificationController;
use App\Http\Controllers\Auth>EmailVerificationPromptController;
use App\Http\Controllers\Auth\NewPasswordController;
use App\Http\Controllers\Auth>PasswordController;
use App\Http\Controllers\Auth>PasswordResetLinkController;
use App\Http\Controllers\Auth\RegisteredUserController;
use App\Http\Controllers\Auth\VerifyEmailController;
use Illuminate\Support\Facades\Route;

Route::middleware('guest')->group(function () {
    Route::get('register', [RegisteredUserController::class, 'create'])->name('register');
    Route::post('register', [RegisteredUserController::class, 'store']);
    Route::get('login', [AuthenticatedSessionController::class, 'create'])->name('login');
    Route::post('login', [AuthenticatedSessionController::class, 'store']);
    Route::get('forgot-password', ...) ->name('password.request');
    Route::post('forgot-password', ...) ->name('password.email');
    Route::get('reset-password/{token}', ...) ->name('password.reset');
    Route::post('reset-password', ...) ->name('password.store');
});

Route::middleware('auth')->group(function () {
    Route::get('verify-email', EmailVerificationPromptController::class)->name('verification.notice');
    Route::get('verify-email/{id}/{hash}', ...) ->name('verification.verify');
    Route::post('email/verification-notification', ...) ->name('verification.send');
    Route::get('confirm-password', ...) ->name('password.confirm');
    Route::post('confirm-password', ...);
    Route::put('password', [PasswordController::class, 'update'])->name('password.update');
    Route::post('logout', [AuthenticatedSessionController::class, 'destroy'])->name('logout');
});
```

Route: web.php

```
routes/web.php

<?php

use App\Http\Controllers\ProfileController;
use App\Http\Controllers\DashboardController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return redirect()->route('login');
});

Route::middleware(['auth', 'verified'])->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index'])
        ->name('dashboard');

    Route::get('/profile', [ProfileController::class, 'edit'])
        ->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])
        ->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])
        ->name('profile.destroy');
});

require __DIR__.'/auth.php';
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Controller: AuthenticatedSessionController

```
app/Http/Controllers/Auth/AuthenticatedSessionController.php Controller

<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class AuthenticatedSessionController extends Controller
{
    public function create(): View
    {
        return view('auth.login');
    }

    public function store(LoginRequest $request): RedirectResponse
    {
        $request->authenticate();
        $request->session()->regenerate();
        return redirect()->intended(route('dashboard', absolute: false));
    }

    public function destroy(Request $request): RedirectResponse
    {
        Auth::guard('web')->logout();
        $request->session()->invalidate();
        $request->session()->regenerateToken();
        return redirect('/');
    }
}
```

Controller: RegisteredUserController

```
app/Http/Controllers/Auth/RegisteredUserController.php Controller

<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;
use Illuminate\View\View;

class RegisteredUserController extends Controller
{
    public function create(): View
    {
        return view('auth.register');
    }

    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:'.User::class],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
            'role' => 'staff',
        ]);

        event(new Registered($user));
        Auth::login($user);
        return redirect(route('dashboard', absolute: false));
    }
}
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Controller: ProfileController

```
app/Http/Controllers/ProfileController.php Controller

<?php

namespace App\Http\Controllers;

use App\Http\Requests\ProfileUpdateRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Redirect;
use Illuminate\View\View;

class ProfileController extends Controller
{
    public function edit(Request $request): View
    {
        return view('profile.edit', ['user' => $request->user()]);
    }

    public function update(ProfileUpdateRequest $request): RedirectResponse
    {
        $request->user()->fill($request->validated());
        if ($request->user()->isDirty('email')) {
            $request->user()->email_verified_at = null;
        }
        $request->user()->save();
        return Redirect::route('profile.edit')
            ->with('status', 'profile-updated');
    }

    public function destroy(Request $request): RedirectResponse
    {
        $request->validateWithBag('userDeletion', [
            'password' => ['required', 'current_password'],
        ]);
        $user = $request->user();
        Auth::logout();
        $user->delete();
        $request->session()->invalidate();
        $request->session()->regenerateToken();
        return Redirect::to('/');
    }
}
```

Request: LoginRequest

```
app/Http/Requests/Auth/LoginRequest.php Request

<?php

namespace App\Http\Requests\Auth;

use Illuminate\Auth\Events\Lockout;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;
use Illuminate\Validation\ValidationException;

class LoginRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'email' => ['required', 'string', 'email'],
            'password' => ['required', 'string'],
        ];
    }

    public function authenticate(): void
    {
        $this->ensureIsNotRateLimited();
        if (! Auth::attempt($this->only('email', 'password'),
            $this->boolean('remember'))) {
            RateLimiter::hit($this->throttleKey());
            throw ValidationException::withMessages([
                'email' => trans('auth.failed'),
            ]);
        }
        RateLimiter::clear($this->throttleKey());
    }
}
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Layout: guest.blade.php

resources/views/layouts/guest.blade.php

Layout

```
<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="csrf-token" content="{{ csrf_token() }}">
    <title>{{ config('app.name', 'Laravel') }}</title>
    <link rel="preconnect" href="https://fonts.bunny.net">
    <link href="https://fonts.bunny.net/css?family=figtree:400,500,600&display=swap" rel="stylesheet" />
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body class="font-sans text-gray-900 antialiased">
    <div class="min-h-screen flex flex-col sm:justify-center items-center pt-6 sm:pt-0 bg-gray-100">
        <div>
            <a href="/">
                <x-application-logo class="w-20 h-20 fill-current text-gray-500" />
            </a>
        </div>
        <div class="w-full sm:max-w-md mt-6 px-6 py-4 bg-white shadow-md overflow-hidden sm:rounded-lg">
            {{ $slot }}
        </div>
    </div>
</body>
</html>
```

View: login.blade.php

resources/views/auth/login.blade.php

View

```
<x-guest-layout>
    <x-auth-session-status class="mb-4" :status="session('status')" />

    <form method="POST" action="{{ route('login') }}">
        @csrf

        <div>
            <x-input-label for="email" :value="__('Email')"/>
            <x-text-input id="email" class="block mt-1 w-full" type="email"
                name="email" :value="old('email')" required autofocus
                autocomplete="username" />
            <x-input-error :messages="$errors->get('email')" class="mt-2" />
        </div>

        <div class="mt-4">
            <x-input-label for="password" :value="__('Password')"/>
            <x-text-input id="password" class="block mt-1 w-full" type="password"
                name="password" required autocomplete="current-password" />
            <x-input-error :messages="$errors->get('password')" class="mt-2" />
        </div>

        <div class="block mt-4">
            <label for="remember_me" class="inline-flex items-center">
                <input id="remember_me" type="checkbox" class="rounded border-gray-300 text-indigo-600 shadow-sm focus:ring-indigo-500" />
                <span class="ms-2 text-sm text-gray-600">{{ __('Remember me') }}</span>
            </label>
        </div>

        <div class="flex items-center justify-end mt-4">
            @if (Route::has('password.request'))
                <a class="underline text-sm text-gray-600 hover:text-gray-900 rounded-md"
                    href="{{ route('password.request') }}">
                    {{ __('Forgot your password?') }}
                </a>
            @endif

            <x-primary-button class="ms-3">{{ __('Log in') }}</x-primary-button>
        </div>
    </form>
</x-guest-layout>
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

View: register.blade.php

resources/views/auth/register.blade.php

View

```
<x-guest-layout>
  <form method="POST" action="{{ route('register') }}">
    @csrf

    <div>
      <x-input-label for="name" :value="__('Name')"/>
      <x-text-input id="name" class="block mt-1 w-full" type="text"
        name="name" :value="old('name')" required autofocus
        autocomplete="name" />
      <x-input-error :messages="$errors->get('name')" class="mt-2" />
    </div>

    <div class="mt-4">
      <x-input-label for="email" :value="__('Email')"/>
      <x-text-input id="email" class="block mt-1 w-full" type="email"
        name="email" :value="old('email')" required
        autocomplete="username" />
      <x-input-error :messages="$errors->get('email')" class="mt-2" />
    </div>

    <div class="mt-4">
      <x-input-label for="password" :value="__('Password')"/>
      <x-text-input id="password" class="block mt-1 w-full" type="password"
        name="password" required autocomplete="new-password" />
      <x-input-error :messages="$errors->get('password')" class="mt-2" />
    </div>

    <div class="mt-4">
      <x-input-label for="password_confirmation" :value="__('Confirm Password')"/>
      <x-text-input id="password_confirmation" class="block mt-1 w-full"
        type="password" name="password_confirmation" required
        autocomplete="new-password" />
      <x-input-error :messages="$errors->get('password_confirmation')" class="mt-2" />
    </div>

    <div class="flex items-center justify-end mt-4">
      <a class="underline text-sm text-gray-600 hover:text-gray-900 rounded-md"
        href="{{ route('login') }}">
        {{ __('Already registered?') }}
      </a>
      <x-primary-button class="ms-4">{{ __('Register') }}</x-primary-button>
    </div>
  </form>
</x-guest-layout>
```

View: dashboard.blade.php

resources/views/dashboard.blade.php

View

```
<x-app-layout>
  @if (session('success'))
    <div class="alert alert-success">
      <i class="fas fa-check-circle"></i> {{ session('success') }}
    </div>
  @endif

  <div class="stat-grid">
    <div class="stat-card" style="background:linear-gradient(135deg,#667eea,#764ba2)">
      <div class="icon"><i class="fas fa-cube"></i></div>
      <div class="label">Total Barang</div>
      <div class="value">{{ $totalBarang }}</div>
    </div>
    <div class="stat-card" style="background:linear-gradient(135deg,#11998e,#38ef7d)">
      <div class="icon"><i class="fas fa-tags"></i></div>
      <div class="label">Total Kategori</div>
      <div class="value">{{ $totalKategori }}</div>
    </div>
    ...
  </div>
</x-app-layout>
```

Controller: DashboardController

```
app/Http/Controllers/DashboardController.php Controller

<?php

namespace App\Http\Controllers;

use App\Models\Barang;
use App\Models\Kategori;
use App\Models\Transaksi;
use App\Models\User;

class DashboardController extends Controller
{
    public function index()
    {
        $totalBarang = Barang::count();
        $totalKategori = Kategori::count();
        $barangDipinjam = Transaksi::where('status', 'dipinjam')->count();
        $barangTersedia = Barang::sum('jumlah');
        $transaksiTerbaru = Transaksi::with(['user', 'barang'])
            ->latest()
            ->take(5)
            ->get();
        $barangStokMenipis = Barang::where('jumlah', '<', 5)
            ->with('kategori')
            ->take(5)
            ->get();

        return view('dashboard', compact(
            'totalBarang', 'totalKategori', 'barangDipinjam',
            'barangTersedia', 'transaksiTerbaru', 'barangStokMenipis'
        ));
    }
}
```

View: profile/edit.blade.php

```
resources/views/profile/edit.blade.php View

<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            {{ __('Profile') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8 space-y-6">
            <div class="p-4 sm:p-8 bg-white shadow sm:rounded-lg">
                <div class="max-w-xl">
                    @include('profile.partials.update-profile-information-form')
                </div>
            </div>

            <div class="p-4 sm:p-8 bg-white shadow sm:rounded-lg">
                <div class="max-w-xl">
                    @include('profile.partials.update-password-form')
                </div>
            </div>

            <div class="p-4 sm:p-8 bg-white shadow sm:rounded-lg">
                <div class="max-w-xl">
                    @include('profile.partials.delete-user-form')
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu mengimplementasikan autentikasi login dan register

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

Acara 9. Middleware dan Role User

Materi Pembelajaran : Middleware, role-based access
Acara Praktikum/Praktik : Minggu 5/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi (TB430805.3)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.0 20.02	Menggunakan SQL	• Mempersiapkan perangkat lunak aplikasi data deskripsi/SQL • Menggunakan fitur aplikasi SQL • Mengisi table • Melakukan operasi relasional • Membuat stored procedure • Membuat function • Membuat trigger • Melakukan perintah commit dan rollback
2	J.620100.0 21.02	Menerapkan Akses Basis Data	• Membuat berbagai operasi terhadap basis data • Membuat prosedur akses terhadap basis data • Membuat prosedur akses terhadap basis data • Menguji program basis data

c. Indikator Penilaian

Mahasiswa mampu membuat middleware role

d. Dasar Teori

Dalam teori pengembangan aplikasi berbasis Laravel, middleware adalah komponen yang berfungsi sebagai lapisan perantara antara request (permintaan dari client) dan response (jawaban dari server). Middleware bekerja dengan cara memproses setiap request yang masuk sebelum diteruskan ke controller, serta memproses response sebelum dikirim kembali ke client. Dengan demikian,

middleware dapat digunakan untuk melakukan berbagai tugas penting seperti autentikasi, otorisasi, logging, validasi, maupun manipulasi data request/response.

Secara konseptual, middleware mendukung prinsip separation of concerns, karena logika yang bersifat umum dan berulang (misalnya pengecekan login atau proteksi CSRF) tidak perlu ditulis berulang di setiap controller, melainkan cukup diletakkan di middleware. Laravel sendiri menyediakan middleware bawaan seperti auth untuk autentikasi, verified untuk verifikasi email, throttle untuk membatasi jumlah request, serta csrf untuk melindungi aplikasi dari serangan cross-site request forgery.

Middleware dapat didefinisikan secara global (berlaku untuk semua request), atau khusus pada route tertentu. Developer juga bisa membuat middleware kustom sesuai kebutuhan, misalnya middleware untuk memeriksa role pengguna, mencatat aktivitas, atau mengatur format response API. Dengan adanya middleware, aplikasi Laravel menjadi lebih modular, aman, dan mudah dipelihara, karena setiap lapisan tanggung jawab dipisahkan dengan jelas. Middleware memfilter HTTP request. Kita akan membuat CheckRole middleware yang membatasi akses berdasarkan role user. Admin dapat mengakses semua halaman, staff hanya halaman tertentu.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Berikut Adalah prosedur Langkah-langkah praktikum :

a) Membuat Middleware

Buat middleware menggunakan Artisan CLI:

```
php artisan make:middleware Role
```

Kemudian isi file app/Http/Middleware/Role.php dengan kode berikut:

```
class Role
{
    public function handle(Request $request, Closure $next, ...$roles): Response
    {
        if (!auth()->check()) {
```

```
        abort(403);
    }

    foreach ($roles as $role) {
        if (auth()->user()->role === $role) {
            return $next($request);
        }
    }

    abort(403);
}
}
```

Penjelasan method handle():

- Parameter ...\$roles menerima satu atau lebih role yang diizinkan
- Pertama dicek apakah user sudah login, jika tidak → abort 403
- Loop setiap role yang diizinkan, jika cocok → lanjutkan request
- Jika tidak ada yang cocok → abort 403

b) Mendaftarkan Middleware

Di Laravel 11+, middleware didaftarkan di bootstrap/app.php. Tambahkan alias middleware pada method withMiddleware():

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role' => \App\Http\Middleware\Role::class,
    ]);
})
```

c) Menggunakan Middleware di Route

Middleware 'role' dapat digunakan di route dengan menambahkan parameter role yang diizinkan. Contoh pada routes/web.php:

```
// Hanya admin yang bisa mengakses route ini
Route::middleware(['auth', 'role:admin'])->group(function () {
    Route::get('/admin/dashboard', ...);
});

// Admin dan staff bisa mengakses
```

```
Route::middleware(['auth', 'role:admin,staff'])->group(function () {  
    Route::resource('barang', BarangController::class);  
});
```

Pada project Inventaris Universitas ini, semua route menggunakan middleware ['auth', 'verified'] tanpa spesifik role, karena pembatasan akses dilakukan di level controller (role-based scoping).

d) Role di Model & Migration

a) Migration: tambahkan kolom role dengan tipe enum di tabel users:

```
$table->enum('role', ['admin', 'staff'])->default('staff');
```

b) Model User: tambahkan 'role' ke \$fillable dan buat method isAdmin():

```
protected $fillable = ['name', 'email', 'password', 'role'];  
  
public function isAdmin(): bool  
{  
    return $this->role === 'admin';  
}
```

e) Role-Based Scoping di Controller

Selain menggunakan middleware, role-based access control juga diterapkan di controller dengan menyaring data berdasarkan role user. Contoh pada TransaksiController:

```
// Staff hanya bisa melihat transaksi miliknya sendiri  
if (Auth::user()->role === 'staff') {  
    $query->where('user_id', Auth::id());  
}
```

Dengan cara ini, staff hanya melihat data transaksi yang mereka buat sendiri, sedangkan admin dapat melihat semua transaksi.

f) Seeder untuk Role

Buat UserSeeder untuk mengisi data user dengan role berbeda:

```
User::create([  
    'name' => 'Admin Inventaris',  
    'email' => 'admin@inventaris.test',  
    'password' => Hash::make('password'),
```

```
'role' => 'admin',
]);

User::create([
    'name' => 'Staff Inventaris',
    'email' => 'staff@inventaris.test',
    'password' => Hash::make('password'),
    'role' => 'staff',
]);
```

Jalankan seeder:

```
php artisan db:seed --class=UserSeeder
```

User baru yang register melalui form akan otomatis mendapatkan role 'staff' seperti yang didefinisikan di RegisteredUserController.

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Middleware, role-based access". Analisis bagaimana konsep middleware dan role user diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Berikut adalah tampilan file terkait Middleware dan Role dari project Inventaris Universitas:

Middleware: Role.php

```
app/Http/Middleware/Role.php

<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class Role
{
    public function handle(
        Request $request,
        Closure $next,
        ...$roles
    ): Response
    {
        if (!auth()->check()) {
            abort(403);
        }

        foreach ($roles as $role) {
            if (auth()->user()->role === $role) {
                return $next($request);
            }
        }

        abort(403);
    }
}
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Model: User.php (role & isAdmin)

```
app/Models/User.php

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class User extends Authenticatable
{
    use HasFactory, Notifiable;

    protected $fillable = [
        'name', 'email', 'password', 'role',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }

    // Relasi: satu user memiliki banyak transaksi
    public function transaksis(): HasMany
    {
        return $this->hasMany(Transaksi::class);
    }

    // Relasi: user memfavoritkan banyak barang (many-to-many)
    public function favorites(): BelongsToMany
    {
        return $this->belongsToMany(Barang::class, 'barang_user');
    }

    // Cek apakah user adalah admin
    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }
}
```

Config: bootstrap/app.php (register alias)

```
bootstrap/app.php

<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->alias([
            'role' => \App\Http\Middleware\Role::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions) {
        //
    })->create();
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Route: web.php (auth middleware group)

routes/web.php

Route

```
<?php

use App\Http\Controllers\ProfileController;
use App\Http\Controllers\DashboardController;
use App\Http\Controllers\KategoriController;
use App\Http\Controllers\BarangController;
use App\Http\Controllers\TransaksiController;
use App\Http\Controllers\LaporanController;
use Illuminate\Support\Facades\Route;

// Redirect root ke Login
Route::get('/', function () {
    return redirect()->route('login');
});

// Route terproteksi: auth + email verified
Route::middleware(['auth', 'verified'])->group(function () {

    Route::get('/dashboard', [DashboardController::class, 'index'])
        ->name('dashboard');

    // Resource controllers
    Route::resource('kategori', KategoriController::class);
    Route::resource('barang', BarangController::class);
    Route::resource('transaksi', TransaksiController::class)
        ->except(['edit', 'update']);

    // Profile
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
});

require __DIR__.'/auth.php';
```

Controller: TransaksiController (role scoping)

app/Http/Controllers/TransaksiController.php

Controller

```
class TransaksiController extends Controller
{
    public function index(Request $request)
    {
        $query = Transaksi::with(['user', 'barang.kategori']);

        // Filter berdasarkan status
        if ($request->filled('status')) {
            $query->where('status', $request->status);
        }

        // Filter pencarian
        if ($request->filled('search')) {
            $search = $request->search;
            $query->whereHas('barang', function ($q) use ($search) {
                $q->where('nama', 'like', "%{$search}%");
            });
        }

        // Role-based scoping: staff hanya lihat transaksinya sendiri
        if (Auth::user()->role === 'staff') {
            $query->where('user_id', Auth::id());
        }

        $transaksis = $query->latest()->paginate(10);
        return view('pages.transaksi.index', compact('transaksis'));
    }

    // ... method lainnya ...
}
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Migration: users table (role column)

```
database/migrations/..._create_users_table.php

public function up(): void
{
    Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->enum('role', ['admin', 'staff'])->default('staff');
        $table->rememberToken();
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('users');
}
```

Seeder: UserSeeder (admin & staff)

```
database/seeder/UserSeeder.php

<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class UserSeeder extends Seeder
{
    public function run(): void
    {
        // Admin
        User::create([
            'name' => 'Admin Inventaris',
            'email' => 'admin@inventaris.test',
            'password' => Hash::make('password'),
            'role' => 'admin',
        ]);

        // Staff
        User::create([
            'name' => 'Staff Inventaris',
            'email' => 'staff@inventaris.test',
            'password' => Hash::make('password'),
            'role' => 'staff',
        ]);
    }
}
```

Controller: RegisteredUserController (role default)

```
app/Http/Controllers/Auth/RegisteredUserController.php

public function store(Request $request): RedirectResponse
{
    $request->validate([
        'name' => ['required', 'string', 'max:255'],
        'email' => ['required', 'string', 'lowercase',
            'email', 'max:255', 'unique:'.User::class],
        'password' => ['required', 'confirmed', Rules\Password::defaults()],
    ]);

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'role' => 'staff', // Default role untuk registrasi baru
    ]);

    event(new Registered($user));
    Auth::login($user);
    return redirect(route('dashboard', absolute: false));
}
```

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat middleware untuk role-based access

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 10. Dashboard Role-Based Access

Materi Pembelajaran : Dashboard, role-based view
Acara Praktikum/Praktik : Minggu 5/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengembangkan modul aplikasi Web Framework menggunakan fitur routing, controller, model, view (TB430805.2)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.62010 0.020.0 2	Menggunakan SQL	• Mempersiapkan perangkat lunak aplikasi data deskripsi/SQL • Menggunakan fitur aplikasi SQL • Mengisi table • Melakukan operasi relasional • Membuat stored procedure • Membuat function • Membuat trigger • Melakukan perintah commit dan rollback
2	J.62010 0.021.0 2	Menerapkan Akses Basis Data	• Membuat berbagai operasi terhadap basis data • Membuat prosedur akses terhadap basis data • Membuat prosedur akses terhadap basis data • Menguji program basis data

c. Indikator Penilaian

Mahasiswa mampu membuat dashboard role-based

d. Dasar Teori

Dashboard bekerja dengan prinsip data visualization dan information management, yaitu mengubah data mentah menjadi informasi yang relevan dan actionable. Dengan dashboard, pengguna dapat melihat tren, pola, serta anomali secara cepat tanpa harus menelusuri data mentah yang panjang.

Dashboard yang berbeda untuk admin dan staff. Admin melihat seluruh data inventaris (total barang, barang dipinjam, pengguna aktif), staff melihat barang yang tersedia dan riwayat peminjamannya.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Install Dependency Frontend

Jalankan perintah berikut untuk menginstall dependency:

```
npm install
```

```
npm install -D tailwindcss postcss autoprefixer @tailwindcss/forms
```

```
npm run dev
```

Membuat Layout Utama (app.blade.php)

Buat file resources/views/layouts/app.blade.php sebagai layout utama. File ini berisi struktur HTML lengkap, CSS styling inline, sidebar, topbar, dan container konten menggunakan {{ \$slot }}.

```
app/Http/Controllers/DashboardController.php

1  <?php
2  namespace App\Http\Controllers;
3  use App\Models\Barang;
4  use App\Models\Kategori;
5  use App\Models\Transaksi;
6  use App\Models\User;
7
8  class DashboardController extends Controller
9  {
10     public function index()
11     {
12         $totalBarang = Barang::count();
13         $totalKategori = Kategori::count();
14         $barangDipinjam = Transaksi::where('status',
15             'dipinjam')->count();
16         $barangTersedia = Barang::sum('jumlah');
17         $totalUser = User::count();
18
19         $transaksiTerbaru = Transaksi::with(['user',
20             'barang'])->latest()->take(5)->get();
21         $barangStokMenipis = Barang::where('jumlah',
22             '<', 5)->with('kategori')->take(5)->get();
23
24         return view('dashboard', compact(
25             'totalBarang', 'totalKategori', 'barangDipinjam',
26             'barangTersedia', 'totalUser', 'transaksiTerbaru',
27             'barangStokMenipis'));
28     }
29 }
```

Membuat Sidebar Navigasi

Sidebar berisi logo, menu navigasi (Dashboard, Kategori, Barang, Transaksi, Laporan, Profile, Logout), dan footer user info. Menggunakan fixed positioning lebar 250px dengan background gradien gelap.

```
app/Http/Middleware/Role.php

1  <?php
2  namespace App\Http\Middleware;
3  use Closure;
4  use Illuminate\Http\Request;
5
6  class Role
7  {
8      public function handle(Request $request,
9          Closure $next, ...$roles)
10     {
11         if (!auth()->check()) {
12             abort(403);
13         }
14
15         foreach ($roles as $role) {
16             if (auth()->user()->role === $role) {
17                 return $next($request);
18             }
19         }
20
21         abort(403);
22     }
23 }
```

Role Based Dashboard Laravel 11 PHP 8.2

Membuat Topbar / Navbar

Topbar menampilkan tombol toggle menu (mobile), tanggal realtime, avatar user, nama, dan role. Sticky di bagian atas halaman.

```
app/Models/User.php

1  <?php
2  namespace App\Models;
3  use Illuminate\Database\Eloquent\Factories\HasFactory;
4  use Illuminate\Foundation\Auth\User as Authenticatable;
5  use Illuminate\Notifications\Notifiable;
6
7  class User extends Authenticatable
8  {
9      use HasFactory, Notifiable;
10
11     protected $fillable = [
12         'name', 'email', 'password', 'role',
13     ];
14
15     public function isAdmin(): bool
16     {
17         return $this->role === 'admin';
18     }
19 }
```

Role Based Dashboard Laravel 11 PHP 8.2

Membuat Halaman Dashboard

Dashboard menampilkan 4 stat cards gradient, tabel transaksi terbaru, daftar stok menipis, dan aksi cepat.

```
resources/views/dashboard.blade.php

1 <x-app-layout>
2 <div class="stat-grid">
3 <div class="stat-card" style="background:
4 linear-gradient(135deg,#667eea,#764ba2)">
5 <div class="icon"><i class="fas fa-cube"></i></div>
6 <div class="label">Total Barang</div>
7 <div class="value">{{ $totalBarang }}</div>
8 </div>
9 <div class="stat-card" style="background:
10 linear-gradient(135deg,#11998e,#38ef7d)">
11 <div class="icon"><i class="fas fa-tags"></i></div>
12 <div class="label">Total Kategori</div>
13 <div class="value">{{ $totalKategori }}</div>
14 </div>
15 <div class="stat-card" ...>
16 <div class="label">Barang Dipinjam</div>
17 <div class="value">{{ $barangDipinjam }}</div>
18 </div>
19 <div class="stat-card" ...>
20 <div class="label">Total Stok</div>
21 <div class="value">{{ $barangTersedia }}</div>
22 </div>
23 </div>

Role Based Dashboard Laravel 11 PHP 8.2
```

Membuat Halaman CRUD

Halaman CRUD untuk Kategori, Barang, dan Transaksi menggunakan layout yang sama dengan `<x-app-layout>`. Masing-masing memiliki halaman Index, Create, Edit, dan Show (khusus Barang).

Kategori Index: Tabel dengan kolom No, Nama, Deskripsi, Jumlah Barang, Aksi (Edit/Hapus)

```
routes/web.php

1 use App\Http\Controllers\DashboardController;
2
3 Route::middleware(['auth', 'verified'])->group(function () {
4
5     Route::get('/dashboard', [
6         DashboardController::class, 'index'
7     ]->name('dashboard'));
8
9     Route::resource('kategori', KategoriController::class);
10    Route::resource('barang', BarangController::class);
11    Route::resource('transaksi', TransaksiController::class)
12        ->except(['edit', 'update']);
13
14    Route::middleware(['role:admin'])->group(function () {
15        Route::get('/laporan/barang/pdf', ...);
16        Route::get('/laporan/transaksi/excel', ...);
17    });
18 });

Role Based Dashboard Laravel 11 PHP 8.2
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Barang Index: Filter card (Kategori, Kondisi, Search), tabel dengan pagination

```
# 1. Buat DashboardController
$ php artisan make:controller DashboardController
Created: app/Http/Controllers/DashboardController.php

# 2. Buat Middleware Role
$ php artisan make:middleware Role
Created: app/Http/Middleware/Role.php

# 3. Daftarkan middleware di bootstrap/app.php
$ php artisan make:seeder UserSeeder
Created: database/seeds/UserSeeder.php
```

Transaksi Index: Tabel dengan kolom Peminjam, Barang, Jumlah, Tanggal, Status, Aksi

```
# Uji Coba Dashboard
$ php artisan serve
Server running on http://127.0.0.1:8000

# Login sebagai Admin
Email: admin@inventaris.test
Password: password
-> Redirect ke /dashboard
-> Lihat 4 stat cards (Total Barang, Kategori, Barang Dipinjam, Stok)
-> Lihat tabel "Transaksi Terbaru" (5 data terakhir)
-> Lihat tabel "Stok Menipis" (barang dengan jumlah < 5)

# Login sebagai Staff
Email: staff@inventaris.test
Password: password
-> Redirect ke /dashboard
-> Lihat hanya data peminjaman milik sendiri
-> Menu tertentu tidak tampil jika bukan admin
```

Konfigurasi Routes

Semua route admin dilindungi middleware auth dan verified. Menggunakan resource controller dan custom route untuk pengembalian serta laporan.

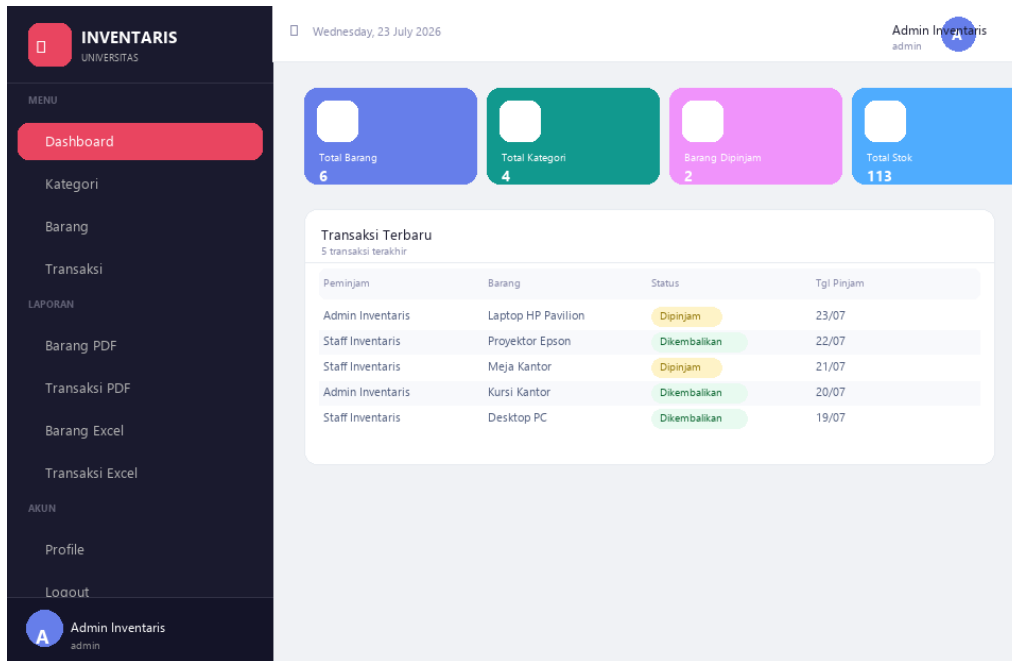
BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Study Kasus

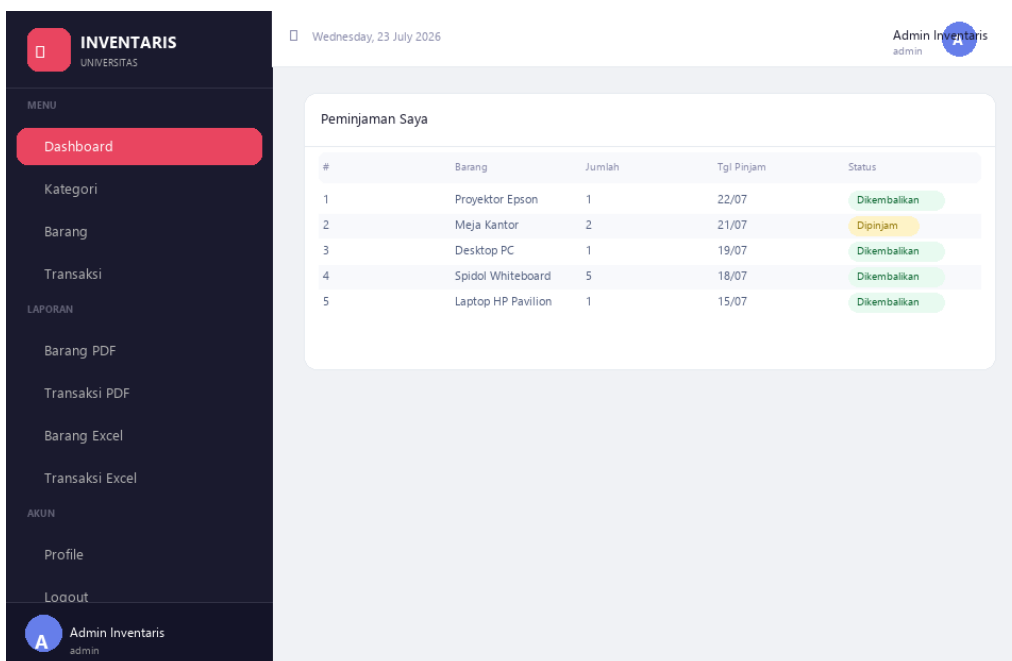
Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Dashboard, role-based view". Analisis bagaimana konsep dashboard role-based access diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Dashboard admin dengan statistik global



Dashboard staff dengan data peminjaman sendiri



BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Dashboard admin

INVENTARIS
UNIVERSITAS

MENU

Dashboard

Kategori

Barang

Transaksi

LAPORAN

Barang PDF

Transaksi PDF

Barang Excel

Transaksi Excel

AKUN

Profile

Logout

A

Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Total Barang
6

Total Kategori
4

Barang Dipinjam
2

Total Stok
113

Transaksi Terbaru

5 transaksi terakhir

Peminjam	Barang	Status	Tgl Pinjam
Admin Inventaris	Laptop HP Pavilion	Dipinjam	23/07
Staff Inventaris	Proyektor Epson	Dikembalikan	22/07
Staff Inventaris	Meja Kantor	Dipinjam	21/07
Admin Inventaris	Kursi Kantor	Dikembalikan	20/07
Staff Inventaris	Desktop PC	Dikembalikan	19/07

Dashboard staff

INVENTARIS
UNIVERSITAS

MENU

Dashboard

Kategori

Barang

Transaksi

LAPORAN

Barang PDF

Transaksi PDF

Barang Excel

Transaksi Excel

AKUN

Profile

Logout

A

Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

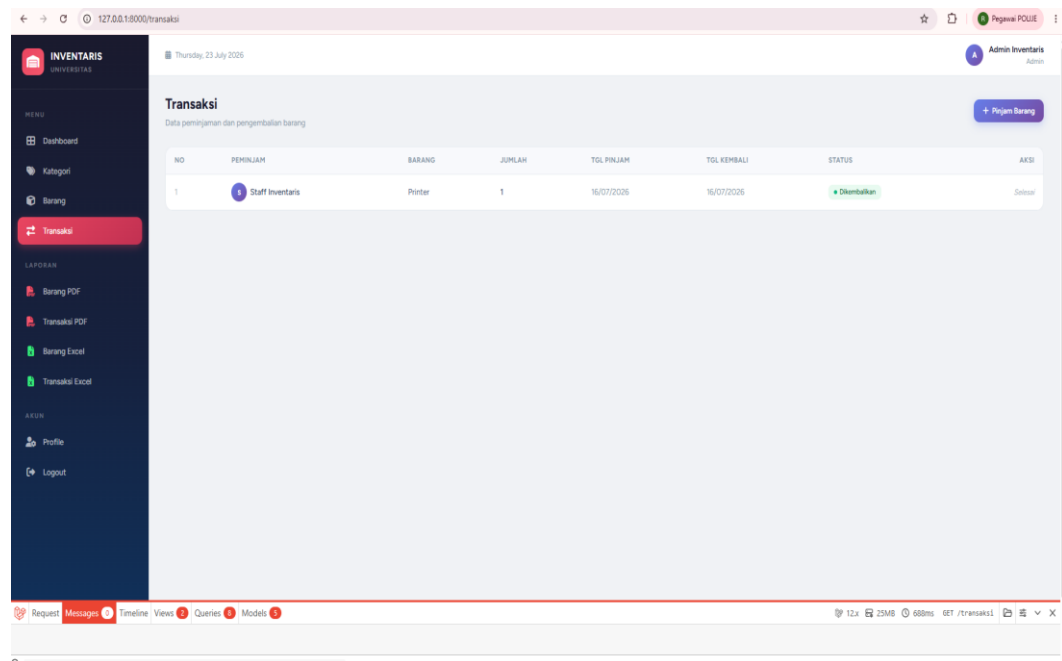
Peminjaman Saya

#	Barang	Jumlah	Tgl Pinjam	Status
1	Proyektor Epson	1	22/07	Dikembalikan
2	Meja Kantor	2	21/07	Dipinjam
3	Desktop PC	1	19/07	Dikembalikan
4	Spidol Whiteboard	5	18/07	Dikembalikan
5	Laptop HP Pavilion	1	15/07	Dikembalikan

92

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman Daftar Transaksi



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat dashboard dinamis berdasarkan role pengguna

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

Acara 11. Template Admin dengan Blade

Materi Pembelajaran : Blade template, layout, @yield
Acara Praktikum/Praktik : Minggu 6/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengembangkan modul aplikasi Web Framework menggunakan fitur routing, controller, model, view (TB430805.2)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.0 17.02	Mengimplementasikan Pemrograman Terstruktur	• Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Membuat program untuk akses file • Membuat program untuk akses file
2	J.620100.0 18.02	Mengimplementasikan Pemrograman Berorientasi Objek	• Membuat program berorientasi objek dengan memanfaatkan class • Menggunakan tipe data dan control program pada metode atau operasi dari suatu kelas • Membuat program dengan konsep berbasis objek • Membuat program object oriented dengan interface dan paket • Mengkompilasi Program

c. Indikator Penilaian

Mahasiswa mampu membuat template admin dengan Blade

d. Dasar Teori

Blade adalah template engine Laravel. Layout utama menggunakan @yield untuk konten dinamis. Kita akan menggunakan template AdminLTE atau membuat sendiri dengan Bootstrap.

Blade bekerja dengan cara mengompilasi file template berakhiran .blade.php menjadi kode PHP murni, kemudian menyajikannya sebagai HTML ke browser. Blade mendukung penggunaan inheritance (pewarisan layout), section, dan component, sehingga developer dapat membuat tampilan yang konsisten di seluruh aplikasi.

- **Fitur Utama Blade**
 - **Template Inheritance** : Memungkinkan penggunaan layout utama (layouts.app) yang dapat diwarisi oleh halaman lain.
 - **Control Structures** : Mendukung sintaks seperti @if, @foreach, @switch, yang lebih ringkas dibandingkan PHP murni.
 - **Echoing Data** : Menampilkan data dengan {{ }} yang otomatis melakukan escaping untuk keamanan (mencegah XSS).
 - **Components & Slots** : Membuat bagian tampilan yang reusable, misalnya tombol atau card.
 - **Directives** : Perintah khusus seperti @csrf, @auth, @guest, yang mempermudah integrasi dengan fitur Laravel.
- **Keunggulan Blade**
 - Ringkas dan mudah dipahami dibandingkan PHP murni.
 - Mendukung modularisasi tampilan dengan layout dan komponen.
 - Aman karena data otomatis di-escape.
 - Terintegrasi penuh dengan fitur Laravel seperti routing, autentikasi, dan middleware.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Membuat DashboardController

Buat controller untuk menangani data dashboard:

```
php artisan make:controller DashboardController
```

- `Barang::count()` — total barang

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

- Transaksi::where('status','dipinjam')->count() — barang dipinjam
- Barang::sum('jumlah') — total stok
- Transaksi::with(['user','barang'])->latest()->take(5) — 5 transaksi terbaru
- Barang::where('jumlah','<','5') — barang dengan stok menipis

DashboardController dengan data statistik

```
app/Http/Controllers/DashboardController.php

1  <?php
2  namespace App\Http\Controllers;
3  use App\Models\Barang;
4  use App\Models\Kategori;
5  use App\Models\Transaksi;
6  use App\Models\User;
7
8  class DashboardController extends Controller
9  {
10     public function index()
11     {
12         $totalBarang = Barang::count();
13         $totalKategori = Kategori::count();
14         $barangDipinjam = Transaksi::where('status',
15             'dipinjam')->count();
16         $barangTersedia = Barang::sum('jumlah');
17         $totalUser = User::count();
18
19         $transaksiTerbaru = Transaksi::with(['user',
20             'barang'])->latest()->take(5)->get();
21         $barangStokMenipis = Barang::where('jumlah',
22             '<', 5)->with('kategori')->take(5)->get();
23
24         return view('dashboard', compact(
25             'totalBarang', 'totalKategori', 'barangDipinjam',
26             'barangTersedia', 'totalUser', 'transaksiTerbaru',
27             'barangStokMenipis'));
28     }
29 }
```

Membuat Middleware Role

Buat middleware untuk memeriksa role user:

php artisan make:middleware Role

- auth()->check() — pastikan user sudah login
- Loop \$roles — periksa apakah role user cocok dengan salah satu role yang diizinkan
- abort(403) — jika tidak cocok, tampilkan Forbidden

Daftarkan middleware di bootstrap/app.php dengan alias:

->withMiddleware(fn(\$m) => \$m->alias(['role' => Role::class]))

Middleware Role untuk otorisasi

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

```
app/Http/Middleware/Role.php

1 <?php
2 namespace App\Http\Middleware;
3 use Closure;
4 use Illuminate\Http\Request;
5
6 class Role
7 {
8     public function handle(Request $request,
9         Closure $next, ...$roles)
10    {
11        if (!auth()->check()) {
12            abort(403);
13        }
14
15        foreach ($roles as $role) {
16            if (auth()->user()->role === $role) {
17                return $next($request);
18            }
19        }
20
21        abort(403);
22    }
23 }
```

Role Based Dashboard Laravel 11 PHP 8.2

Menambahkan Role di User Model

Tambahkan kolom role dan method isAdmin() di User model:

- fillable — tambahkan 'role' ke daftar mass assignment
- isAdmin() — return \$this->role === 'admin'
- Role values: admin, staff (disimpan di kolom role)

User model dengan role & method isAdmin()

```
app/Models/User.php

1 <?php
2 namespace App\Models;
3 use Illuminate\Database\Eloquent\Factories\HasFactory;
4 use Illuminate\Foundation\Auth\User as Authenticatable;
5 use Illuminate\Notifications\Notifiable;
6
7 class User extends Authenticatable
8 {
9     use HasFactory, Notifiable;
10
11     protected $fillable = [
12         'name', 'email', 'password', 'role',
13     ];
14
15     public function isAdmin(): bool
16     {
17         return $this->role === 'admin';
18     }
19 }
```

Role Based Dashboard Laravel 11 PHP 8.2

Membuat View Dashboard

Buat view dashboard dengan Blade dan Tailwind:

- stat-grid — 4 stat cards dengan gradient berbeda

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

- Total Barang (#667eea), Total Kategori (#11998e)
- Barang Dipinjam (#f093fb), Total Stok (#4facfe)
- Tabel "Transaksi Terbaru" — 5 data terakhir dengan status badge
- Tabel "Stok Menipis" — barang dengan jumlah < 5
- Aksi Cepat — shortcut ke create barang, kategori, transaksi

View dashboard dengan statistik cards

```
resources/views/dashboard.blade.php

1 <x-app-layout>
2 <div class="stat-grid">
3 <div class="stat-card" style="background:
4 linear-gradient(135deg,#667eea,#764ba2)">
5 <div class="icon"><i class="fas fa-cube"></i></div>
6 <div class="label">Total Barang</div>
7 <div class="value">{{ $totalBarang }}</div>
8 </div>
9 <div class="stat-card" style="background:
10 linear-gradient(135deg,#11998e,#38ef7d)">
11 <div class="icon"><i class="fas fa-tags"></i></div>
12 <div class="label">Total Kategori</div>
13 <div class="value">{{ $totalKategori }}</div>
14 </div>
15 <div class="stat-card" ...>
16 <div class="label">Barang Dipinjam</div>
17 <div class="value">{{ $barangDipinjam }}</div>
18 </div>
19 <div class="stat-card" ...>
20 <div class="label">Total Stok</div>
21 <div class="value">{{ $barangTersedia }}</div>
22 </div>
23 </div>

Role Based Dashboard Laravel 11 PHP 8.2
```

Mendaftarkan Route

Daftarkan route dashboard dan middleware role:

- Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard')
- Route group middleware ['auth', 'verified'] — harus login & verifikasi email
- Route group middleware ['role:admin'] — hanya admin bisa akses laporan

Route dashboard & middleware role

```
routes/web.php

1 use App\Http\Controllers\DashboardController;
2
3 Route::middleware(['auth', 'verified'])->group(function () {
4
5     Route::get('/dashboard', [
6         DashboardController::class, 'index'
7     ])->name('dashboard');
8
9     Route::resource('kategori', KategoriController::class);
10    Route::resource('barang', BarangController::class);
11    Route::resource('transaksi', TransaksiController::class)
12        ->except(['edit', 'update']);
13
14    Route::middleware(['role:admin'])->group(function () {
15        Route::get('/laporan/barang/pdf', ...);
16        Route::get('/laporan/transaksi/excel', ...);
17    });
18 });

Role Based Dashboard Laravel 11 PHP 8.2
```

Uji Coba Login Admin & Staff

- Login sebagai admin@inventaris.test / password
- Dashboard menampilkan 4 stat cards, transaksi terbaru, stok menipis, aksi cepat
- Login sebagai staff@inventaris.test / password
- Dashboard staff hanya menampilkan peminjaman milik sendiri
- Menu laporan PDF/Excel mungkin tidak tampil untuk staff

Membuat controller & middleware via artisan

```
# 1. Buat DashboardController
$ php artisan make:controller DashboardController
Created: app/Http/Controllers/DashboardController.php

# 2. Buat Middleware Role
$ php artisan make:middleware Role
Created: app/Http/Middleware/Role.php

# 3. Daftarkan middleware di bootstrap/app.php
$ php artisan make:seeder UserSeeder
Created: database/seeds/UserSeeder.php
```

Uji coba dashboard admin & staff

```
# Uji Coba Dashboard
$ php artisan serve
Server running on http://127.0.0.1:8000

# Login sebagai Admin
Email: admin@inventaris.test
Password: password
-> Redirect ke /dashboard
-> Lihat 4 stat cards (Total Barang, Kategori, Barang Dipinjam, Stok)
-> Lihat tabel "Transaksi Terbaru" (5 data terakhir)
-> Lihat tabel "Stok Menipis" (barang dengan jumlah < 5)

# Login sebagai Staff
Email: staff@inventaris.test
Password: password
-> Redirect ke /dashboard
-> Lihat hanya data peminjaman milik sendiri
-> Menu tertentu tidak tampil jika bukan admin
```

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Blade template, layout, @yield". Analisis bagaimana konsep template admin dengan blade diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Layout Utama - app.blade.php

```
app/Http/Controllers/DashboardController.php

1  <?php
2  namespace App\Http\Controllers;
3  use App\Models\Barang;
4  use App\Models\Kategori;
5  use App\Models\Transaksi;
6  use App\Models\User;
7
8  class DashboardController extends Controller
9  {
10     public function index()
11     {
12         $totalBarang = Barang::count();
13         $totalKategori = Kategori::count();
14         $barangDipinjam = Transaksi::where('status',
15             'dipinjam')->count();
16         $barangTersedia = Barang::sum('jumlah');
17         $totalUser = User::count();
18
19         $transaksiTerbaru = Transaksi::with(['user',
20             'barang'])->latest()->take(5)->get();
21         $barangStokMenipis = Barang::where('jumlah',
22             '<', 5)->with('kategori')->take(5)->get();
23
24         return view('dashboard', compact(
25             'totalBarang', 'totalKategori', 'barangDipinjam',
26             'barangTersedia', 'totalUser', 'transaksiTerbaru',
27             'barangStokMenipis'));
28     }
29 }
```

Sidebar Navigasi

```
app/Http/Middleware/Role.php

1  <?php
2  namespace App\Http\Middleware;
3  use Closure;
4  use Illuminate\Http\Request;
5
6  class Role
7  {
8     public function handle(Request $request,
9         Closure $next, ...$roles)
10     {
11         if (!auth()->check()) {
12             abort(403);
13         }
14
15         foreach ($roles as $role) {
16             if (auth()->user()->role === $role) {
17                 return $next($request);
18             }
19         }
20
21         abort(403);
22     }
23 }
```

Topbar / Navbar

```
app/Models/User.php

1  <?php
2  namespace App\Models;
3  use Illuminate\Database\Eloquent\Factories\HasFactory;
4  use Illuminate\Foundation\Auth\User as Authenticatable;
5  use Illuminate\Notifications\Notifiable;
6
7  class User extends Authenticatable
8  {
9      use HasFactory, Notifiable;
10
11      protected $fillable = [
12          'name', 'email', 'password', 'role',
13      ];
14
15      public function isAdmin(): bool
16      {
17          return $this->role === 'admin';
18      }
19  }
```

Role Based Dashboard Laravel 11 PHP 8.2

Halaman Dashboard

```
resources/views/dashboard.blade.php

1  <x-app-layout>
2      <div class="stat-grid">
3          <div class="stat-card" style="background:
4              linear-gradient(135deg,#667eea,#764ba2)">
5              <div class="icon"><i class="fas fa-cube"></i></div>
6              <div class="label">Total Barang</div>
7              <div class="value">{{ $totalBarang }}</div>
8          </div>
9          <div class="stat-card" style="background:
10              linear-gradient(135deg,#11998e,#38ef7d)">
11              <div class="icon"><i class="fas fa-tags"></i></div>
12              <div class="label">Total Kategori</div>
13              <div class="value">{{ $totalKategori }}</div>
14          </div>
15          <div class="stat-card" ...>
16              <div class="label">Barang Dipinjam</div>
17              <div class="value">{{ $barangDipinjam }}</div>
18          </div>
19          <div class="stat-card" ...>
20              <div class="label">Total Stok</div>
21              <div class="value">{{ $barangTersedia }}</div>
22          </div>
23      </div>
```

Role Based Dashboard Laravel 11 PHP 8.2

Halaman Kategori Index

```
routes/web.php

1 use App\Http\Controllers\DashboardController;
2
3 Route::middleware(['auth', 'verified'])->group(function () {
4
5     Route::get('/dashboard', [
6         DashboardController::class, 'index'
7     ])->name('dashboard');
8
9     Route::resource('kategori', KategoriController::class);
10    Route::resource('barang', BarangController::class);
11    Route::resource('transaksi', TransaksiController::class)
12        ->except(['edit', 'update']);
13
14    Route::middleware(['role:admin'])->group(function () {
15        Route::get('/laporan/barang/pdf', ...);
16        Route::get('/laporan/transaksi/excel', ...);
17    });
18 });
```

Role Based Dashboard Laravel 11 PHP 8.2

Halaman Barang Index

```
# 1. Buat DashboardController
$ php artisan make:controller DashboardController
Created: app/Http/Controllers/DashboardController.php

# 2. Buat Middleware Role
$ php artisan make:middleware Role
Created: app/Http/Middleware/Role.php

# 3. Daftarkan middleware di bootstrap/app.php
$ php artisan make:seeder UserSeeder
Created: database/seeder/UserSeeder.php
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman Transaksi Index

```

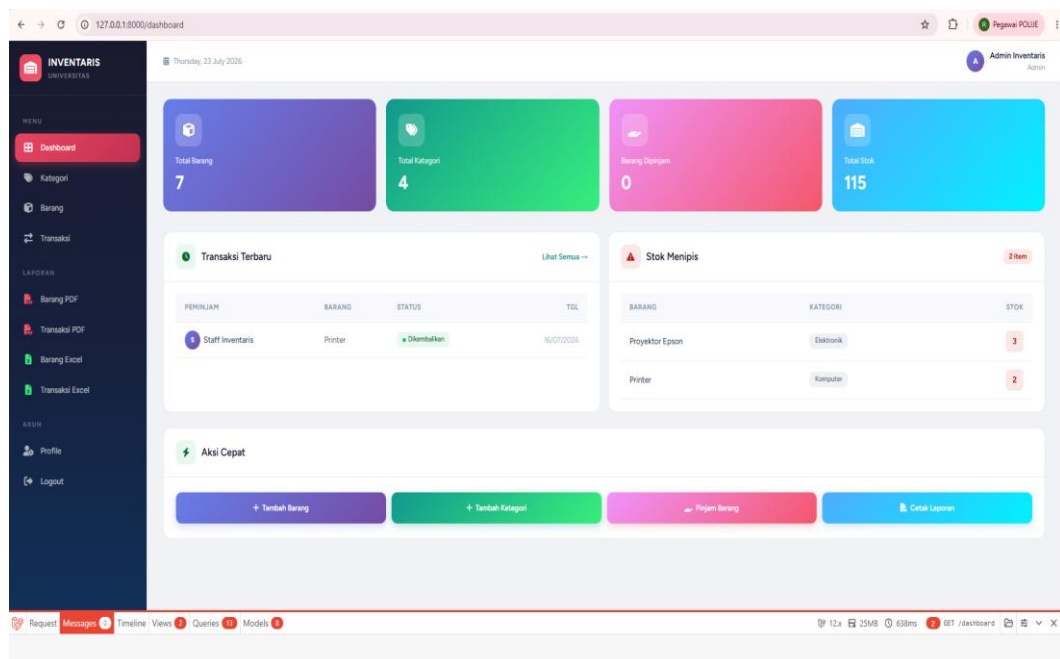
# Uji Coba Dashboard
$ php artisan serve
Server running on http://127.0.0.1:8000

# Login sebagai Admin
Email: admin@inventaris.test
Password: password
-> Redirect ke /dashboard
-> Lihat 4 stat cards (Total Barang, Kategori, Barang Dipinjam, Stok)
-> Lihat tabel "Transaksi Terbaru" (5 data terakhir)
-> Lihat tabel "Stok Menipis" (barang dengan jumlah < 5)

# Login sebagai Staff
Email: staff@inventaris.test
Password: password
-> Redirect ke /dashboard
-> Lihat hanya data peminjaman milik sendiri
-> Menu tertentu tidak tampil jika bukan admin

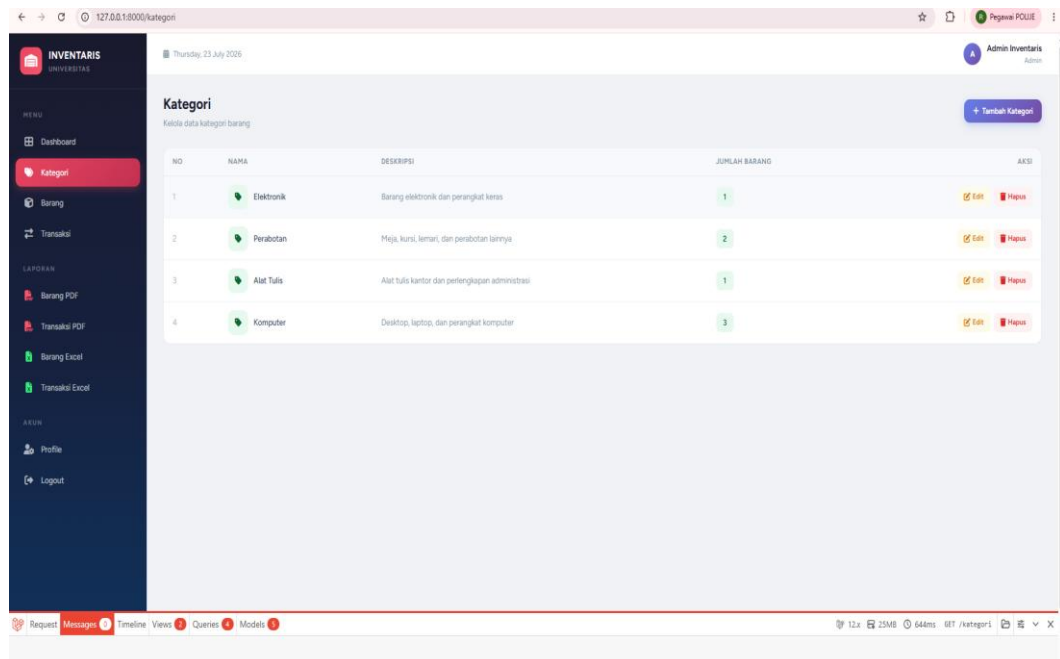
```

Halaman Dashboard dengan Stat Cards



BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman Daftar Kategori



INVENTARIS UNIVERSITAS

Thursday, 23 July 2026

Admin Inventaris Admin

Kategori

Kelola data kategori barang

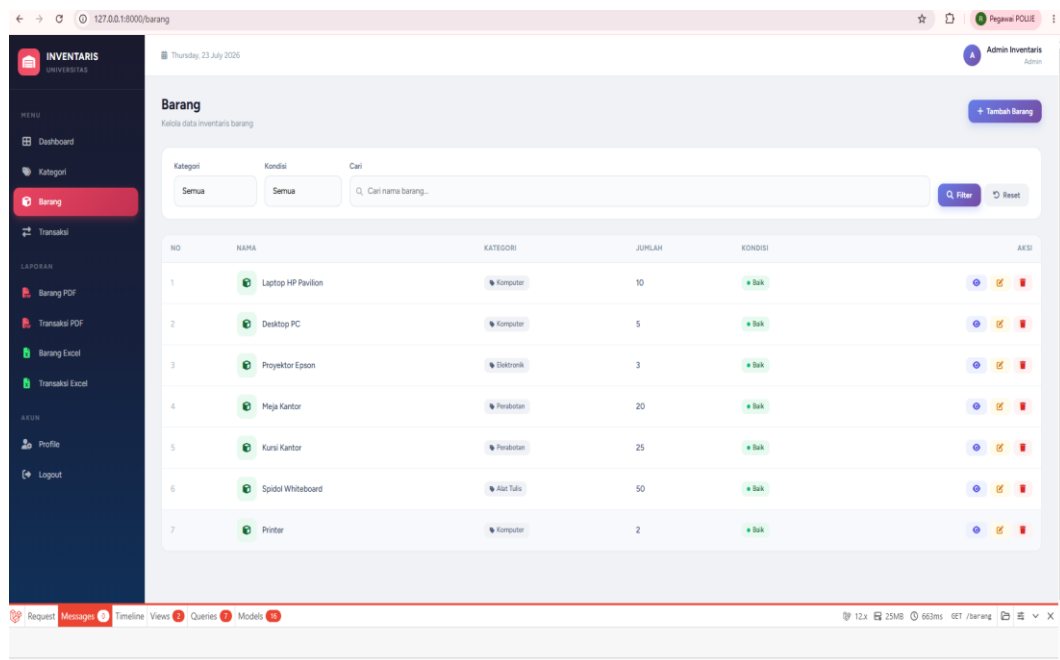
+ Tambah Kategori

NO	NAMA	DESKRIPSI	JUMLAH BARANG	Aksi
1	Elektronik	Barang elektronik dan perangkat keras	1	Edit Hapus
2	Perabotan	Meja, kursi, lemari, dan perabotan lainnya	2	Edit Hapus
3	Alat Tulis	Alat tulis kantor dan perlengkapan administrasi	1	Edit Hapus
4	Komputer	Desktop, laptop, dan perangkat komputer	3	Edit Hapus

Request Messages Timeline Views Queries Models

12x 25MB 64ms GET /kategori

Halaman Daftar Barang dengan Filter



INVENTARIS UNIVERSITAS

Thursday, 23 July 2026

Admin Inventaris Admin

Barang

Kelola data inventaris barang

+ Tambah Barang

Kategori: Semua Kondisi: Semua Cari: Cari nama barang...

FILTER RESET

NO	NAMA	KATEGORI	JUMLAH	KONDISI	Aksi
1	Laptop HP Pavilion	Komputer	10	Stok	Edit Hapus
2	Desktop PC	Komputer	5	Stok	Edit Hapus
3	Proyektor Epson	Elektronik	3	Stok	Edit Hapus
4	Meja Kantor	Perabotan	20	Stok	Edit Hapus
5	Kursi Kantor	Perabotan	25	Stok	Edit Hapus
6	Spidol Whiteboard	Alat Tulis	50	Stok	Edit Hapus
7	Printer	Komputer	2	Stok	Edit Hapus

Request Messages Timeline Views Queries Models

12x 25MB 663ms GET /barang

Form Tambah Barang

The screenshot shows a web browser window with the URL 127.0.0.1:8000/profile. The page title is 'INVENTARIS UNIVERSITAS'. The left sidebar contains a menu with items: Dashboard, Kategori, Barang, Transaksi, Laporan (Barang PDF, Transaksi PDF, Barang Excel, Transaksi Excel), and Akun (Profile, Logout). The main content area has two sections: 'Profile Information' and 'Update Password'. The 'Profile Information' section has fields for Name (Admin Inventaris) and Email (admin@inventaris.test), with a 'SAVE' button. The 'Update Password' section has fields for Current Password, New Password, and Confirm Password.

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat template admin menggunakan Blade Laravel

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

Acara 12. Layout dan Component Blade

Materi Pembelajaran : Blade component, @include, @component
 Acara Praktikum/Praktik : Minggu 6/2
 Tempat : Daring/Luring Politeknik Negeri Jember
 Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengembangkan modul aplikasi Web Framework menggunakan fitur routing, controller, model, view (TB430805.2)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.0 17.02	Mengimplementasikan Pemrograman Terstruktur	• Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Menggunakan tipe data dan control program • Membuat program untuk akses file • Membuat program untuk akses file
2	J.620100.0 18.02	Mengimplementasikan Pemrograman Berorientasi Objek	• Membuat program berorientasi objek dengan memanfaatkan class • Menggunakan tipe data dan control program pada metode atau operasi dari suatu kelas • Membuat program dengan konsep berbasis objek • Membuat program object oriented dengan interface dan paket • Mengkompilasi Program

c. Indikator Penilaian

Mahasiswa mampu membuat komponen Blade reusable

d. Dasar Teori

Blade Component adalah fitur untuk membuat komponen UI yang dapat digunakan ulang. Contoh: card, modal, form input, tabel. Komponen disimpan di resources/views/components/ atau menggunakan php artisan make:component.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Membuat Komponen Card

Jalankan perintah berikut untuk membuat komponen Card:

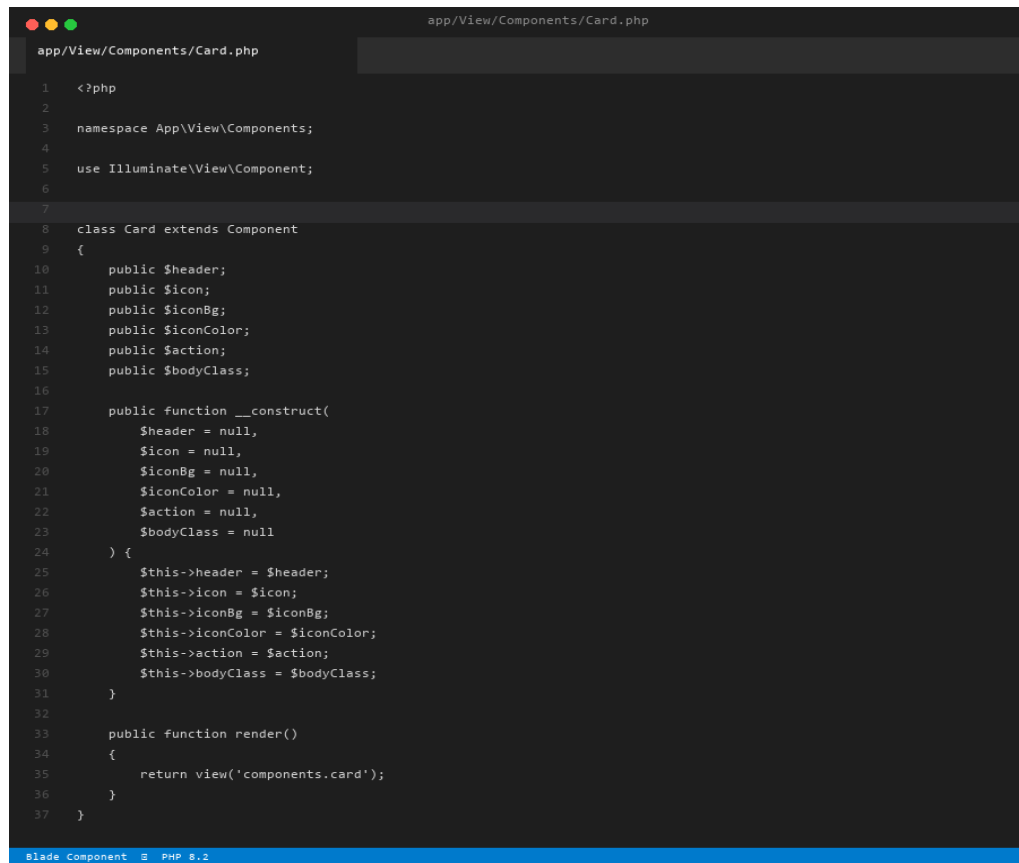
`php artisan make:component Card`

Perintah ini menghasilkan 2 file:

- `app/View/Components/Card.php` — class komponen (dengan properti `header`, `icon`, `iconBg`, `iconColor`, `action`, `bodyClass`)
- `resources/views/components/card.blade.php` — template blade

File `app/View/Components/Card.php`:

Class Card dengan properti dan constructor



```
app/View/Components/Card.php
1  <?php
2
3  namespace App\View\Components;
4
5  use Illuminate\View\Component;
6
7
8  class Card extends Component
9  {
10     public $header;
11     public $icon;
12     public $iconBg;
13     public $iconColor;
14     public $action;
15     public $bodyClass;
16
17     public function __construct(
18         $header = null,
19         $icon = null,
20         $iconBg = null,
21         $iconColor = null,
22         $action = null,
23         $bodyClass = null
24     ) {
25         $this->header = $header;
26         $this->icon = $icon;
27         $this->iconBg = $iconBg;
28         $this->iconColor = $iconColor;
29         $this->action = $action;
30         $this->bodyClass = $bodyClass;
31     }
32
33     public function render()
34     {
35         return view('components.card');
36     }
37 }
```

File `resources/views/components/card.blade.php`:

Template komponen Card dengan header, icon slot, dan body

```
resources/views/components/card.blade.php

1 <div class="card {{ $attributes->get('class') }}">
2     @if(isset($header))
3         <div class="card-header">
4             <div class="flex items-center gap-3">
5                 @if(isset($icon))
6                     <div class="icon-box" style="background:{{ $iconBg ?? '#eef2ff' }};
7                       color:{{ $iconColor ?? '#4f46e5' }}">
8                         <i class="fas {{ $icon }}"></i>
9                     </div>
10                  @endif
11                  <span style="font-weight:600;color:#1a1a2e">{{ $header }}</span>
12              </div>
13              @if(isset($action))
14                  <div>{{ $action }}</div>
15              @endif
16          </div>
17          @endif
18
19      <div class="card-body {{ $bodyClass ?? '' }}">
20          {{ $slot }}
21      </div>
22 </div>
```

Membuat Komponen FormInput

Jalankan perintah:

php artisan make:component FormInput

File resources/views/components/form-input.blade.php:

Komponen FormInput dengan berbagai tipe input (text, select, textarea)

```
resources/views/components/form-input.blade.php

1 <div class="form-group {{ $attributes->get('class') }}">
2     @if(isset($label))
3         <label for="{{ $name }}" class="form-label">
4             {{ $label }}
5             @if($required ?? false)
6                 <span style="color:#dc2626">*</span>
7             @endif
8         </label>
9     @endif
10
11     <div class="input-group">
12         @if(isset($icon))
13             <span class="icon"><i class="fas {{ $icon }}"></i></span>
14         @endif
15
16         @if($type === 'textarea')
17             <textarea name="{{ $name }}" id="{{ $name }}"
18                 {{ $attributes->wire('model') }}
19                 placeholder="{{ $placeholder ?? '' }}"
20                 class="form-control"
21                 rows="{{ $rows ?? 4 }}">{{ $value ?? '' }}</textarea>
22         @elseif($type === 'select')
23             <select name="{{ $name }}" id="{{ $name }}" class="form-control">
24                 {{ $slot }}
25             </select>
26         @else
27             <input type="{{ $type ?? 'text' }}" name="{{ $name }}" id="{{ $name }}"
28                 value="{{ $value ?? old($name) }}"
29                 placeholder="{{ $placeholder ?? '' }}"
30                 class="form-control"
31                 {{ $attributes }}
32         @endif
33     </div>
34
35     @error($name)
36         <p style="font-size:12px;color:#dc2626;margin-top:4px">
37             <i class="fas fa-exclamation-circle"></i> {{ $message }}
38         </p>
39     @enderror
40 </div>
```

Komponen FormInput mendukung:

- label dengan required indicator
- icon di dalam input group
- tipe input: text, number, select, textarea, email, password
- error message otomatis dengan @error
- old() value untuk form repopulation

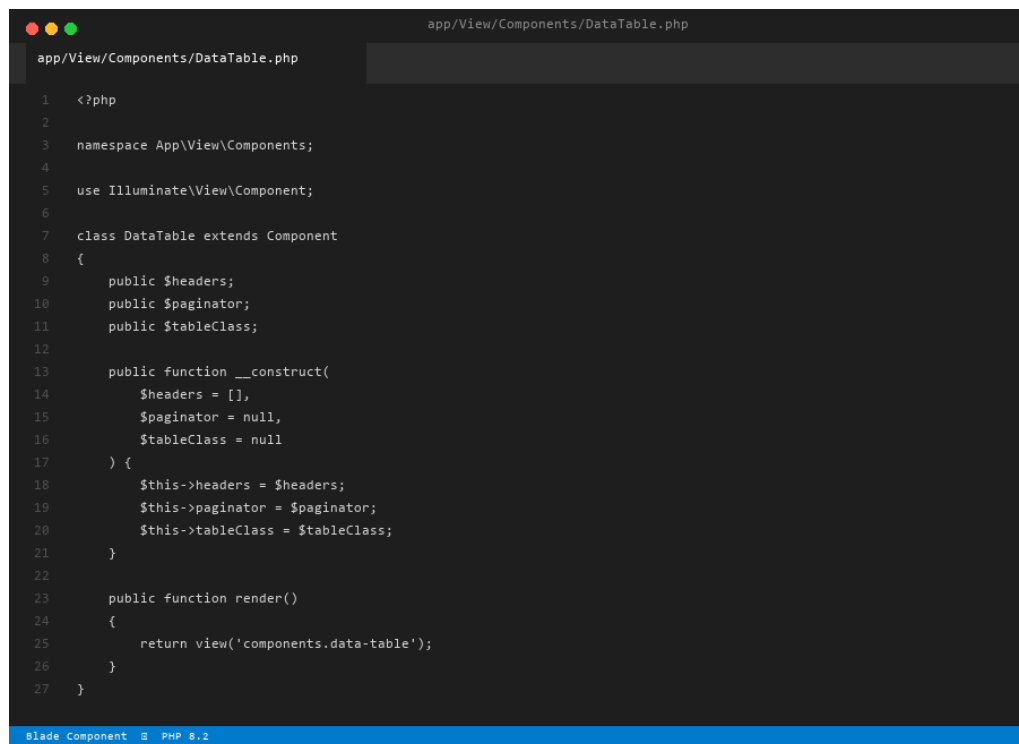
Membuat Komponen DataTable

Jalankan perintah:

php artisan make:component DataTable

File app/View/Components/DataTable.php:

Class DataTable dengan headers array dan paginator



```
app/View/Components/DataTable.php
1  <?php
2
3  namespace App\View\Components;
4
5  use Illuminate\View\Component;
6
7  class DataTable extends Component
8  {
9      public $headers;
10     public $paginator;
11     public $tableClass;
12
13     public function __construct(
14         $headers = [],
15         $paginator = null,
16         $tableClass = null
17     ) {
18         $this->headers = $headers;
19         $this->paginator = $paginator;
20         $this->tableClass = $tableClass;
21     }
22
23     public function render()
24     {
25         return view('components.data-table');
26     }
27 }
```

Blade Component PHP 8.2

File resources/views/components/data-table.blade.php:

Template DataTable dengan headers, slot untuk row, dan pagination

```
resources/views/components/data-table.blade.php

1 <div class="card">
2   <div class="overflow-x-auto">
3     <table class="{{ $tableClass ?? '' }}">
4       @if(isset($headers))
5         <thead>
6           <tr>
7             @foreach($headers as $header)
8               <th class="{{ $header['class'] ?? '' }}"
9                 @if(isset($header['width'])) style="width:{{ $header['width'] }}" @endif
10                {{ $header['label'] ?? $header }}>
11             </th>
12           @endforeach
13         </tr>
14       </thead>
15     @endif
16
17     <tbody class="divide-y">
18       {{ $slot }}
19     </tbody>
20   </table>
21 </div>
22
23 @if(isset($paginator) && $paginator->hasPages())
24 <div class="pagination-wrapper">
25   {{ $paginator->links() }}
26 </div>
27 @endif
28 </div>
```

Menggunakan Komponen di Dashboard

Dashboard menggunakan komponen Card untuk stat cards dan DataTable untuk tabel transaksi:

Dashboard dengan komponen Card dan DataTable

```
resources/views/dashboard.blade.php

1 <x-app-layout>
2   <!-- Stat Cards menggunakan Card component -->
3   @php
4     $stats = [
5       ['label'=>'Total Barang', 'value'=>$totalBarang, 'icon'=>'fa-cube', 'color'=>'#667eea', 'bg'=>'#eef2ff'],
6       ['label'=>'Total Kategori', 'value'=>$totalKategori, 'icon'=>'fa-tags', 'color'=>'#11998e', 'bg'=>'#e8faf0'],
7       ['label'=>'Barang Dipinjam', 'value'=>$barangDipinjam, 'icon'=>'fa-hand-holding', 'color'=>'#f5576c', 'bg'=>'#fd
8       ['label'=>'Total Stok', 'value'=>$barangTersedia, 'icon'=>'fa-warehouse', 'color'=>'#00f2fe', 'bg'=>'#e8faf0'],
9     ];
10   @endphp
11
12   <div class="stat-grid">
13     @foreach($stats as $s)
14       <x-card icon="{{ $s['icon'] }}" icon-bg="{{ $s['bg'] }}"
15         icon-color="{{ $s['color'] }}">
16         <div class="label">{{ $s['label'] }}</div>
17         <div class="value" style="color:{{ $s['color'] }}">{{ $s['value'] }}</div>
18       </x-card>
19     @endforeach
20   </div>
21
22   <!-- Transaksi Terbaru -->
23   <x-card header="Transaksi Terbaru"
24     icon="fa-clock" icon-bg="#e8faf0" icon-color="#0d6832"
25     action="a href="{{ route('transaksi.index') }}">Lihat Semua</a></x-card>
26   <x-data-table :headers="$transaksiHeaders">
27     @foreach($transaksiTerbaru as $t)
28       <tr>
29         ...
30       </tr>
31     @endforeach
32   </x-data-table>
33 </x-app-layout>
```

Keuntungan menggunakan komponen:

- Kode lebih bersih dan mudah dibaca
- Konsistensi tampilan antar halaman
- Mudah di-maintain (ubah di satu tempat)
- Reusable di semua halaman CRUD

Menggunakan Komponen di Halaman CRUD

Halaman form CRUD menggunakan komponen FormInput:

Form Tambah Barang dengan komponen FormInput

```
resources/views/barang/create.blade.php

1  <x-app-layout>
2      <x-card header="Tambah Barang" icon="fa-box" icon-bg="#eef2ff" icon-color="#4f46e5">
3          <form action="{{ route('barang.store') }}" method="POST"
4              enctype="multipart/form-data">
5              @csrf
6
7              <div class="form-grid">
8                  <x-form-input name="nama" label="Nama Barang"
9                      icon="fa-box" placeholder="Nama barang"
10                     :value="old('nama')" required />
11
12                  <x-form-input name="kategori_id" label="Kategori"
13                      type="select" icon="fa-tags" required>
14                      @foreach($kategoris as $k)
15                          <option value="{{ $k->id }}">{{ $k->nama }}</option>
16                      @endforeach
17                  </x-form-input>
18
19                  <x-form-input name="jumlah" label="Jumlah"
20                      type="number" icon="fa-hashtag" required />
21
22                  <x-form-input name="kondisi" label="Kondisi"
23                      type="select" icon="fa-clipboard-check">
24                      <option value="baik">Baik</option>
25                      <option value="rusak">Rusak</option>
26                      <option value="hilang">Hilang</option>
27                  </x-form-input>
28              </div>
29
30              <x-form-input name="deskripsi" label="Deskripsi"
31                  type="textarea" icon="fa-align-left"
32                  placeholder="Deskripsi barang" />
33
34              <div class="flex gap-3" style="margin-top:22px;padding-top:22px;border-top:1px solid #e8ecf1">
35                  <button type="submit" class="btn btn-primary">
36                      <i class="fas fa-save"></i> Simpan
37                  </button>
38                  <a href="{{ route('barang.index') }}" class="btn btn-outline">Batal</a>
39              </div>
40          </form>
41      </x-card>
42  </x-app-layout>
```

Blade Component PHP 8.2

Halaman Kategori menggunakan DataTable:

Halaman Kategori dengan DataTable component

```
resources/views/pages/kategori/index.blade.php

1 <x-app-layout>
2   <div class="page-header">
3     <div>
4       <h2>Kategori</h2>
5       <p>Kelola data kategori barang</p>
6     </div>
7     <a href="{{ route('kategori.create') }}" class="btn btn-primary">
8       <i class="fas fa-plus"></i> Tambah Kategori
9     </a>
10  </div>
11
12  <x-data-table :headers="[
13    ['label'=>'No', 'width'=>'50px'],
14    ['label'=>'Nama'],
15    ['label'=>'Deskripsi'],
16    ['label'=>'Jml Barang', 'class'=>'text-center'],
17    ['label'=>'Aksi', 'class'=>'text-right', 'width'=>'160px'],
18  ]">
19    @forelse ($kategoris as $ket)
20      <tr>
21        <td>{{ $loop->iteration }}</td>
22        <td>{{ $ket->nama }}</td>
23        <td>{{ $ket->deskripsi ?? '-' }}</td>
24        <td class="text-center">{{ $ket->barang_count }}</td>
25        <td class="text-right">
26          <x-button type="edit" :href="route('kategori.edit', $ket)" />
27          <x-button type="delete" :action="route('kategori.destroy', $ket)" />
28        </td>
29      </tr>
30      @empty
31        <tr><td colspan="5" class="empty"><i class="fas fa-tags"></i>Belum ada data</td></tr>
32      @endforelse
33    </x-data-table>
34  </x-app-layout>
```

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Blade component, @include, @component". Analisis bagaimana konsep layout dan component blade diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Terminal - Membuat Komponen

Perintah artisan membuat ketiga komponen

```
terminal

1
2 $ php artisan make:component Card
3 INFO Component [app/View/Components/Card.php] created successfully.
4 INFO Component [resources/views/components/card.blade.php] created successfully.
5
6 $ php artisan make:component FormInput
7 INFO Component [app/View/Components/FormInput.php] created successfully.
8 INFO Component [resources/views/components/form-input.blade.php] created successfully.
9
10 $ php artisan make:component DataTable
11 INFO Component [app/View/Components/DataTable.php] created successfully.
12 INFO Component [resources/views/components/data-table.blade.php] created successfully.
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Class Card (app/View/Components/Card.php)

```
app/View/Components/Card.php

1  <?php
2
3  namespace App\View\Components;
4
5  use Illuminate\View\Component;
6
7
8  class Card extends Component
9  {
10     public $header;
11     public $icon;
12     public $iconBg;
13     public $iconColor;
14     public $action;
15     public $bodyClass;
16
17     public function __construct(
18         $header = null,
19         $icon = null,
20         $iconBg = null,
21         $iconColor = null,
22         $action = null,
23         $bodyClass = null
24     ) {
25         $this->header = $header;
26         $this->icon = $icon;
27         $this->iconBg = $iconBg;
28         $this->iconColor = $iconColor;
29         $this->action = $action;
30         $this->bodyClass = $bodyClass;
31     }
32
33     public function render()
34     {
35         return view('components.card');
36     }
37 }
```

Blade Component PHP 8.2

Template Card (resources/views/components/card.blade.php)

```
resources/views/components/card.blade.php

1  <div class="card {{ $attributes->get('class') }}">
2      @if(isset($header))
3          <div class="card-header">
4              <div class="flex items-center gap-3">
5                  @if(isset($icon))
6                      <div class="icon-box" style="background:{{ $iconBg ?? '#eef2ff' }};
7                          color:{{ $iconColor ?? '#4f46e5' }}">
8                          <i class="fas {{ $icon }}"></i>
9                      </div>
10                 @endif
11                 <span style="font-weight:600;color:#1a1a2e">{{ $header }}</span>
12             </div>
13             @if(isset($action))
14                 <div>{{ $action }}</div>
15             @endif
16         </div>
17     @endif
18
19     <div class="card-body {{ $bodyClass ?? '' }}">
20         {{ $slot }}
21     </div>
22 </div>
```

Blade Component PHP 8.2

Template FormInput (resources/views/components/form-input.blade.php)

```
resources/views/components/form-input.blade.php

1 <div class="form-group" {{ $attributes->get('class') }}">
2     @if(isset($label))
3         <label for="{{ $name }}" class="form-label">
4             {{ $label }}
5             @if($required ?? false)
6                 <span style="color:#dc2626">*</span>
7             @endif
8         </label>
9     @endif
10
11     <div class="input-group">
12         @if(isset($icon))
13             <span class="icon"><i class="fas {{ $icon }}"></i></span>
14         @endif
15
16         @if($type === 'textarea')
17             <textarea name="{{ $name }}" id="{{ $name }}"
18                 {{ $attributes->wire('model') }}
19                 placeholder="{{ $placeholder ?? '' }}"
20                 class="form-control"
21                 rows="{{ $rows ?? 4 }}">{{ $value ?? '' }}</textarea>
22         @elseif($type === 'select')
23             <select name="{{ $name }}" id="{{ $name }}" class="form-control">
24                 {{ $slot }}
25             </select>
26         @else
27             <input type="{{ $type ?? 'text' }}" name="{{ $name }}" id="{{ $name }}"
28                 value="{{ $value ?? old($name) }}"
29                 placeholder="{{ $placeholder ?? '' }}"
30                 class="form-control"
31                 {{ $attributes }}>
32         @endif
33     </div>
34
35     @error($name)
36         <p style="font-size:12px;color:#dc2626;margin-top:4px">
37             <i class="fas fa-exclamation-circle"></i> {{ $message }}
38         </p>
39     @enderror
40 </div>
```

Class DataTable (app/View/Components/DataTable.php)

```
app/View/Components/DataTable.php

1 <?php
2
3 namespace App\View\Components;
4
5 use Illuminate\View\Component;
6
7 class DataTable extends Component
8 {
9     public $headers;
10    public $paginator;
11    public $tableClass;
12
13    public function __construct(
14        $headers = [],
15        $paginator = null,
16        $tableClass = null
17    ) {
18        $this->headers = $headers;
19        $this->paginator = $paginator;
20        $this->tableClass = $tableClass;
21    }
22
23    public function render()
24    {
25        return view('components.data-table');
26    }
27 }
```

Template DataTable (resources/views/components/data-table.blade.php)

```
resources/views/components/data-table.blade.php

1 <div class="card">
2     <div class="overflow-x-auto">
3         <table class="{{ $tableClass ?? '' }}">
4             @if(isset($headers))
5                 <thead>
6                     <tr>
7                         @foreach($headers as $header)
8                             <th class="{{ $header['class'] ?? '' }}"
9                                 @if(isset($header['width'])) style="width:{{ $header['width'] }}" @endif
10                                {{ $header['label'] ?? $header }}
11                            </th>
12                        @endforeach
13                    </tr>
14                </thead>
15            @endif
16
17            <tbody class="divide-y">
18                {{ $slot }}
19            </tbody>
20        </table>
21    </div>
22
23    @if(isset($paginator) && $paginator->hasPages())
24        <div class="pagination-wrapper">
25            {{ $paginator->links() }}
26        </div>
27    @endif
28 </div>
```

Dashboard menggunakan komponen

```
resources/views/dashboard.blade.php

1 <x-app-layout>
2     <!-- Stat Cards menggunakan Card component -->
3     @php
4         $stats = [
5             ['label'=>'Total Barang', 'value'=>$totalBarang, 'icon'=>'fa-cube', 'color'=>'#667eea', 'bg'=>'#eef2ff'],
6             ['label'=>'Total Kategori', 'value'=>$totalKategori, 'icon'=>'fa-tags', 'color'=>'#11998e', 'bg'=>'#e8faf0'],
7             ['label'=>'Barang Dipinjam', 'value'=>$barangDipinjam, 'icon'=>'fa-hand-holding', 'color'=>'#f5576c', 'bg'=>'#fd'],
8             ['label'=>'Total Stok', 'value'=>$barangTersedia, 'icon'=>'fa-warehouse', 'color'=>'#00f2fe', 'bg'=>'#e8faf0'],
9         ];
10     @endphp
11
12     <div class="stat-grid">
13         @foreach($stats as $s)
14             <x-card icon="{{ $s['icon'] }}" icon-bg="{{ $s['bg'] }}"
15                 icon-color="{{ $s['color'] }}">
16                 <div class="label">{{ $s['label'] }}</div>
17                 <div class="value" style="color:{{ $s['color'] }}">{{ $s['value'] }}</div>
18             </x-card>
19         @endforeach
20     </div>
21
22     <!-- Transaksi Terbaru -->
23     <x-card header="Transaksi Terbaru">
24         icon="fa-clock" icon-bg="#e8faf0" icon-color="#0d6832"
25         action="<a href="{{ route('transaksi.index') }}">Lihat Semua</a>"
26         <x-data-table :headers="$transaksiHeaders">
27             @foreach($transaksiTerbaru as $t)
28                 <tr>
29                     ...
30                 </tr>
31             @endforeach
32         </x-data-table>
33     </x-card>
34 </x-app-layout>
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Form CRUD menggunakan komponen

```
resources/views/barang/create.blade.php

1 <x-app-layout>
2 <x-card header="Tambah Barang" icon="fa-box" icon-bg="#eef2ff" icon-color="#4f46e5">
3 <form action="{{ route('barang.store') }}" method="POST"
4     enctype="multipart/form-data">
5     @csrf
6
7     <div class="form-grid">
8         <x-form-input name="nama" label="Nama Barang"
9             icon="fa-box" placeholder="Nama barang"
10            :value="old('nama')" required />
11
12         <x-form-input name="kategori_id" label="Kategori"
13             type="select" icon="fa-tags" required>
14             @foreach($kategoris as $k)
15                 <option value="{{ $k->id }}">{{ $k->nama }}</option>
16             @endforeach
17         </x-form-input>
18
19         <x-form-input name="jumlah" label="Jumlah"
20             type="number" icon="fa-hashtag" required />
21
22         <x-form-input name="kondisi" label="Kondisi"
23             type="select" icon="fa-clipboard-check">
24             <option value="baik">Baik</option>
25             <option value="rusak">Rusak</option>
26             <option value="hilang">Hilang</option>
27         </x-form-input>
28     </div>
29
30     <x-form-input name="deskripsi" label="Deskripsi"
31         type="textarea" icon="fa-align-left"
32         placeholder="Deskripsi barang" />
33
34     <div class="flex gap-3" style="margin-top:22px;padding-top:22px;border-top:1px solid #e8ecf1">
35         <button type="submit" class="btn btn-primary">
36             <i class="fas fa-save"></i> Simpan
37         </button>
38         <a href="{{ route('barang.index') }}" class="btn btn-outline">Batal</a>
39     </div>
40 </form>
41 </x-card>
42 </x-app-layout>
```

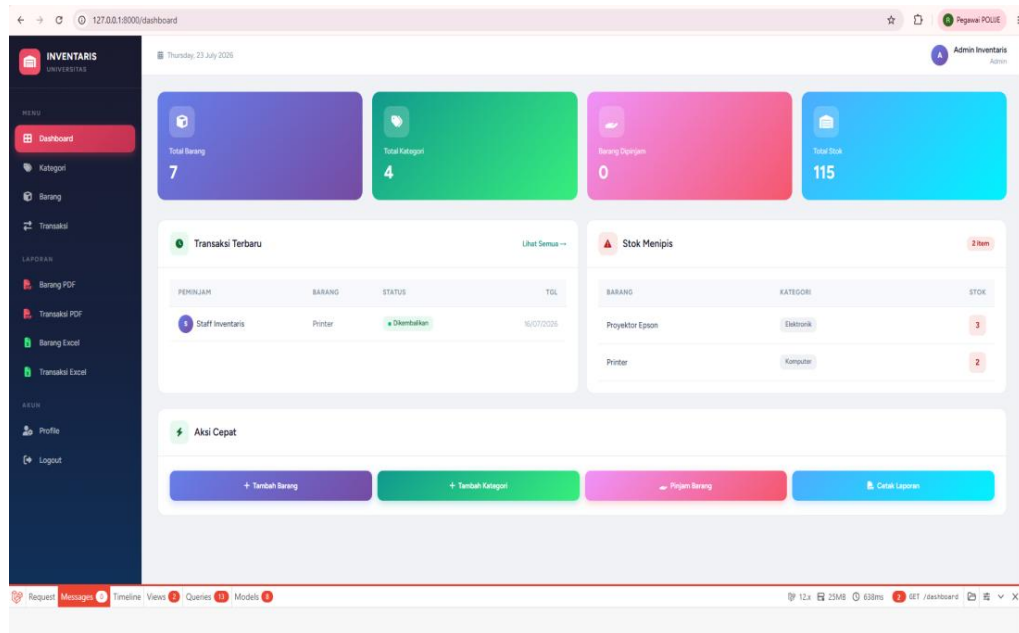
Kategori Index menggunakan DataTable

```
resources/views/pages/kategori/index.blade.php

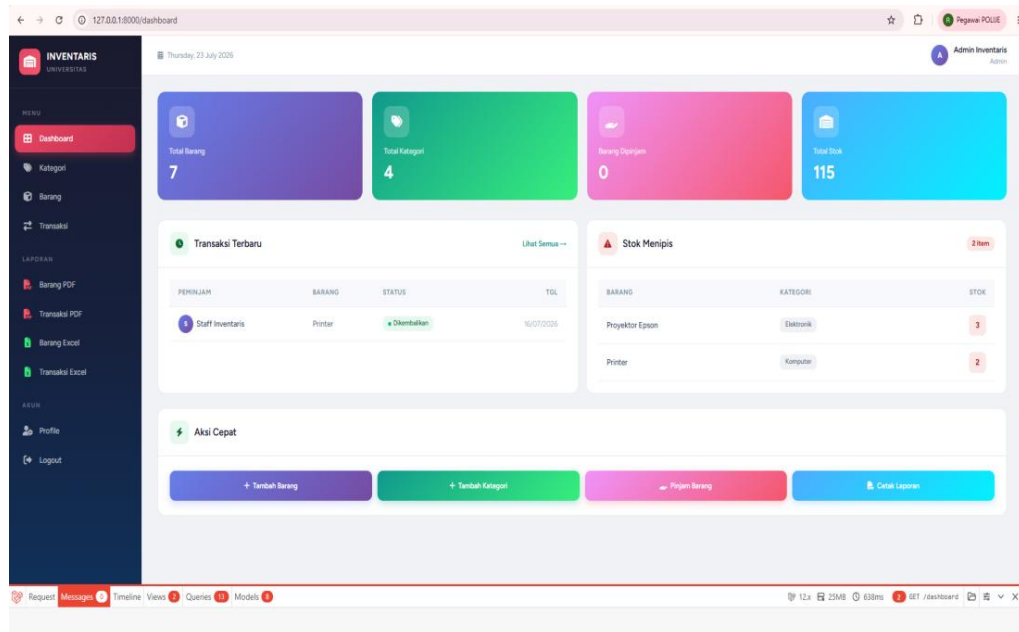
1 <x-app-layout>
2 <div class="page-header">
3 <div>
4 <h2>Kategori</h2>
5 <p>Kelola data kategori barang</p>
6 </div>
7 <a href="{{ route('kategori.create') }}" class="btn btn-primary">
8     <i class="fas fa-plus"></i> Tambah Kategori
9 </a>
10 </div>
11
12 <x-data-table :headers="[
13     ['label'=>'No', 'width'=>'50px'],
14     ['label'=>'Nama'],
15     ['label'=>'Deskripsi'],
16     ['label'=>'Jml Barang', 'class'=>'text-center'],
17     ['label'=>'Aksi', 'class'=>'text-right', 'width'=>'160px'],
18 ]">
19     @forelse ($kategoris as $ket)
20     <tr>
21         <td>{{ $loop->iteration }}</td>
22         <td>{{ $ket->nama }}</td>
23         <td>{{ $ket->deskripsi ?? '-' }}</td>
24         <td class="text-center">{{ $ket->barangs_count }}</td>
25         <td class="text-right">
26             <x-button type="edit" :href="route('kategori.edit', $ket)" />
27             <x-button type="delete" :action="route('kategori.destroy', $ket)" />
28         </td>
29     </tr>
30     @empty
31     <tr><td colspan="5" class="empty"><i class="fas fa-tags"></i>Belum ada data</td></tr>
32     @endforelse
33 </x-data-table>
34 </x-app-layout>
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Komponen Card di Dashboard



Dashboard dengan Card component untuk statistik



BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman Kategori dengan DataTable component

Kategori

Kelola data kategori barang

NO	NAMA	DESKRIPSI	JUMLAH BARANG	Aksi
1	Elektronik	Barang elektronik dan perangkat keras	1	Edit Hapus
2	Perabotan	Majja, kursi, lemari, dan perabotan lainnya	2	Edit Hapus
3	Alat Tulis	Alat tulis kantor dan perlengkapan administrasi	1	Edit Hapus
4	Komputer	Desktop, laptop, dan perangkat komputer	3	Edit Hapus

Form Tambah Barang dengan FormInput component

Profile Information

Update your account's profile information and email address.

Name

Email

[SAVE](#)

Update Password

Ensure your account is using a long, random password to stay secure.

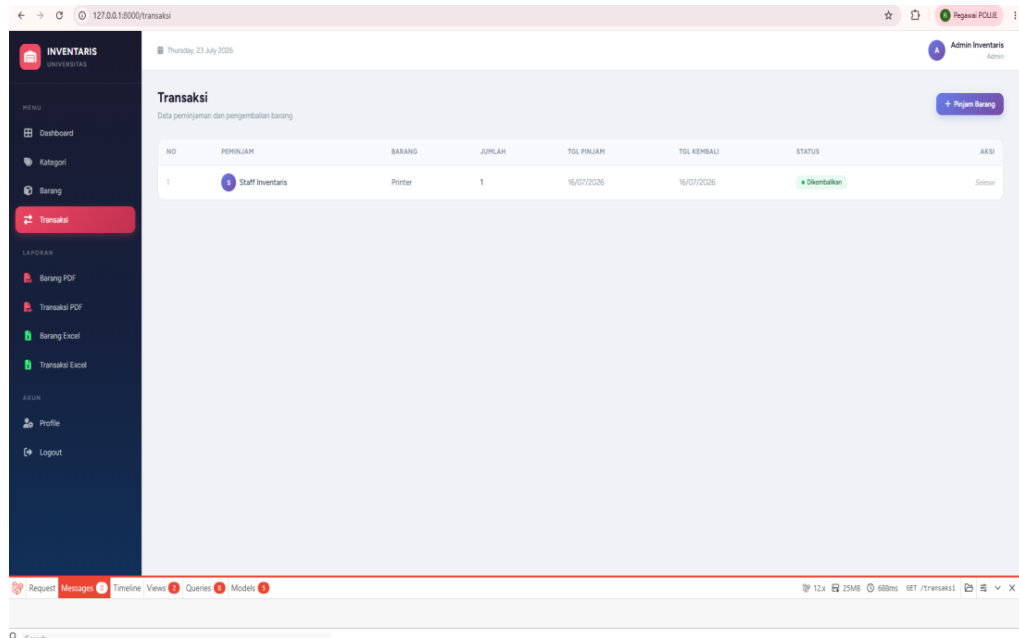
Current Password

New Password

Confirm Password

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman Transaksi dengan DataTable component



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat komponen Blade yang dapat digunakan ulang

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
Total Skor*		Skor x 25			

Acara 13. CRUD Kategori - Create dan Read

Materi Pembelajaran : Resource Controller, migration, form
 Acara Praktikum/Praktik : Minggu 7/1
 Tempat : Daring/Luring Politeknik Negeri Jember
 Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi (TB430805.3)

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.0 20.02	Menggunakan SQL	• Mempersiapkan perangkat lunak aplikasi data deskripsi/SQL • Menggunakan fitur aplikasi SQL • Mengisi table • Melakukan operasi relasional • Membuat stored procedure • Membuat function • Membuat triger • Melakukan perintah commit dan rollback
2	J.620100.0 21.02	Menerapkan Akses Basis Data	• Membuat berbagai operasi terhadap basis data • Membuat prosedur akses terhadap basis data • Membuat prosedur akses terhadap basis data • Menguji program basis data

c. Indikator Penilaian

Mahasiswa mampu membuat fitur Create dan Read Kategori

d. Dasar Teori

Dalam teori pengembangan aplikasi, khususnya dalam konteks CRUD (Create, Read, Update, Delete), istilah Create dan Read merupakan dua operasi dasar yang digunakan untuk mengelola data dalam sistem informasi maupun basis data.

Create adalah proses menambahkan atau memasukkan data baru ke dalam sistem. Operasi ini biasanya dilakukan melalui form input atau API, kemudian data

tersebut disimpan ke dalam tabel database. Contoh sederhana adalah ketika pengguna mendaftarkan akun baru, sistem akan melakukan operasi create untuk menyimpan informasi seperti nama, email, dan password ke dalam tabel pengguna. Secara teori, operasi create mendukung prinsip data persistence, yaitu memastikan data baru tersimpan secara permanen dan dapat diakses kembali.

Read adalah proses mengambil atau menampilkan data yang sudah ada di dalam sistem. Operasi ini memungkinkan pengguna atau aplikasi untuk melakukan query terhadap database dan menampilkan hasilnya dalam bentuk yang mudah dipahami, misalnya daftar produk, profil pengguna, atau laporan transaksi. Read biasanya menggunakan perintah select dalam SQL, dan dalam framework seperti Laravel, operasi ini dilakukan melalui model Eloquent yang mempermudah pengambilan data dengan sintaks berbasis objek.

Secara konseptual, Create dan Read adalah bagian dari siklus CRUD yang menjadi fondasi dalam pengembangan aplikasi berbasis data. Create memastikan sistem dapat menerima dan menyimpan informasi baru, sementara Read memastikan informasi tersebut dapat diakses dan digunakan kembali. Kombinasi keduanya menjadikan aplikasi mampu berfungsi sebagai media interaksi data yang dinamis, konsisten, dan bermanfaat bagi pengguna maupun organisasi.

Kategori barang adalah master data pertama. Fitur: tambah kategori (nama, deskripsi), lihat daftar kategori dalam tabel. Menggunakan Resource Controller dan Form Request Validation.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Membuat Migration

```
php artisan make:migration create_kategoris_table
```

File migration berisi skema tabel kategoris:

id() — primary key auto-increment

string('nama', 100) — nama kategori

text('deskripsi')->nullable() — deskripsi

timestamps() — created_at dan updated_at

File migration create_kategoris_table

```
database/migrations/xxx_create_kategoris_table.php

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('kategoris', function (Blueprint $table) {
12             $table->id();
13             $table->string('nama', 100);
14             $table->text('deskripsi')->nullable();
15             $table->timestamps();
16         });
17     }
18
19     public function down(): void
20     {
21         Schema::dropIfExists('kategoris');
22     }
23 }
```

Menjalankan Migration

php artisan migrate

Perintah ini akan mengeksekusi semua migration yang belum dijalankan, membuat tabel kategoris di database.

Membuat Model Kategori

php artisan make:model Kategori

Model Kategori memiliki properti \$fillable untuk mass assignment dan relasi hasMany ke model Barang.

Model Kategori

```
app/Models/Kategori.php

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\Relations\HasMany;
8
9  class Kategori extends Model
10  {
11      use HasFactory;
12      protected $table = 'kategoris';
13
14      protected $fillable = [
15          'nama',
16          'deskripsi',
17      ];
18
19      public function barangs(): HasMany
20      {
21          return $this->hasMany(Barang::class);
22      }
23 }
```

Membuat KategoriController

php artisan make:controller KategoriController --resource

Flag --resource menghasilkan 7 method CRUD: index, create, store, show, edit, update, destroy.

Controller - method index, create, store

```
app/Http/Controllers/KategoriController.php (1)

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Kategori;
6  use Illuminate\Http\Request;
7
8  class KategoriController extends Controller
9  {
10     public function index()
11     {
12         $kategoris = Kategori::withCount('barangs')->get();
13         return view('pages.kategori.index', compact('kategoris'));
14     }
15
16     public function create()
17     {
18         return view('pages.kategori.create');
19     }
20
21     public function store(Request $request)
22     {
23         $request->validate([
24             'nama' => 'required|string|max:100',
25             'deskripsi' => 'nullable|string',
26         ]);
27
28         Kategori::create($request->only(['nama', 'deskripsi']));
29
30         return redirect()->route('kategori.index')
31             ->with('success', 'Kategori berhasil ditambahkan');
32     }
33 }
```

PHP 8.2 Laravel Blade

Controller - method show, edit, update, destroy

```
app/Http/Controllers/KategoriController.php (2)
app/Http/Controllers/KategoriController.php (2)
1     public function show(Kategori $kategori)
2     {
3         return view('pages.kategori.show', compact('kategori'));
4     }
5
6     public function edit(Kategori $kategori)
7     {
8         return view('pages.kategori.edit', compact('kategori'));
9     }
10
11    public function update(Request $request, Kategori $kategori)
12    {
13        $request->validate([
14            'nama' => 'required|string|max:100',
15            'deskripsi' => 'nullable|string',
16        ]);
17
18        $kategori->update($request->only(['nama', 'deskripsi']));
19
20        return redirect()->route('kategori.index')
21            ->with('success', 'Kategori berhasil diupdate');
22    }
23
24    public function destroy(Kategori $kategori)
25    {
26        if ($kategori->barang()->count() > 0) {
27            return back()->with('error',
28                'Kategori tidak bisa dihapus karena masih memiliki barang');
29        }
30        $kategori->delete();
31        return redirect()->route('kategori.index')
32            ->with('success', 'Kategori berhasil dihapus');
33    }
34 }
```

Implementasi Method Controller

Index: Mengambil semua data kategori dengan jumlah barang (withCount), lalu return ke view.

Create: Menampilkan form tambah kategori.

Store: Validasi request (nama required, deskripsi nullable), simpan ke database, redirect dengan flash message.

Edit/Update: Menampilkan form edit dengan data existing, update dengan validated data.

Destroy: Cek apakah kategori memiliki barang, jika ada tolak penghapusan, jika aman hapus.

Membuat View Index

File resources/views/pages/kategori/index.blade.php:

- Menampilkan tabel dengan kolom No, Nama (icon), Deskripsi, Jumlah Barang, Aksi (Edit/Hapus).
- Tombol "Tambah Kategori" di header.

- Empty state jika belum ada data.
- Konfirmasi hapus dengan JavaScript confirm().

View index kategori

```
resources/views/pages/kategori/index.blade.php

1 <x-app-layout>
2   <div class="page-header">
3     <div style="display:flex;align-items:center;justify-content:space-between">
4       <div>
5         <h2>Kategori</h2>
6         <p>Kelola data kategori barang</p>
7       </div>
8       <a href="{{ route('kategori.create') }}" class="btn btn-primary">
9         <i class="fas fa-plus"></i> Tambah Kategori
10      </a>
11    </div>
12  </div>
13
14  <div class="card">
15    <table>
16      <thead>
17        <tr>
18          <th>No</th><th>Nama</th><th>Deskripsi</th>
19          <th>Jml Barang</th><th>Aksi</th>
20        </tr>
21      </thead>
22      <tbody>
23        @forelse ($kategoris as $ket)
24          <tr>
25            <td>{{ $loop->iteration }}</td>
26            <td>{{ $ket->nama }}</td>
27            <td>{{ $ket->deskripsi ?? '-' }}</td>
28            <td>{{ $ket->barang_count }}</td>
29            <td>
30              <a href="{{ route('kategori.edit', $ket) }}"
31                class="action-btn" style="background:#ffffbe;color:#d97706">
32                <i class="fas fa-edit"></i> Edit
33              </a>
34              <form action="{{ route('kategori.destroy', $ket) }}" method="POST"
35                onsubmit="return confirm('Yakin hapus?')" style="display:inline">
36                @csrf @method('DELETE')
37                <button type="submit" class="action-btn"
38                  style="background:#fef2f2;color:#dc2626">
39                  <i class="fas fa-trash"></i> Hapus
40                </button>
41              </form>
42            </td>
43          </tr>
44        @empty
45          <tr><td colspan="5">Belum ada data</td></tr>
46        @endforelse
47      </tbody>
48    </table>
49  </div>
50 </x-app-layout>
```

Membuat View Create & Edit

File resources/views/pages/kategori/create.blade.php:

- Form dengan input Nama (icon tag) dan Deskripsi (icon align-left).

- Validasi error per-field menggunakan @error.
- Tombol Simpan dan Batal.

View create kategori

```
resources/views/pages/kategori/create.blade.php

1 <x-app-layout>
2   <div class="page-header">
3     <div class="flex items-center gap-4">
4       <a href="{{ route('kategori.index') }}" class="btn"
5         style="background:#f0f2f5;color:#525f7f;padding:10px 14px">
6         <i class="fas fa-arrow-left"></i>
7       </a>
8     </div>
9     <h2>Tambah Kategori</h2>
10    <p>Buat kategori barang baru</p>
11  </div>
12 </div>
13 </div>
14
15 <div class="max-w-2xl">
16   <div class="card">
17     <div class="card-body">
18       <form action="{{ route('kategori.store') }}" method="POST">
19         @csrf
20         <div class="input-group">
21           <span class="icon"><i class="fas fa-tag"></i></span>
22           <input type="text" name="nama"
23             value="{{ old('nama') }}"
24             placeholder="Masukkan nama kategori"
25             class="form-control">
26         </div>
27         @error("nama")
28         <p style="color:#dc2626;font-size:12px">
29           <i class="fas fa-exclamation-circle"></i> {{ $message }}</p>
30         @enderror
31
32         <div class="input-group" style="margin-top:22px">
33           <span class="icon"><i class="fas fa-align-left"></i></span>
34           <textarea name="deskripsi" rows="4"
35             placeholder="Deskripsi kategori (opsional)"
36             class="form-control">{{ old("deskripsi") }}</textarea>
37         </div>
38
39         <div class="flex gap-3" style="margin-top:22px">
40           <button type="submit" class="btn btn-primary">
41             <i class="fas fa-save"></i> Simpan
42           </button>
43           <a href="{{ route('kategori.index') }}" class="btn btn-outline">Batal</a>
44         </div>
45       </form>
46     </div>
47   </div>
48 </div>
49 </x-app-layout>
```

File resources/views/pages/kategori/edit.blade.php:

- Sama seperti create, dengan old() dan data existing dari \$kategori.
- Method PUT menggunakan @method("PUT").

View edit kategori

```
resources/views/pages/kategori/edit.blade.php

1 <x-app-layout>
2   <div class="page-header">
3     <div class="flex items-center gap-4">
4       <a href="{{ route('kategori.index') }}" class="btn" ...>
5         <i class="fas fa-arrow-left"></i>
6       </a>
7     </div>
8     <h2>Edit Kategori</h2>
9     <p>Ubah data kategori {{ $kategori->nama }}</p>
10   </div>
11 </div>
12 </div>
13
14 <div class="max-w-2xl">
15   <div class="card">
16     <div class="card-body">
17       <form action="{{ route('kategori.update', $kategori) }}"
18         method="POST">
19         @csrf @method('PUT')
20
21       <div class="input-group">
22         <span class="icon"><i class="fas fa-tag"></i></span>
23         <input type="text" name="nama"
24           value="{{ old('nama', $kategori->nama) }}"
25           class="form-control">
26       </div>
27
28       <div class="input-group" style="margin-top:22px">
29         <span class="icon"><i class="fas fa-align-left"></i></span>
30         <textarea name="deskripsi" rows="4"
31           class="form-control">{{ old('deskripsi', $kategori->deskripsi) }}</textarea>
32       </div>
33
34       <div class="flex gap-3" style="margin-top:22px">
35         <button type="submit" class="btn btn-primary">
36           <i class="fas fa-save"></i> Update
37         </button>
38         <a href="{{ route('kategori.index') }}" class="btn btn-outline">Batal</a>
39       </div>
40     </form>
41   </div>
42 </div>
43 </div>
44 </x-app-layout>
```

Menambahkan Route

Route::resource('kategori', KategoriController::class)->middleware('auth');

Route resource mendaftarkan 7 route sekaligus:

- GET /kategori → index (kategori.index)
- GET /kategori/create → create (kategori.create)
- POST /kategori → store (kategori.store)
- GET /kategori/{id} → show (kategori.show)
- GET /kategori/{id}/edit → edit (kategori.edit)
- PUT /kategori/{id} → update (kategori.update)
- DELETE /kategori/{id} → destroy (kategori.destroy)

Route resource kategori

```
routes/web.php

1 <?php
2
3 use App\Http\Controllers\KategoriController;
4
5 Route::middleware(['auth', 'verified'])->group(function () {
6     Route::resource('kategori', KategoriController::class);
7     // Ekuivalen dengan:
8     // GET /kategori -> index (nama: kategori.index)
9     // GET /kategori/create -> create (nama: kategori.create)
10    // POST /kategori -> store (nama: kategori.store)
11    // GET /kategori/{kategori} -> show (nama: kategori.show)
12    // GET /kategori/{kategori}/edit -> edit (nama: kategori.edit)
13    // PUT /kategori/{kategori} -> update (nama: kategori.update)
14    // DELETE /kategori/{kategori} -> destroy (nama: kategori.destroy)
15 });
```

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Resource Controller, migration, form". Analisis bagaimana konsep crud kategori - create dan read diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Perintah artisan membuat migration, model, controller

```
terminal

1
2 $ php artisan make:migration create_kategoris_table
3 Created Migration: database/migrations/2026_07_16_000001_create_kategoris_table.php
4
5 $ php artisan migrate
6 Migrating: 2026_07_16_000001_create_kategoris_table
7 Migrated: 2026_07_16_000001_create_kategoris_table (0.02s)
8
9 $ php artisan make:model Kategori
10 Model created successfully.
11
12 $ php artisan make:controller KategoriController --resource
13 Controller created successfully.
```

Migration - create_kategoris_table

```
database/migrations/xxx_create_kategoris_table.php

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('kategoris', function (Blueprint $table) {
12             $table->id();
13             $table->string('nama', 100);
14             $table->text('deskripsi')->nullable();
15             $table->timestamps();
16         });
17     }
18
19     public function down(): void
20     {
21         Schema::dropIfExists('kategoris');
22     }
23 }
```

Model Kategori

```
app/Models/Kategori.php

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\Relations\HasMany;
8
9  class Kategori extends Model
10  {
11      use HasFactory;
12      protected $table = 'kategoris';
13
14      protected $fillable = [
15          'nama',
16          'deskripsi',
17      ];
18
19      public function barang(): HasMany
20      {
21          return $this->hasMany(Barang::class);
22      }
23 }
```

Controller - index, create, store

```
app/Http/Controllers/KategoriController.php (1)

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Kategori;
6  use Illuminate\Http\Request;
7
8  class KategoriController extends Controller
9  {
10     public function index()
11     {
12         $kategoris = Kategori::withCount('barang')->get();
13         return view('pages.kategori.index', compact('kategoris'));
14     }
15
16     public function create()
17     {
18         return view('pages.kategori.create');
19     }
20
21     public function store(Request $request)
22     {
23         $request->validate([
24             'nama' => 'required|string|max:100',
25             'deskripsi' => 'nullable|string',
26         ]);
27
28         Kategori::create($request->only(['nama', 'deskripsi']));
29
30         return redirect()->route('kategori.index')
31             ->with('success', 'Kategori berhasil ditambahkan');
32     }
33 }
```

Controller - show, edit, update, destroy

```
app/Http/Controllers/KategoriController.php (2)

1  public function show(Kategori $kategori)
2  {
3      return view('pages.kategori.show', compact('kategori'));
4  }
5
6  public function edit(Kategori $kategori)
7  {
8      return view('pages.kategori.edit', compact('kategori'));
9  }
10
11 public function update(Request $request, Kategori $kategori)
12 {
13     $request->validate([
14         'nama' => 'required|string|max:100',
15         'deskripsi' => 'nullable|string',
16     ]);
17
18     $kategori->update($request->only(['nama', 'deskripsi']));
19
20     return redirect()->route('kategori.index')
21         ->with('success', 'Kategori berhasil diupdate');
22 }
23
24 public function destroy(Kategori $kategori)
25 {
26     if ($kategori->barang()->count() > 0) {
27         return back()->with('error',
28             'Kategori tidak bisa dihapus karena masih memiliki barang');
29     }
30     $kategori->delete();
31     return redirect()->route('kategori.index')
32         ->with('success', 'Kategori berhasil dihapus');
33 }
34 }
```

View index kategori

```
resources/views/pages/kategori/index.blade.php

1 <x-app-layout>
2   <div class="page-header">
3     <div style="display:flex;align-items:center;justify-content:space-between">
4       <div>
5         <h2>Kategori</h2>
6         <p>Kelola data kategori barang</p>
7       </div>
8       <a href="{{ route('kategori.create') }}" class="btn btn-primary">
9         <i class="fas fa-plus"></i> Tambah Kategori
10      </a>
11    </div>
12  </div>
13
14  <div class="card">
15    <table>
16      <thead>
17        <tr>
18          <th>No</th><th>Nama</th><th>Deskripsi</th>
19          <th>Jml Barang</th><th>Aksi</th>
20        </tr>
21      </thead>
22      <tbody>
23        @forelse ($kategoris as $ket)
24          <tr>
25            <td>{{ $loop->iteration }}</td>
26            <td>{{ $ket->nama }}</td>
27            <td>{{ $ket->deskripsi ?? '-' }}</td>
28            <td>{{ $ket->barang_count }}</td>
29            <td>
30              <a href="{{ route('kategori.edit', $ket) }}"
31                class="action-btn" style="background:#fffbeb;color:#d97706">
32                <i class="fas fa-edit"></i> Edit
33              </a>
34              <form action="{{ route('kategori.destroy', $ket) }}" method="POST"
35                onsubmit="return confirm('Yakin hapus?')" style="display:inline">
36                @csrf @method('DELETE')
37                <button type="submit" class="action-btn"
38                  style="background:#fef2f2;color:#dc2626">
39                  <i class="fas fa-trash"></i> Hapus
40                </button>
41              </form>
42            </td>
43          </tr>
44          @empty
45            <tr><td colspan="5">Belum ada data</td></tr>
46          @endforelse
47        </tbody>
48      </table>
49    </div>
50  </x-app-layout>
```

PHP 8.2 Laravel Blade

View create kategori

```
resources/views/pages/kategori/create.blade.php

1 <x-app-layout>
2   <div class="page-header">
3     <div class="flex items-center gap-4">
4       <a href="{{ route('kategori.index') }}" class="btn"
5         style="background:#f0f2f5;color:#525f7f;padding:10px 14px">
6         <i class="fas fa-arrow-left"></i>
7       </a>
8     </div>
9     <h2>Tambah Kategori</h2>
10    <p>Buat kategori barang baru</p>
11  </div>
12 </div>
13 </div>
14
15 <div class="max-w-2xl">
16   <div class="card">
17     <div class="card-body">
18       <form action="{{ route('kategori.store') }}" method="POST">
19         @csrf
20         <div class="input-group">
21           <span class="icon"><i class="fas fa-tag"></i></span>
22           <input type="text" name="nama"
23             value="{{ old('nama') }}"
24             placeholder="Masukkan nama kategori"
25             class="form-control">
26         </div>
27         @error("nama")
28         <p style="color:#dc2626;font-size:12px">
29           <i class="fas fa-exclamation-circle"></i> {{ $message }}</p>
30         @enderror
31
32         <div class="input-group" style="margin-top:22px">
33           <span class="icon"><i class="fas fa-align-left"></i></span>
34           <textarea name="deskripsi" rows="4"
35             placeholder="Deskripsi kategori (opsional)"
36             class="form-control">{{ old("deskripsi") }}</textarea>
37         </div>
38
39         <div class="flex gap-3" style="margin-top:22px">
40           <button type="submit" class="btn btn-primary">
41             <i class="fas fa-save"></i> Simpan
42           </button>
43           <a href="{{ route('kategori.index') }}" class="btn btn-outline">Batal</a>
44         </div>
45       </form>
46     </div>
47   </div>
48 </div>
49 </x-app-layout>

PHP 8.2  Laravel  Blade
```

View edit kategori

```
resources/views/pages/kategori/edit.blade.php

1 <x-app-layout>
2   <div class="page-header">
3     <div class="flex items-center gap-4">
4       <a href="{{ route('kategori.index') }}" class="btn ...">
5         <i class="fas fa-arrow-left"></i>
6       </a>
7     </div>
8     <h2>Edit Kategori</h2>
9     <p>Ubah data kategori {{ $kategori->nama }}</p>
10  </div>
11 </div>
12 </div>
13
14 <div class="max-w-2xl">
15   <div class="card">
16     <div class="card-body">
17       <form action="{{ route('kategori.update', $kategori) }}"
18         method="POST">
19         @csrf @method('PUT')
20
21       <div class="input-group">
22         <span class="icon"><i class="fas fa-tag"></i></span>
23         <input type="text" name="nama"
24           value="{{ old('nama', $kategori->nama) }}"
25           class="form-control">
26       </div>
27
28       <div class="input-group" style="margin-top:22px">
29         <span class="icon"><i class="fas fa-align-left"></i></span>
30         <textarea name="deskripsi" rows="4"
31           class="form-control">{{ old('deskripsi', $kategori->deskripsi) }}</textarea>
32       </div>
33
34       <div class="flex gap-3" style="margin-top:22px">
35         <button type="submit" class="btn btn-primary">
36           <i class="fas fa-save"></i> Update
37         </button>
38         <a href="{{ route('kategori.index') }}" class="btn btn-outline">Batal</a>
39       </div>
40     </form>
41   </div>
42 </div>
43 </div>
44 </x-app-layout>
```

Route resource

```
routes/web.php

1 <?php
2
3 use App\Http\Controllers\KategoriController;
4
5 Route::middleware(['auth', 'verified'])->group(function () {
6   Route::resource('kategori', KategoriController::class);
7   // Ekuivalen dengan:
8   // GET /kategori -> index (nama: kategori.index)
9   // GET /kategori/create -> create (nama: kategori.create)
10  // POST /kategori -> store (nama: kategori.store)
11  // GET /kategori/{kategori} -> show (nama: kategori.show)
12  // GET /kategori/{kategori}/edit -> edit (nama: kategori.edit)
13  // PUT /kategori/{kategori} -> update (nama: kategori.update)
14  // DELETE /kategori/{kategori} -> destroy (nama: kategori.destroy)
15 });
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman daftar kategori

INVENTARIS
UNIVERSITAS

MENU

- Dashboard
- Kategori**
- Barang
- Transaksi

LAPORAN

- Barang PDF
- Transaksi PDF
- Barang Excel
- Transaksi Excel

AKUN

- Profile

Admin Inventaris
admin

Wednesday, 23 July 2026

Kategori
Kelola data kategori barang

+ Tambah Kategori

No	Nama	Deskripsi	Jml Barang
1	Komputer	Semua jenis komputer dan laptop	
2	Elektronik	Peralatan elektronik	1
3	Perabotan	Meja, kursi, dan lemari	2
4	Alat Tulis	Spidol, penghapus, kertas	1

Form tambah kategori

INVENTARIS
UNIVERSITAS

MENU

- Dashboard
- Kategori**
- Barang
- Transaksi

LAPORAN

- Barang PDF
- Transaksi PDF
- Barang Excel
- Transaksi Excel

AKUN

- Profile

Admin Inventaris
admin

Wednesday, 23 July 2026

Tambah Kategori
Buat kategori barang baru

Tambah Kategori Baru

Nama Kategori:

Deskripsi:

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Form edit kategori

 **INVENTARIS**
UNIVERSITAS

MENU

[Dashboard](#)

[Kategori](#)

[Barang](#)

[Transaksi](#)

LAPORAN

[Barang PDF](#)

[Transaksi PDF](#)

[Barang Excel](#)

[Transaksi Excel](#)

AKUN

 **Admin Inventaris**
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Edit Kategori
Ubah data kategori

Edit Kategori

Nama Kategori

Komputer

Deskripsi

Semua jenis komputer

Simpan

Batal

Notifikasi sukses tambah kategori

 **INVENTARIS**
UNIVERSITAS

MENU

[Dashboard](#)

[Kategori](#)

[Barang](#)

[Transaksi](#)

LAPORAN

[Barang PDF](#)

[Transaksi PDF](#)

[Barang Excel](#)

[Transaksi Excel](#)

AKUN

 **Admin Inventaris**
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Kategori
Kelola data kategori barang

+ Tambah Kategori

Kategori berhasil ditambahkan

No	Nama	Deskripsi	Jml Barang
1	Alat Tulis	Spidol, penghapus, kertas	1
2	Elektronik	Peralatan elektronik	1
3	Komputer	Semua jenis komputer dan laptop	0
4	Furnitur	Barang furniture kantor	0

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat CRUD untuk data master Kategori

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 14. CRUD Kategori – Update dan Delete

Materi Pembelajaran : Form edit, validasi, delete
Acara Praktikum/Praktik : Minggu 7/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi (TB430805.3)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.0 20.02	Menggunakan SQL	• Mempersiapkan perangkat lunak aplikasi data deskripsi/SQL • Menggunakan fitur aplikasi SQL • Mengisi table • Melakukan operasi relasional • Membuat stored procedure • Membuat function • Membuat triger • Melakukan perintah commit dan rollback
2	J.620100.0 21.02	Menerapkan Akses Basis Data	• Membuat berbagai operasi terhadap basis data • Membuat prosedur akses terhadap basis data • Membuat prosedur akses terhadap basis data • Menguji program basis data

c. Indikator Penilaian

Mahasiswa mampu membuat fitur Update dan Delete Kategori

d. Dasar Teori

Dalam teori pengembangan aplikasi berbasis data, Update dan Delete merupakan dua operasi penting dalam siklus CRUD (Create, Read, Update, Delete) yang digunakan untuk mengelola informasi dalam sistem.

Update adalah proses memperbarui atau mengubah data yang sudah ada di dalam database. Operasi ini dilakukan ketika informasi yang tersimpan perlu disesuaikan dengan kondisi terbaru, misalnya memperbarui alamat pengguna,

status pesanan, atau nilai suatu atribut. Secara teknis, update dilakukan dengan perintah SQL UPDATE atau melalui ORM seperti Eloquent di Laravel, yang memungkinkan perubahan data dilakukan secara aman dan terstruktur. Teori update menekankan pada prinsip data consistency, yaitu menjaga agar data yang tersimpan selalu relevan dan sesuai dengan keadaan sebenarnya.

Delete adalah proses menghapus data dari sistem atau database. Operasi ini dilakukan ketika suatu informasi sudah tidak diperlukan lagi, atau demi menjaga integritas sistem, misalnya menghapus akun yang tidak aktif, data transaksi yang salah, atau file yang tidak relevan. Secara teknis, delete dilakukan dengan perintah SQL DELETE atau melalui ORM. Teori delete menekankan pada prinsip data integrity, yaitu memastikan bahwa penghapusan data tidak menimbulkan inkonsistensi atau kerusakan pada hubungan antar tabel.

Secara konseptual, Update dan Delete melengkapi siklus CRUD dengan memberikan kemampuan untuk memelihara kualitas data. Update menjaga agar data tetap akurat dan mutakhir, sedangkan Delete memastikan sistem tidak terbebani oleh data yang tidak relevan. Keduanya mendukung pengelolaan informasi yang efisien, konsisten, dan sesuai kebutuhan organisasi maupun pengguna. Fitur update dan delete untuk data kategori menggunakan Form Request Validation. Update mengubah data yang sudah ada, delete menghapus data dengan konfirmasi.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Langkah 1: Membuat Form Request KategoriRequest

Buat class Form Request untuk validasi data kategori menggunakan perintah artisan:

```
php artisan make:request KategoriRequest
```

```
terminal
1
2 $ php artisan make:request KategoriRequest
3 Request created successfully.
4
5 File: app/Http/Requests/KategoriRequest.php

PHP 8.2  Laravel  Blade
```

Setelah file dibuat, edit file `app/Http/Requests/KategoriRequest.php`.

Implementasikan method `rules()` untuk validasi input dan method `messages()` untuk kustom pesan error:

```
app/Http/Requests/KategoriRequest.php
1 <?php
2
3 namespace App\Http\Requests;
4
5 use Illuminate\Foundation\Http\FormRequest;
6
7 class KategoriRequest extends FormRequest
8 {
9     public function authorize(): bool
10     {
11         return true;
12     }
13
14     public function rules(): array
15     {
16         return [
17             'nama' => 'required|string|max:100',
18             'deskripsi' => 'nullable|string',
19         ];
20     }
21
22     public function messages(): array
23     {
24         return [
25             'nama.required' => 'Nama kategori wajib diisi',
26             'nama.max' => 'Nama kategori maksimal 100 karakter',
27         ];
28     }
29 }
```

Penjelasan validasi:

- `nama => required|string|max:100` — Nama kategori wajib diisi (required), bertipe string, maksimal 100 karakter
- `deskripsi => nullable|string` — Deskripsi bersifat opsional (nullable), bertipe string
- Method `authorize()` mengembalikan `true` agar semua user dapat mengakses request ini

Langkah 2: Implementasi Method `edit()` dan `update()`

Buka file `app/Http/Controllers/KategoriController.php`. Implementasikan method `edit()` yang menerima parameter Kategori `$kategori` (menggunakan Route Model Binding) dan mengembalikan view edit dengan data kategori:

```
app/Http/Controllers/KategoriController.php

1 class KategoriController extends Controller
2 {
3     public function edit(Kategori $kategori)
4     {
5         return view('pages.kategori.edit', compact('kategori'));
6     }
7
8     public function update(KategoriRequest $request, Kategori $kategori)
9     {
10        $kategori->update($request->validated());
11
12        return redirect()->route('kategori.index')
13            ->with('success', 'Kategori berhasil diupdate');
14    }
15
16    public function destroy(Kategori $kategori)
17    {
18        if ($kategori->barang()->count() > 0) {
19            return back()->with('error', 'Kategori tidak bisa
20                dihapus karena masih memiliki barang');
21        }
22
23        $kategori->delete();
24
25        return redirect()->route('kategori.index')
26            ->with('success', 'Kategori berhasil dihapus');
27    }
28 }
```

PHP 8.2 Laravel Blade

Penjelasan method:

- `edit(Kategori $kategori)` — Menggunakan Route Model Binding, Laravel otomatis mencari Kategori berdasarkan ID dari URL
- `update(KategoriRequest $request, ...)` — Menggunakan dependency injection `KategoriRequest`, validasi dijalankan otomatis sebelum method dieksekusi
- `destroy(Kategori $kategori)` — Mengecek relasi barang sebelum delete untuk mencegah error foreign key constraint
- Menggunakan `route()->with('success', ...)` untuk flash message setelah operasi berhasil

Langkah 3: Implementasi Method `destroy()` dengan Pengecekan Relasi

Method `destroy()` memerlukan penanganan khusus karena data kategori mungkin memiliki relasi dengan tabel barang:

```
app/Http/Controllers/KategoriController.php

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Http\Requests\KategoriRequest; // Import FormRequest
6  use App\Models\Kategori;
7
8  class KategoriController extends Controller
9  {
10     public function update(KategoriRequest $request, Kategori $kategori)
11     {
12         // $request->validated() otomatis menjalankan rules()
13         // dari KategoriRequest, tanpa perlu validate() manual
14         $kategori->update($request->validated());
15
16         return redirect()->route('kategori.index')
17             ->with('success', 'Kategori berhasil diupdate');
18     }
19
20     public function destroy(Kategori $kategori)
21     {
22         // Cek relasi sebelum delete
23         if ($kategori->barangs()->count() > 0) {
24             return back()->with('error',
25                 'Tidak bisa dihapus, masih memiliki barang');
26         }
27         $kategori->delete();
28         return redirect()->route('kategori.index')
29             ->with('success', 'Kategori berhasil dihapus');
30     }
31 }
```

Alur pengecekan:

- Hitung jumlah barang yang terkait dengan kategori: `$kategori->barangs()->count()`
- Jika masih ada barang, redirect back dengan error message
- Jika tidak ada barang, hapus kategori dengan `$kategori->delete()`
- Redirect ke index dengan success message

Langkah 4: Membuat View `edit.blade.php`

Buat file `resources/views/pages/kategori/edit.blade.php`. Form menggunakan `method="POST"` dengan `@method('PUT')` untuk mensimulasikan method PUT. Input diisi dengan data existing menggunakan `old()` dan `$kategori`:

```
resources/views/pages/kategori/edit.blade.php

1  <x-app-layout>
2    <div class="page-header">
3      <div class="flex items-center gap-4">
4        <a href="{{ route('kategori.index') }}" class="btn" ...>
5          <i class="fas fa-arrow-left"></i> Kembali
6        </a>
7      </div>
8      <h2>Edit Kategori</h2>
9      <p>Ubah data kategori : {{ $kategori->nama }}</p>
10     </div>
11   </div>
12 </div>
13
14 <div class="max-w-2xl">
15   <div class="card">
16     <div class="card-body">
17       <form action="{{ route('kategori.update', $kategori) }}"
18         method="POST">
19         @csrf @method('PUT')
20
21       <label class="form-label">Nama Kategori</label>
22       <div class="input-group">
23         <span class="icon"><i class="fas fa-tag"></i></span>
24         <input type="text" name="nama"
25           value="{{ old('nama', $kategori->nama) }}"
26           class="form-control">
27       </div>
28       @error('nama')
29       <p style="color:#dc2626;font-size:12px">
30         <i class="fas fa-exclamation-circle"></i> {{ $message }}</p>
31       @enderror
32
33       <label class="form-label" style="margin-top:22px">Deskripsi</label>
34       <div class="input-group">
35         <textarea name="deskripsi" rows="4" class="form-control">
36           {{ old('deskripsi', $kategori->deskripsi) }}</textarea>
37       </div>
38
39       <div class="flex gap-3" style="margin-top:22px;padding-top:22px;
40         border-top:1px solid #e8ecf1">
41         <button type="submit" class="btn btn-primary">
42           <i class="fas fa-save"></i> Update
43         </button>
44         <a href="{{ route('kategori.index') }}" class="btn btn-outline">
45           Batal
46         </a>
47       </div>
48     </form>
49   </div>
50 </div>
51 </div>
52 </x-app-layout>
```

Poin penting dalam view edit:

- @method('PUT') — Laravel membutuhkan method spoofing untuk PUT/PATCH/DELETE
- value="{{ old('nama', \$kategori->nama) }}" — old() untuk menampilkan input sebelumnya jika validasi gagal, fallback ke data existing
- @error('nama') directive untuk menampilkan pesan error validasi per field
- Form action="{{ route('kategori.update', \$kategori) }}" — Route Model Binding di URL

Langkah 5: Menambahkan Tombol Edit dan Delete di Index

Edit file `resources/views/pages/kategori/index.blade.php`. Tambahkan kolom aksi dengan tombol Edit (link ke edit) dan tombol Delete (form dengan `confirm()`):

```
resources/views/pages/kategori/index.blade.php

1      @forelse ($kategoris as $ket)
2          <tr>
3              <td>{{ $loop->iteration }}</td>
4              <td>{{ $ket->nama }}</td>
5              <td>{{ $ket->deskripsi ?? '-' }}</td>
6              <td>{{ $ket->barangs_count }}</td>
7              <td>
8                  <div class="action-group justify-end">
9                      <!-- TOMBOL EDIT -->
10                     <a href="{{ route('kategori.edit', $ket) }}"
11                         class="action-btn"
12                         style="background:#fffbeb;color:#d97706">
13                         <i class="fas fa-edit"></i> Edit
14                     </a>
15
16                     <!-- TOMBOL DELETE dengan KONFIRMASI -->
17                     <form action="{{ route('kategori.destroy', $ket) }}"
18                         method="POST"
19                         onsubmit="return confirm
20                             ('Yakin ingin menghapus kategori
21                              {{ $ket->nama }}?')" style="display:inline">
22                         @csrf @method('DELETE')
23                         <button type="submit" class="action-btn"
24                             style="background:#fef2f2;color:#dc2626">
25                             <i class="fas fa-trash"></i> Hapus
26                         </button>
27                     </form>
28                 </div>
29             </td>
30          </tr>
31      @empty
32          <tr><td colspan="5">
33              <div class="empty-state">
34                  <i class="fas fa-tags"></i>
35                  <p>Belum ada data kategori</p>
36              </div>
37          </td></tr>
38      @endforelse
```

Penjelasan tombol Delete:

- Menggunakan `<form>` dengan method POST + `@method('DELETE')` untuk HTTP DELETE
- `onsubmit="return confirm('...')"` — JavaScript confirm dialog, mengembalikan true/false
- Jika user klik OK, form akan disubmit (method `destroy()` dijalankan)
- Jika user klik Cancel, form tidak disubmit (tidak ada aksi)
- Setiap baris memiliki form sendiri dengan action unik sesuai ID kategori

Langkah 6: Menambahkan Flash Messages

Agar notifikasi sukses/error dari controller ditampilkan di view index, tambahkan flash message handler di bagian atas view `index.blade.php`:

```
resources/views/pages/kategori/index.blade.php

resources/views/pages/kategori/index.blade.php
1 <x-app-layout>
2 <!-- FLASH MESSAGES -->
3 @if (session('success'))
4 <div class="alert alert-success">
5 <i class="fas fa-check-circle"></i>
6 {{ session('success') }}
7 </div>
8 @endif
9
10 @if (session('error'))
11 <div class="alert alert-error">
12 <i class="fas fa-exclamation-circle"></i>
13 {{ session('error') }}
14 </div>
15 @endif
16
17 <div class="page-header">
18 <h2>Kategori</h2>
19 <p>Kelola data kategori barang</p>
20 <a href="{{ route('kategori.create') }}" class="btn btn-primary">
21 <i class="fas fa-plus"></i> Tambah Kategori
22 </a>
23 </div>
24 </x-app-layout>
```

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Form edit, validasi, delete". Analisis bagaimana konsep crud kategori - update dan delete diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Uji Coba Edit Kategori

Jalankan server dan akses halaman kategori. Klik tombol Edit pada salah satu kategori:

INVENTARIS
UNIVERSITAS

WEDNESDAY, 23 JULY 2026

Admin Inventaris
admin

Edit Kategori
Form edit dengan data lama

Edit Kategori

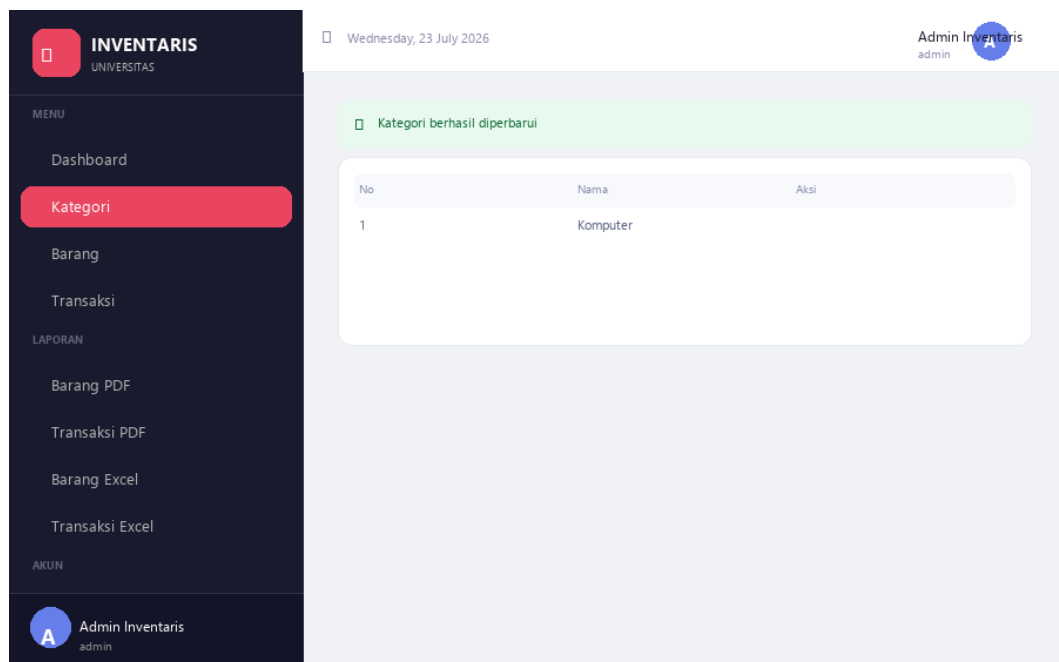
Nama:

Deskripsi:

Admin Inventaris
admin

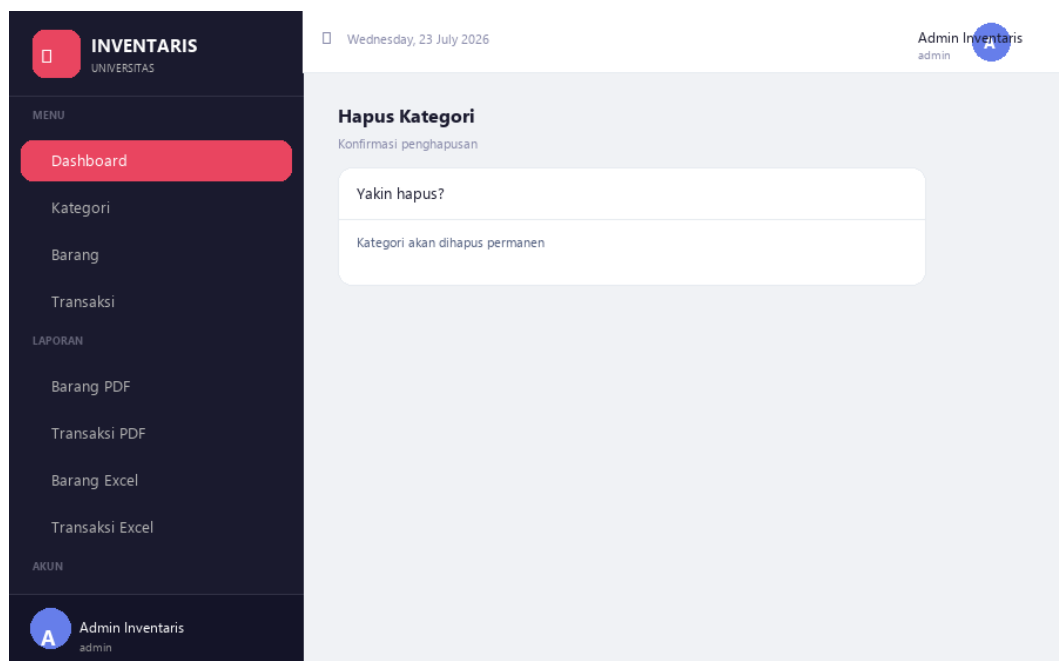
BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Ubah data dan klik tombol Update. Jika berhasil, akan muncul notifikasi:



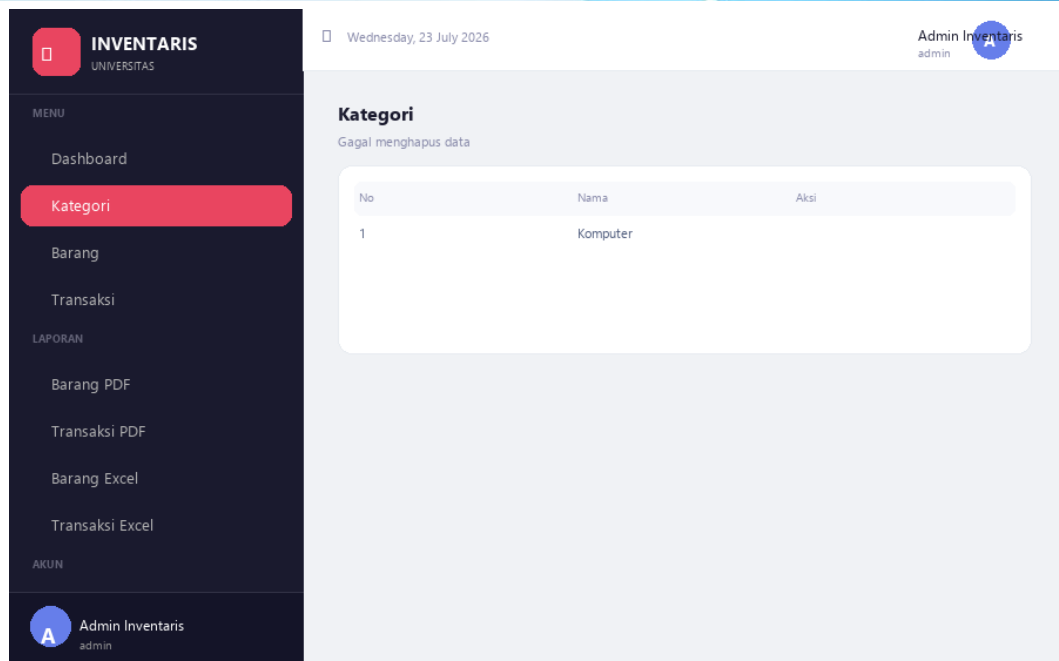
Uji Coba Hapus Kategori

Klik tombol Hapus pada kategori yang ingin dihapus. Akan muncul konfirmasi JavaScript:

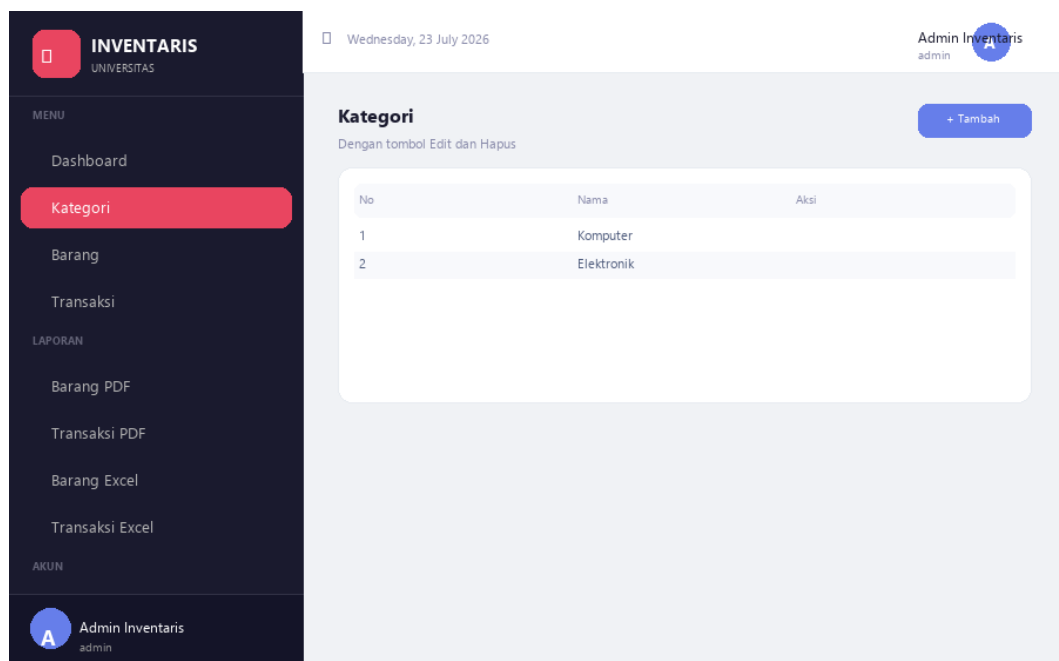


Jika kategori masih memiliki relasi barang, akan muncul pesan Gagal:

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)



Jika kategori tidak memiliki relasi, data akan berhasil dihapus dan redirect ke halaman index.



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat Update dan Delete CRUD kategori

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

Acara 15. CRUD Barang dengan Relasi Kategori

Materi Pembelajaran : Relasi belongsTo, form select
Acara Praktikum/Praktik : Minggu 8/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi (TB430805.3)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.0 20.02	Menggunakan SQL	• Mempersiapkan perangkat lunak aplikasi data deskripsi/SQL • Menggunakan fitur aplikasi SQL • Mengisi table • Melakukan operasi relasional • Membuat stored procedure • Membuat function • Membuat trigger • Melakukan perintah commit dan rollback
2	J.620100.0 21.02	Menerapkan Akses Basis Data	• Membuat berbagai operasi terhadap basis data • Membuat prosedur akses terhadap basis data • Membuat prosedur akses terhadap basis data • Menguji program basis data

c. Indikator Penilaian

Mahasiswa mampu membuat CRUD Barang dengan relasi

d. Dasar Teori

Barang memiliki relasi belongsTo ke Kategori dan hasMany ke Transaksi. Setiap barang terkait dengan satu kategori. Form tambah barang menyertakan select option kategori.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Langkah 1: Membuat Migration Tabel Barang

Buat migration untuk tabel barangs yang memiliki foreign key ke tabel kategoris:

php artisan make:migration create_barangs_table

```
terminal
1
2 $ php artisan make:migration create_barangs_table
3 Created Migration: database/migrations/2026_07_16_000002_create_barangs_table.php
4
5 $ php artisan migrate
6 Migrating: 2026_07_16_000002_create_barangs_table
7 Migrated: 2026_07_16_000002_create_barangs_table

PHP 8.2 Laravel Blade
```

Edit file migration yang telah dibuat. Tambahkan kolom foreignId untuk relasi ke tabel kategoris:

```
database/migrations/xxx_create_barangs_table.php
1 <?php
2
3 use Illuminate\Database\Migrations\Migration;
4 use Illuminate\Database\Schema\Blueprint;
5 use Illuminate\Support\Facades\Schema;
6
7 return new class extends Migration
8 {
9     public function up(): void
10     {
11         Schema::create('barangs', function (Blueprint $table) {
12             $table->id();
13             $table->foreignId('kategori_id')
14                 ->constrained('kategoris')
15                 ->onDelete('cascade');
16             $table->string('nama', 150);
17             $table->integer('jumlah')->default(0);
18             $table->enum('kondisi', ['baik', 'rusak', 'hilang'])
19                 ->default('baik');
20             $table->text('deskripsi')->nullable();
21             $table->string('gambar')->nullable();
22             $table->timestamps();
23         });
24     }
25
26     public function down(): void
27     {
28         Schema::dropIfExists('barangs');
29     }
30 }
```

Kolom penting:

- `foreignId('kategori_id')->constrained('kategoris')->onDelete('cascade')` — Foreign key ke tabel kategoris dengan cascade delete
- `enum('kondisi', ['baik','rusak','hilang'])` — Kolom dengan nilai terbatas (enum)
- `string('gambar')->nullable()` — Kolom opsional untuk path gambar

Jalankan migration:

`php artisan migrate`

Langkah 2: Membuat Model Barang

Buat model Barang menggunakan perintah artisan:

`php artisan make:model Barang`

```
terminal
1
2 $ php artisan make:model Barang
3 Model created successfully.
4
5 $ php artisan make:controller BarangController --resource
6 Controller created successfully.
7
8 File: app/Models/Barang.php
9 File: app/Http/Controllers/BarangController.php

PHP 8.2  Laravel  Blade
```

Setelah file dibuat, tambahkan relasi `belongsTo` ke model Kategori:

```
app/Models/Barang.php
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Database\Eloquent\Model;
7 use Illuminate\Database\Eloquent\Relations\BelongsTo;
8 use Illuminate\Database\Eloquent\Relations\HasMany;
9
10 class Barang extends Model
11 {
12     use HasFactory;
13
14     protected $table = 'barangs';
15
16     protected $fillable = [
17         'kategori_id', 'nama', 'jumlah',
18         'kondisi', 'deskripsi', 'gambar',
19     ];
20
21     // Relasi: Barang belongsTo Kategori
22     public function kategori(): BelongsTo
23     {
24         return $this->belongsTo(Kategori::class);
25     }
26
27     public function transaksis(): HasMany
28     {
29         return $this->hasMany(Transaksi::class);
30     }
31 }
```

Penjelasan:

- \$fillable — Kolom yang boleh diisi mass assignment (kategori_id, nama, jumlah, kondisi, deskripsi, gambar)
- kategori(): BelongsTo — Mendefinisikan relasi belongsTo ke model Kategori
- \$this->belongsTo(Kategori::class) — Satu barang memiliki satu kategori
- transaksis(): HasMany — Relasi ke model Transaksi untuk pengecekan sebelum delete

Langkah 3: Membuat BarangController

Buat controller dengan resource method:

php artisan make:controller BarangController --resource

Method index()

Implementasi method index dengan eager loading kategori dan fitur filter:

```
app/Http/Controllers/BarangController.php
1 class BarangController extends Controller
2 {
3     public function index(Request $request)
4     {
5         $query = Barang::with('kategori');
6
7         if ($request->filled('kategori')) {
8             $query->where('kategori_id', $request->kategori);
9         }
10
11         if ($request->filled('search')) {
12             $query->where('nama', 'like', "%{$request->search}%");
13         }
14
15         $barangs = $query->paginate(10)->withQueryString();
16         $kategoris = Kategori::all();
17
18         return view('pages.barang.index',
19             compact('barangs', 'kategoris'));
20     }
}
```

- Barang::with('kategori') — Eager loading relasi kategori untuk mencegah N+1 query
- where('kategori_id', ...) — Filter berdasarkan kategori
- paginate(10) — Paginasi 10 data per halaman

Method create() dan store()

Method create mengirim data kategori ke view untuk dropdown. Method store memvalidasi data termasuk kategori_id:

```
app/Http/Controllers/BarangController.php

1 public function create()
2 {
3     $kategoris = Kategori::all();
4     return view('pages.barang.create',
5         compact('kategoris'));
6 }
7
8 public function store(Request $request)
9 {
10     $request->validate([
11         'kategori_id' => 'required|exists:kategoris,id',
12         'nama' => 'required|string|max:150',
13         'jumlah' => 'required|integer|min:0',
14         'kondisi' => 'required|in:baik,rusak,hilang',
15         'gambar' => 'nullable|image|mimes:jpeg,png|max:2048',
16     ]);
17
18     $data = $request->except('gambar');
19     if ($request->hasFile('gambar')) {
20         $data['gambar'] = $request->file('gambar')
21             ->store('barang', 'public');
22     }
23
24     Barang::create($data);
25     return redirect()->route('barang.index')
26         ->with('success', 'Barang berhasil ditambahkan');
27 }
```

- create() — Mengirim Kategori::all() ke view untuk select dropdown
- store() — Validasi kategori_id menggunakan rule exists:kategoris,id
- Upload gambar ke storage/public/barang menggunakan storeAs()
- Redirect ke index dengan flash message success

Method edit(), update(), destroy()

```
app/Http/Controllers/BarangController.php

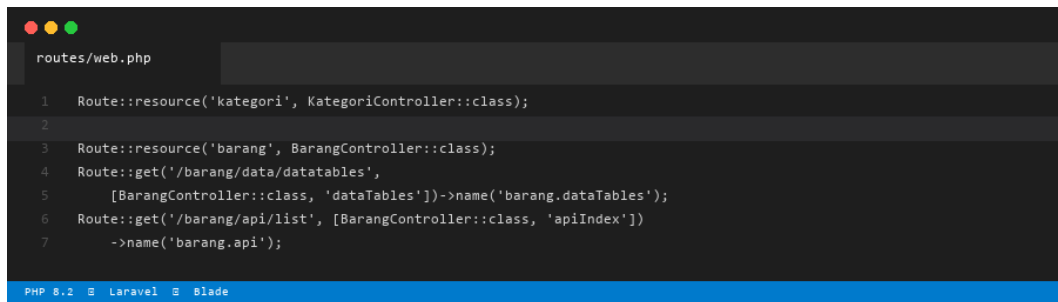
1 public function edit(Barang $barang)
2 {
3     $kategoris = Kategori::all();
4     return view('pages.barang.edit',
5         compact('barang', 'kategoris'));
6 }
7
8 public function update(Request $request, Barang $barang)
9 {
10     $request->validate([
11         'kategori_id' => 'required|exists:kategoris,id',
12         'nama' => 'required|string|max:150',
13     ]);
14     $barang->update($request->except('gambar'));
15     return redirect()->route('barang.index')
16         ->with('success', 'Barang berhasil diupdate');
17 }
18
19 public function destroy(Barang $barang)
20 {
21     if ($barang->transaksis(
22         )->where('status','dipinjam')->count())
23         return back()->with('error', 'Barang sedang dipinjam');
24     $barang->delete();
25     return redirect()->route('barang.index')
26         ->with('success', 'Barang berhasil dihapus');
27 }
```

- Route Model Binding (Barang \$barang) — Laravel otomatis mencari data berdasarkan ID
- exists:kategoris,id — Memastikan kategori_id yang diupdate masih valid
- destroy — Mengecek transaksi dengan status dipinjam sebelum delete

Langkah 4: Menambahkan Route Resource

Tambahkan route resource di routes/web.php:

`Route::resource('barang', BarangController::class);`



```
routes/web.php
1 Route::resource('kategori', KategoriController::class);
2
3 Route::resource('barang', BarangController::class);
4 Route::get('/barang/data/datatables',
5     [BarangController::class, 'dataTables'])->name('barang.dataTables');
6 Route::get('/barang/api/list', [BarangController::class, 'apiIndex'])
7     ->name('barang.api');
```

Route::resource akan menghasilkan 7 route:

- GET /barang → index()
- GET /barang/create → create()
- POST /barang → store()
- GET /barang/{barang} → show()
- GET /barang/{barang}/edit → edit()
- PUT/PATCH /barang/{barang} → update()
- DELETE /barang/{barang} → destroy()

Langkah 5: Membuat View Create dengan Select Kategori

Buat file resources/views/pages/barang/create.blade.php. Tambahkan select dropdown untuk memilih kategori:

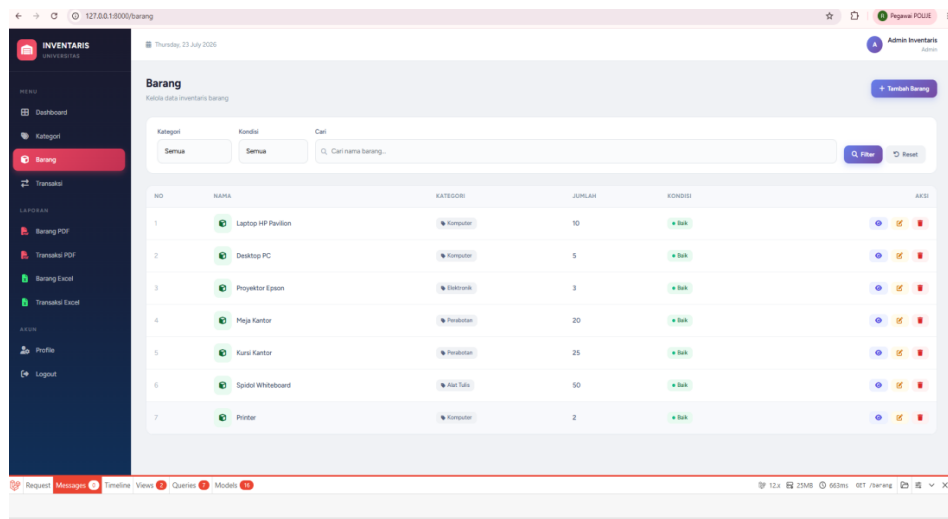
BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

```
resources/views/pages/barang/create.blade.php

1 <x-app-layout>
2 <form action="{{ route('barang.store') }}"
3   method="POST" enctype="multipart/form-data">
4   @csrf
5   <div style="display:grid;grid-template-columns:1fr 1fr;gap:22px">
6     <!-- NAMA BARANG -->
7     <div>
8       <label class="form-label">Nama Barang</label>
9       <div class="input-group">
10        <span class="icon"><i class="fas fa-box"></i></span>
11        <input type="text" name="nama" class="form-control"
12          placeholder="Nama barang">
13      </div>
14    </div>
15    <!-- SELECT KATEGORI (DROPDOWN) -->
16    <div>
17      <label class="form-label">Kategori</label>
18      <div class="input-group">
19        <span class="icon"><i class="fas fa-tags"></i></span>
20        <select name="kategori_id" class="form-control">
21          <option value="">Pilih Kategori</option>
22          @foreach ($kategoris as $k)
23            <option value="{{ $k->id }}"
24              {{ old('kategori_id')==$k->id?
25                'selected':'' }}>
26              {{ $k->nama }}
27            </option>
28          @endforeach
29        </select>
30      </div>
31    </div>
32    <!-- JUMLAH -->
33    <div>
34      <label class="form-label">Jumlah</label>
35      <div class="input-group">
36        <span class="icon"><i class="fas fa-hashtag"></i></span>
37        <input type="number" name="jumlah" class="form-control">
38      </div>
39    </div>
40  </div>
41 </form>
42 </x-app-layout>
```

Poin penting:

- @foreach (\$kategoris as \$k) — Looping data kategori untuk option dropdown
- value="{{ \$k->id }}" — Nilai option adalah ID kategori (foreign key)
- {{ \$k->nama }} — Tampilan yang muncul di dropdown adalah nama kategori
- old('kategori_id') — Menjaga pilihan user saat validasi gagal



Menampilkan Nama Kategori di Index

Edit file index.blade.php untuk menampilkan nama kategori dari relasi:

```
resources/views/pages/barang/index.blade.php

1      @forelse ($barangs as $b)
2      <tr>
3          <td>{{ $loop->iteration }}</td>
4          <td>
5              <div class="flex items-center gap-3">
6                  @if ($b->gambar)
7                      
10                     @endif
11                     <span style="font-weight:500">{{ $b->nama }}</span>
12                 </div>
13             </td>
14             <!-- MENAMPILKAN NAMA KATEGORI -->
15             <td>
16                 <span class="tag">
17                     <i class="fas fa-tag"></i>
18                     {{ $b->kategori->nama }}
19                 </span>
20             </td>
21             <td style="font-weight:600">{{ $b->jumlah }}</td>
22             <td>{{ $b->kondisi }}</td>
23             <td>
24                 <div class="action-group">
25                     <a href="{{ route('barang.edit', $b) }}"
26                         class="action-btn"><i class="fas fa-edit"></i></a>
27                     <form ... onsubmit="return confirm('Yakin?')">
28                         @csrf @method('DELETE')
29                         <button class="action-btn">
30                             <i class="fas fa-trash"></i></button>
31                     </form>
32                 </div>
33             </td>
34         </tr>
35     @empty ... @endforelse
```

Cara mengakses data relasi di Blade:

- `$b->kategori` — Mengakses object Kategori yang terkait
- `$b->kategori->nama` — Mengambil field nama dari kategori
- Eager loading with('kategori') memastikan data sudah di-load, tidak ada query tambahan
- Jika kategori null (data orphan), `$b->kategori` akan null dan akses `->nama` akan error

Study Kasus

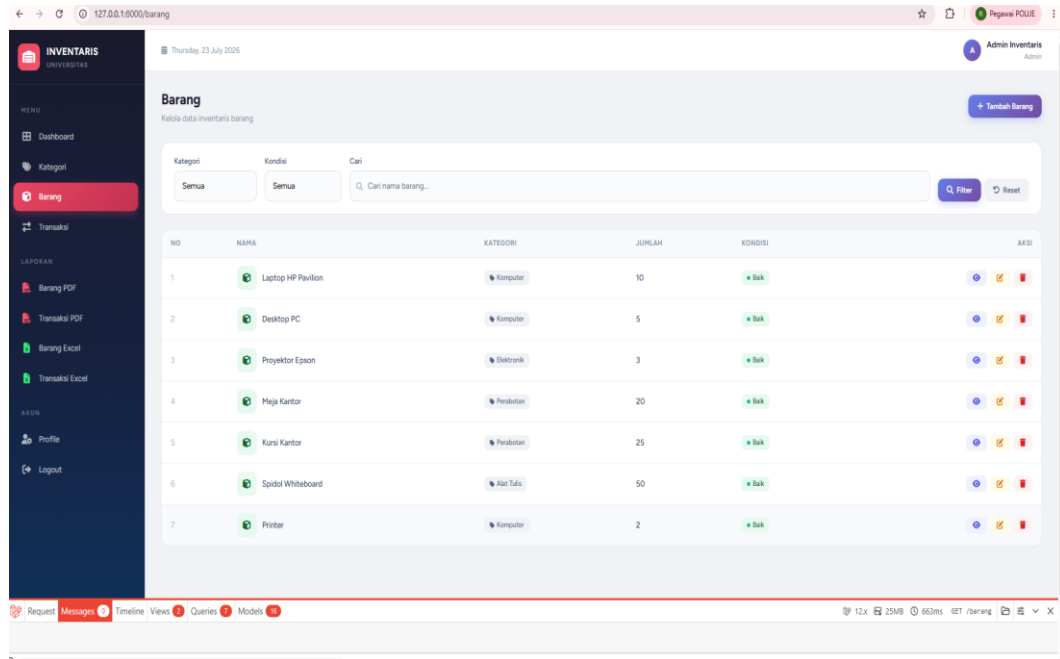
Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Relasi belongsTo, form select". Analisis bagaimana konsep crud barang dengan relasi kategori diterapkan dalam konteks manajemen inventaris barang universitas.

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

g. Hasil dan Pembahasan

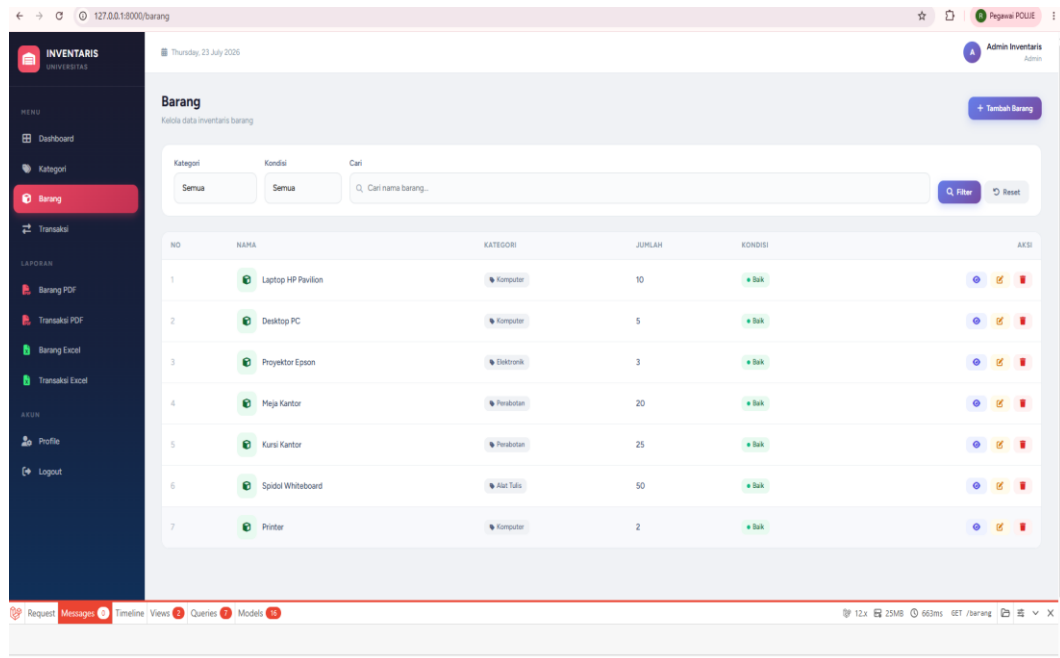
Uji Coba CRUD Barang

Jalankan server dan akses halaman barang. Data akan menampilkan nama kategori dari tabel kategoris:



ID	NAMA	KATEGORI	JUMLAH	KONDISI	Aksi
1	Laptop HP Pavilion	Komputer	10	Baik	[Edit] [Hapus] [Detail]
2	Desktop PC	Komputer	5	Baik	[Edit] [Hapus] [Detail]
3	Proyektor Epson	Elektronik	3	Baik	[Edit] [Hapus] [Detail]
4	Meja Kantor	Perabotan	20	Baik	[Edit] [Hapus] [Detail]
5	Kursi Kantor	Perabotan	25	Baik	[Edit] [Hapus] [Detail]
6	Spidol Whiteboard	Alat Tulis	50	Baik	[Edit] [Hapus] [Detail]
7	Printer	Komputer	2	Baik	[Edit] [Hapus] [Detail]

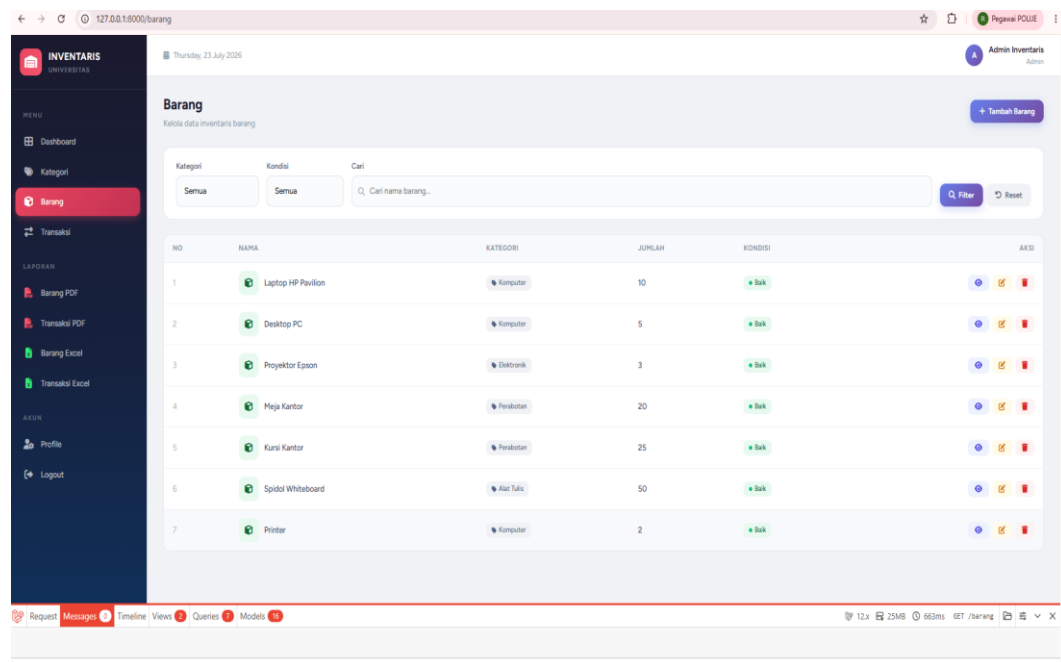
Filter data berdasarkan kategori:



ID	NAMA	KATEGORI	JUMLAH	KONDISI	Aksi
1	Laptop HP Pavilion	Komputer	10	Baik	[Edit] [Hapus] [Detail]
2	Desktop PC	Komputer	5	Baik	[Edit] [Hapus] [Detail]
3	Proyektor Epson	Elektronik	3	Baik	[Edit] [Hapus] [Detail]
4	Meja Kantor	Perabotan	20	Baik	[Edit] [Hapus] [Detail]
5	Kursi Kantor	Perabotan	25	Baik	[Edit] [Hapus] [Detail]
6	Spidol Whiteboard	Alat Tulis	50	Baik	[Edit] [Hapus] [Detail]
7	Printer	Komputer	2	Baik	[Edit] [Hapus] [Detail]

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Detail barang menampilkan informasi lengkap:



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat CRUD Barang dengan relasi belongsTo

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusa	V			
Total Skor*		Skor x 25			

Acara 16. Upload Gambar Barang

Materi Pembelajaran : File upload, storage, validasi file
 Acara Praktikum/Praktik : Minggu 8/2
 Tempat : Daring/Luring Politeknik Negeri Jember
 Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi (TB430805.3)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritikal yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources

4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing • Membuat program perangkat lunak penambahan • diferensiasi
---	-----------------	---------------------------------------	---

c. Indikator Penilaian

Mahasiswa mampu mengimplementasikan upload gambar barang

d. Dasar Teori

Dalam teori pengembangan aplikasi web, upload image atau file adalah proses pengiriman berkas dari sisi pengguna (client) ke server agar dapat disimpan dan digunakan dalam sistem. Proses ini biasanya dilakukan melalui form input atau API dengan metode HTTP POST, di mana file dikirim sebagai bagian dari request. Setelah diterima, server akan melakukan validasi terhadap file tersebut, misalnya memeriksa ukuran, tipe, dan keamanan, sebelum menyimpannya ke direktori tertentu atau ke layanan penyimpanan eksternal seperti cloud storage. Informasi mengenai file yang diunggah, seperti nama, path, dan ukuran, biasanya dicatat dalam database agar mudah diakses kembali. Secara teoritis, upload file mendukung prinsip data persistence karena memastikan data tersimpan secara permanen, serta security management karena harus melibatkan validasi untuk mencegah file berbahaya masuk ke sistem. Dengan demikian, upload image/file menjadi mekanisme penting dalam aplikasi modern, baik untuk menyimpan foto profil, dokumen, maupun data pendukung lain, yang menjamin interaksi pengguna lebih dinamis, aman, dan bermanfaat.

Setiap barang dapat memiliki gambar. Laravel Storage menyimpan file di storage/app/public. Validasi file: tipe gambar (jpeg, png, jpg, gif), maksimal 2MB.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja**Langkah 1: Konfigurasi Storage Link**

Jalankan perintah artisan untuk membuat symbolic link storage:

```
php artisan storage:link
```

```
terminal
1
2 $ php artisan storage:link
3 The [public/storage] directory has been linked.
4
5 $ ls -la public/storage
6 lrwxrwxrwx -> ../../storage/app/public

PHP 8.2 Laravel Blade
```

Setelah perintah dijalankan, folder public/storage akan menjadi link ke storage/app/public. Semua file yang disimpan di storage/app/public dapat diakses melalui URL /storage/...

Langkah 2: Implementasi Upload di Method store()

Buka file BarangController.php dan tambahkan logika upload gambar di method store():

```
app/Http/Controllers/BarangController.php
1 public function store(Request $request)
2 {
3     $request->validate([
4         'kategori_id' => 'required|exists:kategoris,id',
5         'nama' => 'required|string|max:150',
6         'gambar' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
7     ]);
8
9     $data = $request->except('gambar');
10
11     // PROSES UPLOAD GAMBAR
12     if ($request->hasFile('gambar')) {
13         $file = $request->file('gambar');
14         $filename = time() . '_' . $file->getClientOriginalName();
15         $file->storeAs('public/barang', $filename);
16         $data['gambar'] = 'storage/barang/' . $filename;
17     }
18
19     Barang::create($data);
20     return redirect()->route('barang.index')
21         ->with('success', 'Barang berhasil ditambahkan');
22 }
```

Penjelasan kode upload:

- `$request->hasFile('gambar')` — Mengecek apakah user mengupload file
- `$file = $request->file('gambar')` — Mengambil object file yang diupload
- `time() . '_' . $file->getClientOriginalName()` — Membuat nama unik dengan timestamp + nama asli
- `$file->storeAs('public/barang', $filename)` — Menyimpan file ke storage/app/public/barang/
- `$data['gambar'] = 'storage/barang/' . $filename` — Path relatif untuk akses publik via URL
- Data gambar disimpan di database sebagai string path, bukan file binary

Langkah 3: Implementasi Upload di Method update()

Tambahkan logika upload yang sama di method update() untuk menangani perubahan gambar:

```
app/Http/Controllers/BarangController.php

1 public function update(Request $request, Barang $barang)
2 {
3     $request->validate([
4         'gambar' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
5     ]);
6
7     $data = $request->except('gambar');
8
9     // PROSES UPLOAD GAMBAR SAAT UPDATE
10    if ($request->hasFile('gambar')) {
11        $file = $request->file('gambar');
12        $filename = time() . '_' . $file->getClientOriginalName();
13        $file->storeAs('public/barang', $filename);
14        $data['gambar'] = 'storage/barang/' . $filename;
15    }
16
17    $barang->update($data);
18    return redirect()->route('barang.index')
19        ->with('success', 'Barang berhasil diupdate');
20 }
```

Catatan: Saat update, file gambar lama tidak otomatis dihapus. Untuk production, sebaiknya tambahkan logika untuk menghapus file lama sebelum menyimpan yang baru menggunakan `Storage::delete()`.

Langkah 4: Validasi Upload Gambar

Validasi file gambar diperlukan untuk memastikan hanya file gambar valid yang diupload:

```
app/Http/Controllers/BarangController.php

1 $request->validate([
2     'kategori_id' => 'required|exists:kategori,id',
3     'nama' => 'required|string|max:150',
4     'jumlah' => 'required|integer|min:0',
5     'kondisi' => 'required|in:baik,rusak,hilang',
6     'gambar' => [
7         'nullable', // field opsional
8         'image', // harus file gambar
9         'mimes:jpeg,png,jpg,gif', // format yg diizinkan
10        'max:2048' // maks 2MB
11    ],
12 ]);
```

Aturan validasi:

- nullable — Field gambar bersifat opsional, tidak wajib diisi
- image — Memvalidasi bahwa file benar-benar gambar (memeriksa MIME type)
- mimes:jpeg,png,jpg,gif — Membatasi format file yang diizinkan
- max:2048 — Membatasi ukuran maksimum file 2MB (2048 KB)

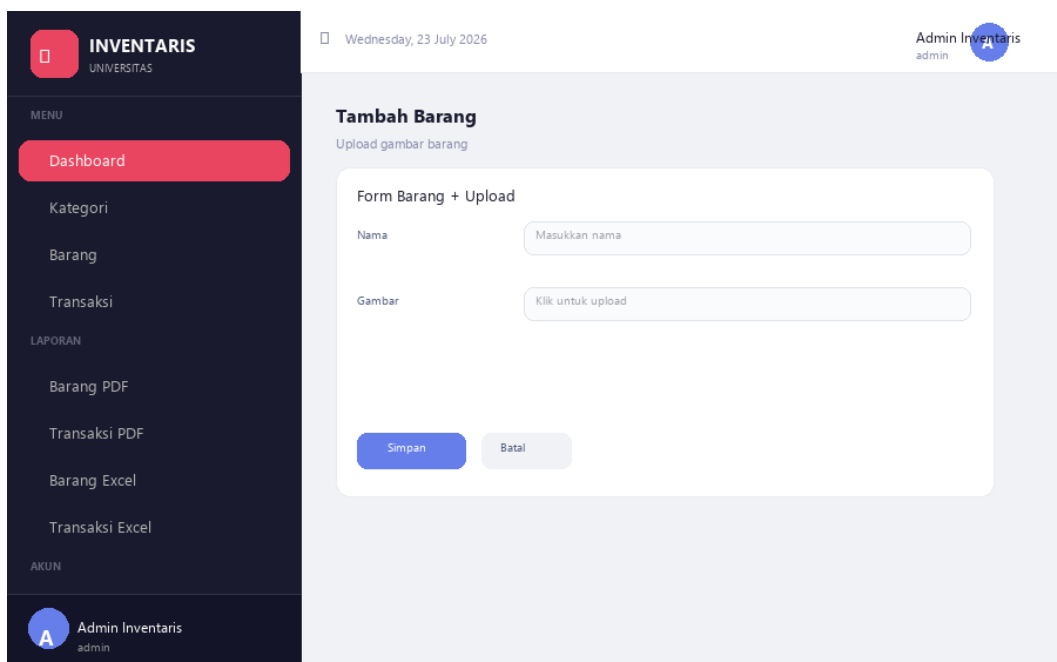
Langkah 5: Menambahkan Input File di Form Create

Edit file create.blade.php untuk menambahkan input file upload. Form harus memiliki atribut enctype="multipart/form-data":

```
resources/views/pages/barang/create.blade.php
1 <form action="{{ route('barang.store') }}"
2     method="POST" enctype="multipart/form-data">
3     @csrf
4     <!-- field lainnya: nama, kategori, jumlah, kondisi -->
5
6     <!-- INPUT FILE UPLOAD GAMBAR -->
7     <div style="margin-top:22px">
8         <label class="form-label">Gambar</label>
9         <div style="border:2px dashed #e0e4e8;border-radius:14px;padding:20px;text-align:center;position:relative">
10             <input type="file" name="gambar" accept="image/*"
11                 style="position:absolute;inset:0;opacity:0;cursor:pointer">
12             <i class="fas fa-cloud-upload-alt" style="font-size:28px;color:#d1d5db"></i>
13             <p style="font-size:13px;color:#8898aa;margin-top:4px">Klik untuk upload gambar</p>
14             <p style="font-size:11px;color:#adb5c4">Format: jpeg, png, jpg, gif. Maks: 2MB</p>
15         </div>
16         @error('gambar')
17             <p style="font-size:12px;color:#dc2626"><i class="fas fa-exclamation-circle"></i> {{ $message }}</p>
18         @enderror
19     </div>
20
21     <button type="submit" class="btn btn-primary"><i class="fas fa-save"></i> Simpan</button>
22 </form>
```

Poin penting:

- enctype="multipart/form-data" — WAJIB agar browser dapat mengirim file
- accept="image/*" — Membatasi dialog file hanya menampilkan file gambar
- input type="file" dengan opacity:0 dan inset:0 — Style untuk custom upload area
- @error('gambar') — Menampilkan pesan error validasi jika upload gagal



Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "File upload, storage, validasi file". Analisis bagaimana konsep upload gambar barang diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

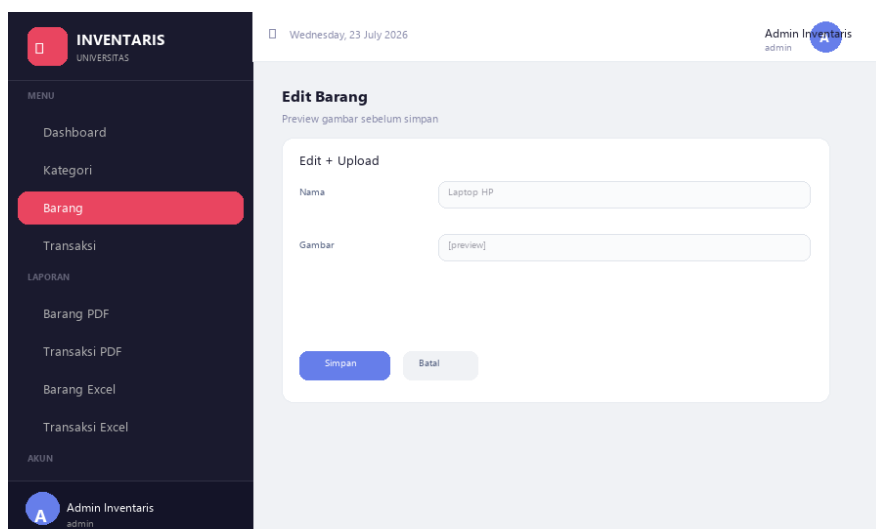
Preview Gambar Existing di Form Edit

Di view edit, tampilkan gambar yang sudah ada sebelumnya dan sediakan opsi untuk menggantinya:

```
resources/views/pages/barang/edit.blade.php
1 <form action="{{ route('barang.update', $barang) }}"
2   method="POST" enctype="multipart/form-data">
3   @csrf @method('PUT')
4
5   <!-- TAMPILKAN GAMBAR EXISTING -->
6   @if ($barang->gambar)
7     <div style="margin-bottom:10px">
8       
10     </div>
11   @endif
12
13   <!-- INPUT FILE UNTUK GANTI GAMBAR -->
14   <div style="border:2px dashed #e0e4e8;border-radius:14px;padding:20px;text-align:center;position:relative">
15     <input type="file" name="gambar" accept="image/*"
16       style="position:absolute;inset:0;opacity:0;cursor:pointer">
17     <i class="fas fa-cloud-upload-alt" style="font-size:28px;color:#d1d5db"></i>
18     <p style="font-size:13px;color:#8898aa;margin-top:4px">Klik untuk ganti gambar</p>
19   </div>
20 </form>
```

Penjelasan:

- @if (\$barang->gambar) — Mengecek apakah barang memiliki gambar
- asset(\$barang->gambar) — URL lengkap ke file gambar
- object-fit:cover — Memastikan gambar thumbnail tidak terdistorsi
- Jika user upload gambar baru, gambar lama akan ditimpa di database (path baru)



Menampilkan Thumbnail di Index

Edit file index.blade.php untuk menampilkan thumbnail gambar di samping nama barang:

```
resources/views/pages/barang/index.blade.php

1      @forelse ($barangs as $b)
2          <tr>
3              <td>{{ $loop->iteration }}</td>
4              <td>
5                  <div style="display:flex;align-items:center;gap:12px">
6                      <!-- TAMPILKAN THUMBNAIL -->
7                      @if ($b->gambar)
8                          
10                      @else
11                          <div style="width:36px;height:36px;background:#e8faf0;border-radius:8px;
12                              display:flex;align-items:center;justify-content:center;color:#0d6832">
13                              <i class="fas fa-cube"></i>
14                          </div>
15                      @endif
16                      <span style="font-weight:500;color:#344767">{{ $b->nama }}</span>
17                  </div>
18              </td>
19          </tr>
20      @endforelse
```

Poin penting:

- @if (\$b->gambar) — Mengecek apakah barang memiliki gambar
- asset(\$b->gambar) — URL thumbnail gambar, jika tidak ada tampilkan icon default
- style="width:36px;height:36px;border-radius:8px;object-fit:cover" — Style thumbnail bulat dengan sudut
- Icon default (fas fa-cube) ditampilkan jika tidak ada gambar

No	Nama	Gambar	Stok
1	Laptop HP	[img]	5
2	Proyektor	[img]	3

Menampilkan Gambar di Detail

Di halaman detail/show, tampilkan gambar dalam ukuran besar:

@if (\$barang->gambar)

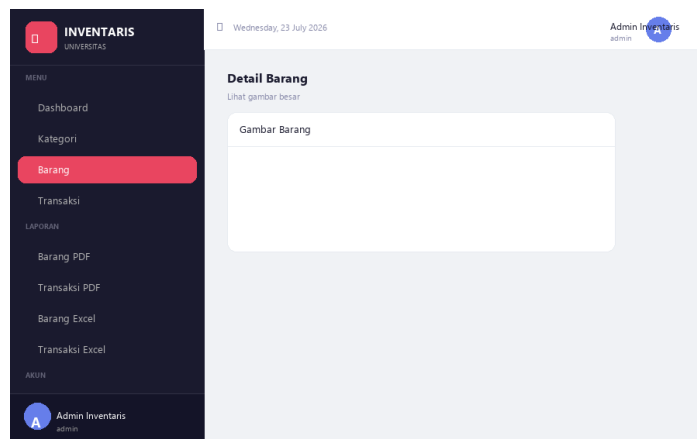
```

```

@else

```
<div style="..."><i class="fas fa-image"></i><p>Tidak ada gambar</p></div>
```

@endif



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu mengimplementasikan upload file gambar

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

Acara 17. CRUD Transaksi Peminjaman

Materi Pembelajaran : Transaksi, peminjaman, stok
Acara Praktikum/Praktik : Minggu 9/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi (TB430805.3)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritis yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources

4	J.620100.047.01	Melakukan Pembaruan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing
---	-----------------	-------------------------------------	---

c. Indikator Penilaian

Mahasiswa mampu membuat fitur peminjaman barang

d. Dasar Teori

Transaksi peminjaman mencatat siapa yang meminjam, barang apa, jumlah, dan tanggal. Saat peminjaman, stok barang berkurang. Status transaksi: "dipinjam". Relasi: Transaksi belongsTo User, belongsTo Barang.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja**Membuat Migration Transaksi**

Jalankan perintah artisan untuk membuat migration:

```
php artisan make:migration create_transaksis_table
```

Struktur tabel transaksis:

- id — primary key auto increment
- user_id — foreign key ke tabel users (peminjam)
- barang_id — foreign key ke tabel barangs (barang yang dipinjam)
- jumlah — integer jumlah barang dipinjam
- tgl_pinjam — date tanggal peminjaman
- tgl_kembali — date nullable tanggal pengembalian
- status — enum('dipinjam', 'dikembalikan') default 'dipinjam'
- timestamps — created_at dan updated_at

Migration tabel transaksi

```
database/migrations/xxxx_create_transaksi_table.php

1 Schema::create('transaksi', function (Blueprint $table) {
2     $table->id();
3
4     $table->foreignId('user_id')
5         ->constrained('users')->onDelete('cascade');
6
7     $table->foreignId('barang_id')
8         ->constrained('barang')->onDelete('cascade');
9
10    $table->integer('jumlah');
11    $table->date('tgl_pinjam');
12    $table->date('tgl_kembali')->nullable();
13
14    $table->enum('status', ['dipinjam', 'dikembalikan'])
15        ->default('dipinjam');
16
17    $table->timestamps();
18 });
```

Transaksi Peminjaman Laravel 11 PHP 8.2

Menjalankan Migrasi

Jalankan perintah berikut untuk menerapkan migrasi ke database:
php artisan migrate

Membuat Model Transaksi

Buat model Transaksi dengan relasi:

php artisan make:model Transaksi

Tambahkan relasi belongsTo ke User dan Barang, serta date casting untuk tgl_pinjam dan tgl_kembali.

Model Transaksi dengan relasi belongsTo

```
app/Models/Transaksi.php

1 class Transaksi extends Model
2 {
3     protected $table = 'transaksi';
4
5     protected $fillable = [
6         'user_id', 'barang_id', 'jumlah',
7         'tgl_pinjam', 'tgl_kembali', 'status',
8     ];
9
10    protected function casts(): array
11    {
12        return [
13            'tgl_pinjam' => 'date',
14            'tgl_kembali' => 'date',
15        ];
16    }
17
18    public function user(): BelongsTo
19    { return $this->belongsTo(User::class); }
20
21    public function barang(): BelongsTo
22    { return $this->belongsTo(Barang::class); }
23 }
```

Transaksi Peminjaman Laravel 11 PHP 8.2

Membuat Controller Transaksi

Buat controller resource:

```
php artisan make:controller TransaksiController --resource
```

Method-method dalam controller:

- index() — menampilkan daftar transaksi dengan eager loading (user, barang.kategori), filter status, pencarian, dan role-based scoping (staff hanya melihat transaksi sendiri)
- create() — menampilkan form peminjaman, hanya barang dengan stok > 0 dan kondisi baik
- store() — validasi input, cek stok mencukupi, simpan transaksi, decrement stok barang
- pengembalian() — update status menjadi dikembalikan, catat tgl_kembali, increment stok
- destroy() — hapus transaksi, kembalikan stok jika status masih dipinjam

Controller - method index (filter, search, role) dan create

```
app/Http/Controllers/TransaksiController.php (1)

1 class TransaksiController extends Controller
2 {
3     public function index(Request $request)
4     {
5         $query = Transaksi::with(['user', 'barang.kategori']);
6
7         if ($request->filled('status'))
8             $query->where('status', $request->status);
9
10        if ($request->filled('search')) {
11            $search = $request->search;
12            $query->whereHas('barang', fn($q) =>
13                $q->where('nama', 'like', "%{$search}%"));
14        }
15
16        if (Auth::user()->role === 'staff')
17            $query->where('user_id', Auth::id());
18
19        $transaksis = $query->latest()->paginate(10)
20            ->withQueryString();
21        return view('pages.transaksi.index', compact('transaksis'));
22    }
23
24    public function create()
25    {
26        $barangs = Barang::where('jumlah', '>', 0)
27            ->where('kondisi', 'baik')->get();
28        return view('pages.transaksi.create', compact('barangs'));
29    }
}
```

Transaksi Peminjaman Laravel 11 PHP 8.2

Controller - method store, pengembalian, destroy

```
app/Http/Controllers/TransaksiController.php (2)

app/Http/Controllers/TransaksiController.php (2)

1     public function store(Request $request)
2     {
3         $request->validate([
4             'barang_id' => 'required|exists:barangs,id',
5             'jumlah' => 'required|integer|min:1',
6             'tgl_pinjam' => 'required|date',
7         ]);
8
9         $barang = Barang::findOrFail($request->barang_id);
10        if ($barang->jumlah < $request->jumlah)
11            return back()->with('error', 'Stok tidak mencukupi');
12
13        Transaksi::create([
14            'user_id' => Auth::id(),
15            'barang_id' => $request->barang_id,
16            'jumlah' => $request->jumlah,
17            'tgl_pinjam' => $request->tgl_pinjam,
18            'status' => 'dipinjam',
19        ]);
20        $barang->decrement('jumlah', $request->jumlah);
21
22        return redirect()->route('transaksi.index')
23            ->with('success', 'Peminjaman berhasil');
24    }
25
26    public function pengembalian(Transaksi $transaksi)
27    {
28        $transaksi->update([
29            'status' => 'dikembalikan',
30            'tgl_kembali' => now(),
31        ]);
32        $transaksi->barang->increment(
33            'jumlah', $transaksi->jumlah);
34
35        return redirect()->route('transaksi.index')
36            ->with('success', 'Barang berhasil dikembalikan');
37    }
38
39    public function destroy(Transaksi $transaksi)
40    {
41        if ($transaksi->status === 'dipinjam')
42            $transaksi->barang->increment(
43                'jumlah', $transaksi->jumlah);
44        $transaksi->delete();
45        return redirect()->route('transaksi.index')
46            ->with('success', 'Transaksi berhasil dihapus');
47    }
48    }
```

Transaksi Peminjaman Laravel 11 PHP 8.2

Mendaftarkan Route

Daftarkan route resource dan route khusus pengembalian di routes/web.php:

Route transaksi (resource + pengembalian)

```
routes/web.php

1 Route::middleware(['auth', 'verified'])->group(function () {
2     Route::get('/', [DashboardController::class, 'index'])
3         ->name('dashboard');
4
5     Route::resource('kategori', KategoriController::class);
6     Route::resource('barang', BarangController::class);
7
8     Route::resource('transaksi', TransaksiController::class)
9         ->except(['edit', 'update']);
10
11     Route::put('/transaksi/{transaksi}/kembali', [
12         TransaksiController::class, 'pengembalian'
13     ]->name('transaksi.pengembalian');
14
15     // Laporan routes...
16 });
```

Transaksi Peminjaman Laravel 11 PHP 8.2

Membuat View Create (Form Peminjaman)

Buat view form peminjaman barang dengan field: pilih barang (dropdown dengan stok), jumlah, dan tanggal pinjam.

View form peminjaman barang

```
resources/views/pages/transaksi/create.blade.php

1 <x-app-layout>
2     <div class="page-header">
3         <h2>Pinjam Barang</h2>
4         <p>Form peminjaman barang inventaris</p>
5     </div>
6
7     <div class="card">
8         <form action="{{ route('transaksi.store') }}"
9             method="POST">
10             @csrf
11
12             <label>Pilih Barang</label>
13             <select name="barang_id" class="form-control">
14                 @foreach ($barang as $b)
15                     <option value="{{ $b->id }}"
16                         data-stok="{{ $b->jumlah }}"
17                         {{ $b->nama }} (Stok: {{ $b->jumlah }})
18                     </option>
19                 @endforeach
20             </select>
21
22             <label>Jumlah</label>
23             <input type="number" name="jumlah" min="1">
24
25             <label>Tanggal Pinjam</label>
26             <input type="date" name="tgl_pinjam">
27
28             <button type="submit">Pinjam</button>
29         </form>
30     </div>
31 </x-app-layout>
```

Transaksi Peminjaman Laravel 11 PHP 8.2

Membuat View Index (Daftar Transaksi)

Buat view daftar transaksi dengan tabel yang menampilkan: nomor, peminjam, barang, jumlah, tanggal pinjam, status, dan aksi. Untuk status "dipinjam" tampilkan tombol "Kembalikan".

View index transaksi dengan status badge dan tombol aksi

```
resources/views/pages/transaksi/index.blade.php

1 <x-app-layout>
2   <h2>Transaksi</h2>
3   <a href="{{ route('transaksi.create') }}">Pinjam Barang</a>
4
5   @if (session('success'))
6     <div class="alert-success">{{ session('success') }}</div>
7   @endif
8
9   <table>
10    <thead>
11      <tr><th>No</th><th>Peminjam</th><th>Barang</th>
12        <th>Jml</th><th>Tgl Pinjam</th>
13        <th>Status</th><th>Aksi</th></tr>
14    </thead>
15    <tbody>
16      @forelse ($transaksis as $t)
17        <tr>
18          <td>{{ $loop->iteration }}</td>
19          <td>{{ $t->user->name }}</td>
20          <td>{{ $t->barang->nama }}</td>
21          <td>{{ $t->jumlah }}</td>
22          <td>{{ $t->tgl_pinjam->format('d/m/Y') }}</td>
23
24          @if ($t->status === 'dipinjam')
25            <form action="{{ route('transaksi.pengembalian', $t) }}"
26              method="POST">
27              @csrf @method('PUT')
28              <button>Kembalikan</button>
29            </form>
30          @else
31            <span>Selesai</span>
32          @endif
33        </tr>
34      @empty
35        <tr><td colspan="7">Belum ada transaksi</td></tr>
36      @endforelse
37    </tbody>
38  </table>
39 </x-app-layout>
```

Uji Coba Peminjaman Barang

Lakukan pengujian peminjaman barang:

- Buka halaman Transaksi, klik tombol "Pinjam Barang"
- Pilih barang yang tersedia (stok > 0)
- Masukkan jumlah dan tanggal pinjam
- Klik tombol "Pinjam"
- Sistem akan menyimpan transaksi dengan status "dipinjam" dan mengurangi stok barang
- Muncul flash message "Peminjaman berhasil"

Uji Coba Pengembalian Barang

Lakukan pengujian pengembalian barang:

- Pada daftar transaksi, klik tombol "Kembalikan" pada transaksi berstatus "Dipinjam"
- Konfirmasi pengembalian
- Sistem mengubah status menjadi "dikembalikan", mencatat tanggal kembali, dan menambah stok barang
- Muncul flash message "Barang berhasil dikembalikan"
- Status berubah menjadi "Dikembalikan" dan tombol aksi berubah menjadi "Selesai"

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Transaksi, peminjaman, stok". Analisis bagaimana konsep crud transaksi peminjaman diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Migration tabel transaksi

```
database/migrations/xxxx_create_transaksiis_table.php

1 Schema::create('transaksis', function (Blueprint $table) {
2     $table->id();
3
4     $table->foreignId('user_id')
5         ->constrained('users')->onDelete('cascade');
6
7     $table->foreignId('barang_id')
8         ->constrained('barang')->onDelete('cascade');
9
10    $table->integer('jumlah');
11    $table->date('tgl_pinjam');
12    $table->date('tgl_kembali')->nullable();
13
14    $table->enum('status', ['dipinjam', 'dikembalikan'])
15        ->default('dipinjam');
16
17    $table->timestamps();
18 });
```

Transaksi Peminjaman Laravel 11 PHP 8.2

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Model Transaksi (app/Models/Transaksi.php)

```
app/Models/Transaksi.php

1  class Transaksi extends Model
2  {
3      protected $table = 'transaksis';
4
5      protected $fillable = [
6          'user_id', 'barang_id', 'jumlah',
7          'tgl_pinjam', 'tgl_kembali', 'status',
8      ];
9
10     protected function casts(): array
11     {
12         return [
13             'tgl_pinjam' => 'date',
14             'tgl_kembali' => 'date',
15         ];
16     }
17
18     public function user(): BelongsTo
19     { return $this->belongsTo(User::class); }
20
21     public function barang(): BelongsTo
22     { return $this->belongsTo(Barang::class); }
23 }
```

Transaksi Peminjaman Laravel 11 PHP 8.2

Controller - method index dan create

```
app/Http/Controllers/TransaksiController.php (1)

1  class TransaksiController extends Controller
2  {
3      public function index(Request $request)
4      {
5          $query = Transaksi::with(['user', 'barang.kategori']);
6
7          if ($request->filled('status'))
8              $query->where('status', $request->status);
9
10         if ($request->filled('search')) {
11             $search = $request->search;
12             $query->whereHas('barang', fn($q) =>
13                 $q->where('nama', 'like', "%{$search}%"));
14         }
15
16         if (Auth::user()->role === 'staff')
17             $query->where('user_id', Auth::id());
18
19         $transaksis = $query->latest()->paginate(10)
20             ->withQueryString();
21         return view('pages.transaksi.index', compact('transaksis'));
22     }
23
24     public function create()
25     {
26         $barang = Barang::where('jumlah', '>', 0)
27             ->where('kondisi', 'baik')->get();
28         return view('pages.transaksi.create', compact('barang'));
29     }
}
```

Transaksi Peminjaman Laravel 11 PHP 8.2

Controller - method store, pengembalian, destroy

```
app/Http/Controllers/TransaksiController.php (2)

app/Http/Controllers/TransaksiController.php (2)
1     public function store(Request $request)
2     {
3         $request->validate([
4             'barang_id' => 'required|exists:barangs,id',
5             'jumlah' => 'required|integer|min:1',
6             'tgl_pinjam' => 'required|date',
7         ]);
8
9         $barang = Barang::findOrFail($request->barang_id);
10        if ($barang->jumlah < $request->jumlah)
11            return back()->with('error', 'Stok tidak mencukupi');
12
13        Transaksi::create([
14            'user_id' => Auth::id(),
15            'barang_id' => $request->barang_id,
16            'jumlah' => $request->jumlah,
17            'tgl_pinjam' => $request->tgl_pinjam,
18            'status' => 'dipinjam',
19        ]);
20        $barang->decrement('jumlah', $request->jumlah);
21
22        return redirect()->route('transaksi.index')
23            ->with('success', 'Peminjaman berhasil');
24    }
25
26    public function pengembalian(Transaksi $transaksi)
27    {
28        $transaksi->update([
29            'status' => 'dikembalikan',
30            'tgl_kembali' => now(),
31        ]);
32        $transaksi->barang->increment(
33            'jumlah', $transaksi->jumlah);
34
35        return redirect()->route('transaksi.index')
36            ->with('success', 'Barang berhasil dikembalikan');
37    }
38
39    public function destroy(Transaksi $transaksi)
40    {
41        if ($transaksi->status === 'dipinjam')
42            $transaksi->barang->increment(
43                'jumlah', $transaksi->jumlah);
44        $transaksi->delete();
45        return redirect()->route('transaksi.index')
46            ->with('success', 'Transaksi berhasil dihapus');
47    }
48    }
```

Transaksi Peminjaman Laravel 11 PHP 8.2

Route transaksi di web.php

```
routes/web.php

routes/web.php
1     Route::middleware(['auth', 'verified'])->group(function () {
2         Route::get('/', [DashboardController::class, 'index'])
3             ->name('dashboard');
4
5         Route::resource('kategori', KategoriController::class);
6         Route::resource('barang', BarangController::class);
7
8         Route::resource('transaksi', TransaksiController::class)
9             ->except(['edit', 'update']);
10
11        Route::put('/transaksi/{transaksi}/kembali', [
12            TransaksiController::class, 'pengembalian'
13        ])->name('transaksi.pengembalian');
14
15        // Laporan routes...
16    });
```

Transaksi Peminjaman Laravel 11 PHP 8.2

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

View form peminjaman (create.blade.php)

```
resources/views/pages/transaksi/create.blade.php

1 <x-app-layout>
2   <div class="page-header">
3     <h2>Pinjam Barang</h2>
4     <p>Form peminjaman barang inventaris</p>
5   </div>
6
7   <div class="card">
8     <form action="{{ route('transaksi.store') }}"
9       method="POST">
10      @csrf
11
12      <label>Pilih Barang</label>
13      <select name="barang_id" class="form-control">
14        @foreach ($barang as $b)
15          <option value="{{ $b->id }}"
16            data-stok="{{ $b->jumlah }}"
17            {{ $b->nama }} (Stok: {{ $b->jumlah }})
18          </option>
19        @endforeach
20      </select>
21
22      <label>Jumlah</label>
23      <input type="number" name="jumlah" min="1">
24
25      <label>Tanggal Pinjam</label>
26      <input type="date" name="tgl_pinjam">
27
28      <button type="submit">Pinjam</button>
29    </form>
30  </div>
31 </x-app-layout>
```

Transaksi Peminjaman Laravel 11 PHP 8.2

View daftar transaksi (index.blade.php)

```
resources/views/pages/transaksi/index.blade.php

1 <x-app-layout>
2   <h2>Transaksi</h2>
3   <a href="{{ route('transaksi.create') }}">Pinjam Barang</a>
4
5   @if (session('success'))
6     <div class="alert-success">{{ session('success') }}</div>
7   @endif
8
9   <table>
10     <thead>
11       <tr><th>No</th><th>Peminjam</th><th>Barang</th>
12       <th>Jml</th><th>Tgl Pinjam</th>
13       <th>Status</th><th>Aksi</th></tr>
14     </thead>
15     <tbody>
16       @foreach ($transaksis as $t)
17         <tr>
18           <td>{{ $loop->iteration }}</td>
19           <td>{{ $t->user->name }}</td>
20           <td>{{ $t->barang->nama }}</td>
21           <td>{{ $t->jumlah }}</td>
22           <td>{{ $t->tgl_pinjam->format('d/m/Y') }}</td>
23
24           @if ($t->status == 'dipinjam')
25             <form action="{{ route('transaksi.pengembalian', $t) }}"
26               method="POST">
27               @csrf @method('PUT')
28               <button>Kembalikan</button>
29             </form>
30           @else
31             <span>Selesai</span>
32           @endif
33         </tr>
34       @empty
35         <tr><td colspan="7">Belum ada transaksi</td></tr>
36       @endforeach
37     </tbody>
38   </table>
39 </x-app-layout>
```

Transaksi Peminjaman Laravel 11 PHP 8.2

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman daftar transaksi dengan filter dan data

INVENTARIS
UNIVERSITAS

MENU

Dashboard

Kategori

Barang

Transaksi

LAPORAN

Barang PDF

Transaksi PDF

Barang Excel

Transaksi Excel

AKUN

Profile

Logout

Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Transaksi

Data peminjaman dan pengembalian

+ Pinjam Barang

No	Peminjam	Barang	Status	Tgl Pinjam
1	Admin	Laptop HP	Dipinjam	23/07
2	Staff	Proyektor	Dikembalikan	22/07

Form peminjaman barang

INVENTARIS
UNIVERSITAS

MENU

Dashboard

Kategori

Barang

Transaksi

LAPORAN

Barang PDF

Transaksi PDF

Barang Excel

Transaksi Excel

AKUN

Profile

Logout

Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Pinjam Barang

Form peminjaman barang inventaris

Form Peminjaman

Pilih Barang

Pilih barang

Jumlah

1

Tanggal Pinjam

23/07/2026

Simpan

Batal

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Flash message sukses peminjaman

The screenshot displays the 'INVENTARIS UNIVERSITAS' web application interface. On the left is a dark sidebar menu with categories: MENU (Dashboard, Kategori, Barang, Transaksi), LAPORAN (Barang PDF, Transaksi PDF, Barang Excel, Transaksi Excel), and AKUN (Profile, Logout). The 'Transaksi' menu item is highlighted in red. The top header shows the date 'Wednesday, 23 July 2026' and the user 'Admin Inventaris admin'. A green flash message at the top right states 'Barang berhasil dipinjam'. Below this, the 'Transaksi' section shows a table of loan data. A blue button '+ Pinjam Barang' is visible. The table has columns for No, Peminjam, Barang, and Status. One transaction is listed: No 1, Peminjam Admin, Barang Laptop HP, and Status 'Dipinjam' (highlighted in yellow).

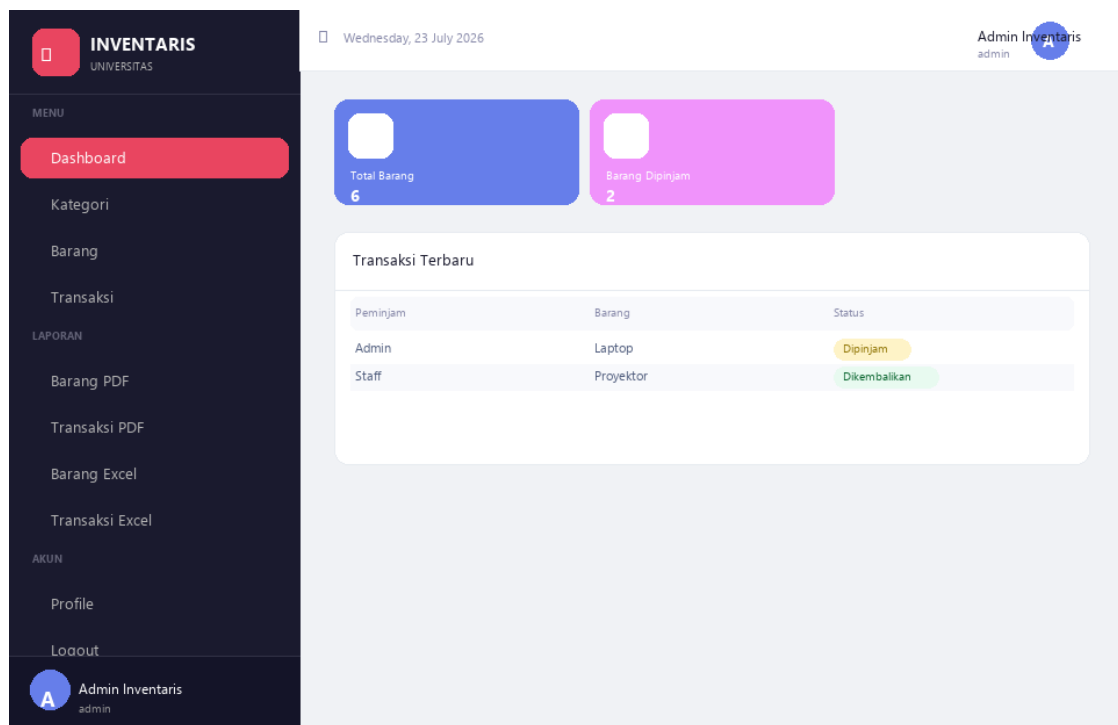
No	Peminjam	Barang	Status
1	Admin	Laptop HP	Dipinjam

Status dikembalikan setelah pengembalian barang

The screenshot shows the same 'INVENTARIS UNIVERSITAS' web application interface. The sidebar menu and top header are identical to the previous screenshot. A confirmation dialog box titled 'Konfirmasi Pengembalian' is displayed in the center. It asks 'Apakah Anda yakin ingin mengembalikan barang ini?' (Are you sure you want to return this item?). There are two buttons: 'Ya, Kembalikan' (Yes, Return) in green and 'Batal' (Cancel) in grey.

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Dashboard dengan statistik dan transaksi terbaru



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat fitur peminjaman barang

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 18. CRUD Transaksi Pengembalian

Materi Pembelajaran : Pengembalian, update stok
Acara Praktikum/Praktik : Minggu 9/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi (TB430805.3)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritikal yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources

4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing
---	-----------------	--	---

c. Indikator Penilaian

Mahasiswa mampu membuat fitur pengembalian barang

d. Dasar Teori

Pengembalian barang mengubah status transaksi dari "dipinjam" menjadi "dikembalikan", mencatat tanggal kembali, dan menambahkan kembali stok barang.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Tambahkan method pengembalian() di TransaksiController:

Method pengembalian() di Controller

Tambahkan method pengembalian() di TransaksiController. Method ini menerima parameter Transaksi (Route Model Binding), mengecek status, mengupdate status menjadi "dikembalikan" dengan tgl_kembali, dan mengembalikan stok barang.

Fungsi method:

- Validasi status transaksi (hanya bisa dikembalikan jika masih "dipinjam")
- Update status menjadi "dikembalikan"
- Catat tanggal pengembalian (now())
- Increment jumlah stok barang sesuai jumlah peminjaman
- Redirect ke index transaksi dengan flash message sukses

Method pengembalian() di TransaksiController

```
app/Http/Controllers/TransaksiController.php

1 public function pengembalian(Transaksi $transaksi)
2 {
3     if ($transaksi->status !== 'dipinjam') {
4         return back()->with('error', 'Barang sudah dikembalikan');
5     }
6
7     $transaksi->update([
8         'status' => 'dikembalikan',
9         'tgl_kembali' => now(),
10    ]);
11
12    $transaksi->barang->increment(
13        'jumlah', $transaksi->jumlah);
14
15    return redirect()->route('transaksi.index')
16        ->with('success', 'Barang berhasil dikembalikan');
17 }
```

Pengembalian Barang Laravel 11 PHP 8.2

Method store() - Stok Berkurang

Method store() menangani peminjaman barang. Validasi stok dilakukan sebelum transaksi dibuat, dan stok barang langsung dikurangi setelah transaksi tersimpan.

Method store() dengan validasi stok dan decrement

```
app/Http/Controllers/TransaksiController.php

1 public function store(Request $request)
2 {
3     $request->validate([
4         'barang_id' => 'required|exists:barangs,id',
5         'jumlah' => 'required|integer|min:1',
6         'tgl_pinjam' => 'required|date',
7     ]);
8
9     $barang = Barang::findOrFail($request->barang_id);
10    if ($barang->jumlah < $request->jumlah)
11        return back()->with('error', 'Stok tidak mencukupi');
12
13    Transaksi::create([
14        'user_id' => Auth::id(),
15        'barang_id' => $request->barang_id,
16        'jumlah' => $request->jumlah,
17        'tgl_pinjam' => $request->tgl_pinjam,
18        'status' => 'dipinjam',
19    ]);
20
21    $barang->decrement('jumlah', $request->jumlah);
22
23    return redirect()->route('transaksi.index')
24        ->with('success', 'Peminjaman berhasil');
25 }
```

Pengembalian Barang Laravel 11 PHP 8.2

Method destroy() - Stok Recovery

Method destroy() menangani penghapusan transaksi. Jika transaksi berstatus "dipinjam", stok barang dikembalikan terlebih dahulu sebelum transaksi dihapus.

Method destroy() dengan stok recovery

```
app/Http/Controllers/TransaksiController.php

1 public function destroy(Transaksi $transaksi)
2 {
3     if ($transaksi->status === 'dipinjam') {
4         $transaksi->barang->increment(
5             'jumlah', $transaksi->jumlah);
6     }
7
8     $transaksi->delete();
9
10    return redirect()->route('transaksi.index')
11        ->with('success', 'Transaksi berhasil dihapus');
12 }
```

Pengembalian Barang Laravel 11 PHP 8.2

Route Pengembalian

Daftarkan route PUT untuk pengembalian di routes/web.php. Gunakan route model binding untuk menerima parameter Transaksi secara otomatis.

```
Route::put('/transaksi/{transaksi}/kembali', [TransaksiController::class,
'pengembalian'])->name('transaksi.pengembalian');
```

Route pengembalian di web.php

```
routes/web.php

1 Route::middleware(['auth', 'verified'])->group(function () {
2     Route::resource('transaksi', TransaksiController::class)
3         ->except(['edit', 'update']);
4
5     Route::put('/transaksi/{transaksi}/kembali', [
6         TransaksiController::class, 'pengembalian'
7     ])->name('transaksi.pengembalian');
8
9     Route::get('/laporan/transaksi/pdf',
10         [LaporanController::class, 'exportPdfTransaksi'])
11         ->name('laporan.transaksi.pdf');
12 });
```

Pengembalian Barang Laravel 11 PHP 8.2

Tombol Kembalikan di View Index

Di view index transaksi, tampilkan tombol "Kembalikan" hanya untuk transaksi berstatus "dipinjam". Tombol mengirimkan request PUT ke route pengembalian dengan konfirmasi JavaScript (confirm dialog).

Kode yang ditambahkan di view index:

- @if(\$t->status === 'dipinjam')
- <form action="{{ route('transaksi.pengembalian', \$t) }}" method="POST">
@csrf @method('PUT')
- <button type="submit">Kembalikan</button>

View index dengan tombol Kembali

```
resources/views/pages/transaksi/index.blade.php

1 <x-app-layout>
2 <h2>Transaksi</h2>
3 <a href="{{ route('transaksi.create') }}">Pinjam Barang</a>
4
5 @if (session('success'))
6 <div>Barang berhasil dikembalikan</div>
7 @endif
8
9 <table>
10 <tr><th>Status</th><th>Aksi</th></tr>
11 @foreach ($transaksis as $t)
12 <tr>
13     @if ($t->status == 'dipinjam')
14     <td>Dipinjam</td>
15     <td>
16         <form action="{{ route('transaksi.pengembalian', $t) }}"
17             method="POST">
18             @csrf @method('PUT')
19             <button>Kembalikan</button>
20         </form>
21     </td>
22     @else
23     <td>Dikembalikan</td>
24     <td>Selesai</td>
25     @endif
26 </tr>
27 @endforeach
28 </table>
29 </x-app-layout>
```

Pengembalian Barang Laravel 11 PHP 8.2

Riwayat Transaksi di Dashboard

Dashboard menampilkan 5 transaksi terbaru melalui DashboardController. Staff hanya melihat transaksi miliknya sendiri (role-based scoping di TransaksiController). Setiap transaksi menampilkan status (Dipinjam/Dikembalikan) dengan badge warna berbeda.

Data yang dikirim ke view dashboard:

- totalBarang — jumlah seluruh barang
- totalKategori — jumlah kategori
- barangDipinjam — transaksi dengan status "dipinjam"
- barangTersedia — total stok semua barang
- transaksiTerbaru — 5 transaksi terbaru dengan eager loading (user, barang)

Uji Coba Pengembalian

Langkah pengujian fitur pengembalian:

- Buka halaman Transaksi dan pastikan ada data dengan status "Dipinjam"
- Klik tombol "Kembalikan" pada transaksi yang akan dikembalikan
- Konfirmasi pengembalian melalui dialog konfirmasi
- Periksa flash message: "Barang berhasil dikembalikan"

- Verifikasi status berubah menjadi "Dikembalikan" dan tombol berubah menjadi "Selesai"
- Cek stok barang bertambah sesuai jumlah yang dikembalikan
- Buka Dashboard dan lihat riwayat transaksi terbaru dengan status terupdate

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Pengembalian, update stok". Analisis bagaimana konsep crud transaksi pengembalian diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Method pengembalian() di TransaksiController

```
app/Http/Controllers/TransaksiController.php

app/Http/Controllers/TransaksiController.php

1 public function pengembalian(Transaksi $transaksi)
2 {
3     if ($transaksi->status !== 'dipinjam') {
4         return back()->with('error', 'Barang sudah dikembalikan');
5     }
6
7     $transaksi->update([
8         'status' => 'dikembalikan',
9         'tgl_kembali' => now(),
10    ]);
11
12    $transaksi->barang->increment(
13        'jumlah', $transaksi->jumlah);
14
15    return redirect()->route('transaksi.index')
16        ->with('success', 'Barang berhasil dikembalikan');
17 }
```

Pengembalian Barang Laravel 11 PHP 8.2

Method store() dengan decrement stok

```
app/Http/Controllers/TransaksiController.php

app/Http/Controllers/TransaksiController.php

1 public function store(Request $request)
2 {
3     $request->validate([
4         'barang_id' => 'required|exists:barangs,id',
5         'jumlah' => 'required|integer|min:1',
6         'tgl_pinjam' => 'required|date',
7     ]);
8
9     $barang = Barang::findOrFail($request->barang_id);
10    if ($barang->jumlah < $request->jumlah)
11        return back()->with('error', 'Stok tidak mencukupi');
12
13    Transaksi::create([
14        'user_id' => Auth::id(),
15        'barang_id' => $request->barang_id,
16        'jumlah' => $request->jumlah,
17        'tgl_pinjam' => $request->tgl_pinjam,
18        'status' => 'dipinjam',
19    ]);
20
21    $barang->decrement('jumlah', $request->jumlah);
22
23    return redirect()->route('transaksi.index')
24        ->with('success', 'Peminjaman berhasil');
25 }
```

Pengembalian Barang Laravel 11 PHP 8.2

Method destroy() dengan stok recovery

```
app/Http/Controllers/TransaksiController.php

app/Http/Controllers/TransaksiController.php

1 public function destroy(Transaksi $transaksi)
2 {
3     if ($transaksi->status === 'dipinjam') {
4         $transaksi->barang->increment(
5             'jumlah', $transaksi->jumlah);
6     }
7
8     $transaksi->delete();
9
10    return redirect()->route('transaksi.index')
11        ->with('success', 'Transaksi berhasil dihapus');
12 }
```

Pengembalian Barang Laravel 11 PHP 8.2

Route pengembalian di web.php

```
routes/web.php

routes/web.php

1 Route::middleware(['auth', 'verified'])->group(function () {
2     Route::resource('transaksi', TransaksiController::class)
3         ->except(['edit', 'update']);
4
5     Route::put('/transaksi/{transaksi}/kembali', [
6         TransaksiController::class, 'pengembalian'
7     ]->name('transaksi.pengembalian');
8
9     Route::get('/laporan/transaksi/pdf',
10         [LaporanController::class, 'exportPdfTransaksi'])
11         ->name('laporan.transaksi.pdf');
12 });
```

Pengembalian Barang Laravel 11 PHP 8.2

View index dengan tombol Kembalikan

```
resources/views/pages/transaksi/index.blade.php

resources/views/pages/transaksi/index.blade.php

1 <x-app-layout>
2     <h2>Transaksi</h2>
3     <a href="{{ route('transaksi.create') }}">Pinjam Barang</a>
4
5     @if (session('success'))
6     <div>Barang berhasil dikembalikan</div>
7     @endif
8
9     <table>
10        <tr><th>Status</th><th>Aksi</th></tr>
11        @foreach ($transaksis as $t)
12            <tr>
13                @if ($t->status === 'dipinjam')
14                <td>Dipinjam</td>
15                <td>
16                    <form action="{{ route('transaksi.pengembalian', $t) }}"
17                        method="POST">
18                        @csrf @method('PUT')
19                        <button>Kembalikan</button>
20                    </form>
21                </td>
22                @else
23                <td>Dikembalikan</td>
24                <td>Selesai</td>
25                @endif
26            </tr>
27        @endforeach
28    </table>
29 </x-app-layout>
```

Pengembalian Barang Laravel 11 PHP 8.2

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Daftar transaksi dengan status dan tombol Kembalikan

MENU

[Dashboard](#)

[Kategori](#)

[Barang](#)

[Transaksi](#)

LAPORAN

[Barang PDF](#)

[Transaksi PDF](#)

[Barang Excel](#)

[Transaksi Excel](#)

AKUN

[Profile](#)

[Logout](#)

Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Transaksi

Daftar peminjaman barang

No	Peminjam	Barang	Status	Aksi
1	Admin	Laptop HP	Dipinjam	
2	Staff	Proyektor	Dikembalikan	

Dialog konfirmasi pengembalian barang

MENU

[Dashboard](#)

[Kategori](#)

[Barang](#)

[Transaksi](#)

LAPORAN

[Barang PDF](#)

[Transaksi PDF](#)

[Barang Excel](#)

[Transaksi Excel](#)

AKUN

[Profile](#)

[Logout](#)

Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Konfirmasi Pengembalian

Klik tombol Kembalikan

Konfirmasi Pengembalian

Apakah Anda yakin ingin mengembalikan barang ini?

[Ya, Kembalikan](#)[Batal](#)

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Flash message sukses setelah pengembalian

The screenshot shows the INVENTARIS web application interface. The left sidebar contains a menu with options: Dashboard, Kategori, Barang, Transaksi (highlighted), LAPORAN (Barang PDF, Transaksi PDF, Barang Excel, Transaksi Excel), and AKUN (Profile, Logout). The main content area displays a success flash message: "Barang berhasil dikembalikan". Below the message is a table with the following data:

No	Barang	Status
1	Laptop HP	Dikembalikan

The top right corner shows the date "Wednesday, 23 July 2026" and the user profile "Admin Inventaris admin".

Dashboard dengan statistik dan riwayat transaksi

The screenshot shows the INVENTARIS web application dashboard. The left sidebar is identical to the previous screenshot, with the "Dashboard" option highlighted. The main content area displays three statistics cards:

- Total Barang: 6
- Dipinjam: 0
- Stok: 113

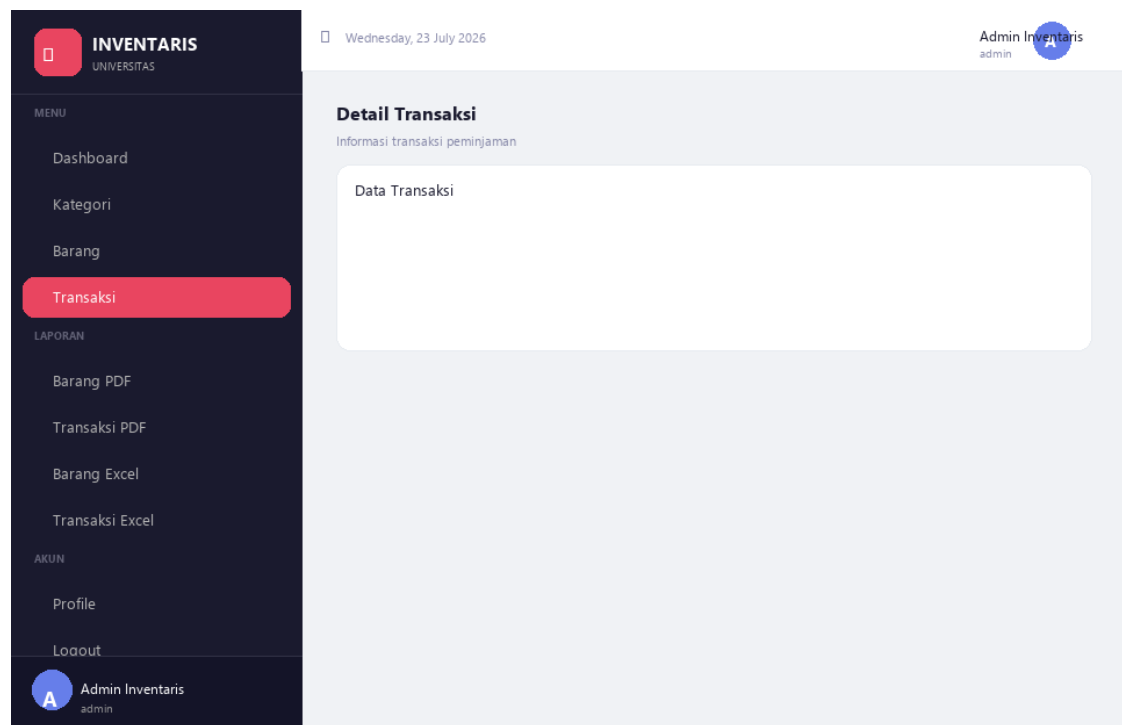
Below the statistics is a section titled "Riwayat Transaksi" with a table showing the transaction history:

Peminjam	Barang	Status
Admin	Laptop	Dikembalikan

The top right corner shows the date "Wednesday, 23 July 2026" and the user profile "Admin Inventaris admin".

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Detail transaksi dengan status Dikembalikan



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat fitur pengembalian barang

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusa	V			
	Total Skor*	Skor x 25			

Acara 19. Relasi One-to-Many dan Query Lanjutan

Materi Pembelajaran : hasMany, eager loading
 Acara Praktikum/Praktik : Minggu 10/1
 Tempat : Daring/Luring Politeknik Negeri Jember
 Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritikal yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources
4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing

c. Indikator Penilaian

Mahasiswa mampu mengimplementasikan relasi one-to-many

d. Dasar Teori

Relasi one-to-many: Kategori hasMany Barang, Barang hasMany Transaksi.
Eager loading dengan with() untuk mengoptimasi query (N+1 problem).

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

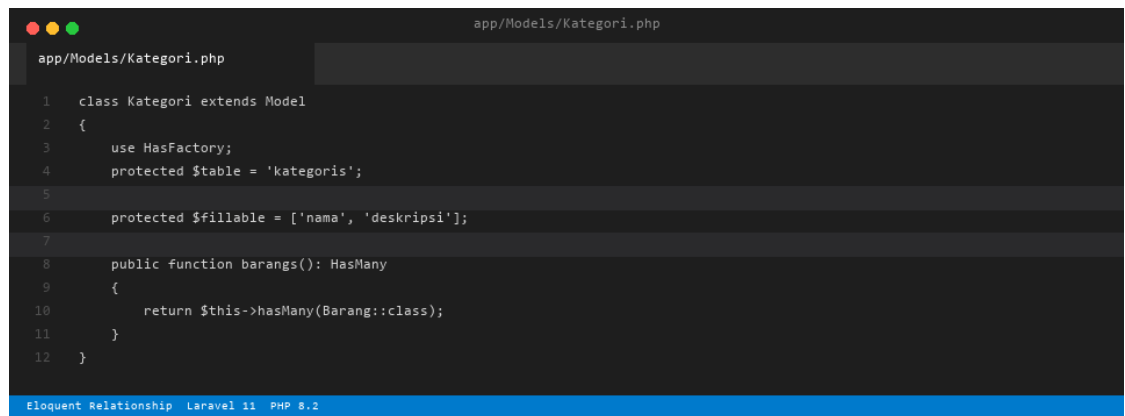
f. Prosedur Kerja

Relasi hasMany di Model Kategori

Tambahkan method barangs() di model Kategori untuk mendefinisikan relasi one-to-many dari Kategori ke Barang. Satu kategori dapat memiliki banyak barang.

```
public function barangs(): HasMany { return $this->hasMany(Barang::class); }
```

Relasi hasMany di model Kategori



```
app/Models/Kategori.php

1  class Kategori extends Model
2  {
3      use HasFactory;
4      protected $table = 'kategoris';
5
6      protected $fillable = ['nama', 'deskripsi'];
7
8      public function barangs(): HasMany
9      {
10         return $this->hasMany(Barang::class);
11     }
12 }
```

Eloquent Relationship Laravel 11 PHP 8.2

Relasi belongsTo + hasMany di Model Barang

Di model Barang, tambahkan relasi belongsTo ke Kategori (setiap barang memiliki satu kategori) dan hasMany ke Transaksi (satu barang memiliki banyak riwayat transaksi).

Relasi belongsTo dan hasMany di model Barang

```
app/Models/Barang.php

1 class Barang extends Model
2 {
3     protected $table = 'barang';
4
5     public function kategori(): BelongsTo
6     {
7         return $this->belongsTo(Kategori::class);
8     }
9
10    public function transaksi(): HasMany
11    {
12        return $this->hasMany(Transaksi::class);
13    }
14
15    public function favoritedBy(): BelongsToMany
16    {
17        return $this->belongsToMany(User::class, 'barang_user');
18    }
19 }
```

Eloquent Relationship Laravel 11 PHP 8.2

Relasi hasMany di Model User

Di model User, tambahkan relasi hasMany ke Transaksi. Satu user dapat melakukan banyak transaksi peminjaman.

Relasi hasMany di model User

```
app/Models/User.php

1 class User extends Authenticatable
2 {
3     use HasFactory, Notifiable;
4
5     public function transaksi(): HasMany
6     {
7         return $this->hasMany(Transaksi::class);
8     }
9
10    public function favorites(): BelongsToMany
11    {
12        return $this->belongsToMany(Barang::class, 'barang_user');
13    }
14
15    public function isAdmin(): bool
16    {
17        return $this->role === 'admin';
18    }
19 }
```

Eloquent Relationship Laravel 11 PHP 8.2

Eager Loading dengan with() di Controller

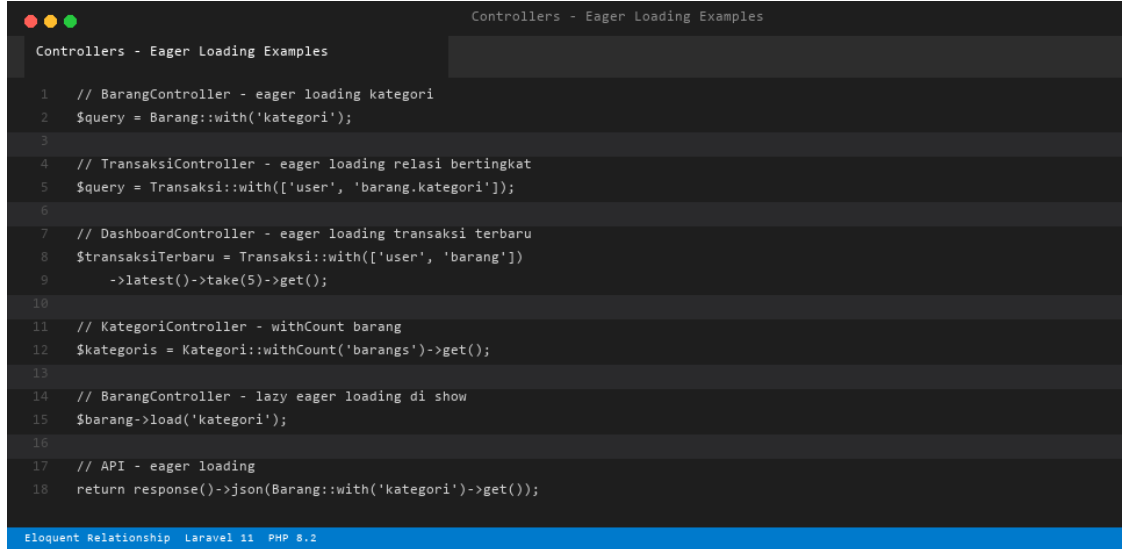
Gunakan method with() di controller untuk memuat relasi secara eager.

Beberapa contoh penggunaan:

- `Barang::with('kategori')` — memuat relasi kategori setiap barang
- `Transaksi::with(['user', 'barang.kategori'])` — memuat relasi bertingkat

- `Transaksi::with(['user', 'barang'])->latest()->take(5)` — eager loading 5 transaksi terbaru
- `$barang->load('kategori')` — lazy eager loading setelah model di-load

Contoh eager loading dengan `with()` dan `withCount()`



```
Controllers - Eager Loading Examples

1 // BarangController - eager loading kategori
2 $query = Barang::with('kategori');
3
4 // TransaksiController - eager loading relasi bertingkat
5 $query = Transaksi::with(['user', 'barang.kategori']);
6
7 // DashboardController - eager loading transaksi terbaru
8 $transaksiTerbaru = Transaksi::with(['user', 'barang'])
9     ->latest()->take(5)->get();
10
11 // KategoriController - withCount barang
12 $kategoris = Kategori::withCount('barangs')->get();
13
14 // BarangController - lazy eager loading di show
15 $barang->load('kategori');
16
17 // API - eager loading
18 return response()->json(Barang::with('kategori')->get());

Eloquent Relationship  Laravel 11  PHP 8.2
```

`withCount()` di `KategoriController`

Gunakan `withCount('barangs')` untuk menampilkan jumlah barang per kategori tanpa memuat seluruh data barang.

```
$kategoris = Kategori::withCount('barangs')->get();
```

Di view, akses melalui `$kategori->barangs_count` yang otomatis tersedia setelah menggunakan `withCount()`.

Menampilkan Data Relasi di View

Di view Blade, akses data relasi menggunakan sintaks panah (`->`) sesuai nama method relasi yang didefinisikan di model.

Contoh di view kategori (menampilkan jumlah barang):

- `{{ $ket->barangs_count }}`

Contoh di view barang (menampilkan nama kategori):

- `{{ $b->kategori->nama }}`

View kategori dengan barangs_count

```
resources/views/pages/kategori/index.blade.php

1 <x-app-layout>
2 <h2>Kategori</h2>
3 <table>
4 <thead>
5 <tr><th>No</th><th>Nama</th>
6 <th>Deskripsi</th>
7 <th>Jumlah Barang</th>
8 <th>Aksi</th></tr>
9 </thead>
10 <tbody>
11 @forelse ($kategoris as $ket)
12 <tr>
13 <td>{{ $loop->iteration }}</td>
14 <td>{{ $ket->nama }}</td>
15 <td>{{ $ket->deskripsi ?? '-' }}</td>
16
17 <td>
18 {{ $ket->barangs_count }}
19 </td>
20
21 <td>
22 <a href="{{ route('kategori.edit', $ket) }}">Edit</a>
23 </td>
24 </tr>
25 @empty
26 <tr><td colspan="5">Belum ada data</td></tr>
27 @endforelse
28 </tbody>
29 </table>
30 </x-app-layout>
```

Eloquent Relationship Laravel 11 PHP 8.2

View barang dengan kategori->nama

```
resources/views/pages/barang/index.blade.php

1 <x-app-layout>
2 <h2>Barang</h2>
3 <table>
4 <thead>
5 <tr><th>No</th><th>Nama</th>
6 <th>Kategori</th><th>Jumlah</th>
7 <th>Kondisi</th><th>Aksi</th></tr>
8 </thead>
9 <tbody>
10 @forelse ($barangs as $b)
11 <tr>
12 <td>{{ $loop->iteration }}</td>
13 <td>{{ $b->nama }}</td>
14
15 <td>
16 {{ $b->kategori->nama }}
17 </td>
18
19 <td>{{ $b->jumlah }}</td>
20 <td>{{ $b->kondisi }}</td>
21 <td>
22 <a href="{{ route('barang.edit', $b) }}">Edit</a>
23 </td>
24 </tr>
25 @empty
26 <tr><td colspan="6">Belum ada data</td></tr>
27 @endforelse
28 </tbody>
29 </table>
30 </x-app-layout>
```

Eloquent Relationship Laravel 11 PHP 8.2

Uji Coba dan Debugbar (N+1 Detection)

Langkah pengujian:

- Install Laravel Debugbar: `composer require barryvdh/laravel-debugbar`
- Buka halaman Barang dan perhatikan jumlah query di debugbar
- Aktifkan N+1 detection di AppServiceProvider:

`Model::preventLazyLoading(!app()->isProduction())`

- Nonaktifkan eager loading sementara dan refresh halaman — debugbar menunjukkan lebih banyak query
- Aktifkan kembali eager loading dan bandingkan jumlah query
- Periksa halaman Kategori dengan `withCount()` — debugbar menunjukkan query `SELECT COUNT`

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "hasMany, eager loading". Analisis bagaimana konsep relasi one-to-many dan query lanjutan diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Model Kategori - hasMany Barang

```
app/Models/Kategori.php

1  class Kategori extends Model
2  {
3      use HasFactory;
4      protected $table = 'kategoris';
5
6      protected $fillable = ['nama', 'deskripsi'];
7
8      public function barangs(): HasMany
9      {
10         return $this->hasMany(Barang::class);
11     }
12 }
```

Eloquent Relationship Laravel 11 PHP 8.2

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Model Barang - belongsTo Kategori, hasMany Transaksi

```
app/Models/Barang.php

1  class Barang extends Model
2  {
3      protected $table = 'barangs';
4
5      public function kategori(): BelongsTo
6      {
7          return $this->belongsTo(Kategori::class);
8      }
9
10     public function transaksi(): HasMany
11     {
12         return $this->hasMany(Transaksi::class);
13     }
14
15     public function favoritedBy(): BelongsToMany
16     {
17         return $this->belongsToMany(User::class, 'barang_user');
18     }
19 }
```

Eloquent Relationship Laravel 11 PHP 8.2

Model User - hasMany Transaksi

```
app/Models/User.php

1  class User extends Authenticatable
2  {
3      use HasFactory, Notifiable;
4
5      public function transaksi(): HasMany
6      {
7          return $this->hasMany(Transaksi::class);
8      }
9
10     public function favorites(): BelongsToMany
11     {
12         return $this->belongsToMany(Barang::class, 'barang_user');
13     }
14
15     public function isAdmin(): bool
16     {
17         return $this->role === 'admin';
18     }
19 }
```

Eloquent Relationship Laravel 11 PHP 8.2

Contoh eager loading di Controllers

```
Controllers - Eager Loading Examples

1  // BarangController - eager loading kategori
2  $query = Barang::with('kategori');
3
4  // TransaksiController - eager loading relasi bertingkat
5  $query = Transaksi::with(['user', 'barang.kategori']);
6
7  // DashboardController - eager loading transaksi terbaru
8  $transaksiTerbaru = Transaksi::with(['user', 'barang'])
9      ->latest()->take(5)->get();
10
11 // KategoriController - withCount barang
12 $kategoris = Kategori::withCount('barangs')->get();
13
14 // BarangController - lazy eager loading di show
15 $barang->load('kategori');
16
17 // API - eager loading
18 return response()->json(Barang::with('kategori')->get());
```

Eloquent Relationship Laravel 11 PHP 8.2

View kategori dengan barangs_count

```
resources/views/pages/kategori/index.blade.php

1  <x-app-layout>
2    <h2>Kategori</h2>
3    <table>
4      <thead>
5        <tr><th>No</th><th>Nama</th>
6          <th>Deskripsi</th>
7          <th>Jumlah Barang</th>
8          <th>Aksi</th></tr>
9      </thead>
10     <tbody>
11       @forelse ($kategoris as $ket)
12         <tr>
13           <td>{{ $loop->iteration }}</td>
14           <td>{{ $ket->nama }}</td>
15           <td>{{ $ket->deskripsi ?? '-' }}</td>
16
17           <td>
18             {{ $ket->barangs_count }}
19           </td>
20
21           <td>
22             <a href="{{ route('kategori.edit', $ket) }}">Edit</a>
23           </td>
24         </tr>
25       @empty
26         <tr><td colspan="5">Belum ada data</td></tr>
27       @endforelse
28     </tbody>
29   </table>
30 </x-app-layout>
```

Eloquent Relationship Laravel 11 PHP 8.2

View barang dengan kategori->nama

```
resources/views/pages/barang/index.blade.php

1  <x-app-layout>
2    <h2>Barang</h2>
3    <table>
4      <thead>
5        <tr><th>No</th><th>Nama</th>
6          <th>Kategori</th><th>Jumlah</th>
7          <th>Kondisi</th><th>Aksi</th></tr>
8      </thead>
9      <tbody>
10        @forelse ($barang as $b)
11          <tr>
12            <td>{{ $loop->iteration }}</td>
13            <td>{{ $b->nama }}</td>
14
15            <td>
16              {{ $b->kategori->nama }}
17            </td>
18
19            <td>{{ $b->jumlah }}</td>
20            <td>{{ $b->kondisi }}</td>
21            <td>
22              <a href="{{ route('barang.edit', $b) }}">Edit</a>
23            </td>
24          </tr>
25        @empty
26          <tr><td colspan="6">Belum ada data</td></tr>
27        @endforelse
28      </tbody>
29    </table>
30 </x-app-layout>
```

Eloquent Relationship Laravel 11 PHP 8.2

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman kategori dengan jumlah barang (withCount)

INVENTARIS
UNIVERSITAS

MENU

Dashboard

Kategori

Barang

Transaksi

LAPORAN

Barang PDF

Transaksi PDF

Barang Excel

Transaksi Excel

AKUN

Profile

Logout

Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Kategori

Kelola data kategori barang

+ Tambah Kategori

No	Nama	Deskripsi	Jml Barang	Aksi
1	Elektronik	Barang elektronik	4	
2	Perabotan	Perabotan kantor	3	
3	Alat Tulis	ATK kantor	8	
4	Komputer	PC dan laptop	2	

Halaman barang dengan nama kategori (belongsTo)

INVENTARIS
UNIVERSITAS

MENU

Dashboard

Kategori

Barang

Transaksi

LAPORAN

Barang PDF

Transaksi PDF

Barang Excel

Transaksi Excel

AKUN

Profile

Logout

Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Barang

Kelola data inventaris barang

+ Tambah Barang

No	Nama	Kategori	Jumlah	Kondisi	Aksi
1	Laptop HP Pavilion	Elektronik	5	Baik	
2	Proyektor Epson	Elektronik	3	Baik	
3	Meja Kantor	Perabotan	10	Baik	
4	Kursi Kantor	Perabotan	8	Baik	
5	Ballpoint	Alat Tulis	50	Baik	

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman transaksi dengan user & barang (hasMany + belongsTo)

 **INVENTARIS**
UNIVERSITAS

MENU

[Dashboard](#)

[Kategori](#)

[Barang](#)

[Transaksi](#)

LAPORAN

[Barang PDF](#)

[Transaksi PDF](#)

[Barang Excel](#)

[Transaksi Excel](#)

AKUN

[Profile](#)

[Logout](#)

 **Admin Inventaris**
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Transaksi

Data peminjaman dengan relasi user dan barang

No	Peminjam	Barang	Jml	Tgl Pinjam	Status
1	Admin Inventaris	Laptop HP Pavilion	1	23/07/2026	Dipinjam
2	Staff Inventaris	Proyektor Epson	1	22/07/2026	Dikembalikan
3	Staff Inventaris	Meja Kantor	2	21/07/2026	Dipinjam
4	Admin Inventaris	Kursi Kantor	1	20/07/2026	Dikembalikan

Dashboard dengan eager loading transaksi terbaru

 **INVENTARIS**
UNIVERSITAS

MENU

[Dashboard](#)

[Kategori](#)

[Barang](#)

[Transaksi](#)

LAPORAN

[Barang PDF](#)

[Transaksi PDF](#)

[Barang Excel](#)

[Transaksi Excel](#)

AKUN

[Profile](#)

[Logout](#)

 **Admin Inventaris**
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Transaksi Terbaru

Menggunakan eager loading with() untuk optimasi query

Peminjam	Barang	Jml	Status	Tanggal
Admin Inventaris	Laptop HP Pavilion	1	Dipinjam	23/07/2026
Staff Inventaris	Proyektor Epson	1	Dikembalikan	22/07/2026
Staff Inventaris	Meja Kantor	2	Dipinjam	21/07/2026

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu mengimplementasikan relasi one-to-many Eloquent

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 20. Relasi Many-to-Many dan Seeder

Materi Pembelajaran : belongsToMany, pivot, seeder
Acara Praktikum/Praktik : Minggu 10/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritikal yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources

4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing
---	-----------------	---------------------------------------	---

c. Indikator Penilaian

Mahasiswa mampu mengimplementasikan relasi many-to-many

d. Dasar Teori

Relasi many-to-many: User favorit Barang (user bisa menyukai banyak barang, barang bisa disukai banyak user). Menggunakan tabel pivot `barang_users`. Seeder untuk data awal.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

1. Buat migration pivot:

Membuat Migration Pivot Table

`php artisan make:migration create_barang_user_table`

Struktur tabel pivot `barang_user`:

- `user_id` — foreign key ke tabel `users` (constrained, cascade delete)
- `barang_id` — foreign key ke tabel `barang` (constrained, cascade delete)
- Primary key komposit (`user_id`, `barang_id`) — mencegah duplikasi
- timestamps — `created_at` dan `updated_at`

Migration pivot table `barang_user`

```
database/migrations/xxxx_create_barang_user_table.php

1 Schema::create('barang_user', function (Blueprint $table) {
2     $table->foreignId('user_id')
3         ->constrained()->onDelete('cascade');
4
5     $table->foreignId('barang_id')
6         ->constrained('barang')->onDelete('cascade');
7
8     $table->primary(['user_id', 'barang_id']);
9     $table->timestamps();
10 });
```

Many to Many Relationship Laravel 11 PHP 8.2

Menjalankan Migrasi

Jalankan perintah migrate untuk menerapkan migrasi ke database:

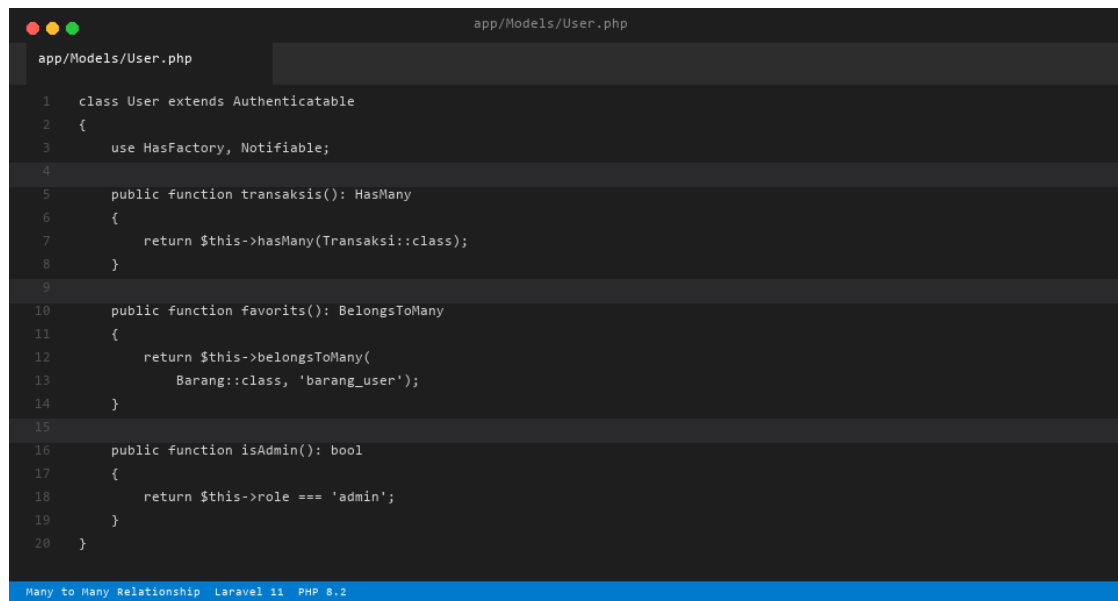
```
php artisan migrate
```

Relasi belongsToMany di Model User

Di model User, tambahkan method `favorites()` yang mendefinisikan relasi `belongsToMany` ke model Barang melalui tabel pivot `barang_user`.

```
public function favorites() { return $this->belongsToMany(Barang::class, 'barang_user'); }
```

Relasi belongsToMany di model User



```
app/Models/User.php

1  class User extends Authenticatable
2  {
3      use HasFactory, Notifiable;
4
5      public function transaksis(): HasMany
6      {
7          return $this->hasMany(Transaksi::class);
8      }
9
10     public function favorites(): BelongsToMany
11     {
12         return $this->belongsToMany(
13             Barang::class, 'barang_user');
14     }
15
16     public function isAdmin(): bool
17     {
18         return $this->role == 'admin';
19     }
20 }
```

Many to Many Relationship Laravel 11 PHP 8.2

Relasi belongsToMany di Model Barang

Di model Barang, tambahkan method `favoritedBy()` untuk relasi kebalikan. Nama method bisa berbeda karena relasi many-to-many simetris.

```
public function favoritedBy() { return $this->belongsToMany(User::class, 'barang_user'); }
```

Relasi belongsToMany di model Barang

```
app/Models/Barang.php

1  class Barang extends Model
2  {
3      protected $table = 'barangs';
4
5      public function kategori(): BelongsTo
6      {
7          return $this->belongsTo(Kategori::class);
8      }
9
10     public function transaksi(): HasMany
11     {
12         return $this->hasMany(Transaksi::class);
13     }
14
15     public function favoritedBy(): BelongsToMany
16     {
17         return $this->belongsToMany(
18             User::class, 'barang_user');
19     }
20 }
```

Many to Many Relationship Laravel 11 PHP 8.2

Membuat DatabaseSeeder

Buat DatabaseSeeder yang memanggil semua seeder secara berurutan. Pastikan urutan: User > Kategori > Barang (karena Barang membutuhkan foreign key ke Kategori).

Buat seeder dengan perintah:

- php artisan make:seeder UserSeeder — seeder untuk tabel users
- php artisan make:seeder KategoriSeeder — seeder untuk tabel kategoris
- php artisan make:seeder BarangSeeder — seeder untuk tabel barangs

DatabaseSeeder dengan semua seeder

```
database/seeder/DatabaseSeeder.php

1  class DatabaseSeeder extends Seeder
2  {
3      public function run(): void
4      {
5          $this->call([
6              UserSeeder::class,
7              KategoriSeeder::class,
8              BarangSeeder::class,
9          ]);
10     }
11 }
```

Many to Many Relationship Laravel 11 PHP 8.2

Menjalankan Seeder

Jalankan perintah seeder untuk mengisi database dengan data awal:

php artisan db:seed

Data yang dihasilkan:

- 2 user (Admin Inventaris + Staff Inventaris)
- 4 kategori (Elektronik, Perabotan, Alat Tulis, Komputer)
- 6 barang (Laptop, Desktop PC, Proyektor, Meja, Kursi, Spidol)

Uji Coba Relasi dengan Tinker

Gunakan Laravel Tinker untuk menguji relasi many-to-many secara interaktif:

- `php artisan tinker` — masuk ke interactive shell
- `$user = User::first()` — ambil user pertama
- `$user->favorites()->attach(Barang::find(1))` — tambahkan barang favorit
- `$user->favorites()->attach(Barang::find(2))` — tambahkan favorit lainnya
- `$user->favorites` — tampilkan semua barang favorit user (query otomatis ke pivot)
- `Barang::find(1)->favoritedBy` — tampilkan user yang memfavoritkan barang

Terminal - migrate, seed, dan uji coba tinker

```
$ php artisan make:migration create_barang_user_table
INFO Migration [database/migrations/xxxx_create_barang_user_table.php] created successfully.

$ php artisan migrate
INFO Running migrations.
xxxx_create_barang_user_table ..... 18ms DONE

$ php artisan db:seed
INFO Seeding database.
Database\Seeders\UserSeeder ..... RUNNING
Database\Seeders\UserSeeder ..... 15ms DONE
Database\Seeders\KategoriSeeder ..... RUNNING
Database\Seeders\KategoriSeeder ..... 12ms DONE
Database\Seeders\BarangSeeder ..... RUNNING
Database\Seeders\BarangSeeder ..... 18ms DONE

$ php artisan tinker
>>> $user = User::first();
>>> $user->favorites()->attach(Barang::find(1));
=> true
>>> $user->favorites()->attach(Barang::find(2));
=> true
>>> $user->favorites;
=> Illuminate\Database\Eloquent\Collection {
  all: [
    App\Models\Barang { id: 1, nama: "Laptop HP Pavilion" },
    App\Models\Barang { id: 2, nama: "Desktop PC" },
  ]
}
```

Study Kasus

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "belongsToMany, pivot, seeder". Analisis bagaimana konsep relasi many-to-many dan seeder diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Berikut adalah kode dari setiap file yang terkait relasi many-to-many:

Migration pivot table barang_user

```
database/migrations/xxxx_create_barang_user_table.php

1 Schema::create('barang_user', function (Blueprint $table) {
2     $table->foreignId('user_id')
3         ->constrained()->onDelete('cascade');
4
5     $table->foreignId('barang_id')
6         ->constrained('barangs')->onDelete('cascade');
7
8     $table->primary(['user_id', 'barang_id']);
9     $table->timestamps();
10 });
```

Many to Many Relationship Laravel 11 PHP 8.2

Model User - belongsToMany Barang (favorites)

```
app/Models/User.php

1 class User extends Authenticatable
2 {
3     use HasFactory, Notifiable;
4
5     public function transaksis(): HasMany
6     {
7         return $this->hasMany(Transaksi::class);
8     }
9
10    public function favorites(): BelongsToMany
11    {
12        return $this->belongsToMany(
13            Barang::class, 'barang_user');
14    }
15
16    public function isAdmin(): bool
17    {
18        return $this->role === 'admin';
19    }
20 }
```

Many to Many Relationship Laravel 11 PHP 8.2

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Model Barang - belongsToMany User (favoritedBy)

```
app/Models/Barang.php

1 class Barang extends Model
2 {
3     protected $table = 'barangs';
4
5     public function kategori(): BelongsTo
6     {
7         return $this->belongsTo(Kategori::class);
8     }
9
10    public function transaksi(): HasMany
11    {
12        return $this->hasMany(Transaksi::class);
13    }
14
15    public function favoritedBy(): BelongsToMany
16    {
17        return $this->belongsToMany(
18            User::class, 'barang_user');
19    }
20 }
```

DatabaseSeeder memanggil semua seeder

```
database/seeder/DatabaseSeeder.php

1 class DatabaseSeeder extends Seeder
2 {
3     public function run(): void
4     {
5         $this->call([
6             UserSeeder::class,
7             KategoriSeeder::class,
8             BarangSeeder::class,
9         ]);
10    }
11 }
```

Terminal - migrate, seed, dan uji coba tinker

```
$ php artisan make:migration create_barang_user_table
INFO Migration [database/migrations/xxxx_create_barang_user_table.php] created successfully.

$ php artisan migrate
INFO Running migrations.
xxxx_create_barang_user_table ..... 10ms DONE

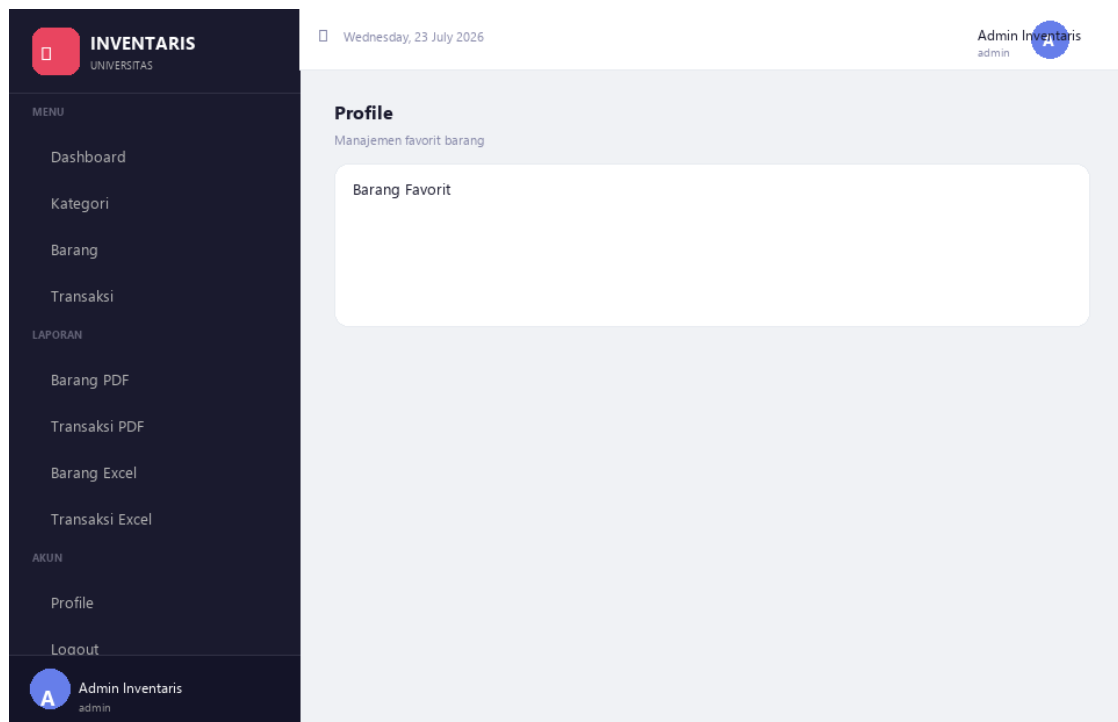
$ php artisan db:seed
INFO Seeding database.
Database\Seeders\UserSeeder ..... RUNNING
Database\Seeders\UserSeeder ..... 15ms DONE
Database\Seeders\KategoriSeeder ..... RUNNING
Database\Seeders\KategoriSeeder ..... 12ms DONE
Database\Seeders\BarangSeeder ..... RUNNING
Database\Seeders\BarangSeeder ..... 18ms DONE

$ php artisan tinker
>>> $user = User::first();
>>> $user->favorites()->attach(Barang::find(1));
=> true
>>> $user->favorites()->attach(Barang::find(2));
=> true
>>> $user->favorites;
=> Illuminate\Database\Eloquent\Collection {
    all: [
        App\Models\Barang { id: 1, nama: "Laptop HP Pavilion" },
        App\Models\Barang { id: 2, nama: "Desktop PC" },
    ]
}
```

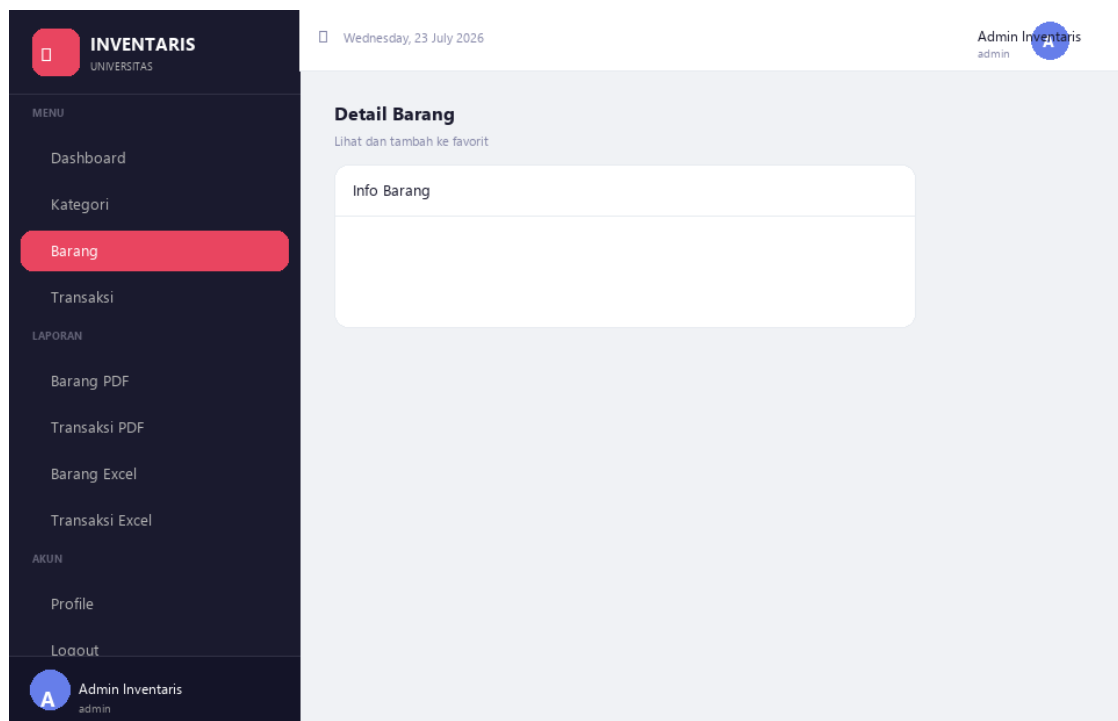
BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Berikut adalah tampilan aplikasi setelah implementasi relasi many-to-many:

Profile user dengan daftar barang favorit



Detail barang dengan daftar user yang memfavoritkan



Daftar barang dengan jumlah favorit

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

INVENTARIS
UNIVERSITAS

Wednesday, 23 July 2026

Admin Inventaris
admin

Barang
Daftar barang dengan status favorit

No	Nama	Kategori	Favorit
1	Laptop HP	Elektronik	☐
2	Proyektor	Elektronik	☒

MENU

- Dashboard
- Kategori
- Barang**
- Transaksi

LAPORAN

- Barang PDF
- Transaksi PDF
- Barang Excel
- Transaksi Excel

AKUN

- Profile
- Logout

Admin Inventaris
admin

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu mengimplementasikan relasi many-to-many Eloquent

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

Acara 21. Ajax Request dengan jQuery

Materi Pembelajaran : Ajax, jQuery, JSON response
 Acara Praktikum/Praktik : Minggu 11/1
 Tempat : Daring/Luring Politeknik Negeri Jember
 Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritis yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources
4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing

c. Indikator Penilaian

Mahasiswa mampu mengimplementasikan Ajax request

d. Dasar Teori

Dalam teori pengembangan aplikasi web, Ajax (Asynchronous JavaScript and XML) adalah teknik yang memungkinkan pengiriman dan penerimaan data antara client dan server tanpa harus melakukan reload halaman secara penuh. Dengan Ajax, pengalaman pengguna menjadi lebih interaktif dan responsif karena hanya bagian tertentu dari halaman yang diperbarui. Secara teknis, Ajax bekerja dengan memanfaatkan objek XMLHttpRequest atau API modern seperti fetch, yang mengirimkan request ke server dan menerima response secara asinkron.

Dalam praktiknya, jQuery Ajax sering digunakan karena menyediakan sintaks yang lebih sederhana untuk melakukan request GET maupun POST. Misalnya, \$.ajax() atau shortcut seperti \$.get() dan \$.post() dapat digunakan untuk mengirim data ke server dan menerima hasilnya. Pada sisi server, khususnya dalam framework seperti Laravel, controller biasanya mengembalikan response dalam format JSON. Data JSON ini kemudian ditangani oleh JavaScript di sisi client untuk memperbarui tampilan halaman sesuai kebutuhan, misalnya menampilkan daftar produk baru, validasi form, atau update status transaksi.

Secara teoritis, Ajax mendukung prinsip asynchronous communication dan user experience enhancement, karena memungkinkan aplikasi web berfungsi layaknya aplikasi desktop dengan interaksi yang cepat dan dinamis. Dengan kombinasi jQuery Ajax dan response JSON dari controller, pengembangan aplikasi menjadi lebih efisien, modular, dan ramah pengguna, sekaligus menjaga pemisahan yang jelas antara logika server dan presentasi di client.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Instalasi jQuery via NPM

Install jQuery menggunakan npm:

```
npm install jquery
```

Setelah instalasi, daftarkan jQuery di file resources/js/bootstrap.js:

```
import jQuery from "jquery";  
window.$ = window.jQuery = jQuery;  
Kemudian compile asset:  
npm run build
```

Install jQuery dan buat view Ajax test

```
$ npm install jquery  
  
added 1 package in 2s  
  
$ npm ls jquery  
proyek@ /var/www/proyek  
?-- jquery@3.7.1  
  
$ php artisan make:controller AjaxController  
INFO Controller [app/Http/Controllers/AjaxController.php] created successfully.  
  
$ php artisan make:view pages.ajax-test  
INFO View [resources/views/pages/ajax-test.blade.php] created successfully.
```

Membuat Route API Endpoint

Daftarkan route GET di routes/web.php yang mengarah ke method apiIndex di BarangController. Juga daftarkan route untuk halaman Ajax test.

```
Route::get('/barang/api/list', [BarangController::class, 'apiIndex'])-  
>name('barang.api');
```

Route API endpoint dan halaman Ajax test

```
routes/web.php  
1 Route::middleware(['auth', 'verified'])->group(function () {  
2     Route::get('/', [DashboardController::class, 'index'])  
3         ->name('dashboard');  
4  
5     Route::get('/barang/api/list', [  
6         BarangController::class, 'apiIndex'  
7     ])->name('barang.api');  
8  
9     Route::get('/ajax-test', function () {  
10         return view('pages.ajax-test');  
11     }->name('ajax.test');  
12  
13     Route::resource('transaksi', TransaksiController::class);  
14 });  
  
Ajax & jQuery Laravel 11 PHP 8.2
```

Method apiIndex() di Controller

Buat method apiIndex() di BarangController yang mengembalikan data barang beserta kategorinya dalam format JSON.

Fungsi method:

- Query barang dengan eager loading kategori menggunakan with('kategori')

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

- Pilih kolom yang diperlukan: id, nama, kategori_id, jumlah
- Kembalikan data sebagai JSON response menggunakan response()->json()
- Data akan otomatis mencakup relasi kategori (nama kategori)

Method apiIndex mengembalikan JSON

```
app/Http/Controllers/BarangController.php

1 class BarangController extends Controller
2 {
3
4     public function apiIndex()
5     {
6         $barangs = Barang::with('kategori')
7             ->select('id', 'nama', 'kategori_id', 'jumlah')
8             ->get();
9
10        return response()->json($barangs);
11    }
12
13    public function index(Request $request)
14    {
15        $barangs = Barang::with('kategori')
16            ->paginate(10);
17        return view('pages.barang.index', ...);
18    }
19 }
```

Membuat View Ajax Test

Buat view blade untuk halaman Ajax test. Di dalamnya terdapat tombol "Load Barang" dan div #result sebagai tempat menampilkan hasil.

View dengan button dan container result

```
resources/views/pages/ajax-test.blade.php

1 <x-app-layout>
2     <div class="page-header">
3         <h2>Ajax Test</h2>
4         <p>Uji coba request Ajax dengan jQuery</p>
5     </div>
6
7     <div class="card" style="padding:24px">
8         <button id="loadData" class="btn btn-primary">
9             <i class="fas fa-sync"></i> Load Barang
10        </button>
11
12        <div id="result" style="margin-top:20px">
13            <p style="color:#adb5c4">
14                Klik tombol untuk memuat data...
15            </p>
16        </div>
17    </div>
18
19    @push('scripts')
20    <script>
21        $(document).ready(function() {
22            $("#loadData").click(function() {
23                $.getJSON("/barang/api/list", function(data) {
24                    let html = "<ul>";
25                    $.each(data, function(i, item) {
26                        html += "<li>" + item.nama
27                            + " (" + item.kategori.nama + ")</li>";
28                    });
29                    html += "</ul>";
30                    $("#result").html(html);
31                });
32            });
33        });
34    </script>
35    @endpush
36 </x-app-layout>
```

Script jQuery untuk Ajax Request

Tambahkan script jQuery di view untuk menangani klik tombol dan melakukan Ajax request ke endpoint API.

Alur script:

- `$(document).ready()` — pastikan DOM sudah siap sebelum script dijalankan
- `$("#loadData").click()` — event handler saat tombol diklik
- `$.getJSON("/barang/api/list", callback)` — GET request ke API endpoint
- Callback memproses data JSON: iterasi data, buat HTML list
- `$("#result").html(html)` — tampilkan hasil di halaman tanpa reload

Uji Coba dan Inspeksi Network

Langkah pengujian:

- Buka halaman Ajax Test (<http://127.0.0.1:8000/ajax-test>)
- Buka browser DevTools (F12), pilih tab Network
- Filter request dengan XHR untuk melihat Ajax request saja
- Klik tombol "Load Barang" — perhatikan request baru muncul di tab Network
- Klik request tersebut untuk melihat response JSON di tab Preview/Response
- Data barang akan tampil di halaman tanpa reload — perhatikan URL tidak berubah

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Ajax, jQuery, JSON response". Analisis bagaimana konsep ajax request dengan jquery diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Berikut adalah kode dari setiap file yang terkait Ajax dan jQuery:

Terminal - Install jQuery dan membuat view

```
$ npm install jquery

added 1 package in 2s

$ npm ls jquery
proyek@ /var/www/proyek
|-- jquery@3.7.1

$ php artisan make:controller AjaxController
INFO Controller [app/Http/Controllers/AjaxController.php] created successfully.

$ php artisan make:view pages.ajax-test
INFO View [resources/views/pages/ajax-test.blade.php] created successfully.
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Route API di web.php

```
routes/web.php

1 Route::middleware(['auth', 'verified'])->group(function () {
2     Route::get('/', [DashboardController::class, 'index'])
3         ->name('dashboard');
4
5     Route::get('/barang/api/list', [
6         BarangController::class, 'apiIndex'
7     ]->name('barang.api'));
8
9     Route::get('/ajax-test', function () {
10         return view('pages.ajax-test');
11     }->name('ajax.test'));
12
13     Route::resource('transaksi', TransaksiController::class);
14 });
```

Method apiIndex di BarangController

```
app/Http/Controllers/BarangController.php

1 class BarangController extends Controller
2 {
3
4     public function apiIndex()
5     {
6         $barang = Barang::with('kategori')
7             ->select('id', 'nama', 'kategori_id', 'jumlah')
8             ->get();
9
10        return response()->json($barang);
11    }
12
13    public function index(Request $request)
14    {
15        $barang = Barang::with('kategori')
16            ->paginate(10);
17        return view('pages.barang.index', ...);
18    }
19 }
```

View ajax-test.blade.php dengan jQuery script

```
resources/views/pages/ajax-test.blade.php

1 <x-app-layout>
2     <div class="page-header">
3         <h2>Ajax Test</h2>
4         <p>Uji coba request Ajax dengan jQuery</p>
5     </div>
6
7     <div class="card" style="padding:24px">
8         <button id="loadData" class="btn btn-primary">
9             <i class="fas fa-sync"></i> Load Barang
10        </button>
11
12        <div id="result" style="margin-top:20px">
13            <p style="color:#adb5c4">
14                Klik tombol untuk memuat data...
15            </p>
16        </div>
17    </div>
18
19    @push('scripts')
20    <script>
21        $(document).ready(function() {
22            $("#loadData").click(function() {
23                $.getJSON("/barang/api/list", function(data) {
24                    let html = "<ul>";
25                    $.each(data, function(i, item) {
26                        html += "<li>" + item.nama
27                            + " (" + item.kategori.nama + ")</li>";
28                    });
29                    html += "</ul>";
30                    $("#result").html(html);
31                });
32            });
33        });
34    </script>
35    @endpush
36 </x-app-layout>
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Berikut adalah tampilan aplikasi saat uji coba Ajax:

Halaman Ajax Test sebelum load data

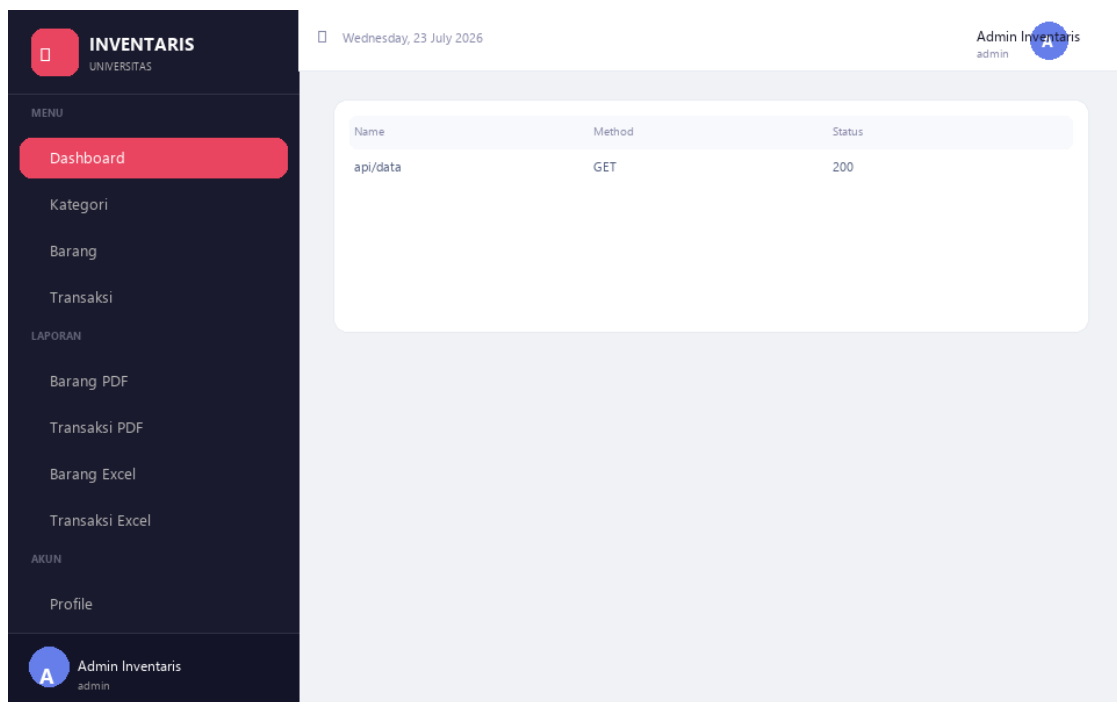
The screenshot shows the 'INVENTARIS UNIVERSITAS' dashboard. The left sidebar contains a menu with 'Dashboard' (highlighted), 'Kategori', 'Barang', 'Transaksi', 'LAPORAN' (with sub-items: 'Barang PDF', 'Transaksi PDF', 'Barang Excel', 'Transaksi Excel'), and 'AKUN' (with 'Profile'). The top right shows the date 'Wednesday, 23 July 2026' and the user 'Admin Inventaris admin'. The main content area is titled 'Ajax Test' with the instruction 'Klik tombol untuk memuat data'. Below this is a form with a label 'Hasil:' and an empty input field.

Data barang berhasil dimuat via Ajax (tanpa reload)

The screenshot shows the same dashboard as above, but the main content area now displays 'Data berhasil dimuat' (Data successfully loaded). Below this message, the form shows 'Data: 5 item dimuat' (Data: 5 items loaded) in the input field, indicating that the data has been successfully retrieved without a page reload.

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Browser DevTools menampilkan request Ajax dan response JSON



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu mengimplementasikan Ajax request

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 22. DataTables Server Side

Materi Pembelajaran : DataTables, server-side processing
Acara Praktikum/Praktik : Minggu 11/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.62010 0.036.0 1	Melaksanakan pengujian kode program secara statis	Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis

c. Indikator Penilaian

Mahasiswa mampu mengimplementasikan Ajax request

d. Dasar Teori

Dalam teori pengembangan aplikasi web, DataTables adalah plugin jQuery yang digunakan untuk membuat tabel interaktif dengan fitur-fitur penting seperti sorting (pengurutan data), searching (pencarian data), dan pagination (pembagian halaman). DataTables bekerja dengan menambahkan fungsi JavaScript pada tabel HTML sehingga pengguna dapat mengelola dan menavigasi data dengan lebih mudah tanpa harus menulis logika kompleks secara manual.

Untuk dataset kecil, DataTables dapat memproses data langsung di sisi client. Namun, ketika berhadapan dengan jumlah data yang sangat besar, digunakan konsep server-side processing, yaitu mekanisme di mana DataTables hanya meminta data yang diperlukan dari server sesuai dengan parameter tertentu (misalnya halaman ke berapa, jumlah baris, kata kunci pencarian). Server kemudian memproses query tersebut dan mengembalikan hasil dalam format JSON yang siap ditampilkan di tabel. Dengan cara ini, performa aplikasi tetap optimal karena beban pemrosesan data besar dialihkan ke server, sementara client hanya menampilkan data yang relevan.

Secara teoritis, penggunaan DataTables dengan server-side processing mendukung prinsip efisiensi data handling dan user experience enhancement. Efisiensi tercapai karena sistem tidak perlu memuat seluruh data sekaligus, sedangkan pengalaman pengguna meningkat karena tabel tetap responsif meskipun data berjumlah ribuan atau jutaan. Dengan demikian, DataTables menjadi solusi praktis untuk aplikasi berbasis data yang membutuhkan tampilan tabel interaktif, scalable, dan mudah digunakan.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Instalasi DataTables dan Yajra

Install package yang diperlukan:

composer require yajra/laravel-datatables-oracle

Install frontend DataTables:

npm install datatables.net-bs5 datatables.net-dt

Install Yajra DataTables dan frontend

```
$ composer require yajra/laravel-datatables-oracle
Installing yajra/laravel-datatables-oracle (^12.7)
- Installing yajra/laravel-datatables-oracle ... DONE

$ npm install datatables.net-bs5 datatables.net-dt
+ datatables.net-bs5@2.2.2
+ datatables.net-dt@2.2.2

$ php artisan vendor:publish --tag=datatables
Copied File [vendor/yajra/.../datatables.php] to [config/datatables.php]
```

Membuat Route DataTables

Daftarkan dua route di web.php: satu untuk endpoint JSON DataTables dan satu untuk halaman yang menampilkan DataTables.

Route DataTables JSON dan halaman

```
routes/web.php

1 Route::middleware(['auth', 'verified'])->group(function () {
2     Route::resource('barang', BarangController::class);
3
4     Route::get('/barang/data/datatables', [
5         BarangController::class, 'dataTables'
6     ])->name('barang.datatables');
7
8     Route::get('/barang/datatables', function () {
9         return view('pages.barang.datatables');
10    });
11
12    Route::get('/barang/api/list', ...);
13 });
```

DataTables Server Side Laravel 11 PHP 8.2

Method dataTables() di Controller

Buat method dataTables() di BarangController yang menggunakan Yajra DataTables. Method ini akan dipanggil oleh DataTables via Ajax untuk setiap interaksi (pagination, search, sorting).

Penjelasan method:

- `Barang::with('kategori')` — eager loading relasi
- `select('barangs.*')` — hindari kolom duplikat dari join
- `DataTables::of($barangs)` — buat instance DataTables dari query
- `addColumn('kategori')` — tambah kolom dari relasi (nama kategori)
- `addColumn('action')` — tambah kolom action dengan HTML buttons
- `rawColumns(['action'])` — agar HTML tidak di-escape
- `make(true)` — generate response JSON

Method dataTables() dengan Yajra

```
app/Http/Controllers/BarangController.php

1 use Yajra\DataTables\Facades\DataTables;
2
3 class BarangController extends Controller
4 {
5
6     public function dataTables(Request $request)
7     {
8         $barangs = Barang::with('kategori')
9             ->select('barangs.*');
10
11         return DataTables::of($barangs)
12             ->addColumn('kategori', fn($b) =>
13                 $b->kategori->nama)
14             ->addColumn('action', function ($b) {
15                 return '<a href="...">Edit</a>';
16             })
17             ->rawColumns(['action'])
18             ->make(true);
19     }
20 }
```

DataTables Server Side Laravel 11 PHP 8.2

Membuat View dengan DataTables

Buat view blade yang menampilkan DataTables dengan konfigurasi server-side.

Tabel hanya perlu <thead> karena data akan diisi oleh DataTables via Ajax.

Konfigurasi DataTables:

- processing: true — tampilkan indikator processing saat request
- serverSide: true — aktifkan mode server-side
- ajax: route('barang.dataTables') — endpoint JSON
- columns — mapping kolom dengan data dari response JSON
- DT_RowIndex — nomor urut otomatis dari server
- orderable: false — nonaktifkan sorting untuk kolom No dan Aksi

View dengan inisialisasi DataTables server-side

```
resources/views/pages/barang/datatables.blade.php

1 <x-app-layout>
2 <h2>Barang - DataTables</h2>
3
4 <link rel="stylesheet" href="https://cdn.datatables.net/2.2.2/css/dataTables.dataTables.css">
5
6 <table id="barangTable" class="display" style="width:100%">
7   <thead>
8     <tr><th>No</th><th>Nama</th>
9     <th>Kategori</th><th>Jumlah</th>
10    <th>Aksi</th></tr>
11  </thead>
12 </table>
13
14 <script src="https://cdn.datatables.net/2.2.2/js/dataTables.js"></script>
15 <script>
16   $(document).ready(function() {
17     $("#barangTable").DataTable({
18       processing: true,
19       serverSide: true,
20       ajax: "{{ route('barang.dataTables') }}",
21       columns: [
22         {data: 'DT_RowIndex', orderable: false},
23         {data: 'nama'},
24         {data: 'kategori'},
25         {data: 'jumlah'},
26         {data: 'action', orderable: false},
27       ]
28     });
29   });
30 </script>
31 </x-app-layout>
```

Uji Coba Sorting, Searching, Pagination

Langkah pengujian:

- Buka halaman DataTables, pastikan tabel menampilkan data barang dari server
- Klik header kolom untuk sorting ascending/descending — perhatikan DataTables mengirim parameter order ke server
- Ketik kata kunci di kotak search — DataTables mencari ke server secara real-time
- Ganti jumlah entries per halaman (10/25/50/100) — DataTables mengirim parameter length
- Navigasi halaman dengan tombol Previous/Next — DataTables mengirim parameter start
- Buka DevTools tab Network, filter XHR, dan lihat request/response DataTables

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "DataTables, server-side processing". Analisis bagaimana konsep datatables server side diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Berikut adalah kode dari setiap file yang terkait DataTables server-side:

Terminal - Install DataTables

```
$ composer require yajra/laravel-datatables-oracle
Installing yajra/laravel-datatables-oracle (^12.7)
- Installing yajra/laravel-datatables-oracle ... DONE

$ npm install datatables.net-bs5 datatables.net-dt
+ datatables.net-bs5@2.2.2
+ datatables.net-dt@2.2.2

$ php artisan vendor:publish --tag=datatables
Copied File [vendor/yajra/.../datatables.php] to [config/datatables.php]
```

Route DataTables di web.php

```
routes/web.php

1 Route::middleware(['auth', 'verified'])->group(function () {
2     Route::resource('barang', BarangController::class);
3
4     Route::get('/barang/data/datatables', [
5         BarangController::class, 'dataTables'
6     ])->name('barang.datatables');
7
8     Route::get('/barang/datatables', function () {
9         return view('pages.barang.datatables');
10    })->name('barang.datatables');
11
12    Route::get('/barang/api/list', ...);
13 });
```

Method dataTable() di BarangController

```
app/Http/Controllers/BarangController.php

1 use Yajra\DataTables\Facades\DataTables;
2
3 class BarangController extends Controller
4 {
5
6     public function dataTable(Request $request)
7     {
8         $barangs = Barang::with('kategori')
9             ->select('barangs.*');
10
11         return DataTables::of($barangs)
12             ->addColumn('kategori', fn($b) =>
13                 $b->kategori->nama)
14             ->addColumn('action', function ($b) {
15                 return '<a href="#">Edit</a>';
16             })
17             ->rawColumns(['action'])
18             ->make(true);
19     }
20 }
```

View DataTables dengan inisialisasi server-side

```
resources/views/pages/barang/datatables.blade.php

1 <x-app-layout>
2   <h2>Barang - DataTables</h2>
3
4   <link rel="stylesheet" href="https://cdn.datatables.net/2.2.2/css/dataTables.dataTables.css">
5
6   <table id="barangTable" class="display" style="width:100%">
7     <thead>
8       <tr><th>No</th><th>Nama</th>
9       <th>Kategori</th><th>Jumlah</th>
10      <th>Aksi</th></tr>
11    </thead>
12  </table>
13
14  <script src="https://cdn.datatables.net/2.2.2/js/dataTables.js"></script>
15  <script>
16    $(document).ready(function() {
17      $("#barangTable").DataTable({
18        processing: true,
19        serverSide: true,
20        ajax: "{{ route('barang.dataTables') }}",
21        columns: [
22          {data: 'DT_RowIndex', orderable: false},
23          {data: 'nama'},
24          {data: 'kategori'},
25          {data: 'jumlah'},
26          {data: 'action', orderable: false},
27        ]
28      });
29    });
30  </script>
31 </x-app-layout>
```

Berikut adalah tampilan aplikasi setelah implementasi DataTables:

DataTables dengan sorting dan pagination server-side

 **INVENTARIS**
UNIVERSITAS

MENU

Dashboard

Kategori

Barang

Transaksi

LAPORAN

Barang PDF

Transaksi PDF

Barang Excel

Transaksi Excel

AKUN

Profile

Logout

 Admin Invenaris
admin

Wednesday, 23 July 2026

Admin Invenaris
admin

Barang

DataTables dengan server-side processing

No	Nama	Kategori	Stok
1	Laptop HP	Elektronik	5
2	Proyektor	Elektronik	3
3	Meja Kantor	Perabotan	10

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Pencarian real-time dengan filter server-side

The screenshot shows the 'INVENTARIS UNIVERSITAS' web application. The left sidebar menu is visible with options like Dashboard, Kategori, Barang (highlighted), Transaksi, and various reports. The main content area displays a search filter for 'laptop' and a table with one result.

Wednesday, 23 July 2026

Admin Inventaris
admin

Search: laptop Kategori: Semua Filter

No	Nama	Kategori	Stok
1	Laptop HP	Elektronik	5

Response JSON dari DataTables endpoint

The screenshot shows the 'INVENTARIS UNIVERSITAS' web application with the 'Dashboard' menu item highlighted. The main content area displays the JSON response from the DataTables endpoint.

Wednesday, 23 July 2026

Admin Inventaris
admin

Key Value

draw	1
recordsTotal	42
data	[...]

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu mengimplementasikan DataTables server-side

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 23. Filter Data dan Pencarian

Materi Pembelajaran : Filter, search, pagination
 Acara Praktikum/Praktik : Minggu 12/1
 Tempat : Daring/Luring Politeknik Negeri Jember
 Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritis yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources
4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing

c. Indikator Penilaian

Mahasiswa mampu membuat filter data dan pencarian dinamis

d. Dasar Teori

Dalam teori pengembangan aplikasi web, DataTables adalah plugin jQuery yang digunakan untuk membuat tabel interaktif dengan fitur-fitur seperti sorting (pengurutan data), searching (pencarian data), dan pagination (pembagian halaman). Fitur ini menjadikan tabel lebih dinamis dan ramah pengguna, karena data dapat diurutkan, dicari, dan ditampilkan secara bertahap tanpa harus memuat ulang seluruh halaman. Untuk dataset kecil, DataTables dapat memproses data langsung di sisi client. Namun, ketika berhadapan dengan jumlah data yang besar, digunakan konsep server-side processing, yaitu mekanisme di mana DataTables hanya meminta data yang relevan dari server sesuai parameter tertentu (misalnya halaman ke berapa, jumlah baris, atau kata kunci pencarian). Server kemudian memproses query tersebut dan mengembalikan hasil dalam format JSON, sehingga performa tetap optimal meskipun data berjumlah ribuan atau jutaan.

Search dalam konteks DataTables adalah proses pencarian data berdasarkan kata kunci yang dimasukkan pengguna. Secara teoritis, search mendukung prinsip information retrieval, yaitu menemukan informasi yang relevan dari kumpulan data besar dengan cepat. Pada client-side, pencarian dilakukan langsung di browser dengan memfilter data yang sudah dimuat. Sedangkan pada server-side, pencarian dilakukan melalui query database, sehingga lebih efisien untuk data berukuran besar.

Pagination adalah teknik membagi data menjadi beberapa halaman agar lebih mudah ditampilkan dan diakses. Secara teoritis, pagination mendukung prinsip data segmentation dan usability, karena pengguna tidak harus melihat seluruh data sekaligus, melainkan dapat menavigasi data secara bertahap. Dalam DataTables, pagination dapat dilakukan secara otomatis di sisi client, atau dikendalikan oleh server melalui server-side processing.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Filter di BarangController

Di method index() BarangController, tambahkan kondisi filter berdasarkan input request. Filter yang tersedia: kategori (dropdown), kondisi (baik/rusak/hilang), dan pencarian (search by nama/deskripsi).

Fungsi filter:

- kategori — filter berdasarkan kategori_id, nilai dari dropdown kategori
- kondisi — filter enum kondisi barang (baik, rusak, hilang)
- search — pencarian dengan LIKE pada kolom nama dan deskripsi

Filter kategori, kondisi, dan search di BarangController

```
app/Http/Controllers/BarangController.php

1 public function index(Request $request)
2 {
3     $query = Barang::with('kategori');
4
5     if ($request->filled('kategori')) {
6         $query->where('kategori_id', $request->kategori);
7     }
8
9     if ($request->filled('kondisi')) {
10        $query->where('kondisi', $request->kondisi);
11    }
12
13    if ($request->filled('search')) {
14        $search = $request->search;
15        $query->where(function ($q) use ($search) {
16            $q->where('nama', 'like', "%{$search}%")
17                ->orWhere('deskripsi', 'like', "%{$search}%");
18        });
19    }
20
21    $barangs = $query->paginate(10)->withQueryString();
22    $kategoris = Kategori::all();
23    return view('pages.barang.index', compact(...));
24 }
```

Filter di TransaksiController

Di TransaksiController, filter ditambahkan untuk status transaksi dan pencarian pada data relasi (nama barang dan nama peminjam). Juga ada role-based filter untuk staff.

Filter status, search relasi, dan role scoping

```
app/Http/Controllers/TransaksiController.php

1 public function index(Request $request)
2 {
3     $query = Transaksi::with(['user', 'barang.kategori']);
4
5     if ($request->filled('status')) {
6         $query->where('status', $request->status);
7     }
8
9     if ($request->filled('search')) {
10        $search = $request->search;
11        $query->whereHas('barang', function ($q) use ($search) {
12            $q->where('nama', 'like', "%{$search}%");
13        })->orWhereHas('user', function ($q) use ($search) {
14            $q->where('name', 'like', "%{$search}%");
15        });
16    }
17
18    if (Auth::user()->role === 'staff') {
19        $query->where('user_id', Auth::id());
20    }
21
22    $transaksis = $query->latest()->paginate(10)
23        ->withQueryString();
24    return view('pages.transaksi.index', compact(...));
25 }
```

Membuat Form Filter di View

Buat form dengan metode GET di view barang/index. Form menggunakan request() helper untuk mempertahankan nilai yang dipilih sebelumnya. Tombol Reset mengarah ke route barang.index tanpa query string.

Elemen form filter:

- Select Kategori — mengambil data dari \$kategoris, menandai selected dengan request('kategori')
- Select Kondisi — opsi: Semua, Baik, Rusak, Hilang
- Input Search — text input untuk kata kunci pencarian
- Button Filter — submit form dengan metode GET
- Link Reset — href ke route barang.index untuk menghapus semua filter

Filter form dengan request() helper

```
resources/views/pages/barang/index.blade.php

1 <x-app-layout>
2 <h2>Barang</h2>
3
4 <form method="GET" class="filter-card">
5 <select name="kategori" class="form-control">
6 <option value="">Semua Kategori</option>
7 @foreach ($kategoris as $k)
8 <option value="{{ $k->id }}"
9 {{ request('kategori') == $k->id ? "selected" : "" }}>
10 {{ $k->nama }}
11 </option>
12 @endforeach
13 </select>
14
15 <select name="kondisi" class="form-control">
16 <option value="">Semua Kondisi</option>
17 <option value="baik" {{ request('kondisi')=="baik"... "selected" }}>Baik</option>
18 <option value="rusak" {{ request('kondisi')=="rusak"... "selected" }}>Rusak</option>
19 <option value="hilang" {{ request('kondisi')=="hilang"... "selected" }}>Hilang</option>
20 </select>
21
22 <input name="search" placeholder="Cari barang..."
23 value="{{ request('search') }}">
24
25 <button type="submit">Filter</button>
26 <a href="{{ route('barang.index') }}">Reset</a>
27 </form>
28
29 <!-- Table barang dengan pagination -->
30 {{ $barangs->links() }}
31 </x-app-layout>
```

Mempertahankan Filter dengan withQueryString()

Method withQueryString() pada pagination memastikan semua query string tetap ada di link pagination. Tanpa ini, filter akan hilang saat pengguna berpindah halaman.

```
$barangs = $query->paginate(10)->withQueryString();
```

withQueryString() mempertahankan filter saat pagination

```
Pagination - withQueryString()

1 // BarangController - pertahankan filter saat pagination
2 $barangs = $query->paginate(10)->withQueryString();
3
4 // Tanpa withQueryString(), pagination akan hilang
5 // saat pindah halaman
6
7 // Di view, links() otomatis menyertakan query string
8 {{ $barangs->links() }}
9
10 // Alternatif: appends() untuk custom parameter
11 {{ $barangs->appends(['sort' => 'nama']->links() }}
12
13 // Akses nilai filter di view
14 request('kategori') // nilai dari query string
15 request('search') // nilai pencarian
16 request('status') // filter status transaksi
```

Filter Transaksi Berdasarkan Status

Transaksi dapat difilter berdasarkan status (dipinjam/dikembalikan) dan pencarian berdasarkan nama barang atau nama peminjam. Filter status menggunakan enum status yang sudah didefinisikan di migration.

Parameter filter transaksi:

- status — enum: "dipinjam" atau "dikembalikan"
- search — LIKE pada relasi barang.nama dan user.name menggunakan whereHas()
- Role staff — otomatis memfilter hanya transaksi milik user yang login

Uji Coba Filter dan Pencarian

Langkah pengujian:

- Buka halaman Barang, pilih kategori tertentu dari dropdown
- Klik Filter — data akan difilter sesuai kategori yang dipilih
- Perhatikan URL berubah: /barang?kategori=1
- Tambahkan pencarian dengan keyword, klik Filter lagi
- Klik link pagination (halaman 2) — pastikan filter tetap aktif
- Klik Reset — semua filter hilang, kembali ke data lengkap
- Uji filter status di halaman Transaksi

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Filter, search, pagination". Analisis bagaimana konsep filter data dan pencarian diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Berikut adalah kode dari setiap file yang terkait filter dan pencarian:

Filter di BarangController

```
app/Http/Controllers/BarangController.php
1 public function index(Request $request)
2 {
3     $query = Barang::with('kategori');
4
5     if ($request->filled('kategori')) {
6         $query->where('kategori_id', $request->kategori);
7     }
8
9     if ($request->filled('kondisi')) {
10        $query->where('kondisi', $request->kondisi);
11    }
12
13    if ($request->filled('search')) {
14        $search = $request->search;
15        $query->where(function ($q) use ($search) {
16            $q->where('nama', 'like', "%{$search}%")
17                ->orWhere('deskripsi', 'like', "%{$search}%");
18        });
19    }
20
21    $barangs = $query->paginate(10)->withQueryString();
22    $kategori = Kategori::all();
23    return view('pages.barang.index', compact(...));
24 }
```

Filter di TransaksiController

```
app/Http/Controllers/TransaksiController.php

1 public function index(Request $request)
2 {
3     $query = Transaksi::with(['user', 'barang.kategori']);
4
5     if ($request->filled('status')) {
6         $query->where('status', $request->status);
7     }
8
9     if ($request->filled('search')) {
10        $search = $request->search;
11        $query->whereHas('barang', function ($q) use ($search) {
12            $q->where('nama', 'like', "%{$search}%");
13        })->orWhereHas('user', function ($q) use ($search) {
14            $q->where('name', 'like', "%{$search}%");
15        });
16    }
17
18    if (Auth::user()->role === 'staff') {
19        $query->where('user_id', Auth::id());
20    }
21
22    $transaksis = $query->latest()->paginate(10)
23        ->withQueryString();
24    return view('pages.transaksi.index', compact(...));
25 }
```

Form filter di view barang

```
resources/views/pages/barang/index.blade.php

1 <x-app-layout>
2     <h2>Barang</h2>
3
4     <form method="GET" class="filter-card">
5         <select name="kategori" class="form-control">
6             <option value="">Semua Kategori</option>
7             @foreach ($kategoris as $k)
8                 <option value="{{ $k->id }}"
9                     {{ request('kategori') == $k->id ? "selected" : "" }}>
10                     {{ $k->nama }}
11             </option>
12         @endforeach
13     </select>
14
15     <select name="kondisi" class="form-control">
16         <option value="">Semua Kondisi</option>
17         <option value="baik" {{ request('kondisi')=="baik"... "selected" }}>Baik</option>
18         <option value="rusak" {{ request('kondisi')=="rusak"... "selected" }}>Rusak</option>
19         <option value="hilang" {{ request('kondisi')=="hilang"... "selected" }}>Hilang</option>
20     </select>
21
22     <input name="search" placeholder="Cari barang..."
23         value="{{ request('search') }}">
24
25     <button type="submit">Filter</button>
26     <a href="{{ route('barang.index') }}">Reset</a>
27 </form>
28
29 <!-- Table barang dengan pagination -->
30 {{ $barang->links() }}
31 </x-app-layout>
```

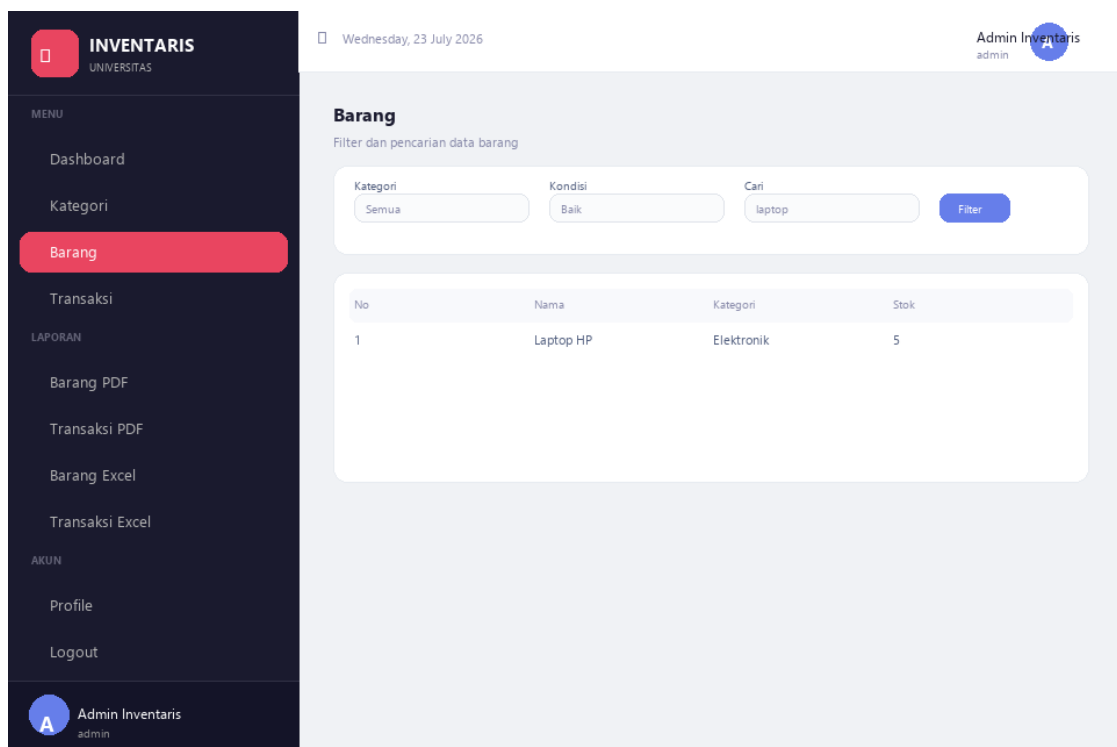
withQueryString() untuk pagination

```
Pagination - withQueryString()

1 // BarangController - pertahankan filter saat pagination
2 $barangs = $query->paginate(10)->withQueryString();
3
4 // Tanpa withQueryString(), pagination akan hilang
5 // saat pindah halaman
6
7 // Di view, links() otomatis menyertakan query string
8 {{ $barangs->links() }}
9
10 // Alternatif: appends() untuk custom parameter
11 {{ $barangs->appends(['sort' => 'nama'])->links() }}
12
13 // Akses nilai filter di view
14 request('kategori') // nilai dari query string
15 request('search')   // nilai pencarian
16 request('status')   // filter status transaksi
```

Berikut adalah tampilan aplikasi setelah implementasi filter:

Halaman barang dengan filter kategori + search aktif



BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Halaman transaksi dengan filter status Dipinjam

The screenshot shows the 'INVENTARIS UNIVERSITAS' web application. The left sidebar contains a menu with 'Transaksi' highlighted. The main content area is titled 'Transaksi' and shows a filter for 'Dipinjam' status. The table below displays one transaction record.

No	Peminjam	Barang	Status
1	Admin	Laptop HP	Dipinjam

URL dengan query string parameter saat filter aktif

The screenshot shows the 'INVENTARIS UNIVERSITAS' web application. The left sidebar contains a menu with 'Dashboard' highlighted. The main content area displays a table of query string parameters.

Parameter	Value
kategori	1
search	laptop

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat filter data dan pencarian

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 24. Export ke PDF dengan DOMPDPF

Materi Pembelajaran : DOMPDPF, export PDF, report
Acara Praktikum/Praktik : Minggu 12/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritikal yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources
4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing

c. Indikator Penilaian

Mahasiswa mampu membuat export data ke PDF

d. Dasar Teori

Dalam teori pengembangan aplikasi web, DOMPDF adalah library PHP yang berfungsi untuk mengonversi tampilan berbasis HTML dan Blade Template di Laravel menjadi dokumen PDF. Dengan DOMPDF, developer dapat membuat laporan yang rapi dan terstruktur tanpa harus menulis kode PDF secara manual, karena sistem cukup merender view Blade yang sudah ada menjadi file PDF siap unduh atau cetak. Hal ini mendukung prinsip document automation, yaitu otomatisasi pembuatan dokumen digital dari data aplikasi.

Dalam konteks sistem informasi inventaris, DOMPDF dapat digunakan untuk menghasilkan laporan berisi daftar barang, jumlah stok, dan kondisi barang. Laporan ini penting untuk mendukung pengelolaan aset organisasi, karena memberikan gambaran jelas mengenai ketersediaan dan kualitas barang.

Selain inventaris, DOMPDF juga dapat digunakan untuk membuat laporan peminjaman barang. Laporan ini biasanya mencakup data peminjam, barang yang dipinjam, tanggal peminjaman, serta status pengembalian. Secara teoritis, laporan peminjaman mendukung prinsip audit trail, yaitu pencatatan jejak penggunaan aset agar dapat

DOMPDF menekankan bahwa ia adalah alat konversi view menjadi dokumen PDF yang mendukung efisiensi, konsistensi, dan profesionalisme dalam penyajian laporan. Laporan inventaris memastikan pengelolaan stok dan kondisi barang berjalan transparan, sedangkan laporan peminjaman menjamin adanya kontrol dan akuntabilitas dalam penggunaan aset. Keduanya menjadi bagian penting dalam sistem informasi manajemen yang terintegrasi.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Instalasi DOMPDF

Install DOMPDF package menggunakan composer:
composer require barryvdh/laravel-dompdf

Install DOMPDF via composer

```
$ composer require barryvdh/laravel-dompdf
- Installing barryvdh/laravel-dompdf
- Installing dompdf/dompdf
Package discovered: barryvdh/laravel-dompdf

$ php artisan vendor:publish --tag=dompdf-config
Copied File [vendor/barryvdh/dompdf/config/dompdf.php] to [config/dompdf.php]
```

Membuat LaporanController

Buat controller untuk menangani export laporan:

php artisan make:controller LaporanController

Method exportPdfBarang:

- Query barang dengan eager loading kategori
- Pdf::loadView() memuat view blade untuk PDF
- download() menghasilkan response file PDF
- Nama file: laporan-barang-YYYY-MM-DD.pdf

LaporanController dengan export PDF

```
app/Http/Controllers/LaporanController.php

1 use Barryvdh\DomPDF\Facade\Pdf;
2
3 class LaporanController extends Controller
4 {
5     public function exportPdfBarang()
6     {
7         $barang = Barang::with('kategori')->get();
8         $pdf = Pdf::loadView(
9             'laporan.pdf.barang', compact('barang'));
10
11         return $pdf->download(
12             'laporan-barang-' . date('Y-m-d') . '.pdf');
13     }
14
15     public function exportPdfTransaksi()
16     {
17         $transaksi = Transaksi::with(['user', 'barang'])
18             ->latest()->get();
19         $pdf = Pdf::loadView(
20             'laporan.pdf.transaksi', compact('transaksi'));
21
22         return $pdf->download(
23             'laporan-transaksi-' . date('Y-m-d') . '.pdf');
24     }
25 }
```

Membuat View PDF Barang

Buat view blade untuk PDF barang dengan struktur HTML murni:

- Styling dengan tag <style>: table border-collapse, header background #f0f0f0
- Mengakses data relasi: \$b->kategori->nama

- Nomor urut otomatis: \$loop->iteration

View PDF barang dengan CSS inline

```
resources/views/laporan/pdf/barang.blade.php

1 <!DOCTYPE html>
2 <html><head>
3   <title>Laporan Barang</title>
4   <style>
5     table { width:100%; border-collapse:collapse; }
6     th, td { border:1px solid #333; padding:8px; }
7     th { background:#f0f0f0; }
8     h2 { text-align:center; }
9   </style>
10 </head><body>
11   <h2>Laporan Inventaris Barang</h2>
12   <p>Tanggal: {{ date('d/m/Y') }}</p>
13   <table>
14     <thead>
15       <tr><th>No</th><th>Nama</th><th>Kategori</th>
16         <th>Jumlah</th><th>Kondisi</th><th>Deskripsi</th></tr>
17     </thead>
18     <tbody>
19       @foreach ($barang as $b)
20         <tr>
21           <td>{{ $loop->iteration }}</td>
22           <td>{{ $b->nama }}</td>
23           <td>{{ $b->kategori->nama }}</td>
24           <td>{{ $b->jumlah }}</td>
25           <td>{{ $b->kondisi }}</td>
26           <td>{{ $b->deskripsi ?? '-' }}</td>
27         </tr>
28       @endforeach
29     </tbody>
30   </table>
31 </body></html>
```

Membuat View PDF Transaksi

View PDF transaksi menampilkan data peminjaman:

- Data dari Transaksi dengan eager loading user dan barang
- Format tanggal dengan ->format('d/m/Y')
- Status transaksi (dipinjam/dikembalikan)
- Tanggal kembali nullable dengan ternary

Mendaftarkan Route

Daftarkan route GET untuk export PDF:

Route export PDF di web.php

```
routes/web.php

1 Route::middleware(['auth', 'verified'])->group(function () {
2
3     Route::get('/laporan/barang/pdf', [
4         LaporanController::class, 'exportPdfBarang'
5     ])->name('laporan.barang.pdf');
6
7     Route::get('/laporan/transaksi/pdf', [
8         LaporanController::class, 'exportPdfTransaksi'
9     ])->name('laporan.transaksi.pdf');
10
11     Route::get('/laporan/barang/excel', ...);
12     Route::get('/laporan/transaksi/excel', ...);
13 });
```

Export PDF Laravel 11 PHP 8.2

Uji Coba Download PDF

- Buka halaman Dashboard, klik tombol "Barang PDF"
- Browser akan mendownload file laporan-barang-2026-07-23.pdf
- Buka file PDF, pastikan tampilan sesuai dengan view Blade
- Uji juga tombol "Transaksi PDF" untuk laporan transaksi
- Periksa bahwa data relasi (kategori, user) muncul dengan benar
- Coba klik tombol langsung dari sidebar menu Laporan PDF

Study Kasus

Inventaris Barang Universitas untuk materi "DOMPDF, export PDF, report". Analisis bagaimana konsep export ke pdf dengan dompdf diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Install DOMPDF

```
$ composer require barryvdh/laravel-dompdf
- Installing barryvdh/laravel-dompdf
- Installing dompdf/dompdf
Package discovered: barryvdh/laravel-dompdf

$ php artisan vendor:publish --tag=dompdf-config
Copied File [vendor/barryvdh/dompdf/config/dompdf.php] to [config/dompdf.php]
```

LaporanController

```
app/Http/Controllers/LaporanController.php

1 use Barryvdh\DomPDF\Facade\Pdf;
2
3 class LaporanController extends Controller
4 {
5     public function exportPdfBarang()
6     {
7         $barang = Barang::with('kategori')->get();
8         $pdf = Pdf::loadView(
9             'laporan.pdf.barang', compact('barang'));
10
11         return $pdf->download(
12             'laporan-barang-' . date('Y-m-d') . '.pdf');
13     }
14
15     public function exportPdfTransaksi()
16     {
17         $transaksi = Transaksi::with(['user', 'barang'])
18             ->latest()->get();
19         $pdf = Pdf::loadView(
20             'laporan.pdf.transaksi', compact('transaksi'));
21
22         return $pdf->download(
23             'laporan-transaksi-' . date('Y-m-d') . '.pdf');
24     }
25 }
```

View PDF barang

```
resources/views/laporan/pdf/barang.blade.php

1 <!DOCTYPE html>
2 <html><head>
3     <title>Laporan Barang</title>
4     <style>
5         table { width:100%; border-collapse:collapse; }
6         th, td { border:1px solid #333; padding:8px; }
7         th { background:#f0f0f0; }
8         h2 { text-align:center; }
9     </style>
10 </head><body>
11     <h2>Laporan Inventaris Barang</h2>
12     <p>Tanggal: {{ date('d/m/Y') }}</p>
13     <table>
14         <thead>
15             <tr><th>No</th><th>Nama</th><th>Kategori</th>
16             <th>Jumlah</th><th>Kondisi</th><th>Deskripsi</th></tr>
17         </thead>
18         <tbody>
19             @foreach ($barang as $b)
20                 <tr>
21                     <td>{{ $loop->iteration }}</td>
22                     <td>{{ $b->nama }}</td>
23                     <td>{{ $b->kategori->nama }}</td>
24                     <td>{{ $b->jumlah }}</td>
25                     <td>{{ $b->kondisi }}</td>
26                     <td>{{ $b->deskripsi ?? '-' }}</td>
27                 </tr>
28             @endforeach
29         </tbody>
30     </table>
31 </body></html>
```

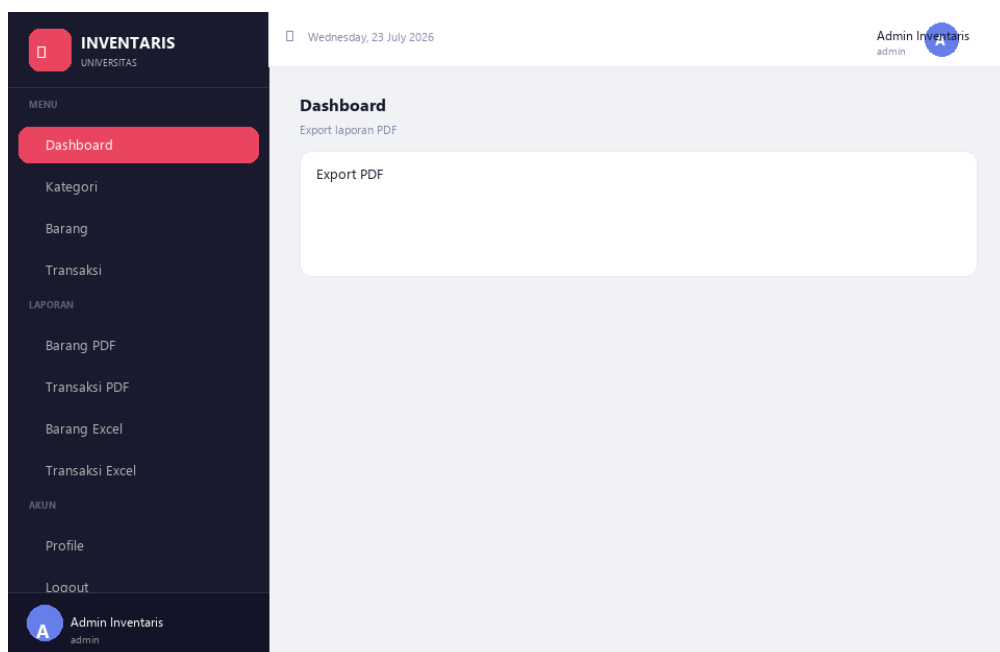
BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Route export PDF

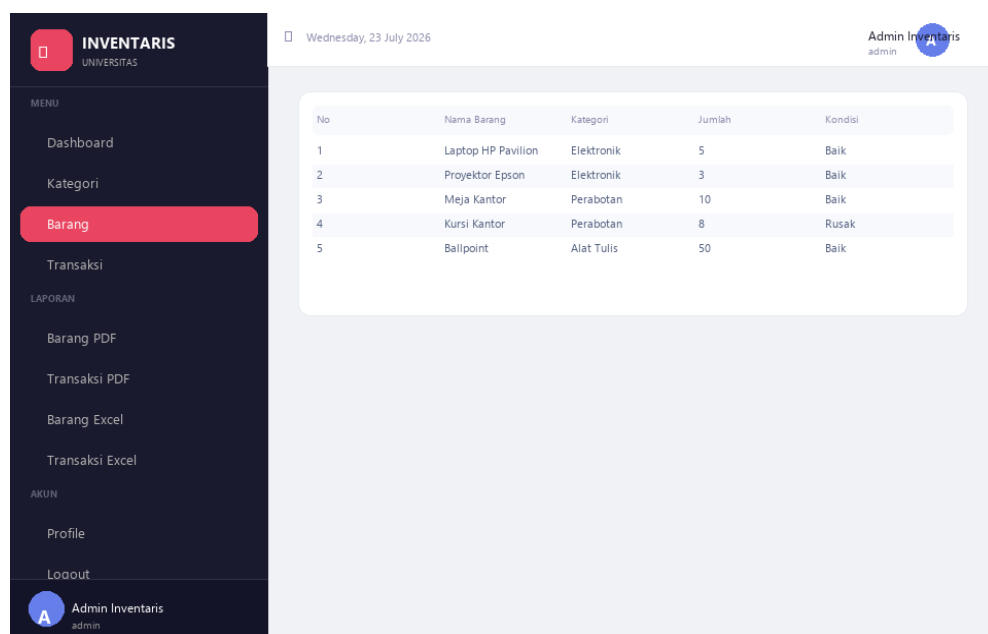
```
routes/web.php
1 Route::middleware(['auth', 'verified'])->group(function () {
2
3     Route::get('/laporan/barang/pdf', [
4         LaporanController::class, 'exportPdfBarang'
5     ])->name('laporan.barang.pdf');
6
7     Route::get('/laporan/transaksi/pdf', [
8         LaporanController::class, 'exportPdfTransaksi'
9     ])->name('laporan.transaksi.pdf');
10
11     Route::get('/laporan/barang/excel', ...);
12     Route::get('/laporan/transaksi/excel', ...);
13 });
```

Export PDF Laravel 11 PHP 8.2

Dashboard dengan tombol export PDF



Hasil PDF laporan barang



BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Hasil PDF laporan transaksi

The screenshot displays the 'INVENTARIS UNIVERSITAS' web application interface. On the left is a dark sidebar menu with options: Dashboard, Kategori, Barang, Transaksi (highlighted in red), LAPORAN (with sub-options: Barang PDF, Transaksi PDF, Barang Excel, Transaksi Excel), and AKUN (with sub-options: Profile, Logout). The main content area shows a table of transactions with columns: No, Peminjam, Barang, Status, and Tanggal. The table contains three rows of data. At the top right of the main area, it says 'Wednesday, 23 July 2026' and 'Admin Inventaris admin' with a user profile icon.

No	Peminjam	Barang	Status	Tanggal
1	Admin Inventaris	Laptop HP	Dipinjam	23/07
2	Staff	Proyektor	Dikembalikan	22/07
3	Staff	Meja Kantor	Dipinjam	21/07

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa berhasil membuat export data ke PDF dengan DOMPDF

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 25. Export ke Excel

Materi Pembelajaran : Laravel Excel, export xlsx
 Acara Praktikum/Praktik : Minggu 13/1
 Tempat : Daring/Luring Politeknik Negeri Jember
 Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritis yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources
4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing

c. Indikator Penilaian

Mahasiswa mampu membuat export data ke Excel

d. Dasar Teori

Dalam teori pengembangan aplikasi Laravel, Maatwebsite/Laravel-Excel adalah package yang memudahkan proses export dan import data ke dalam format Excel. Package ini menyediakan antarmuka yang sederhana untuk mengelola data dalam bentuk spreadsheet, sehingga developer tidak perlu menulis kode kompleks untuk membaca atau menulis file Excel. Dengan Laravel-Excel, data dari database dapat diekspor ke file .xlsx, .csv, atau .xls, dan sebaliknya, file Excel dapat diimpor ke dalam database dengan mudah.

- Export Data

Export adalah proses mengubah data dari sistem menjadi file Excel yang dapat diunduh atau dibagikan. Dalam konteks sistem inventaris, data barang dan data peminjaman dapat diekspor ke format .xlsx agar lebih mudah dianalisis, dicetak, atau dilaporkan. Export mendukung prinsip data portability, yaitu kemampuan membawa data ke luar

- Import Data

Import adalah proses memasukkan data dari file Excel ke dalam sistem. Hal ini berguna untuk mempercepat input data dalam jumlah besar, misalnya menambahkan daftar barang baru atau mencatat riwayat peminjaman. Import mendukung prinsip efficiency in data entry, karena mengurangi pekerjaan manual dan risiko kesalahan input.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Instalasi Laravel Excel

Install package Laravel Excel dan buat export class:

```
composer require maatwebsite/excel
```

- Buat export class untuk Barang dan Transaksi:

```
php artisan make:export BarangExport --model=App\\Models\\Barang
```

php artisan make:export TransaksiExport --model=App\\Models\\Transaksi

Install Laravel Excel dan buat export class

```
$ composer require maatwebsite/excel
- Installing maatwebsite/excel (3.1.x)
- Installing phpoffice/phpspreadsheet
Package discovered: maatwebsite/excel

$ php artisan make:export BarangExport --model=App\\Models\\Barang
Created: app/Exports/BarangExport.php

$ php artisan make:export TransaksiExport --model=App\\Models\\Transaksi
Created: app/Exports/TransaksiExport.php
```

Membuat Export Class Barang

Export class BarangExport menangani export data barang ke Excel:

- collection(): query barang dengan eager loading kategori, return array map
- headings(): header kolom ID, Kategori, Nama, Jumlah, Kondisi, Deskripsi, Dibuat
- Format tanggal: \$barang->created_at->format('d/m/Y')

BarangExport dengan collection dan headings

```
app/Exports/BarangExport.php

1  <?php
2  namespace App\Exports;
3  use App\Models\Barang;
4
5  class BarangExport
6  {
7      public function collection()
8      {
9          return Barang::with('kategori')->get()->map(fn($b) => [
10             $b->id, $b->kategori->nama, $b->nama,
11             $b->jumlah, $b->kondisi, $b->deskripsi ?? '-',
12             $b->created_at->format('d/m/Y'),
13         ]);
14     }
15
16     public function headings(): array
17     {
18         return ['ID', 'Kategori', 'Nama', 'Jumlah',
19             'Kondisi', 'Deskripsi', 'Dibuat'];
20     }
21 }
```

Membuat Export Class Transaksi

Export class TransaksiExport menangani export data transaksi:

- collection(): query transaksi dengan eager loading user dan barang, latest()
- headings(): ID, Peminjam, Barang, Jumlah, Tgl Pinjam, Tgl Kembali, Status
- Ternary untuk tgl_kembali nullable: ? \$t->tgl_kembali->format(...) : '-'

TransaksiExport dengan data relasi

```
app/Exports/TransaksiExport.php

1 <?php
2 namespace App\Exports;
3 use App\Models\Transaksi;
4
5 class TransaksiExport
6 {
7     public function collection()
8     {
9         return Transaksi::with(['user', 'barang'])
10             ->latest()->get()->map(fn($t) => [
11             $t->id, $t->user->name, $t->barang->nama,
12             $t->jumlah, $t->tgl_pinjam->format('d/m/Y'),
13             $t->tgl_kembali ? $t->tgl_kembali->format('d/m/Y') : '-',
14             $t->status,
15         ]);
16     }
17
18     public function headings(): array
19     {
20         return ['ID', 'Peminjam', 'Barang', 'Jumlah',
21             'Tgl Pinjam', 'Tgl Kembali', 'Status'];
22     }
23 }
```

Method di Controller

Tambahkan method exportExcel di LaporanController:

- BarangExport() dan TransaksiExport() di-instantiate untuk mengambil data
- Excel::create() membuat file dengan nama dinamis + date()
- fromArray() mengisi sheet dari collection export
- prependRow() menambahkan header di baris pertama
- row(1, fn) mengatur bold pada header dengan setFontWeight()
- download('xlsx') mengirimkan response file Excel

Method exportExcelBarang dan exportExcelTransaksi

```
app/Http/Controllers/LaporanController.php

1 use App\Exports\BarangExport;
2 use App\Exports\TransaksiExport;
3 use Maatwebsite\Excel\Facades\Excel;
4
5 class LaporanController extends Controller
6 {
7     public function exportExcelBarang()
8     {
9         $export = new BarangExport();
10         return Excel::create('laporan-barang-' . date('Y-m-d'),
11             function ($excel) use ($export) {
12                 $excel->sheet('Barang', fn($s) =>
13                     $s->fromArray($export->collection());
14                 $excel->sheet('Barang', fn($s) =>
15                     $s->prependRow(1, $export->headings());
16                 $excel->sheet('Barang', fn($s) =>
17                     $s->row(1, fn($r) => $r->setFontWeight('bold')));
18             })->download('xlsx');
19     }
20
21     public function exportExcelTransaksi()
22     {
23         $export = new TransaksiExport();
24         return Excel::create('laporan-transaksi-' . date('Y-m-d'),
25             function ($excel) use ($export) {
26                 $excel->sheet('Transaksi', fn($s) =>
27                     $s->fromArray($export->collection());
28                 $excel->sheet('Transaksi', fn($s) =>
29                     $s->prependRow(1, $export->headings());
30                 $excel->sheet('Transaksi', fn($s) =>
31                     $s->row(1, fn($r) => $r->setFontWeight('bold')));
32             })->download('xlsx');
33     }
34 }
```

Mendaftarkan Route

Daftarkan route GET untuk export Excel:

Route export Excel di web.php

```
routes/web.php
1 Route::middleware(['auth', 'verified'])->group(function () {
2
3     Route::get('/laporan/barang/excel', [
4         LaporanController::class, 'exportExcelBarang'
5     ])->name('laporan.barang.excel');
6
7     Route::get('/laporan/transaksi/excel', [
8         LaporanController::class, 'exportExcelTransaksi'
9     ])->name('laporan.transaksi.excel');
10
11     Route::get('/laporan/barang/pdf', ...);
12     Route::get('/laporan/transaksi/pdf', ...);
13 });
```

Export Excel Laravel 11 PHP 8.2

Uji Coba Download Excel

- Buka halaman Dashboard, klik tombol "Barang Excel"
- Browser akan mendownload file laporan-barang-2026-07-23.xlsx
- Buka file Excel, pastikan header bold dan data sesuai
- Uji juga tombol "Transaksi Excel" untuk laporan transaksi
- Periksa bahwa data relasi muncul dengan benar di Excel

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Laravel Excel, export xlsx". Analisis bagaimana konsep export ke excel diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Install Laravel Excel

```
$ composer require maatwebsite/excel
- Installing maatwebsite/excel (3.1.x)
- Installing phpoffice/phpspreadsheet
Package discovered: maatwebsite/excel

$ php artisan make:export BarangExport --model=App\\Models\\Barang
Created: app/Exports/BarangExport.php

$ php artisan make:export TransaksiExport --model=App\\Models\\Transaksi
Created: app/Exports/TransaksiExport.php
```

BarangExport class

```
app/Exports/BarangExport.php

1  <?php
2  namespace App\Exports;
3  use App\Models\Barang;
4
5  class BarangExport
6  {
7      public function collection()
8      {
9          return Barang::with('kategori')->get()->map(fn($b) => [
10              $b->id, $b->kategori->nama, $b->nama,
11              $b->jumlah, $b->kondisi, $b->deskripsi ?? '-',
12              $b->created_at->format('d/m/Y'),
13          ]);
14      }
15
16      public function headings(): array
17      {
18          return ['ID', 'Kategori', 'Nama', 'Jumlah',
19              'Kondisi', 'Deskripsi', 'Dibuat'];
20      }
21  }
```

Export Excel Laravel 11 PHP 8.2

TransaksiExport class

```
app/Exports/TransaksiExport.php

1  <?php
2  namespace App\Exports;
3  use App\Models\Transaksi;
4
5  class TransaksiExport
6  {
7      public function collection()
8      {
9          return Transaksi::with(['user', 'barang'])
10              ->latest()->get()->map(fn($t) => [
11              $t->id, $t->user->name, $t->barang->nama,
12              $t->jumlah, $t->tgl_pinjam->format('d/m/Y'),
13              $t->tgl_kembali ? $t->tgl_kembali->format('d/m/Y') : '-',
14              $t->status,
15          ]);
16      }
17
18      public function headings(): array
19      {
20          return ['ID', 'Peminjam', 'Barang', 'Jumlah',
21              'Tgl Pinjam', 'Tgl Kembali', 'Status'];
22      }
23  }
```

Export Excel Laravel 11 PHP 8.2

Controller export Excel

```
app/Http/Controllers/LaporanController.php

1  use App\Exports\BarangExport;
2  use App\Exports\TransaksiExport;
3  use Maatwebsite\Excel\Facades\Excel;
4
5  class LaporanController extends Controller
6  {
7      public function exportExcelBarang()
8      {
9          $export = new BarangExport();
10         return Excel::create('laporan-barang-' . date('Y-m-d'),
11             function ($excel) use ($export) {
12                 $excel->sheet('Barang', fn($s) =>
13                     $s->fromArray($export->collection());
14                 $excel->sheet('Barang', fn($s) =>
15                     $s->prependRow(1, $export->headings());
16                 $excel->sheet('Barang', fn($s) =>
17                     $s->row(1, fn($r) => $r->setFontWeight('bold')));
18             })->download('xlsx');
19     }
20
21     public function exportExcelTransaksi()
22     {
23         $export = new TransaksiExport();
24         return Excel::create('laporan-transaksi-' . date('Y-m-d'),
25             function ($excel) use ($export) {
26                 $excel->sheet('Transaksi', fn($s) =>
27                     $s->fromArray($export->collection());
28                 $excel->sheet('Transaksi', fn($s) =>
29                     $s->prependRow(1, $export->headings());
30                 $excel->sheet('Transaksi', fn($s) =>
31                     $s->row(1, fn($r) => $r->setFontWeight('bold')));
32             })->download('xlsx');
33     }
34 }
```

Export Excel Laravel 11 PHP 8.2

Route export Excel

```
routes/web.php

1  Route::middleware(['auth', 'verified'])->group(function () {
2
3      Route::get('/laporan/barang/excel', [
4          LaporanController::class, 'exportExcelBarang'
5      ])->name('laporan.barang.excel');
6
7      Route::get('/laporan/transaksi/excel', [
8          LaporanController::class, 'exportExcelTransaksi'
9      ])->name('laporan.transaksi.excel');
10
11     Route::get('/laporan/barang/pdf', ...);
12     Route::get('/laporan/transaksi/pdf', ...);
13 });
```

Export Excel Laravel 11 PHP 8.2

Dashboard dengan tombol export Excel

 **INVENTARIS**
UNIVERSITAS

MENU

Dashboard

Kategori

Barang

Transaksi

LAPORAN

Barang PDF

Transaksi PDF

Barang Excel

Transaksi Excel

AKUN

Profile

Logout

 Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

Dashboard

Export data ke Excel

Export Laporan

File Excel laporan barang

 **INVENTARIS**
UNIVERSITAS

MENU

Dashboard

Kategori

Barang

Transaksi

LAPORAN

Barang PDF

Transaksi PDF

Barang Excel

Transaksi Excel

AKUN

Profile

Logout

 Admin Inventaris
admin

Wednesday, 23 July 2026

Admin Inventaris
admin

No	Nama	Kategori	Stok
1	Laptop HP	Elektronik	5
2	Proyektor	Elektronik	3
3	Meja	Perabotan	10

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

File Excel laporan transaksi

The screenshot shows the 'INVENTARIS UNIVERSITAS' web application. The sidebar menu on the left includes sections for MENU (Dashboard, Kategori, Barang, Transaksi), LAPORAN (Barang PDF, Transaksi PDF, Barang Excel, Transaksi Excel), and AKUN (Profile, Logout). The main content area displays a table of transactions:

No	Peminjam	Barang	Status	Tgl
1	Admin	Laptop HP	Dipinjam	23/07
2	Staff	Proyektor	Dikembalikan	22/07

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat export data ke Excel

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusa	V			
	Total Skor*	Skor x 25			

Acara 26. Debugging dengan Laravel Debugger

Materi Pembelajaran : Debugbar, Logging, error handling
 Acara Praktikum/Praktik : Minggu 13/2
 Tempat : Daring/Luring Politeknik Negeri Jember
 Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mampu melakukan debugging aplikasi Laravel (TB430805.1)

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritis yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources
4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing

c. Indikator Penilaian

- Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna(TB430805.1)
- Mengembangkan modul aplikasi Web Framework menggunakan fitur routing, controller, model, view(TB430805.2)
- Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi(TB430805.3)
- Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

d. Dasar Teori

Laravel Debugbar adalah sebuah package yang berfungsi menampilkan informasi debug secara visual di browser. Debugbar memberikan detail penting seperti daftar queries yang dijalankan, routes yang dipanggil, views yang dirender, serta penggunaan memory. Dengan adanya Debugbar, developer dapat memantau performa aplikasi secara langsung, menemukan bottleneck, dan melakukan optimasi kode. Secara teoritis, Debugbar mendukung prinsip observability dalam sistem, yaitu kemampuan untuk memahami kondisi internal aplikasi melalui data yang ditampilkan.

Selain itu, Laravel menyediakan mekanisme logging melalui Log facade. Logging adalah proses pencatatan informasi, baik berupa error, peringatan, maupun aktivitas normal aplikasi, ke dalam file log yang biasanya tersimpan di direktori storage/logs. Log ini berfungsi sebagai catatan jejak (audit trail) yang dapat digunakan untuk analisis masalah, pemantauan sistem, dan dokumentasi aktivitas. Secara teori, logging mendukung prinsip traceability, yaitu kemampuan melacak peristiwa dalam sistem untuk tujuan debugging maupun keamanan.

Kemudian, Laravel juga memiliki sistem exception handling yang bertugas menangani error secara terstruktur. Exception handling memungkinkan developer mendefinisikan cara aplikasi merespons ketika terjadi kesalahan, misalnya menampilkan pesan error yang ramah pengguna, mencatat error ke log, atau mengarahkan ke halaman khusus. Secara teoritis, exception handling mendukung prinsip fault tolerance, yaitu kemampuan sistem untuk tetap berjalan meskipun terjadi kesalahan, dengan cara mengendalikan dampak error agar tidak merusak keseluruhan aplikasi.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code

- Git dan Github
- Composer.

f. Prosedur Kerja

Instalasi Debugbar

Install Laravel Debugbar sebagai dependency dev:

composer require barryvdh/laravel-debugbar --dev

- Debugbar akan otomatis muncul di browser saat APP_DEBUG=true di .env. Toolbar menampilkan berbagai tab debugging di bagian bawah halaman.

Install Laravel Debugbar

```
$ composer require barryvdh/laravel-debugbar --dev
- Installing barryvdh/laravel-debugbar (3.x)
- Installing maximebf/debugbar
Package discovered: barryvdh/laravel-debugbar

Debugbar otomatis aktif di environment APP_DEBUG=true
```

Logging di Controller

Gunakan Log facade untuk mencatat aktivitas di TransaksiController:

- Log::info() untuk log sukses dengan data konteks (user, barang, jumlah)
- Log::error() untuk log kegagalan dengan pesan exception
- Log masuk ke storage/logs/laravel.log dengan timestamp dan level

Logging info dan error di TransaksiController

```
app/Http/Controllers/TransaksiController.php

1  <?php
2  namespace App\Http\Controllers;
3  use Illuminate\Support\Facades\Log;
4
5  class TransaksiController extends Controller
6  {
7      public function store(TransaksiRequest $request)
8      {
9          try {
10             $transaksi = Transaksi::create($request->validated());
11
12             Log::info('Peminjaman berhasil', [
13                 'user' => auth()->user()->name,
14                 'barang' => $transaksi->barang->nama,
15                 'jumlah' => $transaksi->jumlah,
16             ]);
17
18             return redirect()->route('transaksi.index')
19                 ->with('success', 'Transaksi berhasil');
20         } catch (\Exception $e) {
21
22             Log::error('Gagal peminjaman: ' . $e->getMessage());
23             return back()->with('error', 'Gagal');
24         }
25     }
26 }
```

Laravel Debugbar & Logging Laravel 11 PHP 8.2

Exception Handler

Konfigurasi exception handling di bootstrap/app.php:

- reportable() mendaftarkan closure untuk melaporkan semua exception via Log::error()
- render() menangani NotFoundHttpException (404) dengan custom view
- Exception handler memastikan error terekam di file log

Exception handler dengan Log error

```
bootstrap/app.php

bootstrap/app.php
1 use Illuminate\Foundation\Configuration\Exceptions;
2
3 return Application::configure(basePath: dirname(__DIR__))
4     ->withRouting(...)
5     ->withMiddleware(...)
6     ->withExceptions(function (Exceptions $exceptions) {
7
8         $exceptions->reportable(function (\Throwable $e) {
9             Log::error('Exception: ' . $e->getMessage(), [
10                 'file' => $e->getFile(),
11                 'line' => $e->getLine(),
12             ]);
13         });
14
15         $exceptions->render(function (\NotFoundHttpException $e) {
16             return response()->view('errors.404', [], 404);
17         });
18     })->create();

Laravel Debugbar & Logging  Laravel 11  PHP 8.2
```

Custom View 404

Buat view untuk halaman 404 (Not Found):

resources/views/errors/404.blade.php

Custom error page 404 dengan Tailwind

```
resources/views/errors/404.blade.php

1 <!DOCTYPE html>
2 <html><head>
3     <title>404 - Halaman Tidak Ditemukan</title>
4     @vite(['resources/css/app.css'])
5 </head><body>
6     <x-app-layout>
7         <div class="text-center py-20">
8             <h1 class="text-9xl font-bold text-gray-200">404</h1>
9             <h2 class="text-2xl font-semibold mt-4">Halaman Tidak Ditemukan</h2>
10            <p class="text-gray-500 mt-2">URL yang Anda tuju tidak tersedia.</p>
11            <a href="{{ route('dashboard') }}"
12                class="inline-block mt-6 px-6 py-2 bg-indigo-600 text-white rounded">
13                Kembali ke Dashboard
14            </a>
15        </div>
16    </x-app-layout>
17 </body></html>

Laravel Debugbar & Logging  Laravel 11  PHP 8.2
```

Custom View 500

Buat view untuk halaman 500 (Server Error):

resources/views/errors/500.blade.php

Custom error page 500 dengan Tailwind

```
resources/views/errors/500.blade.php

1 <!DOCTYPE html>
2 <html><head>
3   <title>500 - Server Error</title>
4   @vite(['resources/css/app.css'])
5 </head><body>
6   <x-app-layout>
7     <div class="text-center py-20">
8       <h1 class="text-9xl font-bold text-gray-200">500</h1>
9       <h2 class="text-2xl font-semibold mt-4">Server Error</h2>
10      <p class="text-gray-500 mt-2">Terjadi kesalahan pada server.</p>
11      <a href="{{ route('dashboard') }}"
12        class="inline-block mt-6 px-6 py-2 bg-indigo-600 text-white rounded">
13        Kembali ke Dashboard
14      </a>
15    </div>
16  </x-app-layout>
17 </body></html>
```

Uji Coba Debugbar & Log

- Buka dashboard, Debugbar muncul di bawah halaman dengan tab Queries, Routes, Views
- Klik tab Queries untuk melihat semua query SQL yang dijalankan halaman
- Klik tab Routes untuk melihat daftar route yang terdaftar
- Lakukan transaksi peminjaman, cek log info di Debugbar tab Logs
- Akses URL tidak valid (contoh: /admin), lihat custom 404 page
- Cek file storage/logs/laravel.log untuk lihat semua log aktivitas

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Debugbar, Logging, error handling". Analisis bagaimana konsep debugging dengan laravel debugbar diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Install Debugbar

```
$ composer require barryvdh/laravel-debugbar --dev
- Installing barryvdh/laravel-debugbar (3.x)
- Installing maximebf/debugbar
Package discovered: barryvdh/laravel-debugbar

Debugbar otomatis aktif di environment APP_DEBUG=true
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Logging di Controller

```
app/Http/Controllers/TransaksiController.php

1  <?php
2  namespace App\Http\Controllers;
3  use Illuminate\Support\Facades\Log;
4
5  class TransaksiController extends Controller
6  {
7      public function store(TransaksiRequest $request)
8      {
9          try {
10             $transaksi = Transaksi::create($request->validated());
11
12             Log::info('Peminjaman berhasil', [
13                 'user' => auth()->user()->name,
14                 'barang' => $transaksi->barang->nama,
15                 'jumlah' => $transaksi->jumlah,
16             ]);
17
18             return redirect()->route('transaksi.index')
19                 ->with('success', 'Transaksi berhasil');
20         } catch (\Exception $e) {
21
22             Log::error('Gagal peminjaman: ' . $e->getMessage());
23             return back()->with('error', 'Gagal');
24         }
25     }
26 }
```

Laravel Debugger & Logging Laravel 11 PHP 8.2

Exception Handler

```
bootstrap/app.php

1  use Illuminate\Foundation\Configuration\Exceptions;
2
3  return Application::configure(basePath: dirname(__DIR__))
4      ->withRouting(...)
5      ->withMiddleware(...)
6      ->withExceptions(function (Exceptions $exceptions) {
7
8          $exceptions->reportable(function (\Throwable $e) {
9              Log::error('Exception: ' . $e->getMessage(), [
10                 'file' => $e->getFile(),
11                 'line' => $e->getLine(),
12             ]);
13          });
14
15          $exceptions->render(function (\NotFoundHttpException $e) {
16              return response()->view('errors.404', [], 404);
17          });
18      })->create();
```

Laravel Debugger & Logging Laravel 11 PHP 8.2

Custom 404 view

```
resources/views/errors/404.blade.php

1  <!DOCTYPE html>
2  <html><head>
3      <title>404 - Halaman Tidak Ditemukan</title>
4      @vite(['resources/css/app.css'])
5  </head><body>
6      <x-app-layout>
7          <div class="text-center py-20">
8              <h1 class="text-9xl font-bold text-gray-200">404</h1>
9              <h2 class="text-2xl font-semibold mt-4">Halaman Tidak Ditemukan</h2>
10             <p class="text-gray-500 mt-2">URL yang Anda tuju tidak tersedia.</p>
11             <a href="{{ route('dashboard') }}"
12                 class="inline-block mt-6 px-6 py-2 bg-indigo-600 text-white rounded">
13                 Kembali ke Dashboard
14             </a>
15         </div>
16     </x-app-layout>
17 </body></html>
```

Laravel Debugger & Logging Laravel 11 PHP 8.2

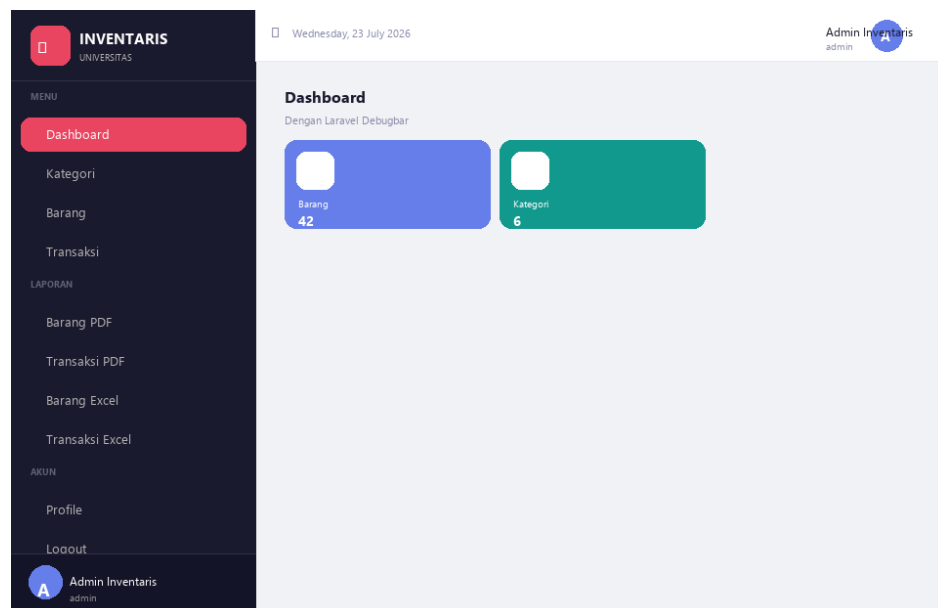
BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Custom 500 view

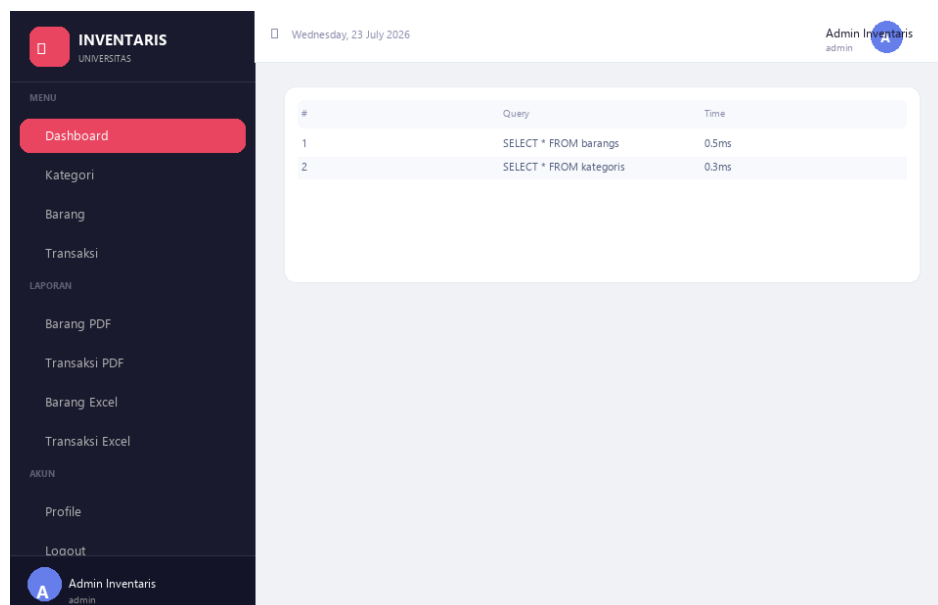
```
resources/views/errors/500.blade.php

1 <!DOCTYPE html>
2 <html><head>
3   <title>500 - Server Error</title>
4   @vite(['resources/css/app.css'])
5 </head><body>
6   <x-app-layout>
7     <div class="text-center py-20">
8       <h1 class="text-9xl font-bold text-gray-200">500</h1>
9       <h2 class="text-2xl font-semibold mt-4">Server Error</h2>
10      <p class="text-gray-500 mt-2">Terjadi kesalahan pada server.</p>
11      <a href="{{ route('dashboard') }}"
12        class="inline-block mt-6 px-6 py-2 bg-indigo-600 text-white rounded">
13        Kembali ke Dashboard
14      </a>
15    </div>
16  </x-app-layout>
17 </body></html>
```

Dashboard dengan Debugbar toolbar

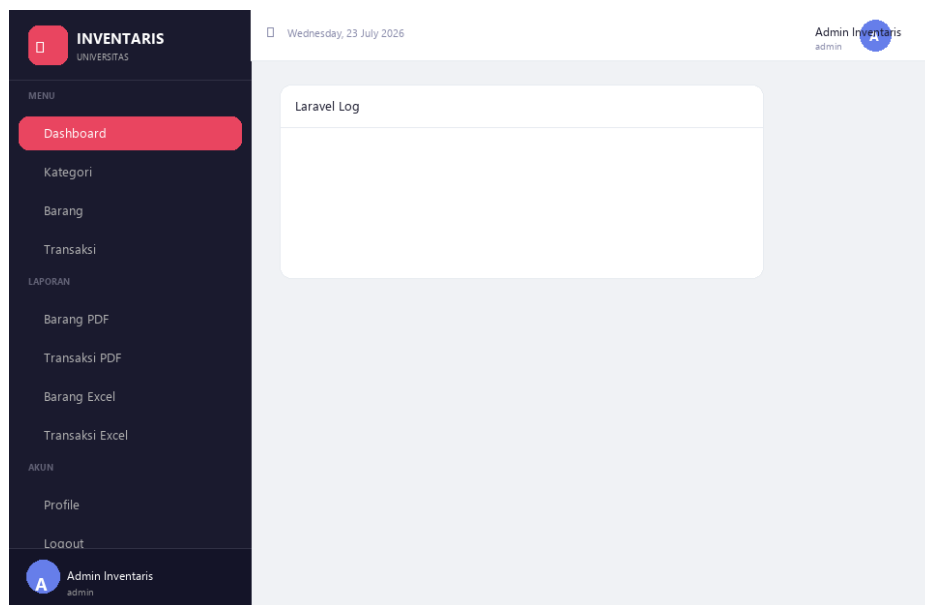


Tab Queries menampilkan SQL



BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

File log di storage/logs



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu melakukan debugging aplikasi Laravel

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak sesuai	V			
	Total Skor*	Skor x 25			

Acara 27. Unit Test dan Feature Test

Materi Pembelajaran : PHPUnit, testing, TDD
Acara Praktikum/Praktik : Minggu 14/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

- Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna(TB430805.1)
- Mengembangkan modul aplikasi Web Framework menggunakan fitur routing, controller, model, view(TB430805.2)
- Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi(TB430805.3)
- Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan

3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritikal yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources
4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing

c. Indikator Penilaian

Mahasiswa mampu membuat unit test dan feature test

d. Dasar Teori

PHPUnit adalah kerangka kerja testing yang digunakan untuk memastikan kualitas dan keandalan kode. Laravel secara bawaan terintegrasi dengan PHPUnit sehingga developer dapat menulis dan menjalankan berbagai jenis pengujian dengan mudah. Testing dalam Laravel mendukung prinsip software reliability dan maintainability, karena setiap perubahan kode dapat diverifikasi melalui serangkaian tes otomatis.

- Unit Test

Unit test adalah pengujian yang dilakukan secara terisolasi terhadap bagian kecil dari aplikasi, biasanya berupa method atau model. Tujuannya adalah memastikan bahwa setiap unit kode berfungsi sesuai dengan yang diharapkan tanpa bergantung pada komponen lain. Secara teoritis, unit test mendukung prinsip modularity dan isolation, karena setiap bagian diuji secara mandiri sehingga kesalahan dapat dilacak dengan cepat.

- Feature Test

Feature test adalah pengujian yang dilakukan terhadap fitur aplikasi secara keseluruhan, mencakup route, controller, dan response. Feature test memastikan bahwa alur kerja aplikasi berjalan sesuai dengan kebutuhan pengguna. Secara teoritis, feature test mendukung prinsip integration testing, yaitu memastikan bahwa berbagai komponen aplikasi dapat bekerja sama dengan baik untuk menghasilkan output yang benar.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code

- Git dan Github
- Composer.

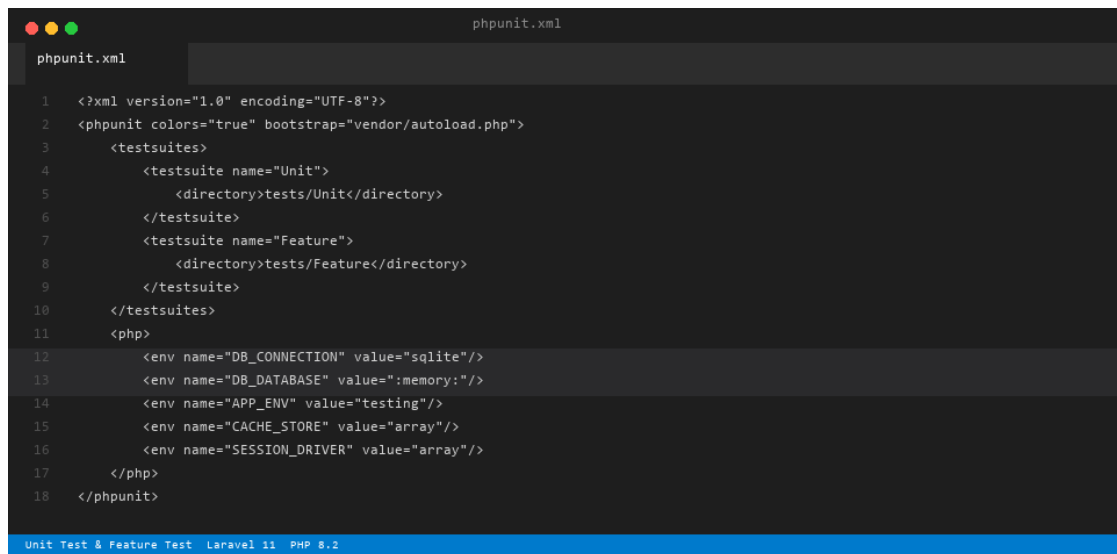
f. Prosedur Kerja

Konfigurasi phpunit.xml

PHPUnit dikonfigurasi melalui phpunit.xml di root project:

- testsuite Unit dan Feature untuk memisahkan jenis test
- DB_CONNECTION=sqlite dan DB_DATABASE=:memory: agar test menggunakan database in-memory
- APP_ENV=testing agar tidak mengganggu environment development
- CACHE_STORE=array dan SESSION_DRIVER=array untuk isolasi test

Konfigurasi PHPUnit dengan SQLite in-memory



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <phpunit colors="true" bootstrap="vendor/autoload.php">
3   <testsuites>
4     <testsuite name="Unit">
5       <directory>tests/Unit</directory>
6     </testsuite>
7     <testsuite name="Feature">
8       <directory>tests/Feature</directory>
9     </testsuite>
10  </testsuites>
11  <php>
12    <env name="DB_CONNECTION" value="sqlite"/>
13    <env name="DB_DATABASE" value=":memory:"/>
14    <env name="APP_ENV" value="testing"/>
15    <env name="CACHE_STORE" value="array"/>
16    <env name="SESSION_DRIVER" value="array"/>
17  </php>
18 </phpunit>
```

Membuat Unit Test Model

Buat unit test untuk menguji model User, Kategori, dan Barang:

php artisan make:test BarangTest --unit

- test_user_can_be_created_with_role: factory create user admin, assertDatabaseHas, isAdmin()
- test_kategori_can_be_created: Kategori::create() langsung, assertDatabaseHas
- test_barang_belongs_to_kategori: factory dengan kategori_id, assertInstanceOf relasi

Unit test untuk model User, Kategori, Barang

```
tests/Unit/ExampleTest.php

1  <?php
2  namespace Tests\Unit;
3  use App\Models\User;
4  use App\Models\Kategori;
5  use App\Models\Barang;
6  use Illuminate\Foundation\Testing\RefreshDatabase;
7  use Tests\TestCase;
8
9  class ExampleTest extends TestCase
10 {
11     use RefreshDatabase;
12
13     public function test_user_can_be_created_with_role(): void
14     {
15         $user = User::factory()->create(['role' => 'admin']);
16         $this->assertDatabaseHas('users',
17             ['id' => $user->id, 'role' => 'admin']);
18         $this->assertTrue($user->isAdmin());
19     }
20
21     public function test_kategori_can_be_created(): void
22     {
23         $kategori = Kategori::create([
24             'nama' => 'Test Kategori', 'deskripsi' => 'Test']);
25         $this->assertDatabaseHas('kategoris',
26             ['nama' => 'Test Kategori']);
27     }
28
29     public function test_barang_belongs_to_kategori(): void
30     {
31         $kategori = Kategori::factory()->create();
32         $barang = Barang::factory()->create(
33             ['kategori_id' => $kategori->id]);
34         $this->assertInstanceOf(Kategori::class,
35             $barang->kategori);
36     }
37 }
```

Membuat Feature Test Auth

Buat feature test untuk menguji alur autentikasi:

php artisan make:test AuthTest

- test_login_screen_can_be_rendered: GET /login, assertStatus(200)
- test_users_can_authenticate: POST /login dengan kredensial benar, assertAuthenticated, assertRedirect
- test_users_can_not_authenticate_with_invalid_password: POST dengan password salah, assertGuest
- test_users_can_logout: POST /logout, assertGuest

Feature test authentication (login, logout)

```
tests/Feature/Auth/AuthenticationTest.php

1  <?php
2  namespace Tests\Feature\Auth;
3  use App\Models\User;
4  use Illuminate\Foundation\Testing\RefreshDatabase;
5  use Tests\TestCase;
6
7  class AuthenticationTest extends TestCase
8  {
9      use RefreshDatabase;
10
11     public function test_login_screen_can_be_rendered(): void
12     {
13         $response = $this->get('/login');
14         $response->assertStatus(200);
15     }
16
17     public function test_users_can_authenticate(): void
18     {
19         $user = User::factory()->create();
20         $response = $this->post('/login', [
21             'email' => $user->email,
22             'password' => 'password',
23         ]);
24         $this->assertAuthenticated();
25         $response->assertRedirect(route('dashboard'));
26     }
27
28     public function test_users_can_not_authenticate
29         with_invalid_password(): void
30     {
31         $user = User::factory()->create();
32         $this->post('/login', [
33             'email' => $user->email,
34             'password' => 'wrong',
35         ]);
36         $this->assertGuest();
37     }
38 }
```

Membuat Feature Test Profile

Buat feature test untuk menguji halaman dan aksi profile:

- test_profile_page_is_displayed: actingAs user, GET /profile, assertOk()
- test_profile_information_can_be_updated: PATCH name & email, assertSessionHasNoErrors, assertSame
- test_user_can_delete_their_account: DELETE /profile, assertGuest

Feature test profile (update, delete akun)

```
tests/Feature/ProfileTest.php

1  <?php
2  namespace Tests\Feature;
3  use App\Models\User;
4  use Illuminate\Foundation\Testing\RefreshDatabase;
5  use Tests\TestCase;
6
7  class ProfileTest extends TestCase
8  {
9      use RefreshDatabase;
10
11     public function test_profile_page_is_displayed(): void
12     {
13         $user = User::factory()->create();
14         $response = $this->actingAs($user)->get('/profile');
15         $response->assertOk();
16     }
17
18     public function test_profile_information_can_be_updated(): void
19     {
20         $user = User::factory()->create();
21         $response = $this->actingAs($user)
22             ->patch('/profile', ['name' => 'Test User',
23                 'email' => 'test@example.com']);
24         $response->assertSessionHasNoErrors()
25             ->assertRedirect('/profile');
26         $user->refresh();
27         $this->assertSame('Test User', $user->name);
28     }
29
30     public function test_user_can_delete_their_account(): void
31     {
32         $user = User::factory()->create();
33         $response = $this->actingAs($user)
34             ->delete('/profile', ['password' => 'password']);
35         $this->assertGuest();
36     }
37 }
```

Unit Test & Feature Test Laravel 11 PHP 8.2

Menjalankan Test

Jalankan semua test dengan perintah:

php artisan test

- Opsi tambahan:
- php artisan test --verbose (detail per test dengan waktu eksekusi)
- php artisan test --filter=AuthTest (jalankan test spesifik)
- php artisan test --testsuite=Feature (hanya feature test)
- vendor/bin/phpunit (jalankan langsung via PHPUnit)

Membuat unit test dan feature test via artisan

```
$ php artisan make:test BarangTest --unit
Created: tests/Unit/BarangTest.php

$ php artisan make:test AuthTest
Created: tests/Feature/AuthTest.php

$ php artisan make:test ProfileTest
Created: tests/Feature/ProfileTest.php
```

Membaca Hasil Test

- PASS menandakan test suite berhasil, FAIL jika ada assertion gagal
- Tanda ✓ (checkmark) untuk test yang lulus, X (cross) untuk gagal
- Info environment: Environment: testing (sqlite://:memory:)
- Ringkasan: Tests: 15 passed, Assertions: 20 assertions

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "PHPUnit, testing, TDD". Analisis bagaimana konsep unit test dan feature test diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Hasil php artisan test — semua passing

```
$ php artisan test

PHPUnit 11.5.0 by Sebastian Bergmann and contributors.

PASS Tests\Unit\ExampleTest
  ☐ test user can be created with role
  ☐ test kategori can be created
  ☐ test barang belongs to kategori

PASS Tests\Feature\ExampleTest
  ☐ test the application redirects to login
  ☐ test login page can be accessed
  ☐ test dashboard requires authentication

PASS Tests\Feature\Auth\AuthenticationTest
  ☐ test login screen can be rendered
  ☐ test users can authenticate
  ☐ test users can not authenticate with invalid password
  ☐ test users can logout

PASS Tests\Feature\ProfileTest
  ☐ test profile page is displayed
  ☐ test profile information can be updated
  ☐ test user can delete their account

Tests: 15 passed
Time: 2.45s
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Detail test dengan verbose output

```
$ php artisan test --verbose

PASS Tests\Unit\ExampleTest
  @ test user can be created with role 0.12s
  @ test kategori can be created 0.08s
  @ test barang belongs to kategori 0.10s

PASS Tests\Feature\Auth\AuthenticationTest
  @ test login screen can be rendered 0.15s
  @ test users can authenticate 0.22s
  @ test users can not authenticate with invalid password 0.18s
  @ test users can logout 0.14s

Assertions: 20 assertions from 15 tests
Time: 2.45s, Memory: 34.50MB

Environment: testing (sqlite://:memory:)
```

phpunit.xml

```
phpunit.xml

1 <?xml version="1.0" encoding="UTF-8"?>
2 <phpunit colors="true" bootstrap="vendor/autoload.php">
3   <testsuites>
4     <testsuite name="Unit">
5       <directory>tests/Unit</directory>
6     </testsuite>
7     <testsuite name="Feature">
8       <directory>tests/Feature</directory>
9     </testsuite>
10  </testsuites>
11  <php>
12    <env name="DB_CONNECTION" value="sqlite"/>
13    <env name="DB_DATABASE" value=":memory:"/>
14    <env name="APP_ENV" value="testing"/>
15    <env name="CACHE_STORE" value="array"/>
16    <env name="SESSION_DRIVER" value="array"/>
17  </php>
18 </phpunit>
```

Unit Test ExampleTest

```
tests/Unit/ExampleTest.php

1 <?php
2 namespace Tests\Unit;
3 use App\Models\User;
4 use App\Models\Kategori;
5 use App\Models\Barang;
6 use Illuminate\Foundation\Testing\RefreshDatabase;
7 use Tests\TestCase;
8
9 class ExampleTest extends TestCase
10 {
11     use RefreshDatabase;
12
13     public function test_user_can_be_created_with_role(): void
14     {
15         $user = User::factory()->create(['role' => 'admin']);
16         $this->assertDatabaseHas('users',
17             ['id' => $user->id, 'role' => 'admin']);
18         $this->assertTrue($user->isAdmin());
19     }
20
21     public function test_kategori_can_be_created(): void
22     {
23         $kategori = Kategori::create([
24             'nama' => 'Test Kategori', 'deskripsi' => 'Test']);
25         $this->assertDatabaseHas('kategoris',
26             ['nama' => 'Test Kategori']);
27     }
28
29     public function test_barang_belongs_to_kategori(): void
30     {
31         $kategori = Kategori::factory()->create();
32         $barang = Barang::factory()->create(
33             ['kategori_id' => $kategori->id]);
34         $this->assertInstanceOf(Kategori::class,
35             $barang->kategori);
36     }
37 }
```

Feature Test Authentication

```
tests/Feature/Auth/AuthenticationTest.php

1  <?php
2  namespace Tests\Feature\Auth;
3  use App\Models\User;
4  use Illuminate\Foundation\Testing\RefreshDatabase;
5  use Tests\TestCase;
6
7  class AuthenticationTest extends TestCase
8  {
9      use RefreshDatabase;
10
11     public function test_login_screen_can_be_rendered(): void
12     {
13         $response = $this->get('/login');
14         $response->assertStatus(200);
15     }
16
17     public function test_users_can_authenticate(): void
18     {
19         $user = User::factory()->create();
20         $response = $this->post('/login', [
21             'email' => $user->email,
22             'password' => 'password',
23         ]);
24         $this->assertAuthenticated();
25         $response->assertRedirect(route('dashboard'));
26     }
27
28     public function test_users_can_not_authenticate
29         with_invalid_password(): void
30     {
31         $user = User::factory()->create();
32         $this->post('/login', [
33             'email' => $user->email,
34             'password' => 'wrong',
35         ]);
36         $this->assertGuest();
37     }
38 }
```

Unit Test & Feature Test Laravel 11 PHP 8.2

Feature Test Profile

```
tests/Feature/ProfileTest.php

1  <?php
2  namespace Tests\Feature;
3  use App\Models\User;
4  use Illuminate\Foundation\Testing\RefreshDatabase;
5  use Tests\TestCase;
6
7  class ProfileTest extends TestCase
8  {
9      use RefreshDatabase;
10
11     public function test_profile_page_is_displayed(): void
12     {
13         $user = User::factory()->create();
14         $response = $this->actingAs($user)->get('/profile');
15         $response->assertOk();
16     }
17
18     public function test_profile_information_can_be_updated(): void
19     {
20         $user = User::factory()->create();
21         $response = $this->actingAs($user)
22             ->patch('/profile', ['name' => 'Test User',
23                 'email' => 'test@example.com']);
24         $response->assertSessionHasNoErrors();
25         $response->assertRedirect('/profile');
26         $user->refresh();
27         $this->assertSame('Test User', $user->name);
28     }
29
30     public function test_user_can_delete_their_account(): void
31     {
32         $user = User::factory()->create();
33         $response = $this->actingAs($user)
34             ->delete('/profile', ['password' => 'password']);
35         $this->assertGuest();
36     }
37 }
```

Unit Test & Feature Test Laravel 11 PHP 8.2

Capture Terminal Output

Membuat test via artisan

```
$ php artisan make:test BarangTest --unit
Created: tests/Unit/BarangTest.php

$ php artisan make:test AuthTest
Created: tests/Feature/AuthTest.php

$ php artisan make:test ProfileTest
Created: tests/Feature/ProfileTest.php
```

Hasil test passing

```
$ php artisan test

PHPUnit 11.5.0 by Sebastian Bergmann and contributors.

PASS Tests\Unit\ExampleTest
  ☐ test user can be created with role
  ☐ test kategori can be created
  ☐ test barang belongs to kategori

PASS Tests\Feature\ExampleTest
  ☐ test the application redirects to login
  ☐ test login page can be accessed
  ☐ test dashboard requires authentication

PASS Tests\Feature\Auth\AuthenticationTest
  ☐ test login screen can be rendered
  ☐ test users can authenticate
  ☐ test users can not authenticate with invalid password
  ☐ test users can logout

PASS Tests\Feature\ProfileTest
  ☐ test profile page is displayed
  ☐ test profile information can be updated
  ☐ test user can delete their account

Tests: 15 passed
Time: 2.45s
```

Detail verbose test

```
$ php artisan test --verbose

PASS Tests\Unit\ExampleTest
  ☐ test user can be created with role 0.12s
  ☐ test kategori can be created 0.08s
  ☐ test barang belongs to kategori 0.10s

PASS Tests\Feature\Auth\AuthenticationTest
  ☐ test login screen can be rendered 0.15s
  ☐ test users can authenticate 0.22s
  ☐ test users can not authenticate with invalid password 0.18s
  ☐ test users can logout 0.14s

Assertions: 20 assertions from 15 tests
Time: 2.45s, Memory: 34.50MB

Environment: testing (sqlite://memory:)
```

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat dan menjalankan testing dengan PHPUnit

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 28. Deployment ke Hosting Lokal

Materi Pembelajaran : XAMPP, deployment, konfigurasi
 Acara Praktikum/Praktik : Minggu 14/2
 Tempat : Daring/Luring Politeknik Negeri Jember
 Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mampu melakukan deployment ke hosting lokal (TB430805.1)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.036.01	Melaksanakan pengujian kode program secara statis	• Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis • Membuat program driver test/stub • Melaksanakan pengujian • Mengukur aplikasi ketika stress test dijalankan • Menganalisis data pengujian • Melaporkan dokumentasi pengujian
2	J.620100.044.01	Menerapkan Alert Notification Jika Aplikasi Bermasalah	• Analisis permasalahan yang memerlukan tindakan cepat • Menentukan mekanisme notification sewaktu timbul suatu permasalahan • Menjalankan mekanisme graceful failure jika terjadi permasalahan
3	J.620100.045.01	Melakukan Pemantauan Resource yang Digunakan Aplikasi	• Analisis resources yang kritikal yang diperlukan aplikasi • Membuat modul visualisasi penggunaan resources
4	J.620100.047.01	Melakukan Pembaharuan Perangkat Lunak	• Menganalisis diferensiasi perangkat lunak yang terbaru dengan yang existing

c. Indikator Penilaian

- Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna(TB430805.1)
- Mengembangkan modul aplikasi Web Framework menggunakan fitur routing, controller, model, view(TB430805.2)
- Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi(TB430805.3)
- Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

d. Dasar Teori

Deployment lokal menggunakan XAMPP adalah proses menyiapkan dan menjalankan project Laravel di lingkungan server lokal sebelum dipindahkan ke server production. XAMPP menyediakan paket lengkap berupa Apache, MySQL/MariaDB, PHP, dan Perl yang memudahkan developer untuk melakukan simulasi server di komputer pribadi. Proses deployment lokal biasanya dilakukan dengan memindahkan project Laravel ke folder htdocs agar dapat diakses melalui web server Apache.

Tahapan penting dalam deployment meliputi pengelolaan database, yaitu melakukan export database dari server development dan import ke server lokal atau sebaliknya. Hal ini memastikan data yang digunakan tetap konsisten antara lingkungan pengembangan dan produksi. Selain itu, developer harus menyesuaikan file environment (.env), misalnya konfigurasi database, URL aplikasi, dan pengaturan lainnya agar sesuai dengan kondisi server production.

Secara teoritis, deployment lokal mendukung prinsip environment consistency dan system portability, yaitu memastikan aplikasi dapat berjalan dengan baik di berbagai lingkungan tanpa mengubah logika inti. Dengan adanya XAMPP, developer dapat melakukan uji coba penuh terhadap aplikasi, termasuk integrasi database, routing, dan tampilan, sebelum dipublikasikan ke server production. Hal ini juga mendukung prinsip risk mitigation, karena potensi error dapat dideteksi lebih awal di lingkungan lokal.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

1. Export database dari phpMyAdmin:

Export Database

Export database dari phpMyAdmin untuk dipindahkan ke environment production:

- Buka phpMyAdmin (<http://localhost/phpmyadmin>)
- Pilih database db_inventaris
- Menu Export -> pilih metode "Kustom"
- Format: SQL -> Go -> simpan file db_inventaris.sql

Copy Project ke httdocs

Copy seluruh folder project ke dalam direktori httdocs XAMPP:

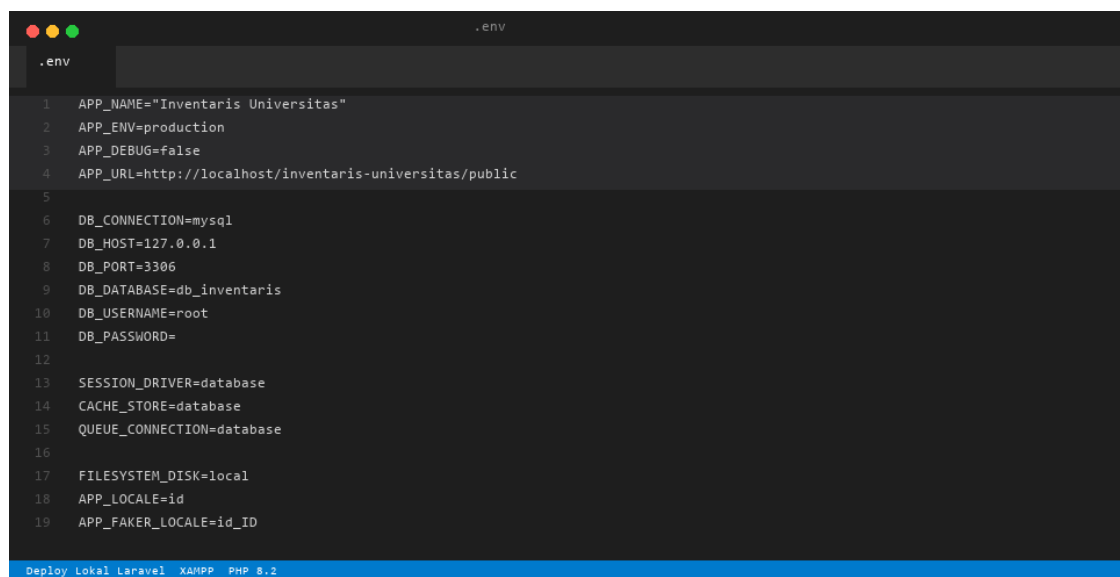
```
cp -r proyek C:/xampp/htdocs/inventaris-universitas
```

Konfigurasi .env

Ubah konfigurasi .env untuk mode production:

- APP_ENV=production (mengubah mode aplikasi)
- APP_DEBUG=false (menonaktifkan error detail)
- APP_URL=<http://localhost/inventaris-universitas/public> (base URL)
- DB_DATABASE=db_inventaris (nama database baru)

Konfigurasi .env untuk production



```
.env
1 APP_NAME="Inventaris Universitas"
2 APP_ENV=production
3 APP_DEBUG=false
4 APP_URL=http://localhost/inventaris-universitas/public
5
6 DB_CONNECTION=mysql
7 DB_HOST=127.0.0.1
8 DB_PORT=3306
9 DB_DATABASE=db_inventaris
10 DB_USERNAME=root
11 DB_PASSWORD=
12
13 SESSION_DRIVER=database
14 CACHE_STORE=database
15 QUEUE_CONNECTION=database
16
17 FILESYSTEM_DISK=local
18 APP_LOCALE=id
19 APP_FAKER_LOCALE=id_ID
```

Deploy Lokal Laravel XAMPP PHP 8.2

Konfigurasi .htaccess

File public/.htaccess digunakan Apache untuk URL rewriting:

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

- mod_rewrite diaktifkan untuk pretty URL Laravel
- RewriteRule ^ index.php [L] mengarahkan semua request ke front controller
- Authorization Header dan X-XSRF-Token ditangani untuk API

Konfigurasi .htaccess untuk URL rewriting

```
public/.htaccess

1 <IfModule mod_rewrite.c>
2   <IfModule mod_negotiation.c>
3     Options -MultiViews -Indexes
4   </IfModule>
5   RewriteEngine On
6
7   RewriteCond %{HTTP:Authorization} .
8   RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
9
10  RewriteCond %{HTTP:x-xsrf-token} .
11  RewriteRule .* - [E=HTTP_X_XSRF_TOKEN:%{HTTP:X-XSRF-Token}]
12
13  RewriteCond %{REQUEST_FILENAME} !-d
14  RewriteCond %{REQUEST_URI} (.+)/$
15  RewriteRule ^ %1 [L,R=301]
16
17  RewriteCond %{REQUEST_FILENAME} !-d
18  RewriteCond %{REQUEST_FILENAME} !-f
19  RewriteRule ^ index.php [L]
20 </IfModule>
```

Deploy Lokal Laravel XAMPP PHP 8.2

VirtualHost Apache

Konfigurasi VirtualHost di httpd-vhosts.conf untuk akses lebih rapi:

C:/xampp/apache/conf/extra/httpd-vhosts.conf

- DocumentRoot diarahkan ke folder public/ Laravel
- AllowOverride All mengizinkan .htaccess bekerja
- Require all granted mengizinkan akses dari semua IP

VirtualHost Apache untuk Laravel

```
C:/xampp/apache/conf/extra/httpd-vhosts.conf

1 <VirtualHost *:80>
2   DocumentRoot "C:/xampp/htdocs/inventaris-universitas/public"
3   ServerName inventaris.local
4
5   <Directory "C:/xampp/htdocs/inventaris-universitas/public">
6     Options Indexes FollowSymLinks
7     AllowOverride All
8     Require all granted
9   </Directory>
10
11   ErrorLog "logs/inventaris-error.log"
12   CustomLog "logs/inventaris-access.log" common
13 </VirtualHost>
```

Deploy Lokal Laravel XAMPP PHP 8.2

Import Database

Import database yang telah di-export ke environment production:

- Buat database baru: `mysql -u root -e "CREATE DATABASE db_inventaris"`
- Import file SQL: `mysql -u root db_inventaris < db_inventaris.sql`
- Atau import melalui phpMyAdmin -> Import -> pilih file .sql -> Go

Import database di phpMyAdmin

Generate Key & Optimasi

Jalankan perintah artisan untuk menyiapkan aplikasi:

- `php artisan key:generate` — menghasilkan APP_KEY baru untuk environment baru
- `php artisan config:cache` — menggabungkan semua config menjadi satu file
- `php artisan route:cache` — caching route untuk mempercepat routing
- `php artisan view:cache` — kompilasi semua Blade view
- `php artisan storage:link` — membuat symlink untuk file upload

Generate key & optimasi caching

```
# 4. Generate application key
$ php artisan key:generate
Application key set successfully.

# 5. Cache optimization
$ php artisan config:cache
Configuration cached successfully.
$ php artisan route:cache
Routes cached successfully.
$ php artisan view:cache
Views cached successfully.

# 6. Storage link
$ php artisan storage:link
The [public/storage] link has been connected.
```

Uji Coba Aplikasi

- Buka browser dan akses `http://localhost/inventaris-universitas/public`
- Aplikasi akan redirect ke halaman login
- Login menggunakan akun yang sudah dibuat sebelumnya
- Pastikan semua fitur berjalan normal (CRUD, laporan, filter)
- Cek bahwa error tidak menampilkan detail (karena APP_DEBUG=false)
- Jika menggunakan VirtualHost, akses via `http://inventaris.local`

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "XAMPP, deployment, konfigurasi". Analisis bagaimana konsep deployment ke hosting lokal diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

.env production

```
.env
1 APP_NAME="Inventaris Universitas"
2 APP_ENV=production
3 APP_DEBUG=false
4 APP_URL=http://localhost/inventaris-universitas/public
5
6 DB_CONNECTION=mysql
7 DB_HOST=127.0.0.1
8 DB_PORT=3306
9 DB_DATABASE=db_inventaris
10 DB_USERNAME=root
11 DB_PASSWORD=
12
13 SESSION_DRIVER=database
14 CACHE_STORE=database
15 QUEUE_CONNECTION=database
16
17 FILESYSTEM_DISK=local
18 APP_LOCALE=id
19 APP_FAKER_LOCALE=id_ID

Deploy Lokal Laravel XAMPP PHP 8.2
```

public/.htaccess

```
public/.htaccess
1 <IfModule mod_rewrite.c>
2 <IfModule mod_negotiation.c>
3 Options -MultiViews -Indexes
4 </IfModule>
5 RewriteEngine On
6
7 RewriteCond %{HTTP:Authorization} .
8 RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
9
10 RewriteCond %{HTTP:X-XSRF-Token} .
11 RewriteRule .* - [E=HTTP_X_XSRF_TOKEN:%{HTTP:X-XSRF-Token}]
12
13 RewriteCond %{REQUEST_FILENAME} !-d
14 RewriteCond %{REQUEST_URI} (.+)/$
15 RewriteRule ^ %1 [L,R=301]
16
17 RewriteCond %{REQUEST_FILENAME} !-d
18 RewriteCond %{REQUEST_FILENAME} !-f
19 RewriteRule ^ index.php [L]
20 </IfModule>

Deploy Lokal Laravel XAMPP PHP 8.2
```

VirtualHost Apache

```
C:/xampp/apache/conf/extra/httpd-vhosts.conf
C:/xampp/apache/conf/extra/httpd-vhosts.conf
1 <VirtualHost *:80>
2 DocumentRoot "C:/xampp/htdocs/inventaris-universitas/public"
3 ServerName inventaris.local
4
5 <Directory "C:/xampp/htdocs/inventaris-universitas/public">
6 Options Indexes FollowSymLinks
7 AllowOverride All
8 Require all granted
9 </Directory>
10
11 ErrorLog "logs/inventaris-error.log"
12 CustomLog "logs/inventaris-access.log" common
13 </VirtualHost>

Deploy Lokal Laravel XAMPP PHP 8.2
```

Terminal export & copy project

```
# 1. Export database dari phpMyAdmin
Buka phpMyAdmin -> pilih db_inventaris
Export -> SQL -> Go -> simpan db_inventaris.sql

# 2. Copy project kehtdocs
$ cp -r proyek C:/xampp/htdocs/inventaris-universitas
Copying files... 12,456 files copied

# 3. Import database
$ mysql -u root -e "CREATE DATABASE db_inventaris"
Query OK, 1 row affected
$ mysql -u root db_inventaris < db_inventaris.sql
Import completed: 15 tables
```

Terminal generate key & caching

```
# 4. Generate application key
$ php artisan key:generate
Application key set successfully.

# 5. Cache optimization
$ php artisan config:cache
Configuration cached successfully.
$ php artisan route:cache
Routes cached successfully.
$ php artisan view:cache
Views cached successfully.

# 6. Storage link
$ php artisan storage:link
The [public/storage] link has been connected.
```

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu melakukan deployment ke hosting lokal

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
Total Skor*		Skor x 25			

Acara 29. Deployment ke Hosting Publik

Materi Pembelajaran : Hosting publik, FTP, domain
Acara Praktikum/Praktik : Minggu 15/1
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mampu melakukan deployment ke hosting publik (TB430805.1)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.62010 0.036.0 1	Melaksanakan pengujian kode program secara statis	Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis

c. Indikator Penilaian

Mahasiswa mampu melakukan deployment ke hosting public

d. Dasar Teori

Deployment ke hosting publik seperti 000webhost, Niagahoster, atau layanan serupa adalah proses memindahkan aplikasi dari lingkungan lokal ke server yang dapat diakses secara global melalui internet. Deployment ini bertujuan agar aplikasi dapat digunakan oleh pengguna secara luas, tidak terbatas pada komputer pengembang.

- Proses Deployment
 - Upload file via FTP : Semua file project Laravel dipindahkan ke server hosting menggunakan protokol FTP (File Transfer Protocol) atau panel file manager. Hal ini mendukung prinsip data transfer reliability, yaitu memastikan seluruh file aplikasi tersalin dengan benar ke server.
 - Import database : Database yang digunakan di lokal diekspor, kemudian diimpor ke server hosting. Proses ini menjaga data consistency, sehingga data yang ada di production sama dengan data di lingkungan pengembangan.

- Konfigurasi environment : File .env disesuaikan dengan konfigurasi server, seperti nama database, username, password, dan URL aplikasi. Penyesuaian ini mendukung prinsip environment adaptability, yaitu kemampuan aplikasi untuk berjalan sesuai dengan kondisi server yang berbeda.
- Setting domain : Domain dikaitkan dengan server hosting agar aplikasi dapat diakses melalui alamat web yang mudah diingat. Hal ini mendukung prinsip usability dan accessibility, karena pengguna dapat mengakses aplikasi dengan cara yang lebih praktis.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Registrasi Hosting & Domain

Pilih dan daftar hosting provider:

- 000webhost (gratis): 1GB storage, PHP 8.2, MySQL, support Laravel
- Niagahoster (berbayar): unlimited storage, SSD, cPanel, Softaculous
- Hostinger: 100GB bandwidth, free SSL, support SSH
- Daftar domain (contoh: inventaris-universitas.com)
- Arahkan DNS A record ke IP server hosting

Registrasi hosting & domain

```
# Hosting Provider Options
1. 000webhost (Free) - 1GB storage, 10GB bandwidth
2. Niagahoster (Paid) - Unlimited storage, SSD
3. Hostinger (Paid) - 100GB bandwidth, Free SSL

# Domain Setup
Purchase domain: inventaris-universitas.com
Point DNS A record to hosting server IP

# PHP & MySQL Version Check
$ php -v | grep PHP
PHP 8.2.12 (minimum Laravel 11: PHP 8.2)
$ mysql --version
mysql Ver 8.0.35 (minimum: MySQL 5.7)
```

Export Database

Export database dari lingkungan development:

```
mysqldump -u root db_inventaris > db_inventaris.sql
```

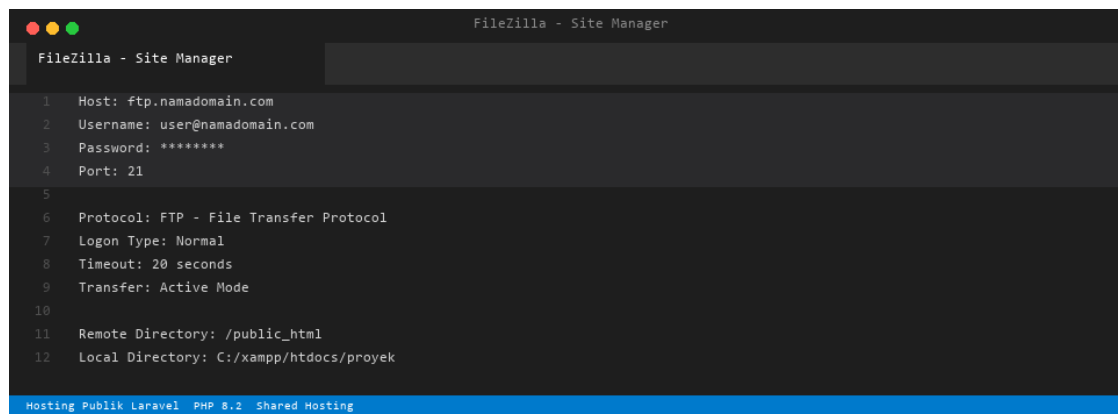
- Atau export via phpMyAdmin -> Export -> SQL -> Go

Upload Project via FTP

Upload project Laravel ke server hosting menggunakan FileZilla:

- Buka FileZilla -> File -> Site Manager -> New Site
- Isi Host: ftp.namadomain.com, Username, Password, Port: 21
- Connect -> Drag folder project ke /public_html/
- Pastikan folder public/ menjadi root directory

Konfigurasi FTP FileZilla



Upload file via FileZilla

Import Database di Hosting

Import database yang telah di-export ke server hosting:

- Buat database baru di cPanel (contoh: u1234567_inventaris)
- Buka phpMyAdmin hosting -> pilih database -> Import
- Pilih file .sql -> Go
- Atau via command line: `mysql -u user -p db_name < file.sql`

Import database di phpMyAdmin hosting

Konfigurasi .env

Sesuaikan .env dengan environment hosting:

- APP_URL=https://inventaris.namadomain.com (ganti dengan domain)
- DB_HOST, DB_DATABASE, DB_USERNAME, DB_PASSWORD sesuai data hosting
- APP_ENV=production, APP_DEBUG=false untuk keamanan

Konfigurasi .env untuk shared hosting

```
.env
1 APP_NAME="Inventaris Universitas"
2 APP_ENV=production
3 APP_DEBUG=false
4 APP_URL=https://inventaris.namadomain.com
5
6 DB_CONNECTION=mysql
7 DB_HOST=localhost
8 DB_DATABASE=u1234567_inventaris
9 DB_USERNAME=u1234567_admin
10 DB_PASSWORD=*****
11
12 SESSION_DRIVER=file
13 CACHE_STORE=file
14 QUEUE_CONNECTION=sync
15
16 FILESYSTEM_DISK=local
17 APP_LOCALE=id

Hosting Publik Laravel PHP 8.2 Shared Hosting
```

.htaccess untuk URL rewriting di hosting

```
public_html/.htaccess
1 <IfModule mod_rewrite.c>
2     <IfModule mod_negotiation.c>
3         Options -MultiViews -Indexes
4     </IfModule>
5     RewriteEngine On
6
7     RewriteCond %{HTTP:Authorization} .
8     RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
9
10    RewriteCond %{REQUEST_FILENAME} !-d
11    RewriteCond %{REQUEST_FILENAME} !-f
12    RewriteRule ^ index.php [L]
13 </IfModule>

Hosting Publik Laravel PHP 8.2 Shared Hosting
```

Generate Key & Optimasi

Jalankan perintah via SSH atau terminal hosting:

- ssh ke server atau gunakan terminal di cPanel
- php artisan key:generate — set application key
- php artisan config:cache — caching konfigurasi
- php artisan route:cache — caching route
- php artisan view:cache — caching Blade view

Generate key, optimasi caching & set permission via SSH

```
# 4. Generate application key
$ ssh user@domain.com "cd public_html && php artisan key:generate"
Application key set successfully.

# 5. Cache optimization
$ ssh user@domain.com "cd public_html && php artisan config:cache"
Configuration cached successfully.
$ ssh user@domain.com "cd public_html && php artisan route:cache"
Routes cached successfully.
$ ssh user@domain.com "cd public_html && php artisan view:cache"
Views cached successfully.

# 6. Set permission
$ ssh user@domain.com "chmod -R 775 public_html/storage"
Permission set: 775
$ ssh user@domain.com "chmod -R 775 public_html/bootstrap/cache"
Permission set: 775
```

Set Permission Storage

Set permission folder agar writable oleh web server:

```
chmod -R 775 storage
```

```
chmod -R 775 bootstrap/cache
```

- Folder storage dan bootstrap/cache harus writable untuk log, upload, cache Uji Coba Aplikasi
- Buka browser dan akses <https://inventaris.namadomain.com>
- Aplikasi akan redirect ke halaman login
- Login dengan akun yang sudah ada
- Pastikan semua fitur berjalan: CRUD, upload, laporan PDF/Excel
- Cek bahwa debug mode sudah non-aktif (APP_DEBUG=false)
- Pastikan HTTPS/SSL berfungsi dengan benar

Export database & upload project ke hosting

```
# 1. Export database
$ mysqldump -u root db_inventaris > db_inventaris.sql
Export completed: 15 tables, 2.3 MB

# 2. Upload via FTP (command line)
$ curl -T proyek.tar.gz ftp://user@domain.com/public_html/
Upload progress: 100% (12,456 files)
$ ssh user@domain.com "cd public_html && tar -xzf proyek.tar.gz"
Extraction completed

# 3. Import database di hosting
$ mysql -h localhost -u u1234567_admin -p u1234567_inventaris < db_inventaris.sql
Import completed: 15 tables
```

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "Hosting publik, FTP, domain". Analisis bagaimana konsep deployment ke hosting publik diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

.env hosting

```
.env
APP_NAME="Inventaris Universitas"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://inventaris.namadomain.com

DB_CONNECTION=mysql
DB_HOST=localhost
DB_DATABASE=u1234567_inventaris
DB_USERNAME=u1234567_admin
DB_PASSWORD=*****

SESSION_DRIVER=file
CACHE_STORE=file
QUEUE_CONNECTION=sync

FILESYSTEM_DISK=local
APP_LOCALE=id
```

Hosting Publik Laravel PHP 8.2 Shared Hosting

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

.htaccess

```
public_html/.htaccess

1 <IfModule mod_rewrite.c>
2   <IfModule mod_negotiation.c>
3     Options -MultiViews -Indexes
4   </IfModule>
5   RewriteEngine On
6
7   RewriteCond %{HTTP:Authorization} .
8   RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
9
10  RewriteCond %{REQUEST_FILENAME} !-d
11  RewriteCond %{REQUEST_FILENAME} !-f
12  RewriteRule ^ index.php [L]
13 </IfModule>
```

Hosting Publik Laravel PHP 8.2 Shared Hosting

Konfigurasi FileZilla

```
FileZilla - Site Manager

1 Host: ftp.namadomain.com
2 Username: user@namadomain.com
3 Password: *****
4 Port: 21
5
6 Protocol: FTP - File Transfer Protocol
7 Logon Type: Normal
8 Timeout: 20 seconds
9 Transfer: Active Mode
10
11 Remote Directory: /public_html
12 Local Directory: C:/xampp/htdocs/proyek
```

Hosting Publik Laravel PHP 8.2 Shared Hosting

Info hosting & domain

```
# Hosting Provider Options
1. 000webhost (Free) - 1GB storage, 10GB bandwidth
2. Niagahoster (Paid) - Unlimited storage, SSD
3. Hostinger (Paid) - 100GB bandwidth, Free SSL

# Domain Setup
Purchase domain: inventaris-universitas.com
Point DNS A record to hosting server IP

# PHP & MySQL Version Check
$ php -v | grep PHP
PHP 8.2.12 (minimum Laravel 11: PHP 8.2)
$ mysql --version
mysql Ver 8.0.35 (minimum: MySQL 5.7)
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

Export & upload via terminal

```
# 1. Export database
$ mysqldump -u root db_inventaris > db_inventaris.sql
Export completed: 15 tables, 2.3 MB

# 2. Upload via FTP (command line)
$ curl -T proyek.tar.gz ftp://user@domain.com/public_html/
Upload progress: 100% (12,456 files)
$ ssh user@domain.com "cd public_html && tar -xzf proyek.tar.gz"
Extraction completed

# 3. Import database di hosting
$ mysql -h localhost -u u1234567_admin -p u1234567_inventaris < db_inventaris.sql
Import completed: 15 tables
```

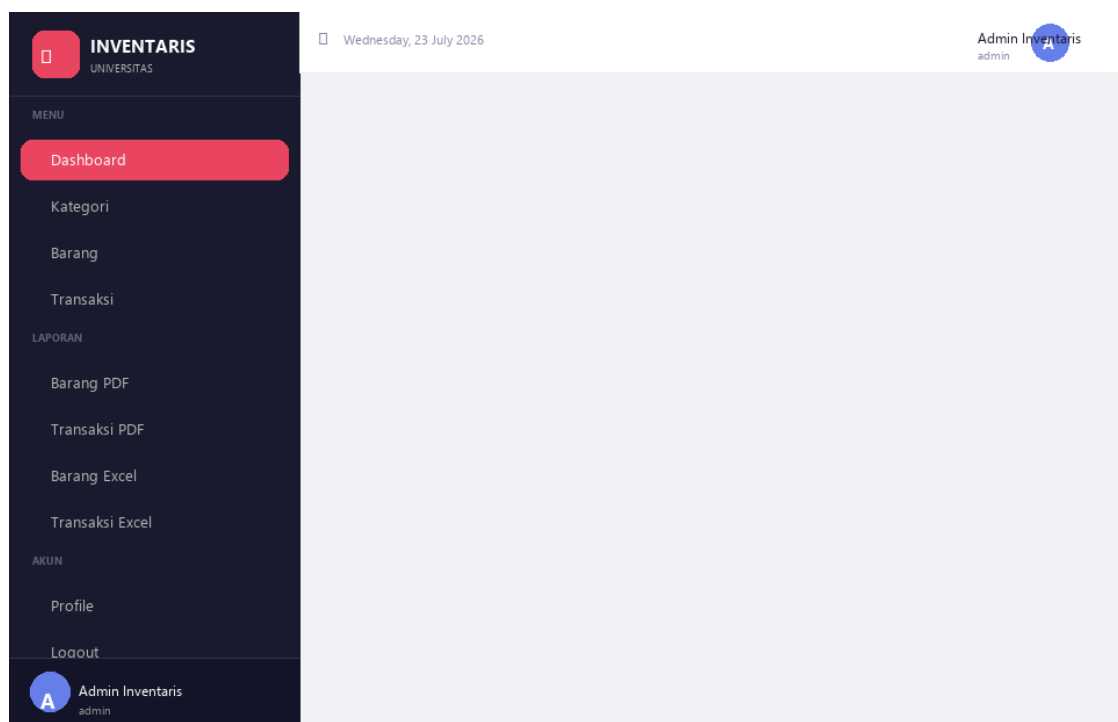
Artisan commands via SSH

```
# 4. Generate application key
$ ssh user@domain.com "cd public_html && php artisan key:generate"
Application key set successfully.

# 5. Cache optimization
$ ssh user@domain.com "cd public_html && php artisan config:cache"
Configuration cached successfully.
$ ssh user@domain.com "cd public_html && php artisan route:cache"
Routes cached successfully.
$ ssh user@domain.com "cd public_html && php artisan view:cache"
Views cached successfully.

# 6. Set permission
$ ssh user@domain.com "chmod -R 775 public_html/storage"
Permission set: 775
$ ssh user@domain.com "chmod -R 775 public_html/bootstrap/cache"
Permission set: 775
```

Aplikasi running di domain



Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu melakukan deployment ke hosting publik

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			

Acara 30. Dokumentasi Proyek (README)

Materi Pembelajaran : README, dokumentasi, GitHub
Acara Praktikum/Praktik : Minggu 15/2
Tempat : Daring/Luring Politeknik Negeri Jember
Alokasi Waktu : 480 menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

- Merancang sistem informasi berbasis Web Framework sesuai kebutuhan pengguna(TB430805.1)
- Mengembangkan modul aplikasi Web Framework menggunakan fitur routing, controller, model, view(TB430805.2)
- Mengintegrasikan middleware, autentikasi, validasi, dan file upload dalam aplikasi(TB430805.3)
- Menerapkan konsep database relasional melalui Eloquent ORM (TB430805.4)

b. Penilaian Bertahap BNSP (*Skill Passport*)

Nama Skema Sertifikasi Pengembang Web/Web Developer (SKM-496-028):

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.62010 0.036.0 1	Melaksanakan pengujian kode program secara statis	Mempersiapkan dokumentasi peralatan dan lingkungan pengujian statis

c. Indikator Penilaian

Mahasiswa mampu membuat dokumentasi proyek

d. Dasar Teori

README.md adalah dokumentasi utama yang biasanya ditempatkan di akar proyek GitHub. File ini berfungsi sebagai pintu masuk pertama bagi pengguna maupun kontributor untuk memahami proyek. README.md ditulis dalam format Markdown sehingga mudah dibaca sekaligus fleksibel untuk menampilkan teks, kode, maupun tautan.

Isi README.md umumnya mencakup deskripsi proyek yang menjelaskan tujuan dan ruang lingkup aplikasi, fitur-fitur utama yang tersedia, serta instruksi instalasi agar pengguna dapat menyiapkan lingkungan kerja dengan benar. Selain itu, README.md

juga memuat cara penggunaan yang menjelaskan langkah-langkah menjalankan aplikasi, struktur database untuk memberikan gambaran skema data yang digunakan, dan panduan kontribusi bagi pengembang lain yang ingin berpartisipasi.

e. Alat dan Bahan

- Web Server - XAMPP
- Text Editor: VS Code
- Git dan Github
- Composer.

f. Prosedur Kerja

Membuat README.md

Buat file README.md di root project dengan informasi lengkap:

- # Judul: Sistem Informasi Inventaris Barang Universitas
- ## Fitur: daftar fitur lengkap aplikasi
- ## Cara Instalasi: langkah-langkah dari clone hingga run
- ## Struktur Database: tabel-tabel utama
- ## Akun Demo: admin dan staff dengan email & password
- ## Lisensi: informasi lisensi project

README.md - Informasi project & instalasi

```
README.md
1 # Sistem Informasi Inventaris Barang Universitas
2
3 Sistem manajemen inventaris barang berbasis web
4 menggunakan Laravel 11.
5
6 ## Fitur
7 - Manajemen Kategori dan Barang
8 - Peminjaman dan Pengembalian Barang
9 - Autentikasi Multi Role (Admin, Staff)
10 - Dashboard Role-Based
11 - Laporan PDF (DOMPDF) dan Excel
12 - AJAX, DataTables Server Side, Filter/Search
13 - Laravel Debugbar, Logging, Error Pages
14 - Unit Test & Feature Test (PHPUnit)
15
16 ## Cara Instalasi
17 1. Clone repository
18    `git clone https://github.com/username/inventaris.git`
19 2. Install dependencies
20    `composer install && npm install && npm run build`
21 3. Copy env `cp .env.example .env`
22 4. Generate key `php artisan key:generate`
23 5. Buat database & konfigurasi .env
24 6. Migration & seed `php artisan migrate --seed`
README.md GitHub Documentation
```

README.md - Database, teknologi & akun demo

```
README.md (lanjutan)

1  ## Struktur Database
2  - `users` - data pengguna & role
3  - `kategoris` - kategori barang
4  - `barangs` - data barang (relasi kategori)
5  - `transaksis` - peminjaman & pengembalian
6  - `barang_user` - favorit (many-to-many)
7
8  ## Teknologi
9  - Laravel 11, PHP 8.2, MySQL 8.0
10 - Tailwind CSS, FontAwesome, DataTables
11 - DOMPDF, Laravel Excel, Debugbar
12 - PHPUnit, Laravel Breeze (Auth)
13
14 ## Akun Demo
15 | Role | Email | Password |
16 |-----|-----|-----|
17 | Admin | admin@inventaris.test | password |
18 | Staff | staff@inventaris.test | password |
19
20 ## Lisensi
21 MIT License

README.md GitHub Documentation
```

Konfigurasi .gitignore

File .gitignore menentukan file/folder yang tidak di-track Git:

- vendor/ dan node_modules/ (dependencies)
- .env (konfigurasi lokal)
- storage/*.key (application key)
- public/build/ (Vite build output)
- *.log, .DS_Store, Thumbs.db (file sistem)

.gitignore untuk Laravel project

```
.gitignore

1  /vendor/
2  /node_modules/
3  /public/build/
4  /public/hot
5  /storage/*.key
6  /.env
7  /.env.backup
8  /.phpunit.result.cache
9  /docker-compose.yml
10 /docker-compose.override.yml
11
12 Homestead.json
13 Homestead.yaml
14 auth.json
15 *.log
16 .DS_Store
17 Thumbs.db

README.md GitHub Documentation
```

Inisialisasi Git & Commit

Inisialisasi repository Git dan commit awal:

- `git init` — memulai repository Git di folder project
- `git add .` — staging semua file (kecuali di `.gitignore`)
- `git commit -m "feat: inventaris barang universitas"` — commit awal

Push ke GitHub

Hubungkan repository lokal ke GitHub dan push:

- Buat repository baru di GitHub (tanpa README)
- `git remote add origin https://github.com/username/inventaris.git`
- `git branch -M main` — rename branch ke main
- `git push -u origin main` — push ke GitHub

Inisialisasi Git, commit, dan push ke GitHub

```
# Inisialisasi Git & Commit Awal
$ git init
Initialized empty Git repository
$ git add .
Changes to be committed: 156 files
$ git commit -m "feat: inventaris barang universitas"
[main (root-commit) a1b2c3d] feat: inventaris barang universitas
156 files changed, 28450 insertions(+)

# Setup Remote & Push
$ git remote add origin https://github.com/username/inventaris.git
Remote "origin" added
$ git branch -M main
$ git push -u origin main
Enumerating objects: 215, done.
To https://github.com/username/inventaris.git
* [new branch] main -> main
```

Menambahkan Screenshot

Tambahkan screenshot aplikasi ke dalam README:

- Buat folder `screenshots/` di root project
- Copy gambar hasil capture ke folder `screenshots/`
- Gunakan markdown: `![alt](screenshots/file.png)`
- Screenshot menunjukkan dashboard, barang, transaksi, laporan

Menambahkan screenshot & update README

```
# Membuat folder screenshots
$ mkdir -p screenshots
$ cp -r storage/captures/*.png screenshots/
Copied 12 screenshot files

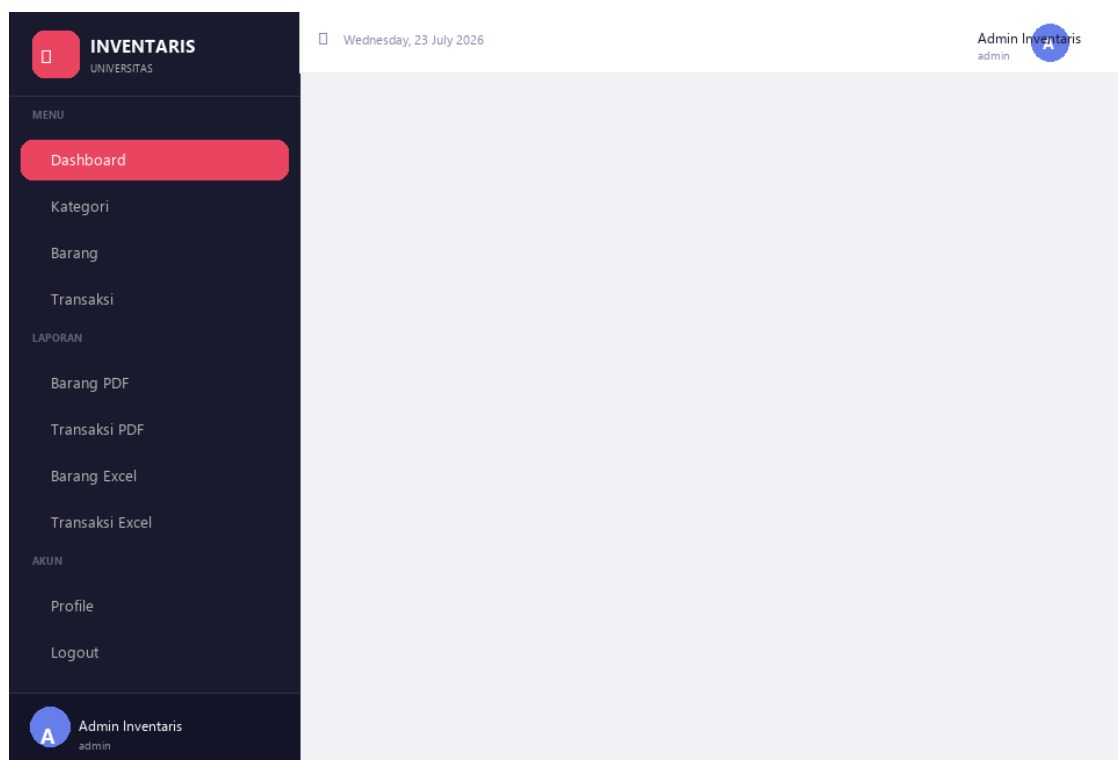
# Update README & Push
$ git add README.md screenshots/
$ git commit -m "docs: update readme with screenshots"
[main 2b3c4d5] docs: update readme with screenshots
$ git push
To https://github.com/username/inventaris.git
main -> main
```

Update README & Badges

Lengkapi README dengan badges dan informasi tambahan:

- Badge build status: ![build](https://img.shields.io/badge/build-passing-green)
- Badge PHP version: ![php](https://img.shields.io/badge/php-8.2-blue)
- Badge license: ![license](https://img.shields.io/badge/license-MIT-yellow)
- Badge version: ![version](https://img.shields.io/badge/version-1.0.0-orange)
- Git add, commit, push perubahan README

README dengan screenshot aplikasi



git push origin main

Study Kasus

Implementasikan langkah-langkah praktikum di atas pada studi kasus Sistem Informasi Inventaris Barang Universitas untuk materi "README, dokumentasi, GitHub". Analisis bagaimana konsep dokumentasi proyek (readme) diterapkan dalam konteks manajemen inventaris barang universitas.

g. Hasil dan Pembahasan

Mahasiswa berhasil membuat dokumentasi proyek README yang lengkap.

README.md bagian atas

```
README.md

1 # Sistem Informasi Inventaris Barang Universitas
2
3 Sistem manajemen inventaris barang berbasis web
4 menggunakan Laravel 11.
5
6 ## Fitur
7 - Manajemen Kategori dan Barang
8 - Peminjaman dan Pengembalian Barang
9 - Autentikasi Multi Role (Admin, Staff)
10 - Dashboard Role-Based
11 - Laporan PDF (DOMPDF) dan Excel
12 - AJAX, DataTables Server Side, Filter/Search
13 - Laravel Debugbar, Logging, Error Pages
14 - Unit Test & Feature Test (PHPUnit)
15
16 ## Cara Instalasi
17 1. Clone repository
18    `git clone https://github.com/username/inventaris.git`
19 2. Install dependencies
20    `composer install && npm install && npm run build`
21 3. Copy env `cp .env.example .env`
22 4. Generate key `php artisan key:generate`
23 5. Buat database & konfigurasi .env
24 6. Migration & seed `php artisan migrate --seed`

README.md  GitHub  Documentation
```

README.md bagian lanjutan

```
README.md (lanjutan)

1 ## Struktur Database
2 - `users` - data pengguna & role
3 - `kategoris` - kategori barang
4 - `barang` - data barang (relasi kategori)
5 - `transaksis` - peminjaman & pengembalian
6 - `barang_user` - favorit (many-to-many)
7
8 ## Teknologi
9 - Laravel 11, PHP 8.2, MySQL 8.0
10 - Tailwind CSS, FontAwesome, DataTables
11 - DOMPDF, Laravel Excel, Debugbar
12 - PHPUnit, Laravel Breeze (Auth)
13
14 ## Akun Demo
15 | Role | Email | Password |
16 |-----|-----|-----|
17 | Admin | admin@inventaris.test | password |
18 | Staff | staff@inventaris.test | password |
19
20 ## Lisensi
21 MIT License

README.md  GitHub  Documentation
```

BKPM - (WORKSHOP SISTEM INFORMASI BERBASIS WEB 2)

.gitignore

```
.gitignore
1 /vendor/
2 /node_modules/
3 /public/build/
4 /public/hot
5 /storage/*.key
6 /.env
7 /.env.backup
8 /.phpunit.result.cache
9 /docker-compose.yml
10 /docker-compose.override.yml
11
12 Homestead.json
13 Homestead.yaml
14 auth.json
15 *.log
16 .DS_Store
17 Thumbs.db

README.md  GitHub  Documentation
```

Git init, commit & push

```
# Inisialisasi Git & Commit Awal
$ git init
  Initialized empty Git repository
$ git add .
  Changes to be committed: 156 files
$ git commit -m "feat: inventaris barang universitas"
[main (root-commit) a1b2c3d] feat: inventaris barang universitas
156 files changed, 28450 insertions(+)

# Setup Remote & Push
$ git remote add origin https://github.com/username/inventaris.git
  Remote "origin" added
$ git branch -M main
$ git push -u origin main
  Enumerating objects: 215, done.
  To https://github.com/username/inventaris.git
  * [new branch] main -> main
```

Tambah screenshot

```
# Membuat folder screenshots
$ mkdir -p screenshots
$ cp -r storage/captures/*.png screenshots/
  Copied 12 screenshot files

# Update README & Push
$ git add README.md screenshots/
$ git commit -m "docs: update readme with screenshots"
[main 2b3c4d5] docs: update readme with screenshots
$ git push
  To https://github.com/username/inventaris.git
  main -> main
```

Tugas

Capture/Ambil screenshot dari setiap hasil luaran/output yang muncul setelah menjalankan langkah-langkah praktikum di atas. Beri nomor dan keterangan pada setiap screenshot. Pastikan screenshot menunjukkan tampilan/output yang sesuai dengan langkah praktikum.

Kesimpulan

- Mahasiswa mampu membuat dokumentasi proyek

h. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan Langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan Langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan Langkah kerja yang dilakukan kurang tepat		V		
4	Keterlambatan waktu pengumpulan dan Langkah kerja tidak seusai	V			
	Total Skor*	Skor x 25			